

DM35418 Linux Library API Documentation

Generated by Doxygen 1.9.1

1 Module Index	1
1.1 Modules	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	5
3.1 File List	5
4 Module Documentation	7
4.1 DM35418 ADC Library Constants	7
4.1.1 Detailed Description	10
4.1.2 Enumeration Type Documentation	10
4.1.2.1 DM35418_Adc_Clock_Events	10
4.1.2.2 DM35418_Gains	11
4.1.2.3 DM35418_Input_Mode	11
4.1.2.4 DM35418_Input_Ranges	12
4.2 DM35418 ADC Public Library Functions	12
4.2.1 Detailed Description	15
4.2.2 Function Documentation	15
4.2.2.1 DM35418_Adc_Channel_Find_Interrupt()	15
4.2.2.2 DM35418_Adc_Channel_Get_Filter()	16
4.2.2.3 DM35418_Adc_Channel_Get_Front_End_Config()	16
4.2.2.4 DM35418_Adc_Channel_Get_Last_Sample()	17
4.2.2.5 DM35418_Adc_Channel_Get_Thresholds()	17
4.2.2.6 DM35418_Adc_Channel_Interrupt_Clear_Status()	18
4.2.2.7 DM35418_Adc_Channel_Interrupt_Get_Config()	19
4.2.2.8 DM35418_Adc_Channel_Interrupt_Get_Status()	19
4.2.2.9 DM35418_Adc_Channel_Interrupt_Set_Config()	20
4.2.2.10 DM35418_Adc_Channel_Reset()	21
4.2.2.11 DM35418_Adc_Channel_Set_Filter()	21
4.2.2.12 DM35418_Adc_Channel_Set_High_Threshold()	22
4.2.2.13 DM35418_Adc_Channel_Set_Low_Threshold()	22
4.2.2.14 DM35418_Adc_Channel_Setup()	23
4.2.2.15 DM35418_Adc_Fifo_Channel_Read()	24
4.2.2.16 DM35418_Adc_Get_Clock_Source_Global()	24
4.2.2.17 DM35418_Adc_Get_Clock_Src()	25
4.2.2.18 DM35418_Adc_Get_Mode_Status()	26
4.2.2.19 DM35418_Adc_Get_Post_Stop_Samples()	26
4.2.2.20 DM35418_Adc_Get_Pre_Trigger_Samples()	27
4.2.2.21 DM35418_Adc_Get_Sample_Count()	27
4.2.2.22 DM35418_Adc_Get_Start_Trigger()	28
4.2.2.23 DM35418_Adc_Get_Stop_Trigger()	28

4.2.2.24	DM35418_Adc_Initialize()	29
4.2.2.25	DM35418_Adc_Interrupt_Clear_Status()	29
4.2.2.26	DM35418_Adc_Interrupt_Get_Config()	30
4.2.2.27	DM35418_Adc_Interrupt_Get_Status()	30
4.2.2.28	DM35418_Adc_Interrupt_Set_Config()	31
4.2.2.29	DM35418_Adc_Open()	31
4.2.2.30	DM35418_Adc_Pause()	32
4.2.2.31	DM35418_Adc_Reset()	32
4.2.2.32	DM35418_Adc_Sample_To_Volts()	33
4.2.2.33	DM35418_Adc_Set_Clk_Divider()	33
4.2.2.34	DM35418_Adc_Set_Clock_Source_Global()	34
4.2.2.35	DM35418_Adc_Set_Clock_Src()	34
4.2.2.36	DM35418_Adc_Set_Post_Stop_Samples()	35
4.2.2.37	DM35418_Adc_Set_Pre_Trigger_Samples()	35
4.2.2.38	DM35418_Adc_Set_Sample_Rate()	36
4.2.2.39	DM35418_Adc_Set_Start_Trigger()	37
4.2.2.40	DM35418_Adc_Set_Stop_Trigger()	37
4.2.2.41	DM35418_Adc_Start()	38
4.2.2.42	DM35418_Adc_Start_Rearm()	38
4.2.2.43	DM35418_Adc_Uninitialize()	39
4.2.2.44	DM35418_Adc_Volts_To_Sample()	39
4.3	DM35418 Board Access Structures	40
4.3.1	Detailed Description	40
4.3.2	Function Documentation	40
4.3.2.1	DM35418_Board_Close()	40
4.3.2.2	DM35418_Board_Open()	41
4.3.2.3	DM35418_Dma()	41
4.3.2.4	DM35418_Modify()	42
4.3.2.5	DM35418_Read()	42
4.3.2.6	DM35418_Write()	43
4.4	DM35418 PCI Region Structures	43
4.4.1	Detailed Description	44
4.4.2	Enumeration Type Documentation	44
4.4.2.1	DM35418_DMA_FUNCTIONS	44
4.4.2.2	dm35418_pci_region_access_size	44
4.4.2.3	dm35418_pci_region_num	45
4.5	DM35418 DAC Library Constants	45
4.5.1	Detailed Description	47
4.5.2	Enumeration Type Documentation	47
4.5.2.1	DM35418_Dac_Clock_Events	47
4.5.2.2	DM35418_Output_Ranges	47
4.6	DM35418 DAC Library Public Functions	48

4.6.1 Detailed Description	50
4.6.2 Function Documentation	50
4.6.2.1 DM35418_Dac_Channel_Clear_Marker_Status()	50
4.6.2.2 DM35418_Dac_Channel_Get_Marker_Config()	50
4.6.2.3 DM35418_Dac_Channel_Get_Marker_Status()	51
4.6.2.4 DM35418_Dac_Channel_Reset()	51
4.6.2.5 DM35418_Dac_Channel_Set_Marker_Config()	52
4.6.2.6 DM35418_Dac_Channel_Setup()	52
4.6.2.7 DM35418_Dac_Conv_To_Volts()	53
4.6.2.8 DM35418_Dac_Fifo_Channel_Write()	53
4.6.2.9 DM35418_Dac_Get_Clock_Div()	54
4.6.2.10 DM35418_Dac_Get_Clock_Src()	54
4.6.2.11 DM35418_Dac_Get_Conversion_Count()	55
4.6.2.12 DM35418_Dac_Get_Last_Conversion()	55
4.6.2.13 DM35418_Dac_Get_Mode_Status()	57
4.6.2.14 DM35418_Dac_Get_Post_Stop_Conversion_Count()	57
4.6.2.15 DM35418_Dac_Get_Start_Trigger()	58
4.6.2.16 DM35418_Dac_Get_Stop_Trigger()	58
4.6.2.17 DM35418_Dac_Interrupt_Clear_Status()	59
4.6.2.18 DM35418_Dac_Interrupt_Get_Config()	59
4.6.2.19 DM35418_Dac_Interrupt_Get_Status()	60
4.6.2.20 DM35418_Dac_Interrupt_Set_Config()	60
4.6.2.21 DM35418_Dac_Open()	61
4.6.2.22 DM35418_Dac_Pause()	61
4.6.2.23 DM35418_Dac_Reset()	62
4.6.2.24 DM35418_Dac_Set_Clock_Div()	62
4.6.2.25 DM35418_Dac_Set_Clock_Source_Global()	63
4.6.2.26 DM35418_Dac_Set_Clock_Src()	63
4.6.2.27 DM35418_Dac_Set_Conversion_Rate()	64
4.6.2.28 DM35418_Dac_Set_Last_Conversion()	64
4.6.2.29 DM35418_Dac_Set_Post_Stop_Conversion_Count()	65
4.6.2.30 DM35418_Dac_Set_Start_Trigger()	65
4.6.2.31 DM35418_Dac_Set_Stop_Trigger()	66
4.6.2.32 DM35418_Dac_Start()	66
4.6.2.33 DM35418_Dac_Volts_To_Conv()	67
4.7 DM35418 DIO Public	67
4.7.1 Detailed Description	68
4.7.2 Function Documentation	68
4.7.2.1 DM35418_Dio_Get_Direction()	68
4.7.2.2 DM35418_Dio_Get_Input_Value()	68
4.7.2.3 DM35418_Dio_Get_Output_Value()	69
4.7.2.4 DM35418_Dio_Open()	70

4.7.2.5 DM35418_Dio_Set_Direction()	70
4.7.2.6 DM35418_Dio_Set_Output_Value()	70
4.8 DM35418 DMA Public Library Constants	71
4.8.1 Detailed Description	72
4.8.2 Enumeration Type Documentation	72
4.8.2.1 DM35418_Fifo_States	72
4.9 DM35418 DMA Public Library Functions	73
4.9.1 Detailed Description	74
4.9.2 Function Documentation	74
4.9.2.1 DM35418_Dma_Buffer_Setup()	75
4.9.2.2 DM35418_Dma_Buffer_Status()	75
4.9.2.3 DM35418_Dma_Check_Buffer_Used()	76
4.9.2.4 DM35418_Dma_Check_For_Error()	76
4.9.2.5 DM35418_Dma_Clear()	78
4.9.2.6 DM35418_Dma_Clear_Interrupt()	78
4.9.2.7 DM35418_Dma_Configure_Interrupts()	79
4.9.2.8 DM35418_Dma_Find_Interrupt()	80
4.9.2.9 DM35418_Dma_Get_Current_Buffer_Count()	81
4.9.2.10 DM35418_Dma_Get_Errors()	81
4.9.2.11 DM35418_Dma_Get_Fifo_Counts()	82
4.9.2.12 DM35418_Dma_Get_Fifo_State()	83
4.9.2.13 DM35418_Dma_Get_Interrupt_Configuration()	83
4.9.2.14 DM35418_Dma_Pause()	84
4.9.2.15 DM35418_Dma_Reset_Buffer()	84
4.9.2.16 DM35418_Dma_Setup()	85
4.9.2.17 DM35418_Dma_Setup_Set_Direction()	86
4.9.2.18 DM35418_Dma_Setup_Set_Used()	86
4.9.2.19 DM35418_Dma_Start()	87
4.9.2.20 DM35418_Dma_Status()	87
4.9.2.21 DM35418_Dma_Stop()	89
4.10 DM35418 Driver Constants	90
4.10.1 Detailed Description	90
4.11 DM35418 Driver Enumerations	90
4.11.1 Detailed Description	90
4.11.2 Enumeration Type Documentation	90
4.11.2.1 dm35418_pci_region_access_dir	90
4.12 DM35418 Driver Structures	91
4.12.1 Detailed Description	91
4.13 DM35418 Example Programs Constants	91
4.13.1 Detailed Description	93
4.13.2 Enumeration Type Documentation	93
4.13.2.1 Help_Options	93

4.14 DM35418 Board Macros	95
4.14.1 Detailed Description	95
4.14.2 Macro Definition Documentation	95
4.14.2.1 CLK_40MHZ	95
4.15 DM35418 Board Library Public Functions	95
4.15.1 Detailed Description	96
4.15.2 Function Documentation	96
4.15.2.1 DM35418_Function_Block_Open()	96
4.15.2.2 DM35418_Function_Block_Open_Module()	96
4.15.2.3 DM35418_Gbc_Ack_Interrupt()	97
4.15.2.4 DM35418_Gbc_Board_Reset()	97
4.15.2.5 DM35418_Gbc_Get_Format()	98
4.15.2.6 DM35418_Gbc_Get_Fpga_Build()	98
4.15.2.7 DM35418_Gbc_Get_Pdp_Number()	99
4.15.2.8 DM35418_Gbc_Get_Revision()	99
4.15.2.9 DM35418_Gbc_Get_Sys_Clock_Freq()	100
4.16 DM35418 ioctl macros	100
4.16.1 Detailed Description	101
4.17 DM35418 Board Access Public Library Functions	101
4.17.1 Detailed Description	101
4.17.2 Function Documentation	101
4.17.2.1 DM35418_Dma_Initialize()	101
4.17.2.2 DM35418_Dma_Read()	102
4.17.2.3 DM35418_Dma_Write()	103
4.17.2.4 DM35418_General_InstallISR()	103
4.17.2.5 DM35418_General_RemoveISR()	104
4.17.2.6 DM35418_General_SetISRPRIORITY()	104
4.17.2.7 DM35418_General_WaitForInterrupt()	105
4.18 DM35418 Register Offsets	105
4.18.1 Detailed Description	110
4.18.2 Macro Definition Documentation	110
4.18.2.1 DM35418_OFFSET_FB_ADC_FIFO	110
4.19 DM35418 SyncBus Library Constants	110
4.19.1 Detailed Description	111
4.20 DM35418 Sync Bus Library Public Functions	111
4.20.1 Detailed Description	112
4.20.2 Function Documentation	112
4.20.2.1 DM35418_Syncbus_Clock_Source_Global_Get()	112
4.20.2.2 DM35418_Syncbus_Clock_Source_Global_Set()	113
4.20.2.3 DM35418_Syncbus_Disable()	113
4.20.2.4 DM35418_Syncbus_Drive_Source_Get()	114
4.20.2.5 DM35418_Syncbus_Drive_Source_Set()	114

4.20.2.6 DM35418_Syncbus_Enable()	115
4.20.2.7 DM35418_Syncbus_Get_Clk_Sel()	116
4.20.2.8 DM35418_Syncbus_Get_PLL_Locked()	116
4.20.2.9 DM35418_Syncbus_Open()	117
4.20.2.10 DM35418_Syncbus_Set_Clk_Sel()	117
4.20.2.11 DM35418_Syncbus_Set_Direction()	118
4.20.2.12 DM35418_Syncbus_Termination_Disable()	118
4.20.2.13 DM35418_Syncbus_Termination_Enable()	119
4.21 DM35418 Temperature	119
4.21.1 Detailed Description	120
4.21.2 Function Documentation	120
4.21.2.1 DM35418_Temperature_Open()	120
4.21.2.2 DM35418_Temperature_Read()	120
4.22 DM35418 Board Types	121
4.22.1 Detailed Description	122
4.23 DM35418 Utility Library Functions	122
4.23.1 Detailed Description	123
4.23.2 Enumeration Type Documentation	123
4.23.2.1 DM35418_Waveforms	123
4.23.3 Function Documentation	123
4.23.3.1 DM35418_Check_Result()	123
4.23.3.2 DM35418_Generate_Signal_Data()	124
4.23.3.3 DM35418_Get_Maskable()	125
4.23.3.4 DM35418_Get_Time_Diff()	125
4.23.3.5 DM35418_Micro_Sleep()	125
5 Data Structure Documentation	127
5.1 DM35418_Board_Descriptor Struct Reference	127
5.1.1 Detailed Description	127
5.1.2 Field Documentation	127
5.1.2.1 file_descriptor	127
5.1.2.2 isr	128
5.1.2.3 pid	128
5.2 dm35418_device_descriptor Struct Reference	128
5.2.1 Detailed Description	129
5.2.2 Field Documentation	129
5.2.2.1 cdev	129
5.2.2.2 device_index	129
5.2.2.3 device_lock	129
5.2.2.4 dma_descr_list	129
5.2.2.5 dma_wait_queue	130
5.2.2.6 int_queue_count	130

5.2.2.7 int_queue_in_marker	130
5.2.2.8 int_queue_missed	130
5.2.2.9 int_queue_out_marker	130
5.2.2.10 int_wait_queue	131
5.2.2.11 interrupt_fb	131
5.2.2.12 irq_number	131
5.2.2.13 name	131
5.2.2.14 pci	131
5.2.2.15 pdev	132
5.2.2.16 reference_count	132
5.2.2.17 remove_isr_flag	132
5.3 DM35418_DMA_Descriptor Struct Reference	132
5.3.1 Detailed Description	132
5.3.2 Field Documentation	133
5.3.2.1 buffer_start_offset	133
5.3.2.2 control_offset	133
5.3.2.3 num_buffers	133
5.4 dm35418_dma_descriptor Struct Reference	133
5.4.1 Detailed Description	134
5.4.2 Field Documentation	134
5.4.2.1 buffer	134
5.4.2.2 buffer_size	134
5.4.2.3 bus_addr	134
5.4.2.4 channel	134
5.4.2.5 fb_num	135
5.4.2.6 list	135
5.4.2.7 virt_addr	135
5.5 DM35418_Function_Block Struct Reference	135
5.5.1 Detailed Description	136
5.5.2 Field Documentation	136
5.5.2.1 control_offset	136
5.5.2.2 dma_channel	136
5.5.2.3 dma_offset	136
5.5.2.4 fb_num	136
5.5.2.5 fb_offset	137
5.5.2.6 num_dma_buffers	137
5.5.2.7 num_dma_channels	137
5.5.2.8 ordinal_fb_type_num	137
5.5.2.9 sub_type	137
5.5.2.10 type	138
5.5.2.11 type_revision	138
5.6 dm35418_ioctl_argument Union Reference	138

5.6.1 Detailed Description	138
5.6.2 Field Documentation	138
5.6.2.1 dma	139
5.6.2.2 interrupt	139
5.6.2.3 modify	139
5.6.2.4 readwrite	139
5.7 dm35418_ioctl_dma Struct Reference	139
5.7.1 Detailed Description	140
5.7.2 Field Documentation	140
5.7.2.1 buffer	140
5.7.2.2 buffer_ptr	140
5.7.2.3 buffer_size	140
5.7.2.4 channel	141
5.7.2.5 fb_num	141
5.7.2.6 function	141
5.7.2.7 num_buffers	141
5.7.2.8 pci	141
5.8 dm35418_ioctl_interrupt_info_request Struct Reference	142
5.8.1 Detailed Description	142
5.8.2 Field Documentation	142
5.8.2.1 error_occurred	142
5.8.2.2 interrupt_fb	142
5.8.2.3 interrupts_remaining	143
5.8.2.4 valid_interrupt	143
5.9 dm35418_ioctl_region_modify Struct Reference	143
5.9.1 Detailed Description	143
5.9.2 Field Documentation	143
5.9.2.1 access	144
5.9.2.2	144
5.9.2.3 mask16	144
5.9.2.4 mask32	144
5.9.2.5 mask8	144
5.10 dm35418_ioctl_region_readwrite Struct Reference	145
5.10.1 Detailed Description	145
5.10.2 Field Documentation	145
5.10.2.1 access	145
5.11 dm35418_pci_access_request Struct Reference	145
5.11.1 Detailed Description	146
5.11.2 Field Documentation	146
5.11.2.1	146
5.11.2.2 data16	146
5.11.2.3 data32	146

5.11.2.4 data8	146
5.11.2.5 offset	147
5.11.2.6 region	147
5.11.2.7 size	147
5.12 dm35418_pci_region Struct Reference	147
5.12.1 Detailed Description	148
5.12.2 Field Documentation	148
5.12.2.1 allocated	148
5.12.2.2 io_addr	148
5.12.2.3 length	148
5.12.2.4 phys_addr	148
5.12.2.5 virt_addr	148
6 File Documentation	149
6.1 examples/dm35418_adc.c File Reference	149
6.1.1 Detailed Description	150
6.1.2 Macro Definition Documentation	150
6.1.2.1 BUFFER_SIZE_BYTES	151
6.1.2.2 BUFFER_SIZE_SAMPLES	151
6.1.2.3 DAC_RATE	151
6.1.2.4 DEFAULT_RANGE	151
6.1.2.5 DEFAULT_RATE	151
6.1.3 Function Documentation	151
6.1.3.1 main()	151
6.1.3.2 sigint_handler()	152
6.1.4 Variable Documentation	152
6.1.4.1 exit_program	152
6.1.4.2 interrupt_count	153
6.1.4.3 program_name	153
6.2 examples/dm35418_adc_basic_ints.c File Reference	153
6.2.1 Detailed Description	154
6.2.2 Macro Definition Documentation	155
6.2.2.1 BUFFER_SIZE_BYTES	155
6.2.2.2 BUFFER_SIZE_SAMPLES	155
6.2.2.3 DAC_RATE	155
6.2.2.4 DEFAULT_RANGE	155
6.2.2.5 DEFAULT_RATE	155
6.2.2.6 TOTAL_INT_SRC_MASK	156
6.2.3 Function Documentation	156
6.2.3.1 main()	156
6.2.3.2 sigint_handler()	157
6.2.4 Variable Documentation	157

6.2.4.1 board	157
6.2.4.2 exit_program	157
6.2.4.3 my_adc	158
6.2.4.4 program_name	158
6.2.4.5 sample_count	158
6.3 examples/dm35418_adc_continuous_dma.c File Reference	158
6.3.1 Detailed Description	160
6.3.2 Macro Definition Documentation	160
6.3.2.1 ASCII_FILE_NAME	160
6.3.2.2 BIN_FILE_NAME	160
6.3.2.3 BUFFER_SIZE_BYTES	161
6.3.2.4 BUFFER_SIZE_SAMPLES	161
6.3.2.5 DAC_RATE	161
6.3.2.6 DEFAULT_NUM_ADC	161
6.3.2.7 DEFAULT_RATE	161
6.3.3 Function Documentation	161
6.3.3.1 convert_bin_to_txt()	161
6.3.3.2 ISR()	162
6.3.3.3 main()	162
6.3.3.4 output_channel_status()	163
6.3.3.5 setup_adcs()	163
6.3.3.6 setup_ctrlc_handler()	164
6.3.3.7 setup_dacs()	164
6.3.3.8 setup_slave_adcs()	165
6.3.3.9 sigint_handler()	165
6.3.4 Variable Documentation	166
6.3.4.1 board	166
6.3.4.2 buffer_count	166
6.3.4.3 buffer_size_bytes	166
6.3.4.4 dma_has_error	166
6.3.4.5 exit_program	167
6.3.4.6 local_buffer	167
6.3.4.7 my_adc	167
6.3.4.8 next_buffer	167
6.3.4.9 program_name	168
6.4 examples/dm35418_adc_sync_trig_dma.c File Reference	168
6.4.1 Detailed Description	169
6.4.2 Macro Definition Documentation	170
6.4.2.1 ASCII_FILE_NAME	170
6.4.2.2 BIN_FILE_NAME	170
6.4.2.3 DAC_BUFF_NUM_SAMPLES	171
6.4.2.4 DAC_BUFF_SIZE_BYTES	171

6.4.2.5 DAC_RATE	171
6.4.2.6 DEFAULT_RATE	171
6.4.2.7 DIO_DIR_OUTPUT	171
6.4.2.8 DIO_TRIGGER_OFF	172
6.4.2.9 DIO_TRIGGER_ON	172
6.4.2.10 NUM_SAMPLES_PER_TRIGGER	172
6.4.2.11 SYNCBUS_TRIGGER_OFF_USEC	172
6.4.2.12 SYNCBUS_TRIGGER_ON_USEC	172
6.4.3 Function Documentation	172
6.4.3.1 ISR()	172
6.4.3.2 main()	173
6.4.3.3 output_channel_status()	173
6.4.3.4 setup_adc()	174
6.4.3.5 setup_dac()	174
6.4.3.6 setup_dio()	175
6.4.3.7 setup_syncbus()	175
6.4.3.8 sigint_handler()	176
6.4.4 Variable Documentation	176
6.4.4.1 board	176
6.4.4.2 buffer_count	177
6.4.4.3 buffer_size_bytes	177
6.4.4.4 dma_has_error	177
6.4.4.5 exit_program	177
6.4.4.6 local_buffer	178
6.4.4.7 my_adc	178
6.4.4.8 next_buffer	178
6.4.4.9 program_name	178
6.5 examples/dm35418_adc_thresh_dma.c File Reference	179
6.5.1 Detailed Description	180
6.5.2 Macro Definition Documentation	180
6.5.2.1 ADC_THRESH_HIGH_VOLTS	180
6.5.2.2 ADC_THRESH_LOW_VOLTS	181
6.5.2.3 ASCII_FILE_NAME	181
6.5.2.4 BIN_FILE_NAME	181
6.5.2.5 DAC_BUFF_NUM_SAMPLES	181
6.5.2.6 DAC_BUFF_SIZE_BYTES	181
6.5.2.7 DAC_RATE	182
6.5.2.8 DEFAULT_RATE	182
6.5.2.9 NUM_SAMPLES_PER_TRIGGER	182
6.5.3 Function Documentation	182
6.5.3.1 ISR()	182
6.5.3.2 main()	183

6.5.3.3 output_channel_status()	183
6.5.3.4 setup_adc()	184
6.5.3.5 setup_dac()	184
6.5.3.6 sigint_handler()	185
6.5.4 Variable Documentation	185
6.5.4.1 board	185
6.5.4.2 buffer_count	185
6.5.4.3 buffer_size_bytes	186
6.5.4.4 dma_has_error	186
6.5.4.5 exit_program	186
6.5.4.6 local_buffer	186
6.5.4.7 my_adc	187
6.5.4.8 next_buffer	187
6.5.4.9 program_name	187
6.6 examples/dm35418_adc_to_dac_dma.c File Reference	187
6.6.1 Detailed Description	189
6.6.2 Macro Definition Documentation	189
6.6.2.1 BUFFER_SIZE_BYTES	189
6.6.2.2 BUFFER_SIZE_SAMPLES	189
6.6.2.3 DEFAULT_RATE	190
6.6.2.4 NUM_FB_TO_USE	190
6.6.3 Function Documentation	190
6.6.3.1 ISR()	190
6.6.3.2 main()	191
6.6.3.3 output_channel_status()	191
6.6.3.4 setup_adcs()	192
6.6.3.5 setup_dacs()	192
6.6.3.6 setup_isr()	193
6.6.3.7 sigint_handler()	193
6.6.3.8 start_slave_adcs()	193
6.6.4 Variable Documentation	194
6.6.4.1 adc_dma_error	194
6.6.4.2 board	194
6.6.4.3 buff_cnt	195
6.6.4.4 exit_program	195
6.6.4.5 local_buffer	195
6.6.4.6 my_adc	195
6.6.4.7 next_buffer	196
6.6.4.8 program_name	196
6.7 examples/dm35418_dac.c File Reference	196
6.7.1 Detailed Description	197
6.7.2 Macro Definition Documentation	197

6.7.2.1 DEFAULT_DAC_NUM	197
6.7.3 Function Documentation	198
6.7.3.1 main()	198
6.7.4 Variable Documentation	199
6.7.4.1 program_name	199
6.8 examples/dm35418_dac_dma.c File Reference	199
6.8.1 Detailed Description	200
6.8.2 Macro Definition Documentation	201
6.8.2.1 BUFFER_SIZE_BYTES	201
6.8.2.2 BUFFER_SIZE_SAMPLES	201
6.8.2.3 DEFAULT_DAC_TO_USE	201
6.8.2.4 DEFAULT_RATE	201
6.8.2.5 NUM_BUFFERS_TO_USE	201
6.8.3 Function Documentation	201
6.8.3.1 main()	201
6.8.3.2 sigint_handler()	202
6.8.4 Variable Documentation	202
6.8.4.1 exit_program	202
6.8.4.2 program_name	203
6.9 examples/dm35418_dac_syncbus.c File Reference	203
6.9.1 Detailed Description	204
6.9.2 Macro Definition Documentation	205
6.9.2.1 BUFFER_SIZE_BYTES	205
6.9.2.2 BUFFER_SIZE_SAMPLES	205
6.9.2.3 DEFAULT_DAC_TO_USE	205
6.9.2.4 DEFAULT_RATE	205
6.9.2.5 NUM_BUFFERS_TO_USE	205
6.9.3 Function Documentation	206
6.9.3.1 main()	206
6.9.3.2 setup_dac()	207
6.9.3.3 setup_syncbus()	207
6.9.3.4 sigint_handler()	208
6.9.4 Variable Documentation	208
6.9.4.1 exit_program	209
6.9.4.2 program_name	209
6.10 examples/dm35418_dio.c File Reference	209
6.10.1 Detailed Description	210
6.10.2 Function Documentation	210
6.10.2.1 main()	210
6.10.3 Variable Documentation	211
6.10.3.1 program_name	211
6.11 examples/dm35418_list_fb.c File Reference	211

6.11.1 Detailed Description	212
6.11.2 Function Documentation	212
6.11.2.1 main()	212
6.11.3 Variable Documentation	213
6.11.3.1 program_name	213
6.12 examples/dm35418_temperature.c File Reference	213
6.12.1 Detailed Description	214
6.12.2 Function Documentation	214
6.12.2.1 main()	214
6.12.2.2 sigint_handler()	215
6.12.3 Variable Documentation	215
6.12.3.1 exit_program	215
6.12.3.2 program_name	215
6.13 include/dm35418.h File Reference	216
6.13.1 Detailed Description	216
6.14 include/dm35418_adc_library.h File Reference	217
6.14.1 Detailed Description	222
6.15 include/dm35418_board_access.h File Reference	223
6.15.1 Detailed Description	223
6.15.2 Macro Definition Documentation	224
6.15.2.1 DM35418LIB_API	224
6.16 include/dm35418_board_access_structs.h File Reference	224
6.16.1 Detailed Description	225
6.17 include/dm35418_dac_library.h File Reference	225
6.17.1 Detailed Description	228
6.18 include/dm35418_dio_library.h File Reference	228
6.18.1 Detailed Description	229
6.19 include/dm35418_dma_library.h File Reference	229
6.19.1 Detailed Description	232
6.20 include/dm35418_driver.h File Reference	232
6.20.1 Detailed Description	233
6.21 include/dm35418_examples.h File Reference	233
6.21.1 Detailed Description	235
6.22 include/dm35418_gbc_library.h File Reference	236
6.22.1 Detailed Description	236
6.23 include/dm35418_ioctl.h File Reference	237
6.23.1 Detailed Description	237
6.24 include/dm35418_os.h File Reference	237
6.24.1 Detailed Description	238
6.25 include/dm35418_registers.h File Reference	238
6.25.1 Detailed Description	243
6.26 include/dm35418_syncbus_library.h File Reference	243

6.26.1 Detailed Description	245
6.27 include/dm35418_temperature_library.h File Reference	245
6.27.1 Detailed Description	246
6.28 include/dm35418_types.h File Reference	246
6.28.1 Detailed Description	248
6.28.2 Enumeration Type Documentation	248
6.28.2.1 DM35418_Clock_Buses	248
6.28.2.2 DM35418_Clock_Sources	248
6.29 include/dm35418_util_library.h File Reference	249
6.29.1 Detailed Description	249
Index	251

Chapter 1

Module Index

1.1 Modules

Here is a list of all modules:

DM35418 ADC Library Constants	7
DM35418 ADC Public Library Functions	12
DM35418 Board Access Structures	40
DM35418 PCI Region Structures	43
DM35418 DAC Library Constants	45
DM35418 DAC Library Public Functions	48
DM35418 DIO Public	67
DM35418 DMA Public Library Constants	71
DM35418 DMA Public Library Functions	73
DM35418 Driver Constants	90
DM35418 Driver Enumerations	90
DM35418 Driver Structures	91
DM35418 Example Programs Constants	91
DM35418 Board Macros	95
DM35418 Board Library Public Functions	95
DM35418 ioctl macros	100
DM35418 Board Access Public Library Functions	101
DM35418 Register Offsets	105
DM35418 SyncBus Library Constants	110
DM35418 Sync Bus Library Public Functions	111
DM35418 Temperature	119
DM35418 Board Types	121
DM35418 Utility Library Functions	122

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

DM35418_Board_Descriptor	DM35418 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable	127
dm35418_device_descriptor	DM35418 Device Descriptor. The identifying info for this particular board	128
DM35418_DMA_Descriptor	Descriptor for the DMA on this board	132
dm35418_dma_descriptor	DM35418 DMA descriptor. This structure holds information about a single DMA buffer	133
DM35418_Function_Block	DM35418 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations	135
dm35418_ioctl_argument	ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call	138
dm35418_ioctl_dma	ioctl() request structure for DMA	139
dm35418_ioctl_interrupt_info_request	ioctl() request structure for interrupt	142
dm35418_ioctl_region_modify	ioctl() request structure for PCI region read/modify/write	143
dm35418_ioctl_region_readwrite	ioctl() request structure for read from or write to PCI region	145
dm35418_pci_access_request	PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions	145
dm35418_pci_region	DM35418 PCI region descriptor. This structure holds information about one of a device's PCI memory regions	147

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

examples/dm35418_adc.c	
Example program which demonstrates the use of the ADC, setting and responding to interrupts	149
examples/dm35418_adc_basic_ints.c	
Example program which demonstrates the use of the non DMA ADC Interrupts	153
examples/dm35418_adc_continuous_dma.c	
Example program which demonstrates the use of the ADC and DMA	158
examples/dm35418_adc_sync_trig_dma.c	
Example program which demonstrates using the Syncbus input as an external trigger to sample with the ADC	168
examples/dm35418_adc_thresh_dma.c	
Demonstrates use of threshold values to control when data is collected	179
examples/dm35418_adc_to_dac_dma.c	
Demonstrates transferring data from ADC DMA to DAC DMA	187
examples/dm35418_dac.c	
Example program which demonstrates the use of the DAC	196
examples/dm35418_dac_dma.c	
Example program which demonstrates the use of the DAC and DMA	199
examples/dm35418_dac_syncbus.c	
Example program which demonstrates the use of Syncbus with the DAC and DMA	203
examples/dm35418_dio.c	
Example program which demonstrates the use of the DIO	209
examples/dm35418_list_fb.c	
Example program which demonstrates use of the library to open a function block for use	211
examples/dm35418_temperature.c	
Example program for read the temperature sensor	213
include/dm35418.h	
Defines for the DM35418 (Device-specific values)	216
include/dm35418_adc_library.h	
Definitions for the DM35418 ADC Library	217
include/dm35418_board_access.h	
Structures for the DM35418 Board Access Library	223
include/dm35418_board_access_structs.h	
Structures for the DM35418 Board Access Library	224
include/dm35418_dac_library.h	
Definitions for the DM35418 DAC Library	225

include/dm35418_dio_library.h	
Definitions for the DM35418 DIO Library	228
include/dm35418_dma_library.h	
Definitions for the DM35418 DMA Library	229
include/dm35418_driver.h	
Structures and defines for the DM35418 driver module	232
include/dm35418_examples.h	
Defines for the DM35418 Example programs. Commonly used constants for the example programs included with the software package	233
include/dm35418_gbc_library.h	
Definitions for the DM35418 Board Library, a library for accessing the board registers	236
include/dm35418_ioctl.h	
DM35418 Low level ioctl() request descriptor structure and request code definitions	237
include/dm35418_os.h	
Function declarations for the DM35418 that are Linux specific	237
include/dm35418_registers.h	
Defines for the DM35418 Registers (Offsets)	238
include/dm35418_syncbus_library.h	
Definitions for the DM35418 Sync Bus Library	243
include/dm35418_temperature_library.h	
Definitions for the DM35418 Temperature Library	245
include/dm35418_types.h	
Defines for the DM35418. Values for the general board, not specific to a particular function block	246
include/dm35418_util_library.h	
Definitions for the DM35418 Utilities library, various helper functions	249

Chapter 4

Module Documentation

4.1 DM35418 ADC Library Constants

Macros

- #define [DM35418_ADC_MODE_RESET](#) 0x00
Register value for ADC Mode Reset.
- #define [DM35418_ADC_MODE_PAUSE](#) 0x01
Register value for ADC Mode Pause.
- #define [DM35418_ADC_MODE_GO_SINGLE_SHOT](#) 0x02
Register value for ADC Mode Go (Single Shot)
- #define [DM35418_ADC_MODE_GO_REARM](#) 0x03
Register value for ADC Mode Go (Rearm after Stop)
- #define [DM35418_ADC_MODE_UNINITIALIZED](#) 0x04
Register value for ADC Mode Uninitialized.
- #define [DM35418_ADC_STAT_STOPPED](#) 0x00
Register value for ADC Status - Stopped.
- #define [DM35418_ADC_STAT_FILLING_PRE_TRIG_BUFF](#) 0x01
Register value for ADC Status - Filling Pre-Start Buffer.
- #define [DM35418_ADC_STAT_WAITING_START_TRIG](#) 0x02
Register value for ADC Status - Waiting for Start Trigger.
- #define [DM35418_ADC_STAT_SAMPLING](#) 0x03
Register value for ADC Status - Sampling Data.
- #define [DM35418_ADC_STAT_FILLING_POST_TRIG_BUFF](#) 0x04
Register value for ADC Status - Filling Post-Stop Buffer.
- #define [DM35418_ADC_STAT_WAIT_REARM](#) 0x05
Register value for ADC Status - Wait for Rearm.
- #define [DM35418_ADC_STAT_DONE](#) 0x07
Register value for ADC Status - Done.
- #define [DM35418_ADC_STAT_UNINITIALIZED](#) 0x08
Register value for ADC Status - Uninitialized.
- #define [DM35418_ADC_STAT_INITIALIZING](#) 0x09
Register value for ADC Status - Initializing.
- #define [DM35418_ADC_INT_SAMPLE_TAKEN_MASK](#) 0x01
Register value for Interrupt Mask - Sample Taken.
- #define [DM35418_ADC_INT_CHAN_THRESHOLD_MASK](#) 0x02

- Register value for Interrupt Mask - Channel Threshold Exceeded.*

 - `#define DM35418_ADC_INT_PRE_BUFF_FULL_MASK 0x04`
- Register value for Interrupt Mask - Pre-Start Buffer Filled.*

 - `#define DM35418_ADC_INT_START_TRIG_MASK 0x08`
- Register value for Interrupt Mask - Start Trigger Occurred.*

 - `#define DM35418_ADC_INT_STOP_TRIG_MASK 0x10`
- Register value for Interrupt Mask - Stop Trigger Occurred.*

 - `#define DM35418_ADC_INT_POST_BUFF_FULL_MASK 0x20`
- Register value for Interrupt Mask - Post-Stop Buffer Filled.*

 - `#define DM35418_ADC_INT_SAMP_COMPL_MASK 0x40`
- Register value for Interrupt Mask - Sampling Complete.*

 - `#define DM35418_ADC_INT_PACER_TICK_MASK 0x80`
- Register value for Interrupt Mask - Pacer Clock Tick Occurred.*

 - `#define DM35418_ADC_INT_ALL_MASK 0xFF`
- Register value for Interrupt Mask - All Bits.*

 - `#define DM35418_ADC_CHAN_INTR_LOW_THRESHOLD_MASK 0x01`
- Register value for Channel Low Threshold Interrupt.*

 - `#define DM35418_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK 0x02`
- Register value for Channel High Threshold Interrupt.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER0 0x0`
- Register value for Channel Filter Order 0.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER1 0x1`
- Register value for Channel Filter Order 1.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER2 0x2`
- Register value for Channel Filter Order 2.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER3 0x3`
- Register value for Channel Filter Order 3.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER4 0x4`
- Register value for Channel Filter Order 4.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER5 0x5`
- Register value for Channel Filter Order 5.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER6 0x6`
- Register value for Channel Filter Order 6.*

 - `#define DM35418_ADC_CHAN_FILTER_ORDER7 0x7`
- Register value for Channel Filter Order 7.*

 - `#define DM35418_ADC_FE_CONFIG_GAIN_05 0x00`
- Register value for setting Half-Gain.*

 - `#define DM35418_ADC_FE_CONFIG_GAIN_1 0x20`
- Register value for setting a Gain of 1.*

 - `#define DM35418_ADC_FE_CONFIG_GAIN_2 0x28`
- Register value for setting a Gain of 2.*

 - `#define DM35418_ADC_FE_CONFIG_GAIN_4 0x30`
- Register value for setting a Gain of 4.*

 - `#define DM35418_ADC_FE_CONFIG_GAIN_8 0x38`
- Register value for setting a Gain of 8.*

 - `#define DM35418_ADC_FE_CONFIG_BIPOLAR 0x00`
- Register value for setting input to Bi-Polar.*

 - `#define DM35418_ADC_FE_CONFIG_UNIPOLAR 0x04`
- Register value for setting input to Uni-Polar.*

 - `#define DM35418_ADC_FE_CONFIG_SINGLE_ENDED 0x01`
- Register value for setting input to Single-Ended.*

- `#define DM35418_ADC_FE_CONFIG_DIFFERENTIAL 0x03`
Register value for setting input to Differential.
- `#define DM35418_ADC_FE_CONFIG_INPUT_VREF_5V 0x00`
Register value for setting input to VREF (5V)
- `#define DM35418_ADC_FE_CONFIG_GAIN_MASK 0x38`
Bit mask for the channel gain bits of the FE Config.
- `#define DM35418_ADC_FE_CONFIG_POLARITY_MASK 0x04`
Bit mask for the polarity bit of the FE Config.
- `#define DM35418_ADC_FE_CONFIG_MODE_MASK 0x03`
Bit mask for the input mode bits of the FE Config.
- `#define DM35418_ADC_MAX_RATE 1538461`
Max allowable rate for the ADC (Hz)
- `#define DM35418_ADC_UNIPOLAR_THRESHOLD_MAX 262144L`
Maximum allowable value to write to the threshold register in unipolar mode.
- `#define DM35418_ADC_UNIPOLAR_THRESHOLD_MIN 0L`
Minimum allowable value to write to the threshold register in unipolar mode.
- `#define DM35418_ADC_BIPOLAR_THRESHOLD_MAX 131071L`
Maximum allowable value to write to the threshold register in unipolar mode.
- `#define DM35418_ADC_BIPOLAR_THRESHOLD_MIN -131072L`
Minimum allowable value to write to the threshold register in unipolar mode.
- `#define DM35418_ADC_BIT_WIDTH_MAX 262144L`
The maximum value represented by the bit width of the ADC.
- `#define DM35418_ADC_BIT_WIDTH_MAX_FLT ((float) DM35418_ADC_BIT_WIDTH_MAX)`
The max value of the ADC as a float.
- `#define DM35418_ADC_RNG_1_25_LSB 0.00000476837158203125`
What each bit is worth at a range of 1.25.
- `#define DM35418_ADC_RNG_2_5_LSB 0.0000095367431640625`
What each bit is worth at a range of 2.5.
- `#define DM35418_ADC_RNG_5_LSB 0.000019073486328125`
What each bit is worth at a range of 5.
- `#define DM35418_ADC_RNG_10_LSB 0.00003814697265625`
What each bit is worth at a range of 10.
- `#define DM35418_ADC_RNG_20_LSB 0.0000762939453125`
What each bit is worth at a range of 20.
- `#define DM35418_ADC_UNIPOLAR_MAX 262143L`
Max possible ADC value when in Unipolar.
- `#define DM35418_ADC_UNIPOLAR_MIN 0`
Min possible ADC value when in Unipolar.
- `#define DM35418_ADC_BIPOLAR_MAX 131071L`
Max possible ADC value when in Bipolar.
- `#define DM35418_ADC_BIPOLAR_MIN -131072L`
Min possible ADC value when in Bipolar.

Enumerations

- `enum DM35418_Adc_Clock_Events {`
`DM35418_ADC_CLK_BUS_SRC_DISABLE = 0x00 , DM35418_ADC_CLK_BUS_SRC_SAMPLE_TAKEN =`
`0x80 , DM35418_ADC_CLK_BUS_SRC_CHAN_THRESH = 0x81 , DM35418_ADC_CLK_BUS_SRC_PRE_START_BUFF_FU`
`= 0x82 ,`
`DM35418_ADC_CLK_BUS_SRC_START_TRIG = 0x83 , DM35418_ADC_CLK_BUS_SRC_STOP_TRIG =`
`0x84 , DM35418_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL = 0x85 , DM35418_ADC_CLK_BUS_SRC_SAMPLING_C`
`= 0x86 ,`
`DM35418_ADC_CLK_BUS_SRC_PACER_TICK = 0x87 }`

Clock events for the global source clocks.

- enum `DM35418_Input_Ranges` {
`DM35418_ADC_RNG_BIPOLAR_10V`, `DM35418_ADC_RNG_BIPOLAR_5V`, `DM35418_ADC_RNG_BIPOLAR_2_5V`,
`DM35418_ADC_RNG_BIPOLAR_1_25V`,
`DM35418_ADC_RNG_BIPOLAR_625mV`, `DM35418_ADC_RNG_UNIPOLAR_5V`, `DM35418_ADC_RNG_UNIPOLAR_10V`,
`DM35418_ADC_RNG_UNIPOLAR_2_5V`,
`DM35418_ADC_RNG_UNIPOLAR_1_25V` }

Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

- enum `DM35418_Input_Mode` { `DM35418_ADC_INPUT_SINGLE_ENDED`, `DM35418_ADC_INPUT_DIFFERENTIAL`,
`DM35418_ADC_INPUT_VREF_5V` }

Input mode of the ADC pin.

- enum `DM35418_Gains` {
`DM35418_ADC_GAIN_05`, `DM35418_ADC_GAIN_1`, `DM35418_ADC_GAIN_2`, `DM35418_ADC_GAIN_4`,
`DM35418_ADC_GAIN_8`, `DM35418_ADC_GAIN_16`, `DM35418_ADC_GAIN_32`, `DM35418_ADC_GAIN_64`,
`DM35418_ADC_GAIN_128` }

Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

4.1.1 Detailed Description

4.1.2 Enumeration Type Documentation

4.1.2.1 DM35418_Adc_Clock_Events

enum `DM35418_Adc_Clock_Events`

Clock events for the global source clocks.

Enumerator

<code>DM35418_ADC_CLK_BUS_SRC_DISABLE</code>	Register value for Clock Event - Disabled.
<code>DM35418_ADC_CLK_BUS_SRC_SAMPLE_TAKEN</code>	Register value for Clock Event - Sample Taken.
<code>DM35418_ADC_CLK_BUS_SRC_CHAN_THRESH</code>	Register value for Clock Event - Channel Threshold Exceeded.
<code>DM35418_ADC_CLK_BUS_SRC_PRE_START_↔ BUFF_FULL</code>	Register value for Clock Event - Pre-Start Buffer Full.
<code>DM35418_ADC_CLK_BUS_SRC_START_TRIG</code>	Register value for Clock Event - Start Trigger Occurred.
<code>DM35418_ADC_CLK_BUS_SRC_STOP_TRIG</code>	Register value for Clock Event - Stop Trigger Occurred.
<code>DM35418_ADC_CLK_BUS_SRC_POST_STOP_↔ BUFF_FULL</code>	Register value for Clock Event - Post-Stop Buffer Full.
<code>DM35418_ADC_CLK_BUS_SRC_SAMPLING_↔ COMPLETE</code>	Register value for Clock Event - Sampling Complete.
<code>DM35418_ADC_CLK_BUS_SRC_PACER_TICK</code>	Register value for Clock Event - Pacer Tick Occurred.

Definition at line 430 of file `dm35418_adc_library.h`.

4.1.2.2 DM35418_Gains

enum [DM35418_Gains](#)

Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35418_ADC_GAIN_05	Input Half-Gain
DM35418_ADC_GAIN_1	Input Gain of 1
DM35418_ADC_GAIN_2	Input Gain of 2
DM35418_ADC_GAIN_4	Input Gain of 4
DM35418_ADC_GAIN_8	Input Gain of 8
DM35418_ADC_GAIN_16	Input Gain of 16
DM35418_ADC_GAIN_32	Input Gain of 32
DM35418_ADC_GAIN_64	Input Gain of 64
DM35418_ADC_GAIN_128	Input Gain of 128

Definition at line 587 of file dm35418_adc_library.h.

4.1.2.3 DM35418_Input_Mode

enum [DM35418_Input_Mode](#)

Input mode of the ADC pin.

Note

Not all values in this enumeration may apply to your board,as this is a shared library. Please consult the board manual for valid values.

Enumerator

DM35418_ADC_INPUT_SINGLE_ENDED	Single-Ended Operation
DM35418_ADC_INPUT_DIFFERENTIAL	Differential Operation
DM35418_ADC_INPUT_VREF_5V	Input line connected to VREF.

Definition at line 557 of file dm35418_adc_library.h.

4.1.2.4 DM35418_Input_Ranges

enum [DM35418_Input_Ranges](#)

Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

Note

Not all values in this enumeration may apply to your board, as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35418_ADC_RNG_BIPOLAR_10V	Bipolar Mode, -10 to 10 V
DM35418_ADC_RNG_BIPOLAR_5V	Bipolar Mode, -5 to 5 V
DM35418_ADC_RNG_BIPOLAR_2_5V	Bipolar Mode, -2.5 to 2.5 V
DM35418_ADC_RNG_BIPOLAR_1_25V	Bipolar Mode, -1.25 to 1.25 V
DM35418_ADC_RNG_BIPOLAR_625mV	Bipolar Mode, -625 mV to 625 mV
DM35418_ADC_RNG_UNIPOLAR_5V	Unipolar Mode, 0 to 5 V
DM35418_ADC_RNG_UNIPOLAR_10V	Unipolar Mode, 0 to 10 V
DM35418_ADC_RNG_UNIPOLAR_2_5V	Unipolar Mode, 0 to 2.5 V
DM35418_ADC_RNG_UNIPOLAR_1_25V	Unipolar Mode, 0 to 1.25 V

Definition at line 499 of file `dm35418_adc_library.h`.

4.2 DM35418 ADC Public Library Functions

Functions

- [DM35418LIB_API](#) int [DM35418_Adc_Open](#) (struct [DM35418_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35418_Function_Block](#) *func_block)
Open the ADC indicated, and determine register locations of control blocks needed to control it.
- [DM35418LIB_API](#) int [DM35418_Adc_Get_Start_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, uint8_t *start_trigger)
Get the start trigger for data collection.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Start_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, uint8_t start_trigger)
Set the start trigger for data collection.
- [DM35418LIB_API](#) int [DM35418_Adc_Get_Stop_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, uint8_t *stop_trigger)
Get the stop trigger for data collection.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Stop_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, uint8_t stop_trigger)
Set the stop trigger for data collection.

- [DM35418LIB_API](#) int [DM35418_Adc_Get_Pre_Trigger_Samples](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *count)
Get the amount of data to capture prior to start trigger.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Pre_Trigger_Samples](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t count)
Set the amount of data to capture prior to start trigger.
- [DM35418LIB_API](#) int [DM35418_Adc_Get_Post_Stop_Samples](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *count)
Get the amount of data to capture after stop trigger.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Post_Stop_Samples](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t count)
Set the amount of data to capture after stop trigger.
- [DM35418LIB_API](#) int [DM35418_Adc_Get_Clock_Src](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) *source)
Get the clock source for the ADC.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Clock_Src](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) source)
Set the clock source for the ADC.
- [DM35418LIB_API](#) int [DM35418_Adc_Initialize](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Clk_Divider](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t divider)
Set the Clock Divider for the ADC function block.
- [DM35418LIB_API](#) int [DM35418_Adc_Set_Sample_Rate](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t rate, uint32_t *actual_rate)
Set the sampling rate for the ADC.
- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Get_Front_End_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint16_t *fe_config)
Get the front-end config register contents.
- [DM35418LIB_API](#) int [DM35418_Adc_Interrupt_Set_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t int_source, int enable)
Configure the interrupts for the ADC.
- [DM35418LIB_API](#) int [DM35418_Adc_Interrupt_Get_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for the ADC.
- [DM35418LIB_API](#) int [DM35418_Adc_Start](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Start.
- [DM35418LIB_API](#) int [DM35418_Adc_Start_Rearm](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Start-Rearm.
- [DM35418LIB_API](#) int [DM35418_Adc_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Reset.
- [DM35418LIB_API](#) int [DM35418_Adc_Pause](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Pause.
- [DM35418LIB_API](#) int [DM35418_Adc_Uninitialize](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Uninitialized.
- [DM35418LIB_API](#) int [DM35418_Adc_Get_Mode_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *mode_status)

Get the ADC mode-status value.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Get_Last_Sample](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t *value)

Get the last sample taken from the ADC.

- [DM35418LIB_API](#) int [DM35418_Adc_Get_Sample_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)

Get the count of number of samples taken.

- [DM35418LIB_API](#) int [DM35418_Adc_Interrupt_Get_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *value)

Get the interrupt status register.

- [DM35418LIB_API](#) int [DM35418_Adc_Interrupt_Clear_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t value)

Clear the interrupt status register.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, enum [DM35418_Input_Ranges](#) input_range, enum [DM35418_Input_Mode](#) input_mode)

Setup the channel input for the ADC.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)

Reset the channel front-end config.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Interrupt_Set_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t interrupts_to_↵ set, int enable)

Setup the channel interrupts.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Interrupt_Get_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_intr_↵ enable)

Get the channel interrupt configuration.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Interrupt_Get_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_intr_↵ status)

Get the channel interrupt status.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Interrupt_Clear_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t chan_intr_status)

Clear the interrupt status for this channel.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Find_Interrupt](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int *channel_with_interrupt, int *channel_↵ has_interrupt, uint8_t *channel_intr_status, uint8_t *channel_intr_enable)

Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Set_Filter](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t chan_filter)

Set the filter value for the channel.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Get_Filter](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_filter)

Get the filter value for the channel.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Set_Low_Threshold](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t threshold)

Set the lower threshold for this channel.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Set_High_Threshold](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t threshold)

Set the high threshold for this channel.

- [DM35418LIB_API](#) int [DM35418_Adc_Channel_Get_Thresholds](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t *low_threshold, int32_t *high_threshold)

Get both thresholds for this channel.

- `DM35418LIB_API int DM35418_Adc_Fifo_Channel_Read` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel, int32_t *value)

Read an ADC sample stored in the onboard FIFO.

- `DM35418LIB_API int DM35418_Adc_Set_Clock_Source_Global` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, enum `DM35418_Clock_Sources` clock_select, enum `DM35418_Adc_Clock_Events` clock_driver)

Set the global clock source for the ADC.

- `DM35418LIB_API int DM35418_Adc_Get_Clock_Source_Global` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, int clock_select, int *clock_source)

Get the global clock source for the selected clock.

- `DM35418LIB_API int DM35418_Adc_Sample_To_Volts` (enum `DM35418_Input_Ranges` input_range, int32_t adc_sample, float *volts)

Convert an ADC sample to a volts value.

- `DM35418LIB_API int DM35418_Adc_Volts_To_Sample` (enum `DM35418_Input_Ranges` input_range, float volts, int32_t *adc_sample)

Convert volts to an ADC value.

4.2.1 Detailed Description

DM35418_Adc_Library_Constants

4.2.2 Function Documentation

4.2.2.1 DM35418_Adc_Channel_Find_Interrupt()

```
DM35418LIB_API int DM35418_Adc_Channel_Find_Interrupt (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int * channel_with_interrupt,
    int * channel_has_interrupt,
    uint8_t * channel_intr_status,
    uint8_t * channel_intr_enable )
```

Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel_with_interrupt</i>	Pointer to the returned channel that has an interrupt (if any)
<i>channel_has_interrupt</i>	Pointer to boolean indicating whether returned channel has interrupt or not.
<i>channel_intr_status</i>	Pointer to the channel's interrupt status.
<i>channel_intr_enable</i>	Pointer to the channel's interrupt enable register.

Return values

0	Success.
Non-zero	Failure.

4.2.2.2 DM35418_Adc_Channel_Get_Filter()

```
DM35418LIB_API int DM35418_Adc_Channel_Get_Filter (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t * chan_filter )
```

Get the filter value for the channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to get filter of
<i>chan_filter</i>	Pointer to returned channel filter value. Reference the user's manual for valid filter values.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.2.2.3 DM35418_Adc_Channel_Get_Front_End_Config()

```
DM35418LIB_API int DM35418_Adc_Channel_Get_Front_End_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint16_t * fe_config )
```

Get the front-end config register contents.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
---------------	--

Parameters

<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel the FE Config is requested for.
<i>fe_config</i>	Pointer to the returned FE Config register value

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.2.2.4 DM35418_Adc_Channel_Get_Last_Sample()

```
DM35418LIB_API int DM35418_Adc_Channel_Get_Last_Sample (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int32_t * value )
```

Get the last sample taken from the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to get sample from.
<i>value</i>	Pointer to returned sample value.

Return values

0	Success.
Non-zero	Failure.

4.2.2.5 DM35418_Adc_Channel_Get_Thresholds()

```
DM35418LIB_API int DM35418_Adc_Channel_Get_Thresholds (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
```

```

    unsigned int channel,
    int32_t * low_threshold,
    int32_t * high_threshold )

```

Get both thresholds for this channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to set the high threshold of.
<i>low_threshold</i>	Pointer to signed integer value of threshold.
<i>high_threshold</i>	Pointer to signed integer value of threshold.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameters are returned as 32-bit integers. After getting the value from the register, it will be left-shifted 16-bits.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by main().

4.2.2.6 DM35418_Adc_Channel_Interrupt_Clear_Status()

```

DM35418LIB_API int DM35418_Adc_Channel_Interrupt_Clear_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t chan_intr_status )

```

Clear the interrupt status for this channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to clear.
<i>chan_intr_status</i>	Bit mask indicating which interrupts to clear.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.2.2.7 DM35418_Adc_Channel_Interrupt_Get_Config()

```
DM35418LIB_API int DM35418_Adc_Channel_Interrupt_Get_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t * chan_intr_enable )
```

Get the channel interrupt configuration.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to get configuration.
<i>chan_intr_enable</i>	Pointer to interrupt configuration being returned.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by main().

4.2.2.8 DM35418_Adc_Channel_Interrupt_Get_Status()

```
DM35418LIB_API int DM35418_Adc_Channel_Interrupt_Get_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t * chan_intr_status )
```

Get the channel interrupt status.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to get configuration.
<i>chan_intr_status</i>	Pointer to interrupt status being returned.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.2.2.9 DM35418_Adc_Channel_Interrupt_Set_Config()

```
DM35418LIB_API int DM35418_Adc_Channel_Interrupt_Set_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t interrupts_to_set,
    int enable )
```

Setup the channel interrupts.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to configure.
<i>interrupts_to_set</i>	A bit mask indicating which interrupts to set
<i>enable</i>	A boolean value indicating if selected interrupts should be enabled or disabled.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested, or requested input mode is not possible on this ADC subtype.

Referenced by main().

4.2.2.10 DM35418_Adc_Channel_Reset()

```
DM35418LIB_API int DM35418_Adc_Channel_Reset (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Reset the channel front-end config.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to reset.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.2.2.11 DM35418_Adc_Channel_Set_Filter()

```
DM35418LIB_API int DM35418_Adc_Channel_Set_Filter (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t chan_filter )
```

Set the filter value for the channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to set filter
<i>chan_filter</i>	Channel filter value. Reference the user's manual for valid filter values.

Return values

0	Success.
---	----------

Return values

<i>Non-zero</i>	Failure. errno may be set as follows: • EINVAL Invalid channel requested.
-----------------	---

4.2.2.12 DM35418_Adc_Channel_Set_High_Threshold()

```
DM35418LIB_API int DM35418_Adc_Channel_Set_High_Threshold (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int32_t threshold )
```

Set the high threshold for this channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to set the high threshold of.
<i>threshold</i>	Signed high threshold value for this channel.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameter is accepted as a 32-bit integer. Before writing the value, it will be right-shifted 16 bits.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • EINVAL Invalid channel requested.

Referenced by main(), and setup_adc().

4.2.2.13 DM35418_Adc_Channel_Set_Low_Threshold()

```
DM35418LIB_API int DM35418_Adc_Channel_Set_Low_Threshold (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
unsigned int channel,
int32_t threshold )
```

Set the lower threshold for this channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to set the lower threshold of.
<i>threshold</i>	Signed lower threshold value for this channel.

Note

The threshold register is only 16-bits. Thus, the threshold value really only represents the top 16-bits of the 24-bit ADC value. For convenience, the threshold parameter is accepted as a 32-bit integer. Before writing the value, it will be right-shifted 16 bits.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by setup_adc().

4.2.2.14 DM35418_Adc_Channel_Setup()

```
DM35418LIB_API int DM35418_Adc_Channel_Setup (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel,
    enum DM35418_Input_Ranges input_range,
    enum DM35418_Input_Mode input_mode )
```

Setup the channel input for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to configure.
<i>input_range</i>	An enumerated value representing the input voltage range of the input.
<i>input_mode</i>	An enumerated value representing the mode to set the input line to.

Note

The input line mode and input voltage ranges available for the board is dependent on the ADC subtype on the board. Review the user's guide to see what values the ADC can be set to.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • <code>EINVAL</code> Invalid channel requested, or requested mode/range is not possible on this ADC subtype.

Referenced by `main()`, `setup_adc()`, and `setup_adcs()`.

4.2.2.15 DM35418_Adc_Fifo_Channel_Read()

```
DM35418LIB_API int DM35418_Adc_Fifo_Channel_Read (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int32_t * value )
```

Read an ADC sample stored in the onboard FIFO.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to get sample from.
<i>value</i>	Pointer to returned sample value.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure.

4.2.2.16 DM35418_Adc_Get_Clock_Source_Global()

```
DM35418LIB_API int DM35418_Adc_Get_Clock_Source_Global (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
int clock_select,
int * clock_source )
```

Get the global clock source for the selected clock.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>clock_select</i>	Which global clock source to get
<i>clock_source</i>	Pointer to the returned clock source for the selected global clock

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid clock select or source..

4.2.2.17 DM35418_Adc_Get_Clock_Src()

```
DM35418LIB_API int DM35418_Adc_Get_Clock_Src (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    enum DM35418_Clock_Sources * source )
```

Get the clock source for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>source</i>	Pointer to returned clock source.

Return values

0	Success.
Non-zero	Failure.

4.2.2.18 DM35418_Adc_Get_Mode_Status()

```
DM35418LIB_API int DM35418_Adc_Get_Mode_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint8_t * mode_status )
```

Get the ADC mode-status value.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>mode_status</i>	Pointer to the mode_status value to return.

Return values

0	Success.
Non-zero	Failure.

Referenced by main().

4.2.2.19 DM35418_Adc_Get_Post_Stop_Samples()

```
DM35418LIB_API int DM35418_Adc_Get_Post_Stop_Samples (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * count )
```

Get the amount of data to capture after stop trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>count</i>	Pointer to the returned count.

Return values

0	Success.
Non-zero	Failure.

4.2.2.20 DM35418_Adc_Get_Pre_Trigger_Samples()

```
DM35418LIB_API int DM35418_Adc_Get_Pre_Trigger_Samples (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * count )
```

Get the amount of data to capture prior to start trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>count</i>	Pointer to the returned capture count

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure.

4.2.2.21 DM35418_Adc_Get_Sample_Count()

```
DM35418LIB_API int DM35418_Adc_Get_Sample_Count (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * value )
```

Get the count of number of samples taken.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>value</i>	Pointer to returned sample count.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure.

4.2.2.22 DM35418_Adc_Get_Start_Trigger()

```
DM35418LIB_API int DM35418_Adc_Get_Start_Trigger (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    uint8_t * start_trigger )
```

Get the start trigger for data collection.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>start_trigger</i>	Pointer to the returned trigger value.

Return values

0	Success.
Non-zero	Failure.

4.2.2.23 DM35418_Adc_Get_Stop_Trigger()

```
DM35418LIB_API int DM35418_Adc_Get_Stop_Trigger (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    uint8_t * stop_trigger )
```

Get the stop trigger for data collection.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>stop_trigger</i>	Pointer to the returned trigger value

Return values

0	Success.
Non-zero	Failure.

4.2.2.24 DM35418_Adc_Initialize()

```
DM35418LIB_API int DM35418_Adc_Initialize (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Note

In many cases, several other steps have to occur before initialization is attempted, or the device will not initialize correctly or at all. Please review the user's manual for the correct steps to take.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • EPERM Attempted to initialize an ADC with no active channels. EBUSY Device did not complete initialization in the time expected (timeout).

Referenced by `main()`, `setup_slave_adcs()`, and `start_slave_adcs()`.

4.2.2.25 DM35418_Adc_Interrupt_Clear_Status()

```
DM35418LIB_API int DM35418_Adc_Interrupt_Clear_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t value )
```

Clear the interrupt status register.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>value</i>	Bit mask of which interrupts to clear.

Return values

<i>0</i>	Success.
----------	----------

Return values

<i>Non-zero</i>	Failure.
-----------------	----------

4.2.2.26 DM35418_Adc_Interrupt_Get_Config()

```
DM35418LIB_API int DM35418_Adc_Interrupt_Get_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t * interrupt_ena )
```

Get the interrupt configuration for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>interrupt_ena</i>	Pointer to the interrupt configuration register.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure.

Referenced by main().

4.2.2.27 DM35418_Adc_Interrupt_Get_Status()

```
DM35418LIB_API int DM35418_Adc_Interrupt_Get_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t * value )
```

Get the interrupt status register.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>value</i>	Pointer to returned interrupt status.

Return values

0	Success.
Non-zero	Failure.

Referenced by ISR().

4.2.2.28 DM35418_Adc_Interrupt_Set_Config()

```
DM35418LIB_API int DM35418_Adc_Interrupt_Set_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t int_source,
    int enable )
```

Configure the interrupts for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>int_source</i>	The interrupts to configure. The bits indicate specific interrupts. Consult the user's manual for a description.
<i>enable</i>	Boolean indicating to enable or disable the selected interrupts.

Return values

0	Success.
Non-zero	Failure.

Referenced by main().

4.2.2.29 DM35418_Adc_Open()

```
DM35418LIB_API int DM35418_Adc_Open (
    struct DM35418_Board_Descriptor * handle,
    unsigned int number_of_type,
    struct DM35418_Function_Block * func_block )
```

Open the ADC indicated, and determine register locations of control blocks needed to control it.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>number_of_type</i>	Which ADC to open. The first ADC on the board will be 0.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	Success.
-1	Failure.

Referenced by `main()`, `setup_adc()`, and `setup_adcs()`.

4.2.2.30 DM35418_Adc_Pause()

```
DM35418LIB_API int DM35418_Adc_Pause (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the ADC mode to Pause.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	Success.
Non-zero	Failure.

4.2.2.31 DM35418_Adc_Reset()

```
DM35418LIB_API int DM35418_Adc_Reset (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the ADC mode to Reset.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	Success.
Non-zero	Failure.

4.2.2.32 DM35418_Adc_Sample_To_Volts()

```
DM35418LIB_API int DM35418_Adc_Sample_To_Volts (
    enum DM35418_Input_Ranges input_range,
    int32_t adc_sample,
    float * volts )
```

Convert an ADC sample to a volts value.

Parameters

<i>input_range</i>	Enumerated value indicating what range the ADC channel has been set to, or NULL if the ADC does not have selectable input ranges.
<i>adc_sample</i>	Signed value from the ADC that we want to convert to volts.
<i>volts</i>	Pointer to the returned float value in volts.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: • ENODEV The function block passed in is the wrong subtype

4.2.2.33 DM35418_Adc_Set_Clk_Divider()

```
DM35418LIB_API int DM35418_Adc_Set_Clk_Divider (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t divider )
```

Set the Clock Divider for the ADC function block.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>divider</i>	The requested clock divider.

Return values

0	Success.
-1	Failure.

4.2.2.34 DM35418_Adc_Set_Clock_Source_Global()

```
DM35418LIB_API int DM35418_Adc_Set_Clock_Source_Global (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    enum DM35418_Clock_Sources clock_select,
    enum DM35418_Adc_Clock_Events clock_driver )
```

Set the global clock source for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>clock_select</i>	Which global clock source to set
<i>clock_driver</i>	Source to set global clock to (what is driving it?)

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid clock select or source..

Referenced by main().

4.2.2.35 DM35418_Adc_Set_Clock_Src()

```
DM35418LIB_API int DM35418_Adc_Set_Clock_Src (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    enum DM35418_Clock_Sources source )
```

Set the clock source for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>source</i>	Clock source to use for the ADC. Consult the user's manual for the list of available sources.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • <code>EINVAL</code> The clock source selected is not valid.

Referenced by `main()`, `setup_adc()`, and `setup_adcs()`.

4.2.2.36 `DM35418_Adc_Set_Post_Stop_Samples()`

```
DM35418LIB_API int DM35418_Adc_Set_Post_Stop_Samples (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t count )
```

Set the amount of data to capture after stop trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>count</i>	Number of samples to capture after the stop trigger.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure.

Referenced by `main()`, and `setup_adc()`.

4.2.2.37 `DM35418_Adc_Set_Pre_Trigger_Samples()`

```
DM35418LIB_API int DM35418_Adc_Set_Pre_Trigger_Samples (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t count )
```

Set the amount of data to capture prior to start trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>count</i>	Number of samples to capture prior to the start trigger.

Note

The amount of data that can be captured prior to the start trigger is limited by the size of the FIFO. Consult the user's manual for this information.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • <code>EINVAL</code> The size is not within the valid value range.

Referenced by `main()`, and `setup_adc()`.

4.2.2.38 `DM35418_Adc_Set_Sample_Rate()`

```
DM35418LIB_API int DM35418_Adc_Set_Sample_Rate (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t rate,
    uint32_t * actual_rate )
```

Set the sampling rate for the ADC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>rate</i>	The requested sampling rate for the ADC (Hz).
<i>actual_rate</i>	Pointer to the actual rate achieved by the ADC (Hz). Due to divider and clock values, the actual rate will rarely ever be the exact same as the requested rate.

Return values

<i>0</i>	Success.
----------	----------

Return values

<i>Non-zero</i>	Failure. errno may be set as follows: • <code>EINVAL</code> Asked for an invalid sampling rate (negative or 0) ERANGE Requested sampling rate is outside of the possible range for this ADC.
-----------------	--

Referenced by `main()`, `setup_adc()`, `setup_slave_adcs()`, and `start_slave_adcs()`.

4.2.2.39 DM35418_Adc_Set_Start_Trigger()

```
DM35418LIB_API int DM35418_Adc_Set_Start_Trigger (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    uint8_t start_trigger )
```

Set the start trigger for data collection.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>start_trigger</i>	Trigger to start capturing values. See the hardware manual for valid trigger values.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • <code>EINVAL</code> An invalid value was passed for a start trigger

Referenced by `main()`, `setup_adc()`, `setup_slave_adcs()`, and `start_slave_adcs()`.

4.2.2.40 DM35418_Adc_Set_Stop_Trigger()

```
DM35418LIB_API int DM35418_Adc_Set_Stop_Trigger (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    uint8_t stop_trigger )
```

Set the stop trigger for data collection.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>stop_trigger</i>	Trigger to stop capturing values. See the hardware manual for valid trigger values.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure. errno may be set as follows: • <code>EINVAL</code> An invalid value was passed for a stop trigger

Referenced by `main()`, `setup_adc()`, `setup_slave_adcs()`, and `start_slave_adcs()`.

4.2.2.41 DM35418_Adc_Start()

```
DM35418LIB_API int DM35418_Adc_Start (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the ADC mode to Start.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	Failure.

Referenced by `main()`, `setup_slave_adcs()`, and `start_slave_adcs()`.

4.2.2.42 DM35418_Adc_Start_Rearm()

```
DM35418LIB_API int DM35418_Adc_Start_Rearm (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the ADC mode to Start-Rearm.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	Success.
Non-zero	Failure.

4.2.2.43 DM35418_Adc_Uninitialize()

```
DM35418LIB_API int DM35418_Adc_Uninitialize (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the ADC mode to Uninitialized.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.

Return values

0	Success.
Non-zero	Failure.

4.2.2.44 DM35418_Adc_Volts_To_Sample()

```
DM35418LIB_API int DM35418_Adc_Volts_To_Sample (
    enum DM35418_Input_Ranges input_range,
    float volts,
    int32_t * adc_sample )
```

Convert volts to an ADC value.

Parameters

<i>input_range</i>	Enumerated value indicating what range the ADC channel has been set to
<i>volts</i>	Value to be converted to counts.
<i>adc_sample</i>	Pointer to the returned ADC count value.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: • ENODEV The function block passed in is the wrong subtype

Referenced by `main()`, and `setup_adc()`.

4.3 DM35418 Board Access Structures

Data Structures

- struct [DM35418_DMA_Descriptor](#)
Descriptor for the DMA on this board.
- struct [DM35418_Function_Block](#)
DM35418 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Functions

- [DM35418LIB_API](#) int [DM35418_Board_Open](#) (uint8_t dev_num, struct [DM35418_Board_Descriptor](#) **handle)
Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.
- [DM35418LIB_API](#) int [DM35418_Board_Close](#) (struct [DM35418_Board_Descriptor](#) *handle)
Close the board, closing the open handle for the device file, and freeing the memory allocated for the descriptor.
- [DM35418LIB_API](#) int [DM35418_Read](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Read from the board.
- [DM35418LIB_API](#) int [DM35418_Write](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Write to the board.
- [DM35418LIB_API](#) int [DM35418_Modify](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Read/Modify/Write to the board.
- int [DM35418_Dma](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Perform a DMA operation.

4.3.1 Detailed Description

4.3.2 Function Documentation

4.3.2.1 DM35418_Board_Close()

```
DM35418LIB_API int DM35418_Board_Close (
    struct DM35418_Board_Descriptor * handle )
```

Close the board, closing the open handle for the device file, and freeing the memory allocated for the descriptor.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
---------------	--

Return values

0	Success.
-1	Failure. errno may be set as follows: • ENODATA Device handle is null.

Referenced by main().

4.3.2.2 DM35418_Board_Open()

```
DM35418LIB_API int DM35418_Board_Open (
    uint8_t dev_num,
    struct DM35418_Board_Descriptor ** handle )
```

Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.

Parameters

<i>dev_num</i>	The minor number of the device being opened.
<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.

Return values

0	Success.
-1	Failure. errno may be set as follows: • EBUSY Cannot open specified device. ENOMEM Cannot allocate memory for device descriptor.

Referenced by main().

4.3.2.3 DM35418_Dma()

```
int DM35418_Dma (
    struct DM35418_Board_Descriptor * handle,
    union dm35418_ioctl_argument * ioctl_request )
```

Perform a DMA operation.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>ioctl_request</i>	Structure holding all required information for request to complete, including a DMA descriptor.

Return values

0	Success.
-1	Failure.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

4.3.2.4 DM35418_Modify()

```
DM35418LIB_API int DM35418_Modify (
    struct DM35418_Board_Descriptor * handle,
    union dm35418_ioctl_argument * ioctl_request )
```

Read/Modify/Write to the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>ioctl_request</i>	Structure holding all required information for the modify to complete, including register offset, data size, PCI region number, value mask, and data to write

Return values

0	Success.
-1	Failure.

4.3.2.5 DM35418_Read()

```
DM35418LIB_API int DM35418_Read (
    struct DM35418_Board_Descriptor * handle,
    union dm35418_ioctl_argument * ioctl_request )
```

Read from the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>ioctl_request</i>	Structure holding all required information for the read to complete, including register offset, data size, PCI region number, and pointer for returned data.

Return values

0	Success.
-1	Failure.

4.3.2.6 DM35418_Write()

```
DM35418LIB_API int DM35418_Write (
    struct DM35418_Board_Descriptor * handle,
    union dm35418_ioctl_argument * ioctl_request )
```

Write to the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>ioctl_request</i>	Structure holding all required information for the write to complete, including register offset, data size, PCI region number, and data to write.

Return values

0	Success.
-1	Failure.

4.4 DM35418 PCI Region Structures

Data Structures

- struct [dm35418_pci_access_request](#)
PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.
- struct [dm35418_ioctl_region_readwrite](#)
ioctl() request structure for read from or write to PCI region
- struct [dm35418_ioctl_region_modify](#)
ioctl() request structure for PCI region read/modify/write
- struct [dm35418_ioctl_interrupt_info_request](#)
ioctl() request structure for interrupt
- struct [dm35418_ioctl_dma](#)

- ioctl()* request structure for DMA
 - union `dm35418_ioctl_argument`

ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the *ioctl()* call.

Enumerations

- enum `dm35418_pci_region_num` { `DM35418_PCI_REGION_GBC` = 0 , `DM35418_PCI_REGION_GBC2` , `DM35418_PCI_REGION_FB` }

Standard PCI region number.
- enum `dm35418_pci_region_access_size` { `DM35418_PCI_REGION_ACCESS_8` = 0 , `DM35418_PCI_REGION_ACCESS_16` , `DM35418_PCI_REGION_ACCESS_32` }

Desired size in bits of access to standard PCI region.
- enum `DM35418_DMA_FUNCTIONS` { `DM35418_DMA_INITIALIZE` , `DM35418_DMA_READ` , `DM35418_DMA_WRITE` }

4.4.1 Detailed Description

4.4.2 Enumeration Type Documentation

4.4.2.1 DM35418_DMA_FUNCTIONS

enum `DM35418_DMA_FUNCTIONS`

Enumeration for DMA functions that can be requested for the driver to perform.

Enumerator

<code>DM35418_DMA_INITIALIZE</code>	Initialize the DMA buffers
<code>DM35418_DMA_READ</code>	Read from the DMA buffers (transfer to user space)
<code>DM35418_DMA_WRITE</code>	Write to the DMA buffers (transfer from user space)

Definition at line 96 of file `dm35418_board_access_structs.h`.

4.4.2.2 dm35418_pci_region_access_size

enum `dm35418_pci_region_access_size`

Desired size in bits of access to standard PCI region.

Enumerator

DM35418_PCI_REGION_ACCESS_8	8-bit access
DM35418_PCI_REGION_ACCESS_16	16-bit access
DM35418_PCI_REGION_ACCESS_32	32-bit access

Definition at line 68 of file dm35418_board_access_structs.h.

4.4.2.3 dm35418_pci_region_num

```
enum dm35418_pci_region_num
```

Standard PCI region number.

Enumerator

DM35418_PCI_REGION_GBC	General Board Control Registers (BAR0)
DM35418_PCI_REGION_GBC2	General Board Control Registers (64-bit) (BAR1)
DM35418_PCI_REGION_FB	Functional Blocks Registers (BAR2)

Definition at line 40 of file dm35418_board_access_structs.h.

4.5 DM35418 DAC Library Constants

Macros

- #define [DM35418_DAC_INT_CONVERSION_SENT_MASK](#) 0x01
Register value for Interrupt Mask - Conversion Sent.
- #define [DM35418_DAC_INT_CHAN_MARKER_MASK](#) 0x02
Register value for Interrupt Mask - Channel has enabled marker.
- #define [DM35418_DAC_INT_START_TRIG_MASK](#) 0x08
Register value for Interrupt Mask - Start Trigger Occurred.
- #define [DM35418_DAC_INT_STOP_TRIG_MASK](#) 0x10
Register value for Interrupt Mask - Stop Trigger Occurred.
- #define [DM35418_DAC_INT_POST_STOP_DONE_MASK](#) 0x20
Register value for Interrupt Mask - Post-Stop Conversions Completed.
- #define [DM35418_DAC_INT_PACER_TICK_MASK](#) 0x80
Register value for Interrupt Mask - Pacer Clock Tick.
- #define [DM35418_DAC_INT_ALL_MASK](#) 0xBB
Register value for Interrupt Mask - All Bits.
- #define [DM35418_DAC_MODE_RESET](#) 0x00
Register value for Mode - Reset.
- #define [DM35418_DAC_MODE_PAUSE](#) 0x01
Register value for Mode - Pause.
- #define [DM35418_DAC_MODE_GO_SINGLE_SHOT](#) 0x02

- Register value for Mode - Go (Single Shot)*
 - #define `DM35418_DAC_MODE_GO_REARM` 0x03
- Register value for Mode - Go (Re-arm)*
 - #define `DM35418_DAC_STATUS_STOPPED` 0x00
- Register value for DAC Status - Stopped.*
 - #define `DM35418_DAC_STATUS_WAITING_START_TRIG` 0x02
- Register value for DAC Status - Waiting for Start Trigger.*
 - #define `DM35418_DAC_STATUS_CONVERTING` 0x03
- Register value for DAC Status - Converting Data.*
 - #define `DM35418_DAC_STATUS_OUTPUT_POST` 0x04
- Register value for DAC Status - Outputting Post-Stop conversions.*
 - #define `DM35418_DAC_STATUS_WAITING_REARM` 0x05
- Register value for DAC Status - Waiting for Re-Arm.*
 - #define `DM35418_DAC_STATUS_DONE` 0x07
- Register value for DAC Status - Done.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_1` 0x00
- Register value for setting a Gain of 1.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_2` 0x01
- Register value for setting a Gain of 2.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_4` 0x02
- Register value for setting a Gain of 4.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_MASK` 0x03
- Bit Mask for the gain bits of the FE Config.*
 - #define `DM35418_DAC_MAX_RATE` 1000000
- Max allowable rate for the DAC (Hz)*
 - #define `DM35418_DAC_BIPOLAR_MIN` -32768
- DAC Min value in Bipolar mode.*
 - #define `DM35418_DAC_BIPOLAR_MAX` 32767
- DAC Max value in Bipolar mode.*
 - #define `DM35418_DAC_RNG_5_LSB` 0.0000762939453125f
- What each bit is worth at a range of 5.*
 - #define `DM35418_DAC_RNG_10_LSB` 0.000152587890625f
- What each bit is worth at a range of 10.*
 - #define `DM35418_DAC_RNG_20_LSB` 0.00030517578125f
- What each bit is worth at a range of 20.*

Enumerations

- enum `DM35418_Dac_Clock_Events` {
 `DM35418_DAC_CLK_BUS_SRC_DISABLE` = 0x00 , `DM35418_DAC_CLK_BUS_SRC_CONVERSION_SENT`
 = 0x80 , `DM35418_DAC_CLK_BUS_SRC_CHAN_MARKER` = 0x81 , `DM35418_DAC_CLK_BUS_SRC_START_TRIG`
 = 0x83 ,
 `DM35418_DAC_CLK_BUS_SRC_STOP_TRIG` = 0x84 , `DM35418_DAC_CLK_BUS_SRC_CONV_COMPL`
 = 0x85 }
Clocking events that can be used as the global clock sources.
- enum `DM35418_Output_Ranges` { `DM35418_DAC_RNG_BIPOLAR_2_5V` , `DM35418_DAC_RNG_BIPOLAR_5V`
 , `DM35418_DAC_RNG_BIPOLAR_10V` }
Output range of the DAC pin.

4.5.1 Detailed Description

Functions

4.5.2 Enumeration Type Documentation

4.5.2.1 DM35418_Dac_Clock_Events

```
enum DM35418_Dac_Clock_Events
```

Clocking events that can be used as the global clock sources.

Enumerator

DM35418_DAC_CLK_BUS_SRC_DISABLE	Register value for Clock Event - Disabled.
DM35418_DAC_CLK_BUS_SRC_CONVERSION_↔ SENT	Register value for Clock Event - Conversion Sent.
DM35418_DAC_CLK_BUS_SRC_CHAN_MARKER	Register value for Clock Event - Channel has enabled marker.
DM35418_DAC_CLK_BUS_SRC_START_TRIG	Register value for Clock Event - Start Trigger Occurred.
DM35418_DAC_CLK_BUS_SRC_STOP_TRIG	Register value for Clock Event - Stop Trigger Occurred.
DM35418_DAC_CLK_BUS_SRC_CONV_COMPL	Register value for Clock Event - Conversions Complete.

Definition at line 216 of file dm35418_dac_library.h.

4.5.2.2 DM35418_Output_Ranges

```
enum DM35418_Output_Ranges
```

Output range of the DAC pin.

Note

Not all values in this enumeration may apply to your board, as this is a shared library. Please consult the board manual for legal values.

Enumerator

DM35418_DAC_RNG_BIPOLAR_2_5V	Range -2.5 V to 2.5 V
DM35418_DAC_RNG_BIPOLAR_5V	Range -5 V to 5 V
DM35418_DAC_RNG_BIPOLAR_10V	Range -10 V to 10 V

Definition at line 267 of file dm35418_dac_library.h.

4.6 DM35418 DAC Library Public Functions

Functions

- [DM35418LIB_API](#) `int DM35418_Dac_Open` (struct [DM35418_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35418_Function_Block](#) *func_block)
Open the DAC indicated, and determine register locations of control blocks needed to control it.
- [DM35418LIB_API](#) `int DM35418_Dac_Set_Clock_Src` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) source)
Set the clock source of the DAC.
- [DM35418LIB_API](#) `int DM35418_Dac_Get_Clock_Src` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) *source)
Get the clock source of the DAC.
- [DM35418LIB_API](#) `int DM35418_Dac_Get_Clock_Div` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *divider)
Get the clock divider value.
- [DM35418LIB_API](#) `int DM35418_Dac_Set_Clock_Div` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t divider)
Set the clock divider value.
- [DM35418LIB_API](#) `int DM35418_Dac_Set_Conversion_Rate` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t requested_rate, uint32_t *actual_rate)
Set the conversion rate of this DAC.
- [DM35418LIB_API](#) `int DM35418_Dac_Interrupt_Set_Config` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t interrupt_src, int enable)
Set the interrupt configuration for this DAC.
- [DM35418LIB_API](#) `int DM35418_Dac_Interrupt_Get_Config` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for this DAC.
- [DM35418LIB_API](#) `int DM35418_Dac_Set_Start_Trigger` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t trigger_value)
Set the start trigger.
- [DM35418LIB_API](#) `int DM35418_Dac_Set_Stop_Trigger` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t trigger_value)
Set the stop trigger.
- [DM35418LIB_API](#) `int DM35418_Dac_Get_Start_Trigger` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *trigger_value)
Get the start trigger.
- [DM35418LIB_API](#) `int DM35418_Dac_Get_Stop_Trigger` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *trigger_value)
Get the stop trigger.
- [DM35418LIB_API](#) `int DM35418_Dac_Start` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the DAC Mode to Start.
- [DM35418LIB_API](#) `int DM35418_Dac_Reset` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the DAC Mode to Reset.
- [DM35418LIB_API](#) `int DM35418_Dac_Pause` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the DAC Mode to Pause.

- [DM35418LIB_API](#) int [DM35418_Dac_Get_Mode_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *mode_status)
Get the Mode and Status of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Last_Conversion](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker, int16_t *value)
Get the value of the last conversion of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Last_Conversion](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t marker, int16_t value)
Set a value to be converted by the DAC immediately.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Conversion_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get a count of the number of conversions that DAC has executed.
- [DM35418LIB_API](#) int [DM35418_Dac_Interrupt_Get_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *value)
Get a interrupt status register of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Interrupt_Clear_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t value)
Clear the interrupt status register of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Post_Stop_Conversion_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t value)
Set the number of conversions the DAC will make after a stop trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Post_Stop_Conversion_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get the number of conversions the DAC will make after a stop trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Clock_Source_Global](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) clock, enum [DM35418_Dac_Clock_Events](#) clock_driver)
Set the source that will drive the global clock.
- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, enum [DM35418_Output_Ranges](#) output_range)
Setup the selected DAC channel.
- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel)
Reset the DAC channel, by writing all zeros to the front-end config.
- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Set_Marker_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_enable)
Set the configuration of the marker interrupts for this channel.
- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Get_Marker_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_enable)
Get the configuration of the marker interrupts for this channel.
- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Get_Marker_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_status)
Get the status of the marker interrupts for this channel.
- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Clear_Marker_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_to_clear)
Clear the marker interrupts for this channel.
- [DM35418LIB_API](#) int [DM35418_Dac_Fifo_Channel_Write](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t value)
Write a value to the onboard FIFO.
- [DM35418LIB_API](#) int [DM35418_Dac_Volts_To_Conv](#) (enum [DM35418_Output_Ranges](#) output_range, float volts, int16_t *dac_conversion)
Convert a value in volts to a DAC equivalent signed value.

- `DM35418LIB_API int DM35418_Dac_Conv_To_Volts (enum DM35418_Output_Ranges output_range, int16_t conversion, float *volts)`

Convert a DAC conversion value to volts.

4.6.1 Detailed Description

DM35418_Dac_Library_Constants

4.6.2 Function Documentation

4.6.2.1 DM35418_Dac_Channel_Clear_Marker_Status()

```
DM35418LIB_API int DM35418_Dac_Channel_Clear_Marker_Status (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t marker_to_clear )
```

Clear the marker interrupts for this channel.

Parameters

<i>handle</i>	Pointer to the board handle
<i>func_block</i>	Pointer to the function block.
<i>channel</i>	The channel to change.
<i>marker_to_clear</i>	Bit values indicating which bits in register to clear.

Return values

<i>0</i>	Success.
<i>Non-zero</i>	

4.6.2.2 DM35418_Dac_Channel_Get_Marker_Config()

```
DM35418LIB_API int DM35418_Dac_Channel_Get_Marker_Config (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t * marker_enable )
```

Get the configuration of the marker interrupts for this channel.

Parameters

<i>handle</i>	Pointer to the board handle
<i>func_block</i>	Pointer to the function block.
<i>channel</i>	The channel to change.
<i>marker_enable</i>	Pointer to returned marker interrupt config.

Return values

0	Success.
Non-zero	

4.6.2.3 DM35418_Dac_Channel_Get_Marker_Status()

```
DM35418LIB_API int DM35418_Dac_Channel_Get_Marker_Status (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t * marker_status )
```

Get the status of the marker interrupts for this channel.

Parameters

<i>handle</i>	Pointer to the board handle
<i>func_block</i>	Pointer to the function block.
<i>channel</i>	The channel to change.
<i>marker_status</i>	Pointer to returned marker status.

Return values

0	Success.
Non-zero	

4.6.2.4 DM35418_Dac_Channel_Reset()

```
DM35418LIB_API int DM35418_Dac_Channel_Reset (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Reset the DAC channel, by writing all zeros to the front-end config.

Parameters

<i>handle</i>	Pointer to the board handle
<i>func_block</i>	Pointer to the function block we want to reset.
<i>channel</i>	The channel to reset.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid channel.

4.6.2.5 DM35418_Dac_Channel_Set_Marker_Config()

```
DM35418LIB_API int DM35418_Dac_Channel_Set_Marker_Config (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t marker_enable )
```

Set the configuration of the marker interrupts for this channel.

Parameters

<i>handle</i>	Pointer to the board handle
<i>func_block</i>	Pointer to the function block.
<i>channel</i>	The channel to change.
<i>marker_enable</i>	Bit values indicating whether to enable marker interrupts (1) or disable (0).

Return values

0	Success.
Non-zero	

4.6.2.6 DM35418_Dac_Channel_Setup()

```
DM35418LIB_API int DM35418_Dac_Channel_Setup (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * func_block,
    unsigned int channel,
    enum DM35418_Output_Ranges output_range )
```

Setup the selected DAC channel.

Parameters

<i>handle</i>	Pointer to the board handle
<i>func_block</i>	Pointer to the function block we want to change the output range on.
<i>channel</i>	The channel to set the output range of.
<i>output_range</i>	Enumerated value defining desired output range.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Invalid range selected or channel.

4.6.2.7 DM35418_Dac_Conv_To_Volts()

```
DM35418LIB_API int DM35418_Dac_Conv_To_Volts (
    enum DM35418_Output_Ranges output_range,
    int16_t conversion,
    float * volts )
```

Convert a DAC conversion value to volts.

Parameters

<i>output_range</i>	The output range the channel was set to.
<i>conversion</i>	DAC converter signed value.
<i>volts</i>	The volts equivalent of the converter value.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Function called by an unsupported function block.

4.6.2.8 DM35418_Dac_Fifo_Channel_Write()

```
DM35418LIB_API int DM35418_Dac_Fifo_Channel_Write (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
unsigned int channel,
int32_t value )
```

Write a value to the onboard FIFO.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor, which contains the offsets to command sections of the board.
<i>channel</i>	Channel to write the data to.
<i>value</i>	value to write.

Return values

0	Success.
Non-zero	Failure.

4.6.2.9 DM35418_Dac_Get_Clock_Div()

```
DM35418LIB_API int DM35418_Dac_Get_Clock_Div (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * divider )
```

Get the clock divider value.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>divider</i>	Pointer to the clock divider returned.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.10 DM35418_Dac_Get_Clock_Src()

```
DM35418LIB_API int DM35418_Dac_Get_Clock_Src (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
enum DM35418_Clock_Sources * source )
```

Get the clock source of the DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>source</i>	Pointer to the returned clock set for this DAC.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.6.2.11 DM35418_Dac_Get_Conversion_Count()

```
DM35418LIB_API int DM35418_Dac_Get_Conversion_Count (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * value )
```

Get a count of the number of conversions that DAC has executed.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>value</i>	Pointer to the returned count of conversions executed.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.6.2.12 DM35418_Dac_Get_Last_Conversion()

```
DM35418LIB_API int DM35418_Dac_Get_Last_Conversion (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
```

```
uint8_t * marker,  
int16_t * value )
```

Get the value of the last conversion of the DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DAC channel that we want the last conversion value from.
<i>marker</i>	Pointer to the returned value for the marker byte.
<i>value</i>	Pointer to the returned signed value of the last conversion.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.13 DM35418_Dac_Get_Mode_Status()

```
DM35418LIB_API int DM35418_Dac_Get_Mode_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint8_t * mode_status )
```

Get the Mode and Status of the DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>mode_status</i>	Pointer to the value of the returned mode_status register.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.14 DM35418_Dac_Get_Post_Stop_Conversion_Count()

```
DM35418LIB_API int DM35418_Dac_Get_Post_Stop_Conversion_Count (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * value )
```

Get the number of conversions the DAC will make after a stop trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>value</i>	Pointer to the returned number of conversions.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.15 DM35418_Dac_Get_Start_Trigger()

```
DM35418LIB_API int DM35418_Dac_Get_Start_Trigger (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint8_t * trigger_value )
```

Get the start trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>trigger_value</i>	Pointer to the returned trigger value (event) that will initiate conversions on this DAC.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.16 DM35418_Dac_Get_Stop_Trigger()

```
DM35418LIB_API int DM35418_Dac_Get_Stop_Trigger (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint8_t * trigger_value )
```

Get the stop trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
---------------	--

Parameters

<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>trigger_value</i>	Pointer to the returned trigger value (event) that will halt conversions on this DAC.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.17 DM35418_Dac_Interrupt_Clear_Status()

```
DM35418LIB_API int DM35418_Dac_Interrupt_Clear_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t value )
```

Clear the interrupt status register of the DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>value</i>	Bitmask indicating which interrupts to clear.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.18 DM35418_Dac_Interrupt_Get_Config()

```
DM35418LIB_API int DM35418_Dac_Interrupt_Get_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t * interrupt_ena )
```

Get the interrupt configuration for this DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>interrupt_ena</i>	Pointer to the returned bitmask indicating which interrupts are set.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.6.2.19 DM35418_Dac_Interrupt_Get_Status()

```
DM35418LIB_API int DM35418_Dac_Interrupt_Get_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t * value )
```

Get a interrupt status register of the DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>value</i>	Pointer to the returned interrupt status.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.6.2.20 DM35418_Dac_Interrupt_Set_Config()

```
DM35418LIB_API int DM35418_Dac_Interrupt_Set_Config (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint16_t interrupt_src,
    int enable )
```

Set the interrupt configuration for this DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>interrupt_src</i>	Bitmask indicating which interrupts to set.
<i>enable</i>	A boolean value indicating whether selected interrupts are to be enabled or disabled.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.21 DM35418_Dac_Open()

```
DM35418LIB_API int DM35418_Dac_Open (
    struct DM35418_Board_Descriptor * handle,
    unsigned int number_of_type,
    struct DM35418_Function_Block * func_block )
```

Open the DAC indicated, and determine register locations of control blocks needed to control it.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>number_of_type</i>	Which DAC to open. The first DAC on the board will be 0.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_dacs().

4.6.2.22 DM35418_Dac_Pause()

```
DM35418LIB_API int DM35418_Dac_Pause (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the DAC Mode to Pause.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	Success.
---	----------

Return values

<i>Non-Zero</i>	Failure.
-----------------	----------

4.6.2.23 DM35418_Dac_Reset()

```
DM35418LIB_API int DM35418_Dac_Reset (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the DAC Mode to Reset.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.6.2.24 DM35418_Dac_Set_Clock_Div()

```
DM35418LIB_API int DM35418_Dac_Set_Clock_Div (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t divider )
```

Set the clock divider value.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>divider</i>	Divider value to set this DAC clock to.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.6.2.25 DM35418_Dac_Set_Clock_Source_Global()

```
DM35418LIB_API int DM35418_Dac_Set_Clock_Source_Global (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    enum DM35418_Clock_Sources clock,
    enum DM35418_Dac_Clock_Events clock_driver )
```

Set the source that will drive the global clock.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>clock</i>	Which global clock to set.
<i>clock_driver</i>	Source to drive global clock.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid clock or source requested.

4.6.2.26 DM35418_Dac_Set_Clock_Src()

```
DM35418LIB_API int DM35418_Dac_Set_Clock_Src (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    enum DM35418_Clock_Sources source )
```

Set the clock source of the DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>source</i>	The clock source that we want to set for this DAC.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.27 DM35418_Dac_Set_Conversion_Rate()

```
DM35418LIB_API int DM35418_Dac_Set_Conversion_Rate (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t requested_rate,
    uint32_t * actual_rate )
```

Set the conversion rate of this DAC.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>requested_rate</i>	Requested rate of conversion for the DAC (Hz).
<i>actual_rate</i>	Pointer to the returned value of the actual rate achieved (Hz).

Note

The actual obtainable rate depends on many board-specific values and clocks, and so the returned rate will rarely be the exact same as the requested rate.

Return values

0	Success.
Non-Zero	Failure.

errno may be set as follows:

- `EINVAL` Invalid rate requested.

4.6.2.28 DM35418_Dac_Set_Last_Conversion()

```
DM35418LIB_API int DM35418_Dac_Set_Last_Conversion (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint8_t marker,
    int16_t value )
```

Set a value to be converted by the DAC immediately.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DAC channel that we want the last conversion set to.
<i>marker</i>	Value of the marker bits (top 8 bits)
<i>value</i>	Value to be converted by DAC and set on its output pin.

Note

The DAC will set its output value to the last conversion register value only if the DAC is in Reset mode.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.29 DM35418_Dac_Set_Post_Stop_Conversion_Count()

```
DM35418LIB_API int DM35418_Dac_Set_Post_Stop_Conversion_Count (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t value )
```

Set the number of conversions the DAC will make after a stop trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>value</i>	Number of conversions.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.30 DM35418_Dac_Set_Start_Trigger()

```
DM35418LIB_API int DM35418_Dac_Set_Start_Trigger (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
uint8_t trigger_value )
```

Set the start trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>trigger_value</i>	Trigger value (event) that will initiate conversions on this DAC.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.31 DM35418_Dac_Set_Stop_Trigger()

```
DM35418LIB_API int DM35418_Dac_Set_Stop_Trigger (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint8_t trigger_value )
```

Set the stop trigger.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>trigger_value</i>	Trigger value (event) that will halt conversions on this DAC.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.32 DM35418_Dac_Start()

```
DM35418LIB_API int DM35418_Dac_Start (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Set the DAC Mode to Start.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	Success.
Non-Zero	Failure.

4.6.2.33 DM35418_Dac_Volts_To_Conv()

```
DM35418LIB_API int DM35418_Dac_Volts_To_Conv (
    enum DM35418_Output_Ranges output_range,
    float volts,
    int16_t * dac_conversion )
```

Convert a value in volts to a DAC equivalent signed value.

Parameters

<i>output_range</i>	The output range this channel was set to.
<i>volts</i>	The volts value we want the DAC to output.
<i>dac_conversion</i>	Pointer to signed value representing the equivalent of the volts.

Return values

0	Success.
Non-zero	Failure. errno may be set as follows: EINVAL Function called by an unsupported function block.

4.7 DM35418 DIO Public

Functions

- DM35418LIB_API int DM35418_Dio_Open (struct DM35418_Board_Descriptor *handle, unsigned int number_of_type, struct DM35418_Function_Block *func_block)
Open the DIO indicated, and determine register locations of control blocks needed to control it.
- DM35418LIB_API int DM35418_Dio_Set_Direction (struct DM35418_Board_Descriptor *handle, const struct DM35418_Function_Block *func_block, uint32_t direction)
Set the direction of the DIO pins.

- [DM35418LIB_API](#) [int DM35418_Dio_Get_Direction](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *direction)
Get the direction of the DIO pins.
- [DM35418LIB_API](#) [int DM35418_Dio_Get_Input_Value](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get the input value of the DIO.
- [DM35418LIB_API](#) [int DM35418_Dio_Get_Output_Value](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get the current value of the output register.
- [DM35418LIB_API](#) [int DM35418_Dio_Set_Output_Value](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t value)
Set the value to be put on output pins.

4.7.1 Detailed Description

Library Functions

4.7.2 Function Documentation

4.7.2.1 DM35418_Dio_Get_Direction()

```
DM35418LIB_API int DM35418_Dio_Get_Direction (
    struct DM35418\_Board\_Descriptor * handle,
    const struct DM35418\_Function\_Block * func_block,
    uint32_t * direction )
```

Get the direction of the DIO pins.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block representing the DIO.
<i>direction</i>	Bitmask representing the directions of the pins. (0 = input, 1 = output)

Return values

0	Success.
Non-Zero	Failure.

4.7.2.2 DM35418_Dio_Get_Input_Value()

```
DM35418LIB_API int DM35418_Dio_Get_Input_Value (
    struct DM35418\_Board\_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
uint32_t * value )
```

Get the input value of the DIO.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block representing the DIO.
<i>value</i>	Pointer to returned value.

Note

The value of pins that are set to output will be zero.

Return values

0	Success.
Non-Zero	Failure.

4.7.2.3 DM35418_Dio_Get_Output_Value()

```
DM35418LIB_API int DM35418_Dio_Get_Output_Value (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t * value )
```

Get the current value of the output register.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block representing the DIO.
<i>value</i>	Pointer to returned value.

Note

The value of pins that are set to input will be zero.

Return values

0	Success.
Non-Zero	Failure.

4.7.2.4 DM35418_Dio_Open()

```
DM35418LIB_API int DM35418_Dio_Open (
    struct DM35418_Board_Descriptor * handle,
    unsigned int number_of_type,
    struct DM35418_Function_Block * func_block )
```

Open the DIO indicated, and determine register locations of control blocks needed to control it.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>number_of_type</i>	Which DIO to open. The first DIO on the board will be 0.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.

Return values

0	Success.
Non-Zero	Failure.

4.7.2.5 DM35418_Dio_Set_Direction()

```
DM35418LIB_API int DM35418_Dio_Set_Direction (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    uint32_t direction )
```

Set the direction of the DIO pins.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block representing the DIO.
<i>direction</i>	Bitmask representing the directions to set the pins. (0 = input, 1 = output)

Return values

0	Success.
Non-Zero	Failure.

4.7.2.6 DM35418_Dio_Set_Output_Value()

```
DM35418LIB_API int DM35418_Dio_Set_Output_Value (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
uint32_t value )
```

Set the value to be put on output pins.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block representing the DIO.
<i>value</i>	Value to be written to output pins.

Note

Writing a bit to a pin set to input will have no effect.

Return values

0	Success.
Non-Zero	Failure.

4.8 DM35418 DMA Public Library Constants

Macros

- `#define DM35418_DMA_ACTION_CLEAR 0x00`
Register value for DMA clear action.
- `#define DM35418_DMA_ACTION_GO 0x01`
Register value for DMA go action.
- `#define DM35418_DMA_ACTION_PAUSE 0x02`
Register value for DMA pause action.
- `#define DM35418_DMA_ACTION_HALT 0x03`
Register value for DMA halt action.
- `#define DM35418_DMA_SETUP_DIRECTION_READ 0x04`
Register value to set DMA to READ direction.
- `#define DM35418_DMA_SETUP_DIRECTION_WRITE 0x00`
Register value to set DMA to WRITE direction.
- `#define DM35418_DMA_SETUP_DIRECTION_MASK 0x04`
Register value to set DMA to READ direction.
- `#define DM35418_DMA_SETUP_IGNORE_USED 0x08`
Register value to tell DMA to ignore used buffers.
- `#define DM35418_DMA_SETUP_NOT_IGNORE_USED 0x00`
Register value to tell DMA to not ignore used buffers.
- `#define DM35418_DMA_SETUP_IGNORE_USED_MASK 0x08`
Bit mask for Ignore Used bit in setup register.
- `#define DM35418_DMA_SETUP_INT_ENABLE 0x01`
Register value to enabled interrupts in the setup register.
- `#define DM35418_DMA_SETUP_INT_DISABLE 0x00`
Register value to disable interrupts in the setup register.

- `#define DM35418_DMA_SETUP_INT_MASK 0x01`
Bit mask for the interrupt bit in the setup register.
- `#define DM35418_DMA_SETUP_ERR_INT_ENABLE 0x02`
Register value to enable the error interrupt.
- `#define DM35418_DMA_SETUP_ERR_INT_DISABLE 0x00`
Register value to disable the error interrupt.
- `#define DM35418_DMA_SETUP_ERR_INT_MASK 0x02`
Bit mask for the error interrupt bit in the setup register.
- `#define DM35418_DMA_STATUS_CLEAR 0x00`
Register value to write to status registers to clear them.
- `#define DM35418_DMA_CTRL_CLEAR 0x00`
Register value to write to control register to clear it.
- `#define DM35418_DMA_BUFFER_STATUS_CLEAR 0x00`
Register value to write to the buffer status register to clear it.
- `#define DM35418_DMA_BUFFER_CTRL_CLEAR 0x00`
Register value to write to the buffer control register to clear it.
- `#define DM35418_DMA_BUFFER_STATUS_USED_MASK 0x01`
Bit mask for the used buffer bit in the buffer status register.
- `#define DM35418_DMA_BUFFER_STATUS_TERM_MASK 0x02`
Bit mask for the terminated buffer bit in the buffer status register.
- `#define DM35418_DMA_BUFFER_CTRL_VALID 0x01`
Register value to write to buffer control register to mark it as valid.
- `#define DM35418_DMA_BUFFER_CTRL_HALT 0x02`
Register value to write to buffer control register to tell DMA to halt after processing this buffer.
- `#define DM35418_DMA_BUFFER_CTRL_LOOP 0x04`
Register value to write to buffer control register to tell DMA to loop back to buffer 0 after using this buffer.
- `#define DM35418_DMA_BUFFER_CTRL_INTR 0x08`
Register value to write to buffer control register to tell DMA to issue an interrupt after using this buffer.
- `#define DM35418_DMA_BUFFER_CTRL_PAUSE 0x10`
Register value to write to buffer control register to tell DMA to pause after processing this buffer.
- `#define DM35418_DMA_CTRL_BLOCK_SIZE 0x10`
Constant value indicating DMA control block size.
- `#define DM35418_DMA_BUFFER_CTRL_BLOCK_SIZE 0x10`
Constant value indicating DMA buffer control block size.
- `#define DM35418_BIT_MASK_DMA_BUFFER_SIZE 0xFFFFFFFF`
Bit mask for the DMA buffer size, since it is 24-bits of a 32-bit register.

Enumerations

- `enum DM35418_Fifo_States { DM35418_FIFO_UNKNOWN , DM35418_FIFO_EMPTY , DM35418_FIFO_FULL , DM35418_FIFO_HAS_DATA }`
Descriptions of the possible states the FIFO might be in.

4.8.1 Detailed Description

4.8.2 Enumeration Type Documentation

4.8.2.1 DM35418_Fifo_States

`enum DM35418_Fifo_States`

Descriptions of the possible states the FIFO might be in.

Enumerator

DM35418_FIFO_UNKNOWN	State of FIFO is unknown.
DM35418_FIFO_EMPTY	FIFO is empty.
DM35418_FIFO_FULL	FIFO is full
DM35418_FIFO_HAS_DATA	FIFO is between empty and full

Definition at line 237 of file dm35418_dma_library.h.

4.9 DM35418 DMA Public Library Functions

Functions

- [DM35418LIB_API](#) `int DM35418_Dma_Start (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel)`
Start the DMA.
- [DM35418LIB_API](#) `int DM35418_Dma_Stop (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel)`
Stop the DMA.
- [DM35418LIB_API](#) `int DM35418_Dma_Pause (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel)`
Pause the DMA.
- [DM35418LIB_API](#) `int DM35418_Dma_Clear (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel)`
Clear the DMA.
- [DM35418LIB_API](#) `int DM35418_Dma_Get_Fifo_Counts (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, uint16_t *write_count, uint16_t *read_count)`
Get the Read and Write FIFO count values.
- [DM35418LIB_API](#) `int DM35418_Dma_Get_Fifo_State (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, enum DM35418\_Fifo\_States *state)`
Get the state of the FIFO.
- [DM35418LIB_API](#) `int DM35418_Dma_Configure_Interrupts (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, int enable, int error_enable)`
Configure the interrupts for the DMA channel.
- [DM35418LIB_API](#) `int DM35418_Dma_Get_Interrupt_Configuration (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, int *enable, int *error_enable)`
Get the configuration of the interrupts for the DMA channel.
- [DM35418LIB_API](#) `int DM35418_Dma_Setup (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, int direction, int ignore_used)`
Setup the DMA channel, specifically the direction and if used buffers are ignored.
- [DM35418LIB_API](#) `int DM35418_Dma_Setup_Set_Direction (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, int direction)`
Set the direction of the DMA, read or write.
- [DM35418LIB_API](#) `int DM35418_Dma_Setup_Set_Used (struct DM35418\_Board\_Descriptor *handle, const struct DM35418\_Function\_Block *func_block, unsigned int channel, int ignore_used)`
Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.

- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Get_Errors](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid)
Get the current value of the DMA channel error registers.
- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count, int *current_action, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid, int *stat_complete)
Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.
- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Get_Current_Buffer_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count)
Get the current buffer and buffer count in use by the DMA.
- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Check_For_Error](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int *has_error)
Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.
- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Buffer_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t ctrl)
Setup the DMA buffer for use.
- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Buffer_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t *status, uint8_t *control, uint32_t *size)
Get the status of the buffer. This gets the status, control, and size registers.
- [DM35418LIB_API](#) [int](#) [DM35418_Dma_Check_Buffer_Used](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_num, int *is_used)
Check if the indicated buffer has the "Used" flag set.
- [int](#) [DM35418_Dma_Find_Interrupt](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int *channel, int *channel_complete, int *channel_error)
Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.
- [int](#) [DM35418_Dma_Clear_Interrupt](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int clear_overflow, int clear_underflow, int clear_used, int clear_invalid, int clear_complete)
Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.
- [int](#) [DM35418_Dma_Reset_Buffer](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer)
Reset the DMA buffer, preparing it to be used again by the DMA engine.

4.9.1 Detailed Description

DM35418_Dma_Library_Constants

4.9.2 Function Documentation

4.9.2.1 DM35418_Dma_Buffer_Setup()

```
DM35418LIB_API int DM35418_Dma_Buffer_Setup (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer,
    uint8_t ctrl )
```

Setup the DMA buffer for use.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to set.
<i>buffer</i>	DMA buffer to set.
<i>ctrl</i>	Unsigned short containing control bits. Will be written to control register.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure. errno may be set as follows: EINVAL Invalid channel or buffer requested.

Referenced by setup_adc(), and setup_adcs().

4.9.2.2 DM35418_Dma_Buffer_Status()

```
DM35418LIB_API int DM35418_Dma_Buffer_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer,
    uint8_t * status,
    uint8_t * control,
    uint32_t * size )
```

Get the status of the buffer. This gets the status, control, and size registers.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to get.
<i>buffer</i>	DMA buffer to get.
<i>status</i>	Pointer to returned DMA buffer status register value.
<i>control</i>	Pointer to returned DMA buffer control register value.
<i>size</i>	Pointer to returned DMA buffer size (in bytes).

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel or buffer requested.

Referenced by `setup_adc()`, and `setup_adcs()`.

4.9.2.3 DM35418_Dma_Check_Buffer_Used()

```
DM35418LIB_API int DM35418_Dma_Check_Buffer_Used (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer_num,
    int * is_used )
```

Check if the indicated buffer has the "Used" flag set.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to get.
<i>buffer_num</i>	Which buffer in the DMA channel to check.
<i>is_used</i>	Pointer to returned boolean indicating if the buffer has the Used flag set.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by `ISR()`.

4.9.2.4 DM35418_Dma_Check_For_Error()

```
DM35418LIB_API int DM35418_Dma_Check_For_Error (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int * has_error )
```

Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to set.
<i>has_error</i>	Pointer to returned boolean value indicating if the DMA channel has an error condition.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by ISR().

4.9.2.5 DM35418_Dma_Clear()

```
DM35418LIB_API int DM35418_Dma_Clear (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Clear the DMA.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to clear.

Return values

0	Success.
-1	Failure. errno may be set as follows: EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

4.9.2.6 DM35418_Dma_Clear_Interrupt()

```
int DM35418_Dma_Clear_Interrupt (
    struct DM35418_Board_Descriptor * handle,
```

```

const struct DM35418_Function_Block * func_block,
unsigned int channel,
int clear_overflow,
int clear_underflow,
int clear_used,
int clear_invalid,
int clear_complete )

```

Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA channel to clear.
<i>clear_overflow</i>	Boolean indicating whether or not to clear the overflow interrupt status.
<i>clear_underflow</i>	Boolean indicating whether or not to clear the underflow interrupt status.
<i>clear_used</i>	Boolean indicating whether or not to clear the used interrupt status.
<i>clear_invalid</i>	Boolean indicating whether or not to clear the invalid buffer interrupt status.
<i>clear_complete</i>	Boolean indicating whether or not to clear the buffer completed interrupt status.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

Referenced by ISR().

4.9.2.7 DM35418_Dma_Configure_Interrupts()

```

DM35418LIB_API int DM35418_Dma_Configure_Interrupts (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int enable,
    int error_enable )

```

Configure the interrupts for the DMA channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to configure interrupts for.
<i>enable</i>	Boolean value indicating if interrupt is to be enabled or disabled.
<i>error_enable</i>	Boolean value indicating if interrupts for error conditions are to be enabled or disabled.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by `setup_adc()`, and `setup_adcs()`.

4.9.2.8 DM35418_Dma_Find_Interrupt()

```
int DM35418_Dma_Find_Interrupt (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int * channel,
    int * channel_complete,
    int * channel_error )
```

Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	Pointer to returned DMA channel with interrupt, if any.
<i>channel_complete</i>	Pointer to returned boolean indicating that the interrupt on this channel was from using a buffer with the interrupt bit set.
<i>channel_error</i>	Pointer to returned boolean indicating that the interrupt on this channel was from an error condition.

Return values

0	Success.
Non-Zero	Failure.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

4.9.2.9 DM35418_Dma_Get_Current_Buffer_Count()

```
DM35418LIB_API int DM35418_Dma_Get_Current_Buffer_Count (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint32_t * current_buffer,
    uint32_t * current_count )
```

Get the current buffer and buffer count in use by the DMA.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	Channel to get buffer info from.
<i>current_buffer</i>	Pointer to the returned current buffer the DMA is using.
<i>current_count</i>	Pointer to the returned count for the current buffer. This indicates how far into the buffer the DMA is.

Return values

0	Success.
Non-Zero	Failure.

4.9.2.10 DM35418_Dma_Get_Errors()

```
DM35418LIB_API int DM35418_Dma_Get_Errors (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int * stat_overflow,
    int * stat_underflow,
    int * stat_used,
    int * stat_invalid )
```

Get the current value of the DMA channel error registers.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
---------------	--

Parameters

<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to configure interrupts for.
<i>stat_overflow</i>	Pointer to the returned boolean indicating if overflow has occurred.
<i>stat_underflow</i>	Pointer to the returned boolean indicating if underflow has occurred.
<i>stat_used</i>	Pointer to the returned boolean indicating if the DMA attempted to use an already used buffer.
<i>stat_invalid</i>	Pointer to the returned boolean indicating if the DMA attempted to use an invalid buffer.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.9.2.11 DM35418_Dma_Get_Fifo_Counts()

```
DM35418LIB_API int DM35418_Dma_Get_Fifo_Counts (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint16_t * write_count,
    uint16_t * read_count )
```

Get the Read and Write FIFO count values.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to get counts for.
<i>write_count</i>	Pointer to the returned number of bytes of space available in the FIFO.
<i>read_count</i>	Pointer to the returned number of bytes of data available in the FIFO.

Note

These counts are valid regardless of the direction of the DMA.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.9.2.12 DM35418_Dma_Get_Fifo_State()

```
DM35418LIB_API int DM35418_Dma_Get_Fifo_State (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    enum DM35418_Fifo_States * state )
```

Get the state of the FIFO.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to get state for.
<i>state</i>	Pointer to the returned FIFO state enumeration.

Note

This is just a convenience function that infers a FIFO state from the FIFO counts.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.9.2.13 DM35418_Dma_Get_Interrupt_Configuration()

```
DM35418LIB_API int DM35418_Dma_Get_Interrupt_Configuration (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int * enable,
    int * error_enable )
```

Get the configuration of the interrupts for the DMA channel.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to configure interrupts for.
<i>enable</i>	Pointer to returned value indicating if interrupt is enabled or disabled.
<i>error_enable</i>	Pointer to returned boolean value indicating if interrupts for error conditions are enabled or disabled.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.9.2.14 DM35418_Dma_Pause()

```
DM35418LIB_API int DM35418_Dma_Pause (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Pause the DMA.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to pause.

Return values

0	Success.
-1	Failure. errno may be set as follows: EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

4.9.2.15 DM35418_Dma_Reset_Buffer()

```
int DM35418_Dma_Reset_Buffer (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer )
```

Reset the DMA buffer, preparing it to be used again by the DMA engine.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA channel containing the buffer.
<i>buffer</i>	Buffer in channel to clear.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel or buffer requested.

Warning

This function is not compatible with the Windows driver package and is therefore not included in the Windows DLL.

Referenced by ISR().

4.9.2.16 DM35418_Dma_Setup()

```
DM35418LIB_API int DM35418_Dma_Setup (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int direction,
    int ignore_used )
```

Setup the DMA channel, specifically the direction and if used buffers are ignored.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to set.
<i>direction</i>	Direction for DMA. See the DMA constants for possible values.
<i>ignore_used</i>	Boolean value indicating if used buffers should be ignored.

Note

This is a convenience function that accomplishes two of the setup steps in one function. This function is identical to calling the direction and ignore used library functions separately.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested, or wrong direction requested for function block type.

Referenced by setup_adc(), and setup_adcs().

4.9.2.17 DM35418_Dma_Setup_Set_Direction()

```
DM35418LIB_API int DM35418_Dma_Setup_Set_Direction (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int direction )
```

Set the direction of the DMA, read or write.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to set.
<i>direction</i>	Direction for DMA. See the DMA constants for possible values.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested, or wrong direction requested for function block type.

4.9.2.18 DM35418_Dma_Setup_Set_Used()

```
DM35418LIB_API int DM35418_Dma_Setup_Set_Used (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    int ignore_used )
```

Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to set.
<i>ignore_used</i>	Boolean value indicating if used buffers should be ignored.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel requested.

4.9.2.19 DM35418_Dma_Start()

```
DM35418LIB_API int DM35418_Dma_Start (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Start the DMA.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to start.

Return values

0	Success.
-1	Failure. errno may be set as follows: EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

Referenced by main(), setup_slave_adcs(), and start_slave_adcs().

4.9.2.20 DM35418_Dma_Status()

```
DM35418LIB_API int DM35418_Dma_Status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    uint32_t * current_buffer,
    uint32_t * current_count,
    int * current_action,
    int * stat_overflow,
    int * stat_underflow,
    int * stat_used,
    int * stat_invalid,
    int * stat_complete )
```

Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to configure interrupts for.
<i>current_buffer</i>	Pointer to the returned current buffer the DMA is using.
<i>current_count</i>	Pointer to the returned count for the current buffer. This indicates how far into the buffer the DMA is.
<i>current_action</i>	Pointer to the returned action the DMA is currently taking.
<i>stat_overflow</i>	Pointer to the returned boolean indicating if overflow has occurred.
<i>stat_underflow</i>	Pointer to the returned boolean indicating if underflow has occurred.
<i>stat_used</i>	Pointer to the returned boolean indicating if the DMA attempted to use an already used buffer.
<i>stat_invalid</i>	Pointer to the returned boolean indicating if the DMA attempted to use an invalid buffer.
<i>stat_complete</i>	Pointer to the returned boolean indicating if the DMA has completed using a buffer that had an interrupt set.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel requested.

Referenced by output_channel_status().

4.9.2.21 DM35418_Dma_Stop()

```
DM35418LIB_API int DM35418_Dma_Stop (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Stop the DMA.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to stop.

Return values

0	Success.
-1	Failure. errno may be set as follows: EBUSY The action was not executed before timeout. EINVAL Invalid channel requested.

4.10 DM35418 Driver Constants

Macros

- #define `DM35418_MAX_NUM_DEVICES` 8
DM35418 Maximum number of devices.
- #define `DM35418_NUM_MINORS_PER_DEVICE` 1
DM35418 Number of minors per device.
- #define `DM35418_NAME_LENGTH` 200
DM35418 Max possible board name length.
- #define `DM35418_PCI_NUM_REGIONS` PCI_ROM_RESOURCE
Number of standard PCI regions.
- #define `DM35418_INT_QUEUE_SIZE` 256
Number of interrupts to hold in a queue for processing.

4.10.1 Detailed Description

4.11 DM35418 Driver Enumerations

Enumerations

- enum `dm35418_pci_region_access_dir` { `DM35418_PCI_REGION_ACCESS_READ` = 0 , `DM35418_PCI_REGION_ACCESS_WRITE` }
Direction of access to standard PCI region.

4.11.1 Detailed Description

DM35418_Driver_Constants

4.11.2 Enumeration Type Documentation

4.11.2.1 dm35418_pci_region_access_dir

```
enum dm35418_pci_region_access_dir
```

Direction of access to standard PCI region.

Enumerator

DM35418_PCI_REGION_ACCESS_READ	Read from the region
DM35418_PCI_REGION_ACCESS_WRITE	Write to the region

Definition at line 92 of file dm35418_driver.h.

4.12 DM35418 Driver Structures

Data Structures

- struct [dm35418_pci_region](#)
DM35418 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.
- struct [dm35418_dma_descriptor](#)
DM35418 DMA descriptor. This structure holds information about a single DMA buffer.
- struct [dm35418_device_descriptor](#)
DM35418 Device Descriptor. The identifying info for this particular board.

Variables

- static struct file_operations [dm35418_file_ops](#)
Placeholder prototype for file ops struct.

4.12.1 Detailed Description

DM35418_Driver_Enumerations

4.13 DM35418 Example Programs Constants

Macros

- #define [BUFFER_VALID](#) 1
Boolean indicating buffer valid.
- #define [BUFFER_NO_VALID](#) 0
Boolean indicating buffer not valid.
- #define [BUFFER_HALT](#) 1
Boolean indicating buffer halt set.
- #define [BUFFER_NO_HALT](#) 0
Boolean indicating buffer halt not set.
- #define [BUFFER_LOOP](#) 1
Boolean indicating buffer loop set.
- #define [BUFFER_NO_LOOP](#) 0
Boolean indicating buffer loop not set.
- #define [BUFFER_INTERRUPT](#) 1
Boolean indicating buffer interrupt.
- #define [BUFFER_NO_INTERRUPT](#) 0
Boolean indicating no buffer interrupt.
- #define [BUFFER_PAUSE](#) 1
Boolean indicating buffer should pause when filled.
- #define [BUFFER_NO_PAUSE](#) 0

- Boolean indicating buffer should not pause when filled.*

 - #define `IGNORE_USED` 1

Boolean indicating ignore used buffers.
- #define `NOT_IGNORE_USED` 0

Boolean indicating not ignore used buffers.
- #define `CLEAR_INTERRUPT` 1

Boolean indicating to clear an interrupt.
- #define `NO_CLEAR_INTERRUPT` 0

Boolean indicating to not clear an interrupt.
- #define `INTERRUPT_ENABLE` 1

Boolean indicating interrupt enable.
- #define `INTERRUPT_DISABLE` 0

Boolean indicating interrupt disable.
- #define `ERROR_INTR_ENABLE` 1

Boolean indicating error interrupt enable.
- #define `ERROR_INTR_DISABLE` 0

Boolean indicating error interrupt disable.
- #define `SYNCBUS_NONE` 0

Value indicating no Syncbus option was chosen.
- #define `SYNCBUS_MASTER` 1

Value indicating Syncbus Master was chosen.
- #define `SYNCBUS_SLAVE` 2

Value indicating Syncbus Slave was chosen.
- #define `CHANNEL_0` 0

Constant for selecting Channel 0.
- #define `CHANNEL_1` 1

Constant for selecting Channel 1.
- #define `CHANNEL_2` 2

Constant for selecting Channel 2.
- #define `CHANNEL_3` 3

Constant for selecting Channel 3.
- #define `BUFFER_0` 0

Constant for selecting Buffer 0.
- #define `BUFFER_1` 1

Constant for selecting Buffer 1.
- #define `ADC_0` 0

Constant for selecting ADC 0.
- #define `ADC_1` 1

Constant for selecting ADC 1.
- #define `DAC_0` 0

Constant for selecting DAC 0.
- #define `DAC_1` 1

Constant for selecting DAC 1.
- #define `DAC_2` 2

Constant for selecting DAC 2.
- #define `DAC_3` 3

Constant for selecting DAC 3.
- #define `REF_0` 0

Constant for selecting REF 0.
- #define `REF_1` 1

Constant for selecting REF 1.

- `#define DIO_0 0`
Constant for selecting DIO 0.
- `#define ADIO_0 0`
Constant for selecting ADIO 0.
- `#define ENABLED 1`
Constant to indicate an Enabled value.
- `#define DISABLED 0`
Constant to indicate a Disabled value.

Enumerations

- `enum Help_Options {`
`HELP_OPTION = 1 , MINOR_OPTION , RATE_OPTION , CHANNELS_OPTION ,`
`FILE_OPTION , START_OPTION , WAVE_OPTION , TEST_OPTION ,`
`NOSTOP_OPTION , SYNCBUS_OPTION , DUMP_OPTION , HOURS_OPTION ,`
`OUTPUT_RMS_OPTION , OUTPUT_ADC_OPTION , ADC_NUM_OPTION , DAC_NUM_OPTION ,`
`ADC_OPTION , DAC_OPTION , PATTERN_OPTION , SAMPLES_OPTION ,`
`MODE_OPTION , AD_MODE_OPTION , REF_NUM_OPTION , BINARY_OPTION ,`
`SENDER_OPTION , RECEIVER_OPTION , RANGE_OPTION , REFILL_FIFO_OPTION ,`
`LOW_THRESHOLD_OPTION , PORT_OPTION , BAUD_OPTION , EXTERNAL_OPTION ,`
`SIZE_OPTION , VERBOSE_OPTION , USER_ID_OPTION , COUNT_OPTION ,`
`NUM_OPTION , SYNC_TERM_OPTION , BIN2TXT_OPTION , STORE_OPTION ,`
`TERM_OPTION , REFCLK_OPTION , OFILE_OPTION , PACKED_OPTION ,`
`MASTER_OPTION , SLAVE_OPTION , SYNC_CONN_OPTION }`
Constants used for parsing command line parameters of example programs.

4.13.1 Detailed Description

4.13.2 Enumeration Type Documentation

4.13.2.1 Help_Options

`enum Help_Options`

Constants used for parsing command line parameters of example programs.

Note

This value won't be seen by the user (except in code), so the value can be used for any desired option.

Enumerator

HELP_OPTION	Command line parameter <code>--help</code> .
MINOR_OPTION	Command line parameter <code>--minor</code> .
RATE_OPTION	Command line parameter <code>--rate</code> .
CHANNELS_OPTION	Command line parameter <code>--chan</code> .
FILE_OPTION	Command line parameter for including a file.
START_OPTION	Command line parameter <code>--start</code> .

Enumerator

WAVE_OPTION	Command line parameter –wave.
TEST_OPTION	Command line parameter –test.
NOSTOP_OPTION	Command line parameter –nostop.
SYNCBUS_OPTION	Command line parameter –syncbus.
DUMP_OPTION	Command line parameter –dump.
HOURS_OPTION	Command line parameter –hours.
OUTPUT_RMS_OPTION	Command line parameter –output_rms.
OUTPUT_ADC_OPTION	Command line parameter –output_adc.
ADC_NUM_OPTION	Command line parameter –num_adc.
DAC_NUM_OPTION	Command line parameter –num_dac.
ADC_OPTION	Command line parameter –adc.
DAC_OPTION	Command line parameter –dac.
PATTERN_OPTION	Command line parameter –pattern.
SAMPLES_OPTION	Command line parameter –samples.
MODE_OPTION	Command line parameter –mode.
AD_MODE_OPTION	Command line parameter –ad_mode.
REF_NUM_OPTION	Command line parameter –ref.
BINARY_OPTION	Command line parameter –binary.
SENDER_OPTION	Command line parameter –sender.
RECEIVER_OPTION	Command line parameter –receiver.
RANGE_OPTION	Command line parameter –range.
REFILL_FIFO_OPTION	Command line parameter –refill.
LOW_THRESHOLD_OPTION	Command line parameter –low.
PORT_OPTION	Command line parameter –port.
BAUD_OPTION	Command line parameter –baud.
EXTERNAL_OPTION	Command line parameter –external.
SIZE_OPTION	Command line parameter –size.
VERBOSE_OPTION	Command line parameter –verbose.
USER_ID_OPTION	Command line parameter –userid.
COUNT_OPTION	Command line parameter –count.
NUM_OPTION	Command line parameter –num.
SYNC_TERM_OPTION	Command line parameter –syncterm.
BIN2TXT_OPTION	Command line parameter –bin2txt.
STORE_OPTION	Command line parameter –store.
TERM_OPTION	Command line parameter –term (Termination)
REFCLK_OPTION	Command line parameter –refclk (Reference clock selection)
OFIELD_OPTION	Command line parameter –ofile (Output file selection)
PACKED_OPTION	Command line parameter –packed (Packed, 16-bit samples selection)
MASTER_OPTION	Master minor option for synchronization.
SLAVE_OPTION	Slave minor option for synchronization.
SYNC_CONN_OPTION	Syncbus connector option.

Definition at line 43 of file dm35418_examples.h.

4.14 DM35418 Board Macros

Macros

- #define `CLK_40MHZ` 40000000

4.14.1 Detailed Description

4.14.2 Macro Definition Documentation

4.14.2.1 CLK_40MHZ

```
#define CLK_40MHZ 40000000
```

This is the standard clock of the DM35x18 boards

Definition at line 52 of file `dm35418_gbc_library.h`.

4.15 DM35418 Board Library Public Functions

Functions

- `DM35418LIB_API` int `DM35418_Gbc_Board_Reset` (struct `DM35418_Board_Descriptor` *handle)
Write the reset value to the correct register to initiate a board-level reset.
- `DM35418LIB_API` int `DM35418_Gbc_Ack_Interrupt` (struct `DM35418_Board_Descriptor` *handle)
Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.
- `DM35418LIB_API` int `DM35418_Function_Block_Open` (struct `DM35418_Board_Descriptor` *handle, unsigned int number, struct `DM35418_Function_Block` *func_block)
Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.
- `DM35418LIB_API` int `DM35418_Function_Block_Open_Module` (struct `DM35418_Board_Descriptor` *handle, uint32_t fb_type, unsigned int number_of_type, struct `DM35418_Function_Block` *func_block)
Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.
- `DM35418LIB_API` int `DM35418_Gbc_Get_Format` (struct `DM35418_Board_Descriptor` *handle, uint8_t *format_id)
Get the format ID of the board.
- `DM35418LIB_API` int `DM35418_Gbc_Get_Revision` (struct `DM35418_Board_Descriptor` *handle, uint8_t *rev)
Get the PDP revision number of the board.
- `DM35418LIB_API` int `DM35418_Gbc_Get_Pdp_Number` (struct `DM35418_Board_Descriptor` *handle, uint32_t *pdp_num)
Get PDP Number of the board.
- `DM35418LIB_API` int `DM35418_Gbc_Get_Fpga_Build` (struct `DM35418_Board_Descriptor` *handle, uint32_t *fpga_build)
Get the FPGA Build number of the board.
- `DM35418LIB_API` int `DM35418_Gbc_Get_Sys_Clock_Freq` (struct `DM35418_Board_Descriptor` *handle, uint32_t *clock_freq, int *is_std_clk)
Get the measured frequency of the system clock of the board.

4.15.1 Detailed Description

DM35418_Board_Macros

4.15.2 Function Documentation

4.15.2.1 DM35418_Function_Block_Open()

```
DM35418LIB_API int DM35418_Function_Block_Open (
    struct DM35418_Board_Descriptor * handle,
    unsigned int number,
    struct DM35418_Function_Block * func_block )
```

Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>number</i>	Which function block to open. The first function block on the board is at number 0.
<i>func_block</i>	Pointer to the function block descriptor. When the function block info is successfully read from the device, then this descriptor will be allocated to hold the data.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

Referenced by main().

4.15.2.2 DM35418_Function_Block_Open_Module()

```
DM35418LIB_API int DM35418_Function_Block_Open_Module (
    struct DM35418_Board_Descriptor * handle,
    uint32_t fb_type,
    unsigned int number_of_type,
    struct DM35418_Function_Block * func_block )
```

Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
---------------	--

Parameters

<i>fb_type</i>	Type of function block you want to open. ADC, DAC, DIO, etc. The constant values are in the dm35418_types.h file.
<i>number_of_type</i>	Ordinal number of that particular type of function block that you wish to access. The first instance of that type is 0th.
<i>func_block</i>	Pointer to the function block descriptor. When the function block info is successfully read from the device, then this descriptor will be allocated to hold the data.

Return values

0	Success.
Non-Zero	Failure.

4.15.2.3 DM35418_Gbc_Ack_Interrupt()

```
DM35418LIB_API int DM35418_Gbc_Ack_Interrupt (
    struct DM35418_Board_Descriptor * handle )
```

Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
---------------	--

Return values

0	Success.
Non-Zero	Failure.

Referenced by ISR().

4.15.2.4 DM35418_Gbc_Board_Reset()

```
DM35418LIB_API int DM35418_Gbc_Board_Reset (
    struct DM35418_Board_Descriptor * handle )
```

Write the reset value to the correct register to initiate a board-level reset.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
---------------	--

Return values

0	Success.
Non-Zero	Failure.

Referenced by main().

4.15.2.5 DM35418_Gbc_Get_Format()

```
DM35418LIB_API int DM35418_Gbc_Get_Format (
    struct DM35418_Board_Descriptor * handle,
    uint8_t * format_id )
```

Get the format ID of the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>format_id</i>	Pointer to the returned format ID value.

Return values

0	Success.
Non-Zero	Failure.

4.15.2.6 DM35418_Gbc_Get_Fpga_Build()

```
DM35418LIB_API int DM35418_Gbc_Get_Fpga_Build (
    struct DM35418_Board_Descriptor * handle,
    uint32_t * fpga_build )
```

Get the FPGA Build number of the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>fpga_build</i>	Pointer to the returned FPGA Build number.

Return values

0	Success.
Non-Zero	Failure.

Referenced by main().

4.15.2.7 DM35418_Gbc_Get_Pdp_Number()

```
DM35418LIB_API int DM35418_Gbc_Get_Pdp_Number (
    struct DM35418_Board_Descriptor * handle,
    uint32_t * pdp_num )
```

Get PDP Number of the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>pdp_num</i>	Pointer to the returned PDP Number.

Return values

0	Success.
Non-Zero	Failure.

Referenced by main().

4.15.2.8 DM35418_Gbc_Get_Revision()

```
DM35418LIB_API int DM35418_Gbc_Get_Revision (
    struct DM35418_Board_Descriptor * handle,
    uint8_t * rev )
```

Get the PDP revision number of the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>rev</i>	Pointer to the returned revision value.

Return values

0	Success.
Non-Zero	Failure.

Referenced by main().

4.15.2.9 DM35418_Gbc_Get_Sys_Clock_Freq()

```
DM35418LIB_API int DM35418_Gbc_Get_Sys_Clock_Freq (
    struct DM35418_Board_Descriptor * handle,
    uint32_t * clock_freq,
    int * is_std_clk )
```

Get the measured frequency of the system clock of the board.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>clock_freq</i>	Pointer to the returned system clock frequency (in Hz)
<i>is_std_clk</i>	Boolean value indicating if the clock read is a standard value. If true, then this function will always return the same value upon every call. If false, then this function will return the clock frequency actually read from the register. Note: If the value read from the GBC register is not a standard clock, then the clock frequency returned can change from read to read by slight variations.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.16 DM35418 ioctl macros

Macros

- #define **DM35418_IOCTL_MAGIC** 'D'
Unique 8-bit value used to generate unique ioctl() request codes.
- #define **DM35418_IOCTL_REQUEST_BASE** 0x00
First ioctl() request number.
- #define **DM35418_IOCTL_REGION_READ**
ioctl() request code for reading from a PCI region
- #define **DM35418_IOCTL_REGION_WRITE**
ioctl() request code for writing to a PCI region
- #define **DM35418_IOCTL_REGION_MODIFY**
ioctl() request code for PCI region read/modify/write
- #define **DM35418_IOCTL_DMA_FUNCTION**
ioctl() request code for DMA function
- #define **DM35418_IOCTL_WAKEUP**
ioctl() request code for User ISR thread wake up
- #define **DM35418_IOCTL_INTERRUPT_GET**
ioctl() request code to retrieve interrupt status information

4.16.1 Detailed Description

4.17 DM35418 Board Access Public Library Functions

Data Structures

- struct [DM35418_Board_Descriptor](#)

DM35418 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Functions

- int [DM35418_Dma_Initialize](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int num_buffers, uint32_t buffer_size)
Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.
- int [DM35418_Dma_Read](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_to_read_from, uint32_t buffer_size, void *local_↔buffer_ptr)
Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.
- int [DM35418_Dma_Write](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_to_write_to, uint32_t buffer_size, void *local_buffer↔_ptr)
Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.
- int [DM35418_General_RemoveISR](#) (struct [DM35418_Board_Descriptor](#) *handle)
Remove the ISR from the system interrupt.
- void * [DM35418_General_WaitForInterrupt](#) (void *ptr)
Loop/Poll and wait for an interrupt to happen, then take action.
- int [DM35418_General_InstallISR](#) (struct [DM35418_Board_Descriptor](#) *handle, void(*isr_fnct))
Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.
- int [DM35418_General_SetISRPriority](#) (struct [DM35418_Board_Descriptor](#) *handle, int priority)
Set the priority of the user ISR thread.

4.17.1 Detailed Description

4.17.2 Function Documentation

4.17.2.1 DM35418_Dma_Initialize()

```
int DM35418_Dma_Initialize (
    struct DM35418\_Board\_Descriptor * handle,
    const struct DM35418\_Function\_Block * func_block,
    unsigned int channel,
    unsigned int num_buffers,
    uint32_t buffer_size )
```

Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA Channel to get state for.
<i>num_buffers</i>	Number of DMA buffers to allocate and initialize.
<i>buffer_size</i>	The size in bytes to allocate for each buffer.

Return values

0	Success.
-1	Failure. errno may be set as follows: EINVAL Invalid channel or buffer requested. ENOMEM Memory could not be allocated for the DMA buffers.

Referenced by `setup_adc()`, and `setup_adcs()`.

4.17.2.2 DM35418_Dma_Read()

```
int DM35418_Dma_Read (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer_to_read_from,
    uint32_t buffer_size,
    void * local_buffer_ptr )
```

Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA channel containing the buffer to be read.
<i>buffer_to_read_from</i>	Buffer in channel to read.
<i>buffer_size</i>	Number of bytes to read from the DMA buffer. In most cases, this should be equal to the allocated size of the buffer.
<i>local_buffer_ptr</i>	Pointer to local memory buffer already allocated that data will be copied into.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel or buffer requested.

Referenced by ISR().

4.17.2.3 DM35418_Dma_Write()

```
int DM35418_Dma_Write (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel,
    unsigned int buffer_to_write_to,
    uint32_t buffer_size,
    void * local_buffer_ptr )
```

Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the function block descriptor. The descriptor holds the information about the function block, including offsets.
<i>channel</i>	DMA channel containing the buffer to be written to.
<i>buffer_to_write_to</i>	Buffer in channel to write to.
<i>buffer_size</i>	Number of bytes to write to the DMA buffer. In most cases, this should be equal to the allocated size of the buffer.
<i>local_buffer_ptr</i>	Pointer to local memory buffer already allocated where data will come from.

Return values

0	Success.
Non-Zero	Failure. errno may be set as follows: EINVAL Invalid channel or buffer requested.

Referenced by main().

4.17.2.4 DM35418_General_InstallISR()

```
int DM35418_General_InstallISR (
    struct DM35418_Board_Descriptor * handle,
    void * isr_fnct )
```

Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>isr_fnct</i>	Pointer to the user ISR function that will be executed when an interrupt happens.

Return values

0	Success.
-1	Failure. errno may be set as follows: EFAULT Could not create thread.

Referenced by `main()`, and `setup_isr()`.

4.17.2.5 DM35418_General_RemoveISR()

```
int DM35418_General_RemoveISR (
    struct DM35418_Board_Descriptor * handle )
```

Remove the ISR from the system interrupt.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
---------------	--

Return values

0	Success.
-1	Failure. errno may be set as follows: EFAULT User ISR was already removed.

4.17.2.6 DM35418_General_SetISRPriority()

```
int DM35418_General_SetISRPriority (
    struct DM35418_Board_Descriptor * handle,
    int priority )
```

Set the priority of the user ISR thread.

Parameters

<i>handle</i>	Pointer to the device descriptor, which contains the open file id.
<i>priority</i>	Attempt to set the priority of the user ISR thread.

Note

This may require root privileges

Return values

0	Success.
-1	Failure. errno may be set as follows: EFAULT User ISR did not exist.

4.17.2.7 DM35418_General_WaitForInterrupt()

```
void* DM35418_General_WaitForInterrupt (
    void * ptr )
```

Loop/Poll and wait for an interrupt to happen, then take action.

Parameters

<i>ptr</i>	A void pointer for the board descriptor.
------------	--

Return values

0	Success.
-1	Failure. errno may be set as follows: ENODATA File descriptor is missing or unreadable. EIO There was no interrupt allocated to this device.

4.18 DM35418 Register Offsets

Macros

- #define [DM35418_OFFSET_GBC_FORMAT](#) 0x00
Offset to General Board Control (BAR0) Format ID register.
- #define [DM35418_OFFSET_GBC_REV](#) 0x01
Offset to General Board Control (BAR0) Format ID register.
- #define [DM35418_OFFSET_GBC_END_INTERRUPT](#) 0x02
Offset to General Board Control (BAR0) EOI (End of Interrupt) register.
- #define [DM35418_OFFSET_GBC_BOARD_RESET](#) 0x03
Offset to General Board Control (BAR0) Board Reset register.
- #define [DM35418_OFFSET_GBC_PDP_NUMBER](#) 0x04
Offset to General Board Control (BAR0) PDP Number register.
- #define [DM35418_OFFSET_GBC_FPGA_BUILD](#) 0x08
Offset to General Board Control (BAR0) FPGA Build register.
- #define [DM35418_OFFSET_GBC_SYS_CLK_FREQ](#) 0x0c
Offset to General Board Control (BAR0) System Clock register.
- #define [DM35418_OFFSET_GBC_IRQ_STATUS](#) 0x10

- Offset to General Board Control (BAR0) IRQ Status register. Each bit corresponds to a function block.*

 - #define `DM35418_OFFSET_GBC_DMA_IRQ_STATUS` 0x18
- Offset to General Board Control (BAR0) DMA IRQ Status register. Each bit corresponds to a function block.*

 - #define `DM35418_OFFSET_GBC_FB_START` 0x20
- Offset to the beginning of the Function Blocks section of the GBC.*

 - #define `DM35418_GBC_FB_BLK_SIZE` 0x10
- Size of the function block entries in the GBC.*

 - #define `DM35418_OFFSET_GBC_FB_ID` 0x00
- Offset to Function Block ID, from the start of the function block section.*

 - #define `DM35418_FB_ID_TYPE_MASK` 0x0000FFFF
- Bit mask for TYPE portion of FB ID.*

 - #define `DM35418_FB_ID_SUBTYPE_MASK` 0x00FF0000
- Bit mask for SUBTYPE portion of FB ID.*

 - #define `DM35418_FB_ID_TYPE_REV_MASK` 0xFF000000
- Bit mask for TYPE REV portion of FB ID.*

 - #define `DM35418_OFFSET_GBC_FB_OFFSET` 0x04
- Offset to the FB Offset in the GBC, from the start of the FB data block.*

 - #define `DM35418_OFFSET_GBC_FB_DMA_OFFSET` 0x08
- Offset to the FB DMA Offset in the GBC, from the start of the FB data block.*

 - #define `DM35418_OFFSET_DMA_ACTION` 0x00
- Offset to the DMA Action Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_SETUP` 0x01
- Offset to the DMA Setup Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_STAT_OVERFLOW` 0x02
- Offset to the DMA Status (Overflow) Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_STAT_UNDERFLOW` 0x03
- Offset to the DMA Status (Underflow) Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_CURRENT_COUNT` 0x04
- Offset to the DMA Current Count Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_CURRENT_BUFFER` 0x07
- Offset to the DMA Current Buffer Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_WR_FIFO_CNT` 0x08
- Offset to the DMA Write FIFO Count Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_RD_FIFO_CNT` 0x0A
- Offset to the DMA Read FIFO Count Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_STAT_USED` 0x0C
- Offset to the DMA Status (Used) Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_STAT_INVALID` 0x0D
- Offset to the DMA Status (Invalid) Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_STAT_COMPLETE` 0x0E
- Offset to the DMA Status (Complete) Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_LAST_ACTION` 0x0F
- Offset to the DMA Last Action Register (BAR2)*

 - #define `DM35418_OFFSET_DMA_BUFF_START` 0x10
- Offset to the start of the buffer control section (BAR2)*

 - #define `DM35418_OFFSET_DMA_BUFFER_STAT` 0x02
- Offset to the buffer status register, from the start of the buffer control section (BAR2)*

 - #define `DM35418_OFFSET_DMA_BUFFER_CTRL` 0x03
- Offset to the buffer control register, from the start of the buffer control section (BAR2)*

 - #define `DM35418_OFFSET_DMA_BUFFER_SIZE` 0x04
- Offset to the buffer size register, from the start of the buffer control section (BAR2)*

- #define [DM35418_OFFSET_DMA_BUFFER_ADDRESS](#) 0x08
Offset to the buffer address register, from the start of the buffer control section (BAR2)
- #define [DM35418_OFFSET_FB_DMA_CHANNELS](#) 0x06
Offset to the DMA Channels count of the function block (BAR2)
- #define [DM35418_OFFSET_FB_DMA_BUFFERS](#) 0x07
Offset to the DMA buffers count of the function block (BAR2)
- #define [DM35418_OFFSET_FB_CTRL_START](#) 0x08
Offset to the beginning of the Function Block control section in BAR2.
- #define [DM35418_OFFSET_ADC_MODE_STATUS](#) 0x00
Offset to the ADC Mode-Status register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_SRC](#) 0x01
Offset to the ADC Clock Source register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_START_TRIG](#) 0x02
Offset to the ADC Start Trigger register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_STOP_TRIG](#) 0x03
Offset to the ADC Stop Trigger register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_DIV](#) 0x04
Offset to the ADC Clock Divider register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_DIV_COUNTER](#) 0x08
Offset to the ADC Clock Divider Counter register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_PRE_CAPT_COUNT](#) 0x0c
Offset to the ADC Pre-Start Capture Count register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_POST_CAPT_COUNT](#) 0x10
Offset to the ADC Post-Stop Capture Count register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_SAMPLE_COUNT](#) 0x14
Offset to the ADC Sample Count register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_INT_ENABLE](#) 0x18
Offset to the ADC Interrupt Enable register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_INT_STAT](#) 0x1e
Offset to the ADC Interrupt Status register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_BUS2](#) 0x22
Offset to the ADC Clock Bus 2, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_BUS3](#) 0x23
Offset to the ADC Clock Bus 3 register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_BUS4](#) 0x24
Offset to the ADC Clock Bus 4 register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_BUS5](#) 0x25
Offset to the ADC Clock Bus 5 register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_BUS6](#) 0x26
Offset to the ADC Clock Bus 6 register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CLK_BUS7](#) 0x27
Offset to the ADC Clock Bus 7 register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_AD_CONFIG](#) 0x28
Offset to the ADC AD Config register, from the start of the ADC control section.
- #define [DM35418_OFFSET_ADC_CHAN_CTRL_BLK_START](#) 0x2c
Offset to the start of the Channel Control Section, from the start of the ADC control section.
- #define [DM35418_ADC_CHAN_CTRL_BLK_SIZE](#) 0x18
Constant size of ADC channel section in function block.
- #define [DM35418_OFFSET_ADC_CHAN_FRONT_END_CONFIG](#) 0x00
Offset to the Channel Front End Config register, from the start of the ADC channel control section.
- #define [DM35418_OFFSET_ADC_CHAN_DATA_COUNT](#) 0x04

- Offset to the Channel FIFO Data count register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_FILTER](#) 0x09
- Offset to the Channel Filter register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_INTR_STAT](#) 0x0a
- Offset to the Channel Interrupt Status register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_INTR_ENABLE](#) 0x0b
- Offset to the Channel Interrupt Enable register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_LOW_THRESHOLD](#) 0x0c
- Offset to the Channel Low Threshold register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_HIGH_THRESHOLD](#) 0x10
- Offset to the Channel High Threshold register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_LAST_SAMPLE](#) 0x14
- Offset to the Channel Last Sample register, from the start of the ADC channel control section.*

 - #define [DM35418_OFFSET_ADC_FIFO_CTRL_BLK_START](#) 0x334
- Offset to the start of the FIFO Control Section, from the start of the ADC control section.*

 - #define [DM35418_ADC_FIFO_CTRL_BLK_SIZE](#) 0x4
- Constant size of ADC FIFO section in function block.*

 - #define [DM35418_OFFSET_FB_ADC_FIFO](#) 0x0334
- Offset to the FIFO for non-DMA read and write operations.*

 - #define [DM35418_OFFSET_DAC_MODE_STATUS](#) 0x00
- Offset to the Mode/Status register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_SRC](#) 0x01
- Offset to the Clock Source register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_START_TRIG](#) 0x02
- Offset to the Start Trigger register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_STOP_TRIG](#) 0x03
- Offset to the Stop Trigger register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_DIV](#) 0x04
- Offset to the Clock Divider register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_DIV_COUNT](#) 0x08
- Offset to the Clock Divider Counter register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_POST_STOP_CONV](#) 0x10
- Offset to the Post-Stop Conversion Count register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CONV_COUNT](#) 0x14
- Offset to the Conversion Count register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_INT_ENABLE](#) 0x18
- Offset to the Interrupt Enable register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_INT_STAT](#) 0x1e
- Offset to the Interrupt Status register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS2](#) 0x22
- Offset to the Clock Bus 2 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS3](#) 0x23
- Offset to the Clock Bus 3 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS4](#) 0x24
- Offset to the Clock Bus 4 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS5](#) 0x25
- Offset to the Clock Bus 5 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS6](#) 0x26
- Offset to the Clock Bus 6 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS7](#) 0x27
- Offset to the Clock Bus 7 register, from the start of the DAC control section.*

- #define [DM35418_OFFSET_DAC_DA_CONFIG](#) 0x28
Offset to the DA Config register, from the start of the DAC control section.
- #define [DM35418_OFFSET_DAC_CHAN_CTRL_BLK_START](#) 0x2c
Offset to the start of the DAC channel control section, from the start of the DAC control section.
- #define [DM35418_DAC_CHAN_CTRL_BLK_SIZE](#) 0x14
Constant size of channel control section in function block.
- #define [DM35418_OFFSET_DAC_CHAN_FRONT_END_CONFIG](#) 0x00
Offset to the Front-End Config register, from the start of the DAC channel control section.
- #define [DM35418_OFFSET_DAC_CHAN_MARKER_STATUS](#) 0x0a
Offset to the Channel marker Interrupt Status register, from the start of the DAC channel control section.
- #define [DM35418_OFFSET_DAC_CHAN_MARKER_ENABLE](#) 0x0b
Offset to the Channel marker Interrupt Enable register, from the start of the DAC channel control section.
- #define [DM35418_OFFSET_DAC_CHAN_LAST_CONVERSION](#) 0x10
Offset to the Channel Last Conversion register, from the start of the DAC channel control section.
- #define [DM35418_OFFSET_DAC_FIFO_CTRL_BLK_START](#) 0x84
Offset to the start of the DAC FIFO control section, from the start of the DAC control section.
- #define [DM35418_OFFSET_DAC_FIFO_CTRL_BLK_SIZE](#) 0x4
Constant size of FIFO control section in function block.
- #define [DM35418_OFFSET_DIO_INPUT_VAL](#) 0x00
Offset to the Input Value register, from the start of the DIO control section.
- #define [DM35418_OFFSET_DIO_OUTPUT_VAL](#) 0x04
Offset to the Output Value register, from the start of the DIO control section.
- #define [DM35418_OFFSET_DIO_DIRECTION](#) 0x08
Offset to the Direction register, from the start of the DIO control section.
- #define [DM35418_OFFSET_TEMPERATURE](#) 0x00
Offset to the Temperature register, from the start of the Temperature control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_SEL](#) 0x00
Offset to the Clock Select register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_PLL_LOCKED](#) 0x01
Offset to the PLL Locked register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_TERM_ENABLE](#) 0x02
Offset to the Termination Enable register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_ENABLE](#) 0x03
Offset to the Enable register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_0_SRC](#) 0x04
Offset to the Line 0 Source register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_1_SRC](#) 0x05
Offset to the Line 1 Source register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_2_SRC](#) 0x06
Offset to the Line 2 Source register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_DIRECTION](#) 0x07
Offset to the Direction register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_BUS2](#) 0x0a
Offset to the Clock Bus 2 register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_BUS3](#) 0x0b
Offset to the Clock Bus 3 register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_BUS4](#) 0x0c
Offset to the Clock Bus 4 register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_BUS5](#) 0x0d
Offset to the Clock Bus 5 register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_BUS6](#) 0x0e
Offset to the Clock Bus 6 register, from the start of the SyncBus control section.
- #define [DM35418_OFFSET_SYNCBUS_CLK_BUS7](#) 0x0f
Offset to the Clock Bus 7 register, from the start of the SyncBus control section.

4.18.1 Detailed Description

4.18.2 Macro Definition Documentation

4.18.2.1 DM35418_OFFSET_FB_ADC_FIFO

```
#define DM35418_OFFSET_FB_ADC_FIFO 0x0334
```

Offset to the FIFO for non-DMA read and write operations.

Note

This value should be used directly. It is used in conjunction with a channel number.

Definition at line 495 of file dm35418_registers.h.

4.19 DM35418 SyncBus Library Constants

Macros

- #define [DM35418_SYNCBUS_CLK_BUS_SRC_DISABLE](#) 0x00
Register value for Clock Event - Disable.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0](#) 0x80
Register value for Clock Event - SyncBus 0.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_1](#) 0x81
Register value for Clock Event - SyncBus 1.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2](#) 0x82
Register value for Clock Event - Syncbus 2.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_IMMEDIATE](#) 0x00
Register value for Clock Source - Immediate.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_NEVER](#) 0x01
Register value for Clock Source - Never.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL2](#) 0x02
Register value for Clock Source - Global 2.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL3](#) 0x03
Register value for Clock Source - Global 3.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL4](#) 0x04
Register value for Clock Source - Global 4.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL5](#) 0x05
Register value for Clock Source - Global 5.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL6](#) 0x06
Register value for Clock Source - Global 6.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL7](#) 0x07
Register value for Clock Source - Global 7.
- #define [DM35418_SYNCBUS_CLK_BUS_SRC_GBL2_INV](#) 0x0a
Register value for Clock Source - Global 2 Inverted.

- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL3_INV 0x0b`
Register value for Clock Source - Global 3 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL4_INV 0x0c`
Register value for Clock Source - Global 4 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL5_INV 0x0d`
Register value for Clock Source - Global 5 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL6_INV 0x0e`
Register value for Clock Source - Global 6 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL7_INV 0x0f`
Register value for Clock Source - Global 7 Inverted.
- `#define DM35418_SYNCBUS_0_CLK_SRC 0x00`
Value for selecting SyncBus 0 Clock Source.
- `#define DM35418_SYNCBUS_1_CLK_SRC 0x01`
Value for selecting SyncBus 1 Clock Source.
- `#define DM35418_SYNCBUS_2_CLK_SRC 0x02`
Value for selecting SyncBus 2 Clock Source.
- `#define DM35418_SYNCBUS_DIR_OUTPUT 1`
Value for setting syncbus line to output.
- `#define DM35418_SYNCBUS_DIR_INPUT 0`
Value for setting syncbus line to input.
- `#define DM35418_SYNCBUS_CLK_SEL_LOCAL 0`
Value for setting the clock select to local clock.
- `#define DM35418_SYNCBUS_CLK_SEL_SYNC 1`
Value for setting the clock select to syncbus clock.

4.19.1 Detailed Description

4.20 DM35418 Sync Bus Library Public Functions

Functions

- `DM35418LIB_API int DM35418_Syncbus_Open` (struct `DM35418_Board_Descriptor` *handle, unsigned int number, struct `DM35418_Function_Block` *func_block)
Open the Syncbus functional block, making it available for operations.
- `DM35418LIB_API int DM35418_Syncbus_Enable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Enable the Syncbus drivers.
- `DM35418LIB_API int DM35418_Syncbus_Disable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Disable the Syncbus drivers.
- `DM35418LIB_API int DM35418_Syncbus_Termination_Enable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Enable the 100 Ohm differential termination on the Syncbus.
- `DM35418LIB_API int DM35418_Syncbus_Termination_Disable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Disable the 100 Ohm differential termination on the Syncbus.
- `DM35418LIB_API int DM35418_Syncbus_Set_Direction` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, int dir_0, int dir_1, int dir_2, int dir_3)

Set the direction of the individual Syncbus lines. There must be only one board that has output direction enabled (the master), the other board should be configured as input (slave). The syncbus0..syncbus2 lines are data (trigger) lines, which can be connected to the internal clock bus. The syncbus3 is the 10MHz clock line. One of the boards must be the master, driving that line, the others are slave, synchronizing their PLL from the Syncbus clock.

- [DM35418LIB_API](#) `int DM35418_Syncbus_Set_Clk_Sel` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clkssel)

Set master clock source of the board. It can be either the local onboard oscillator (clkssel=0) or the syncbus clock line (clkssel=1, provided that the PLL is locked).

- [DM35418LIB_API](#) `int DM35418_Syncbus_Get_Clk_Sel` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int *clkssel)

Get master clock source of the board. It can be either the local onboard oscillator (clkssel=0) or the syncbus clock line (clkssel=1, provided that the PLL is locked).

- [DM35418LIB_API](#) `int DM35418_Syncbus_Get_PLL_Locked` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int *pll_locked)

Get the syncbus clock PLL lock status.

- [DM35418LIB_API](#) `int DM35418_Syncbus_Clock_Source_Global_Set` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clock_select, int clock_source)

Set a syncbus line as a source (input) to one of the internal global clock bus lines. The specified syncbus line must be set as input.

- [DM35418LIB_API](#) `int DM35418_Syncbus_Clock_Source_Global_Get` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clock_select, int *clock_source)

Get the source syncbus line of the specified global clock bus line.

- [DM35418LIB_API](#) `int DM35418_Syncbus_Drive_Source_Set` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int syncbus_select, int clock_source)

Set the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

- [DM35418LIB_API](#) `int DM35418_Syncbus_Drive_Source_Get` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int syncbus_select, int *clock_source)

Get the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

4.20.1 Detailed Description

DM35418_Syncbus_Library_Constants

4.20.2 Function Documentation

4.20.2.1 DM35418_Syncbus_Clock_Source_Global_Get()

```
DM35418LIB_API int DM35418_Syncbus_Clock_Source_Global_Get (
    struct DM35418\_Board\_Descriptor * handle,
    const struct DM35418\_Function\_Block * func_block,
    int clock_select,
    int * clock_source )
```

Get the source syncbus line of the specified global clock bus line.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>clock_select</i>	The identifier of the global clock bus line to be queried. Refer to the board manual for valid values
<i>clock_source</i>	Preallocated integer variable, output the identifier of the syncbus line to be used as source. DM35418_SYNCBUS_CLK_EVENT_DISABLE means the specified global clock bus line is not driven by the yncbus functional block. Refer to the board manual for valid values.

Return values

0	Success.
Non-Zero	Failure.

4.20.2.2 DM35418_Syncbus_Clock_Source_Global_Set()

```
DM35418LIB_API int DM35418_Syncbus_Clock_Source_Global_Set (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    int clock_select,
    int clock_source )
```

Set a syncbus line as a source (input) to one of the internal global clock bus lines. The specified syncbus line must be set as input.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>clock_select</i>	The identifier of the global clock bus line to be used as a target.
<i>clock_source</i>	The identifier syncbus line to be used as a source. DM35418_SYNCBUS_CLK_EVENT_DISABLE means the specified global clock bus is not driven by the syncbus functional block.

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.3 DM35418_Syncbus_Disable()

```
DM35418LIB_API int DM35418_Syncbus_Disable (
```

```
struct DM35418_Board_Descriptor * handle,
const struct DM35418_Function_Block * func_block )
```

Disable the Syncbus drivers.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.20.2.4 DM35418_Syncbus_Drive_Source_Get()

```
DM35418LIB_API int DM35418_Syncbus_Drive_Source_Get (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    int syncbus_select,
    int * clock_source )
```

Get the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>syncbus_select</i>	The identifier of the syncbus line to be queried.
<i>clock_source</i>	Preallocated integer variable, output the identifier of the global clock bus line to be used as source. Refer to the board manual for valid values.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

4.20.2.5 DM35418_Syncbus_Drive_Source_Set()

```
DM35418LIB_API int DM35418_Syncbus_Drive_Source_Set (
    struct DM35418_Board_Descriptor * handle,
```

```
const struct DM35418_Function_Block * func_block,
int syncbus_select,
int clock_source )
```

Set the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>syncbus_select</i>	The identifier of the syncbus line to be used as target.
<i>clock_source</i>	The identifier global clock bus line to be used as a source. Refer to the board manual for valid values.

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.6 DM35418_Syncbus_Enable()

```
DM35418LIB_API int DM35418_Syncbus_Enable (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Enable the Syncbus drivers.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.

Note

The direction of the driver is decided by DM35418_Syncbus_Set_Output_Enable().

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.7 DM35418_Syncbus_Get_Clk_Sel()

```
DM35418LIB_API int DM35418_Syncbus_Get_Clk_Sel (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    int * clkssel )
```

Get master clock source of the board. It can be either the local onboard oscillator (clkssel=0) or the syncbus clock line (clkssel=1, provided that the PLL is locked).

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>clkssel</i>	Preallocated boolean variable for clk select output. 0 (or false): onboard oscillator 1 (or true): syncbus clock PLL

Return values

0	Success.
Non-Zero	Failure.

4.20.2.8 DM35418_Syncbus_Get_PLL_Locked()

```
DM35418LIB_API int DM35418_Syncbus_Get_PLL_Locked (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    int * pll_locked )
```

Get the syncbus clock PLL lock status.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>pll_locked</i>	Preallocated boolean variable for the PLL lock status 0 (or false): syncbus clock PLL is not locked 1 (or true): syncbus clock PLL is locked

Return values

0	Success.
Non-Zero	Failure.

4.20.2.9 DM35418_Syncbus_Open()

```
DM35418LIB_API int DM35418_Syncbus_Open (
    struct DM35418_Board_Descriptor * handle,
    unsigned int number,
    struct DM35418_Function_Block * func_block )
```

Open the Syncbus functional block, making it available for operations.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>number</i>	Ordinal number of syncbus function block to open
<i>func_block</i>	Pointer to the returned function block descriptor.

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.10 DM35418_Syncbus_Set_Clk_Sel()

```
DM35418LIB_API int DM35418_Syncbus_Set_Clk_Sel (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    int clkssel )
```

Set master clock source of the board. It can be either the local onboard oscillator (clkssel=0) or the syncbus clock line (clkssel=1, provided that the PLL is locked).

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>clkssel</i>	boolean value of clk select. 0 (or false): onboard oscillator 1 (or true): syncbus clock PLL

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.11 DM35418_Syncbus_Set_Direction()

```
DM35418LIB_API int DM35418_Syncbus_Set_Direction (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    int dir_0,
    int dir_1,
    int dir_2,
    int dir_3 )
```

Set the direction of the individual Syncbus lines. There must be only one board that has output direction enabled (the master), the other board should be configured as input (slave). The syncbus0..syncbus2 lines are data (trigger) lines, which can be connected to the internal clock bus. The syncbus3 is the 10MHz clock line. One of the boards must be the master, driving that line, the others are slave, synchronizing their PLL from the Syncbus clock.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.
<i>dir_0</i>	Direction of syncbus 0.
<i>dir_1</i>	Direction of syncbus 1.
<i>dir_2</i>	Direction of syncbus 2.
<i>dir_3</i>	Direction of syncbus 3.

Note

For Direction, 0 == Input, 1 == Output

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.12 DM35418_Syncbus_Termination_Disable()

```
DM35418LIB_API int DM35418_Syncbus_Termination_Disable (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Disable the 100 Ohm differential termination on the Syncbus.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.

Note

The termination must be enabled on both end of the bus, but must not be enabled on the boards in the middle of the bus.

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.20.2.13 DM35418_Syncbus_Termination_Enable()

```
DM35418LIB_API int DM35418_Syncbus_Termination_Enable (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block )
```

Enable the 100 Ohm differential termination on the Syncbus.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>func_block</i>	Pointer to the open function block descriptor.

Note

The termination must be enabled on both end of the bus, but must not be enabled on the boards in the middle of the bus.

Return values

0	Success.
Non-Zero	Failure.

Referenced by setup_syncbus().

4.21 DM35418 Temperature**Functions**

- **DM35418LIB_API int DM35418_Temperature_Open** (struct [DM35418_Board_Descriptor](#) *handle, unsigned int ordinal_to_open, struct [DM35418_Function_Block](#) *fb_temp)
Open the temperature function block, getting address values that will be used later by other library functions.
- **DM35418LIB_API int DM35418_Temperature_Read** (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *temp_fb, float *temperature)
Read the temperature of the board, in Celsius degrees.

4.21.1 Detailed Description

Public Library Functions

4.21.2 Function Documentation

4.21.2.1 DM35418_Temperature_Open()

```
DM35418LIB_API int DM35418_Temperature_Open (
    struct DM35418_Board_Descriptor * handle,
    unsigned int ordinal_to_open,
    struct DM35418_Function_Block * fb_temp )
```

Open the temperature function block, getting address values that will be used later by other library functions.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>ordinal_to_open</i>	Which function block on the board to open (0th, 1st, 2nd, etc)
<i>fb_temp</i>	Pointer to function block structure that will hold register offset values.

Return values

<i>0</i>	Success.
<i>Non-Zero</i>	Failure.

Referenced by main().

4.21.2.2 DM35418_Temperature_Read()

```
DM35418LIB_API int DM35418_Temperature_Read (
    struct DM35418_Board_Descriptor * handle,
    struct DM35418_Function_Block * temp_fb,
    float * temperature )
```

Read the temperature of the board, in Celsius degrees.

Parameters

<i>handle</i>	Address of the handle pointer, which will contain the device descriptor.
<i>temp_fb</i>	Pointer to function block we want to read.
<i>temperature</i>	Pointer to a preallocated float. On output holds the temperature in Celsius degree.

Return values

0	Success.
Non-Zero	Failure.

Referenced by main().

4.22 DM35418 Board Types

Macros

- #define [DM35418_SUBTYPE_00](#) 0
Constant for FB subtype 0.
- #define [DM35418_SUBTYPE_01](#) 1
Constant for FB subtype 1.
- #define [DM35418_SUBTYPE_02](#) 2
Constant for FB subtype 2.
- #define [DM35418_SUBTYPE_03](#) 3
Constant for FB subtype 3.
- #define [DM35418_SUBTYPE_INVALID](#) 0xFF
Constant value indicating an invalid subtype.
- #define [DM35418_FUNC_BLOCK_INVALID](#) 0x0000
Constant value indicating an invalid function block.
- #define [DM35418_FUNC_BLOCK_INVALID2](#) 0xFFFF
Constant value indicating an invalid function block.
- #define [DM35418_FUNC_BLOCK_SYNCBUS](#) 0x0001
Function Block Constant for SyncBus.
- #define [DM35418_FUNC_BLOCK_EXT_CLOCKING](#) 0x0002
Function Block Constant for Global Clocking.
- #define [DM35418_FUNC_BLOCK_CLK0003](#) 0x0003
Function Block Constant for External Clocking (0003)
- #define [DM35418_FUNC_BLOCK_CAPTWIN](#) 0x0005
Function Block Constant for Capture Window.
- #define [DM35418_FUNC_BLOCK_ADC](#) 0x1000
Function Block Constant for ADC.
- #define [DM35418_FUNC_BLOCK_ADC1001](#) 0x1001
Function Block Constant for 10 MHz ADC (1001)
- #define [DM35418_FUNC_BLOCK_DAC](#) 0x2000
Function Block Constant for DAC.
- #define [DM35418_FUNC_BLOCK_DAC2001](#) 0x2001
Function Block Constant for High Speed DAC (2001)
- #define [DM35418_FUNC_BLOCK_DIO](#) 0x3000
Function Block Constant for DIO.
- #define [DM35418_FUNC_BLOCK_ADIO](#) 0x3001
Function Block Constant for ADIO.
- #define [DM35418_FUNC_BLOCK_ADIO3010](#) 0x3010
Function Block Constant for ADIO3010.
- #define [DM35418_FUNC_BLOCK_USART](#) 0x4000

- Function Block Constant for Synchronous/Asynchronous Serial Port.*

 - #define `DM35418_FUNC_BLOCK_REF_ADJUST` 0xF000
- Function Block Constant for Reference Adjustment.*

 - #define `DM35418_FUNC_BLOCK_TEMPERATURE_SENSOR` 0xF001
- Function Block Constant for Temperature Sensor.*

 - #define `DM35418_FUNC_BLOCK_FLASH_PROGRAMMER` 0xF002
- Function Block Constant for Module Firmware Programmer.*

 - #define `DM35418_FUNC_BLOCK_CLK_GEN` 0xF003
- Function Block Constant for Clock Generator.*

 - #define `DM35418_FUNC_BLOCK_DIN3011` 0x3011
- Function Block Constant for Digital Input (3011)*

 - #define `DM35418_FUNC_BLOCK_DOT3012` 0x3012
- Function Block Constant for Digital Output (3012)*

 - #define `DM35418_FUNC_BLOCK_INC3200` 0x3200
- Function Block Constant for Incremental Encoder (3200)*

 - #define `DM35418_FUNC_BLOCK_PWM3100` 0x3100
- Function Block Constant for PWM (3100)*

 - #define `DM35418_FUNC_BLOCK_CLK0004` 0x0004
- Function Block Constant for Programmable Clock (0004)*

 - #define `DM35418_FUNC_BLOCK_I2C3300` 0x3300
- Function Block Constant for I2C Bus Master (3300)*

 - #define `DM35418_MAX_FB` 62
- Maximum possible number of function blocks on a board.*

 - #define `MAX_DMA_BUFFERS` 16
- Maximum possible number of DMA buffers for any function block.*

 - #define `MAX_DMA_CHANNELS` 32
- Maximum possible number of DMA channels for any function block.*

 - #define `DM35418_DMA_MAX_BUFFER_SIZE` 0xFFFFFC
- Maximum possible DMA buffer size.*

 - #define `DM35418_BOARD_ACK_INTERRUPT` 0x1
- Value to write to the EOI register to acknowledge interrupts.*

 - #define `DM35418_BOARD_RESET_VALUE` 0xAA
- Value to write to the Reset register in order to reset the board.*

 - #define `DM35418_FIFO_ACCESS_FB_REVISION` 0x01
- Minimum function block revision that supports direct FIFO read/write access.*

4.22.1 Detailed Description

4.23 DM35418 Utility Library Functions

Enumerations

- enum `DM35418_Waveforms` { `DM35418_SINE_WAVE` , `DM35418_SQUARE_WAVE` , `DM35418_SAWTOOTH_WAVE` }
- List of possible waveforms that can be generated for DAC purposes.*

Functions

- uint32_t [DM35418_Get_Maskable](#) (uint16_t data, uint16_t mask)
Return a 32-bit maskable register value from the data and mask.
- void [DM35418_Micro_Sleep](#) (unsigned long microsecs)
Sleep for a specified number of microseconds.
- long [DM35418_Get_Time_Diff](#) (struct timeval last, struct timeval first)
Calculate the time difference between the two timeval structs, in microseconds.
- int [DM35418_Generate_Signal_Data](#) (enum [DM35418_Waveforms](#) waveform, int32_t *data, uint32_t data_count, int32_t max, int32_t minimum, int32_t offset, uint32_t mask)
Generate data with a specific wave pattern. This is useful for producing recognizable waves for DAC output.
- void [DM35418_Check_Result](#) (int return_val, char *message)
Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

4.23.1 Detailed Description

4.23.2 Enumeration Type Documentation

4.23.2.1 DM35418_Waveforms

```
enum DM35418\_Waveforms
```

List of possible waveforms that can be generated for DAC purposes.

Enumerator

DM35418_SINE_WAVE	A simple sine wave.
DM35418_SQUARE_WAVE	A square wave starting at max value
DM35418_SAWTOOTH_WAVE	A sawtooth wave going for min to max

Definition at line 46 of file dm35418_util_library.h.

4.23.3 Function Documentation

4.23.3.1 DM35418_Check_Result()

```
void DM35418\_Check\_Result (
    int return_val,
    char * message )
```

Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

Parameters

<i>return_val</i>	Value to be evaluated. Non-zero values will be considered an error.
<i>message</i>	Pointer to string that will be output if an error condition exists.

Return values

<i>None</i>	
-------------	--

Referenced by ISR(), main(), output_channel_status(), setup_adc(), setup_adcs(), setup_isr(), setup_slave_adcs(), setup_synbus(), and start_slave_adcs().

4.23.3.2 DM35418_Generate_Signal_Data()

```
int DM35418_Generate_Signal_Data (
    enum DM35418_Waveforms waveform,
    int32_t * data,
    uint32_t data_count,
    int32_t max,
    int32_t minimum,
    int32_t offset,
    uint32_t mask )
```

Generate data with a specific wave pattern. This is useful for producing recognizeable waves for DAC output.

Parameters

<i>waveform</i>	Enumerated value indicating what waveform to produce.
<i>data</i>	Pointer to pre-allocated memory to hold resulting data values.
<i>data_count</i>	Number of data samples to produce
<i>max</i>	The maximum value in this generated data.
<i>minimum</i>	The minimum value in this generated data.
<i>offset</i>	Offset from 0 that will be the median value of this wave.
<i>mask</i>	Bitmask that is applied to every calculated value. This allows for handling of generated data that is less than 32 bits. To use all 32-bits, the mask would be 0xFFFFFFFF.

Note

No matter the data count, the returned data will only contain 1 period of the waveform. A higher data count will result in a "finer" set of data.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure

4.23.3.3 DM35418_Get_Maskable()

```
uint32_t DM35418_Get_Maskable (
    uint16_t data,
    uint16_t mask )
```

Return a 32-bit maskable register value from the data and mask.

Parameters

<i>data</i>	Data portion (upper 16-bits) of the maskable.
<i>mask</i>	Mask portion (lower 16-bits) of the maskable

Return values

<i>maskable</i>	Maskable register value.
-----------------	--------------------------

4.23.3.4 DM35418_Get_Time_Diff()

```
long DM35418_Get_Time_Diff (
    struct timeval last,
    struct timeval first )
```

Calculate the time difference between the two timeval structs, in microseconds.

Parameters

<i>last</i>	The last (most recent) timeval to compare.
<i>first</i>	The first (least recent) timeval to compare.

Return values

<i>difference</i>	The difference between the two timevals, in microseconds.
-------------------	---

4.23.3.5 DM35418_Micro_Sleep()

```
void DM35418_Micro_Sleep (
    unsigned long microsecs )
```

Sleep for a specified number of microseconds.

Parameters

<i>microsecs</i>	Length of sleep (microseconds)
------------------	--------------------------------

Return values

<i>None</i>	
-------------	--

Referenced by `main()`.

Chapter 5

Data Structure Documentation

5.1 DM35418_Board_Descriptor Struct Reference

DM35418 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

```
#include <dm35418_os.h>
```

Data Fields

- int [file_descriptor](#)
- void(* [isr](#))()
- pthread_t [pid](#)

5.1.1 Detailed Description

DM35418 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Definition at line 50 of file dm35418_os.h.

5.1.2 Field Documentation

5.1.2.1 file_descriptor

```
int file_descriptor
```

File descriptor for device returned from open()

Definition at line 55 of file dm35418_os.h.

5.1.2.2 isr

```
void(* isr) ()
```

Function pointer to the user ISR callback function.

Definition at line 60 of file dm35418_os.h.

5.1.2.3 pid

```
pthread_t pid
```

Process ID of the child process which will monitor DMA done interrupts.

Definition at line 65 of file dm35418_os.h.

The documentation for this struct was generated from the following file:

- [include/dm35418_os.h](#)

5.2 dm35418_device_descriptor Struct Reference

DM35418 Device Descriptor. The identifying info for this particular board.

```
#include <dm35418_driver.h>
```

Data Fields

- char [name](#) [[DM35418_NAME_LENGTH](#)]
- int [device_index](#)
- struct [dm35418_pci_region](#) [pci](#) [[PCI_ROM_RESOURCE](#)]
- struct pci_dev * [pdev](#)
- spinlock_t [device_lock](#)
- struct cdev [cdev](#)
- uint8_t [reference_count](#)
- unsigned int [irq_number](#)
- uint8_t [remove_isr_flag](#)
- wait_queue_head_t [int_wait_queue](#)
- wait_queue_head_t [dma_wait_queue](#)
- int [interrupt_fb](#) [[DM35418_INT_QUEUE_SIZE](#)]
- unsigned int [int_queue_missed](#)
- unsigned int [int_queue_count](#)
- unsigned int [int_queue_in_marker](#)
- unsigned int [int_queue_out_marker](#)
- struct list_head [dma_descr_list](#)

5.2.1 Detailed Description

DM35418 Device Descriptor. The identifying info for this particular board.

Definition at line 222 of file dm35418_driver.h.

5.2.2 Field Documentation

5.2.2.1 cdev

```
struct cdev cdev
```

Character device

Definition at line 251 of file dm35418_driver.h.

5.2.2.2 device_index

```
int device_index
```

Device index

Definition at line 234 of file dm35418_driver.h.

5.2.2.3 device_lock

```
spinlock_t device_lock
```

Concurrency control

Definition at line 251 of file dm35418_driver.h.

5.2.2.4 dma_descr_list

```
struct list_head dma_descr_list
```

A list of all allocated DMA buffers

Definition at line 319 of file dm35418_driver.h.

5.2.2.5 dma_wait_queue

```
wait_queue_head_t dma_wait_queue
```

Queue of processes waiting to be woken up when an interrupt occurs

Definition at line 288 of file dm35418_driver.h.

5.2.2.6 int_queue_count

```
unsigned int int_queue_count
```

Number of interrupts currently in the queue

Definition at line 307 of file dm35418_driver.h.

5.2.2.7 int_queue_in_marker

```
unsigned int int_queue_in_marker
```

Where in the queue new entries are put

Definition at line 313 of file dm35418_driver.h.

5.2.2.8 int_queue_missed

```
unsigned int int_queue_missed
```

Number of interrupts missed because of a full queue

Definition at line 301 of file dm35418_driver.h.

5.2.2.9 int_queue_out_marker

```
unsigned int int_queue_out_marker
```

Where in the queue entries are pulled from

Definition at line 319 of file dm35418_driver.h.

5.2.2.10 int_wait_queue

```
wait_queue_head_t int_wait_queue
```

Queue of processes waiting to be woken up when an interrupt occurs

Definition at line 282 of file dm35418_driver.h.

5.2.2.11 interrupt_fb

```
int interrupt_fb[DM35418_INT_QUEUE_SIZE]
```

Interrupt queue containing which functional blocks caused interrupts

Definition at line 294 of file dm35418_driver.h.

5.2.2.12 irq_number

```
unsigned int irq_number
```

IRQ line number

Definition at line 269 of file dm35418_driver.h.

5.2.2.13 name

```
char name[DM35418_NAME_LENGTH]
```

Device name used when requesting resources; a NUL terminated string of the form rtd-dm35418-x where x is the device minor number.

Definition at line 229 of file dm35418_driver.h.

5.2.2.14 pci

```
struct dm35418_pci_region pci[PCI_ROM_RESOURCE]
```

Information about each of the standard PCI regions

Definition at line 234 of file dm35418_driver.h.

5.2.2.15 pdev

```
struct pci_dev* pdev
```

PCI device pointer

Definition at line 245 of file dm35418_driver.h.

5.2.2.16 reference_count

```
uint8_t reference_count
```

Number of entities which have the device file open. Used to enforce single open semantics.

Definition at line 263 of file dm35418_driver.h.

5.2.2.17 remove_isr_flag

```
uint8_t remove_isr_flag
```

Used to assist poll in shutting down the thread waiting for interrupts

Definition at line 276 of file dm35418_driver.h.

The documentation for this struct was generated from the following file:

- [include/dm35418_driver.h](#)

5.3 DM35418_DMA_Descriptor Struct Reference

Descriptor for the DMA on this board.

```
#include <dm35418_board_access.h>
```

Data Fields

- uint32_t [control_offset](#)
- uint8_t [num_buffers](#)
- uint32_t [buffer_start_offset](#) [[MAX_DMA_BUFFERS](#)]

5.3.1 Detailed Description

Descriptor for the DMA on this board.

Definition at line 65 of file dm35418_board_access.h.

5.3.2 Field Documentation

5.3.2.1 buffer_start_offset

```
uint32_t buffer_start_offset [MAX_DMA_BUFFERS]
```

Offset to the beginning of the buffer control section.

Definition at line 80 of file dm35418_board_access.h.

5.3.2.2 control_offset

```
uint32_t control_offset
```

Offset to the DMA control register section

Definition at line 70 of file dm35418_board_access.h.

5.3.2.3 num_buffers

```
uint8_t num_buffers
```

Number of buffers for this DMA channel.

Definition at line 75 of file dm35418_board_access.h.

The documentation for this struct was generated from the following file:

- include/dm35418_board_access.h

5.4 dm35418_dma_descriptor Struct Reference

DM35418 DMA descriptor. This structure holds information about a single DMA buffer.

```
#include <dm35418_driver.h>
```

Data Fields

- uint32_t [fb_num](#)
- int [channel](#)
- int [buffer](#)
- void * [virt_addr](#)
- dma_addr_t [bus_addr](#)
- unsigned int [buffer_size](#)
- struct list_head [list](#)

5.4.1 Detailed Description

DM35418 DMA descriptor. This structure holds information about a single DMA buffer.

Definition at line 172 of file dm35418_driver.h.

5.4.2 Field Documentation

5.4.2.1 buffer

```
int buffer
```

DMA buffer number this descriptor represents.

Definition at line 187 of file dm35418_driver.h.

5.4.2.2 buffer_size

```
unsigned int buffer_size
```

Size of this allocated buffer

Definition at line 204 of file dm35418_driver.h.

5.4.2.3 bus_addr

```
dma_addr_t bus_addr
```

Bus memory address for buffer.

Definition at line 198 of file dm35418_driver.h.

5.4.2.4 channel

```
int channel
```

DMA channel this buffer is in.

Definition at line 182 of file dm35418_driver.h.

5.4.2.5 fb_num

```
uint32_t fb_num
```

Function block number this DMA is associated with.

Definition at line 177 of file dm35418_driver.h.

5.4.2.6 list

```
struct list_head list
```

List head so that descriptors can be kept in a linked list.

Definition at line 204 of file dm35418_driver.h.

5.4.2.7 virt_addr

```
void* virt_addr
```

System memory address for buffer

Definition at line 192 of file dm35418_driver.h.

The documentation for this struct was generated from the following file:

- [include/dm35418_driver.h](#)

5.5 DM35418_Function_Block Struct Reference

DM35418 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

```
#include <dm35418_board_access.h>
```

Data Fields

- [uint16_t type](#)
- [uint16_t sub_type](#)
- [uint16_t type_revision](#)
- [uint32_t fb_offset](#)
- [uint32_t dma_offset](#)
- [int fb_num](#)
- [int ordinal_fb_type_num](#)
- [uint8_t num_dma_buffers](#)
- [uint8_t num_dma_channels](#)
- [uint32_t control_offset](#)
- [struct DM35418_DMA_Descriptor dma_channel](#) [[MAX_DMA_CHANNELS](#)]

5.5.1 Detailed Description

DM35418 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Definition at line 93 of file dm35418_board_access.h.

5.5.2 Field Documentation

5.5.2.1 control_offset

```
uint32_t control_offset
```

Offset to the beginning of the control registers for this function block

Definition at line 145 of file dm35418_board_access.h.

5.5.2.2 dma_channel

```
struct DM35418_DMA_Descriptor dma_channel[MAX_DMA_CHANNELS]
```

Array of descriptors for each DMA channel

Definition at line 149 of file dm35418_board_access.h.

5.5.2.3 dma_offset

```
uint32_t dma_offset
```

Offset to the beginning of the DMA registers for this function block

Definition at line 119 of file dm35418_board_access.h.

5.5.2.4 fb_num

```
int fb_num
```

Function block num (as identified in GBC)

Definition at line 124 of file dm35418_board_access.h.

Referenced by ISR(), main(), and output_channel_status().

5.5.2.5 fb_offset

```
uint32_t fb_offset
```

Offset to the beginning of the function block registers

Definition at line 114 of file dm35418_board_access.h.

5.5.2.6 num_dma_buffers

```
uint8_t num_dma_buffers
```

Number of DMA buffers in this function block

Definition at line 135 of file dm35418_board_access.h.

Referenced by ISR(), main(), setup_adc(), and setup_adcs().

5.5.2.7 num_dma_channels

```
uint8_t num_dma_channels
```

Number of DMA channels in this function block

Definition at line 140 of file dm35418_board_access.h.

Referenced by main(), setup_adc(), and setup_adcs().

5.5.2.8 ordinal_fb_type_num

```
int ordinal_fb_type_num
```

The ordinal number of this particular function block type (0th, 1st, etc)

Definition at line 130 of file dm35418_board_access.h.

5.5.2.9 sub_type

```
uint16_t sub_type
```

Type of specific function block (ADC1, ADC2, ADC3, etc)

Definition at line 103 of file dm35418_board_access.h.

Referenced by main().

5.5.2.10 type

`uint16_t type`

Type of function block (ADC, DAC, DIO, etc)

Definition at line 98 of file `dm35418_board_access.h`.

Referenced by `main()`.

5.5.2.11 type_revision

`uint16_t type_revision`

Revision of subtype (internal use only)

Definition at line 109 of file `dm35418_board_access.h`.

The documentation for this struct was generated from the following file:

- [include/dm35418_board_access.h](#)

5.6 dm35418_ioctl_argument Union Reference

`ioctl()` request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the `ioctl()` call.

```
#include <dm35418_board_access_structs.h>
```

Data Fields

- struct [dm35418_ioctl_region_readwrite](#) `readwrite`
- struct [dm35418_ioctl_region_modify](#) `modify`
- struct [dm35418_ioctl_interrupt_info_request](#) `interrupt`
- struct [dm35418_ioctl_dma](#) `dma`

5.6.1 Detailed Description

`ioctl()` request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the `ioctl()` call.

Definition at line 320 of file `dm35418_board_access_structs.h`.

5.6.2 Field Documentation

5.6.2.1 dma

```
struct dm35418_ioctl_dma dma
```

DMA Configuration and Control

Definition at line 309 of file dm35418_board_access_structs.h.

5.6.2.2 interrupt

```
struct dm35418_ioctl_interrupt_info_request interrupt
```

Interrupt request structure

Definition at line 309 of file dm35418_board_access_structs.h.

5.6.2.3 modify

```
struct dm35418_ioctl_region_modify modify
```

PCI region read/modify/write

Definition at line 309 of file dm35418_board_access_structs.h.

5.6.2.4 readwrite

```
struct dm35418_ioctl_region_readwrite readwrite
```

PCI region read and write

Definition at line 309 of file dm35418_board_access_structs.h.

The documentation for this union was generated from the following file:

- [include/dm35418_board_access_structs.h](#)

5.7 dm35418_ioctl_dma Struct Reference

ioctl() request structure for DMA

```
#include <dm35418_board_access_structs.h>
```

Data Fields

- enum [DM35418_DMA_FUNCTIONS](#) function
- int [num_buffers](#)
- uint32_t [buffer_size](#)
- uint32_t [fb_num](#)
- int [channel](#)
- int [buffer](#)
- struct [dm35418_pci_access_request](#) pci
- void * [buffer_ptr](#)

5.7.1 Detailed Description

ioctl() request structure for DMA

Definition at line 269 of file dm35418_board_access_structs.h.

5.7.2 Field Documentation

5.7.2.1 buffer

```
int buffer
```

Buffer in DMA channel that DMA is meant for.

Definition at line 299 of file dm35418_board_access_structs.h.

5.7.2.2 buffer_ptr

```
void* buffer_ptr
```

Pointer to user-space buffer for read or write.

Definition at line 309 of file dm35418_board_access_structs.h.

5.7.2.3 buffer_size

```
uint32_t buffer_size
```

Size (in bytes) to allocate for buffers

Definition at line 284 of file dm35418_board_access_structs.h.

5.7.2.4 channel

```
int channel
```

Channel in function with DMA operation is for.

Definition at line 294 of file dm35418_board_access_structs.h.

5.7.2.5 fb_num

```
uint32_t fb_num
```

Function Block DMA is for.

Definition at line 289 of file dm35418_board_access_structs.h.

5.7.2.6 function

```
enum DM35418_DMA_FUNCTIONS function
```

Requested DMA function to perform.

Definition at line 260 of file dm35418_board_access_structs.h.

5.7.2.7 num_buffers

```
int num_buffers
```

Number of buffers to initialize for DMA

Definition at line 279 of file dm35418_board_access_structs.h.

5.7.2.8 pci

```
struct dm35418_pci_access_request pci
```

PCI Address of DMA registers for this operation

Definition at line 299 of file dm35418_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/[dm35418_board_access_structs.h](#)

5.8 dm35418_ioctl_interrupt_info_request Struct Reference

ioctl() request structure for interrupt

```
#include <dm35418_board_access_structs.h>
```

Data Fields

- int [interrupts_remaining](#)
- int [valid_interrupt](#)
- int [error_occurred](#)
- int [interrupt_fb](#)

5.8.1 Detailed Description

ioctl() request structure for interrupt

Definition at line 238 of file dm35418_board_access_structs.h.

5.8.2 Field Documentation

5.8.2.1 error_occurred

```
int error_occurred
```

Boolean if error occurred during interrupt

Definition at line 254 of file dm35418_board_access_structs.h.

Referenced by ISR().

5.8.2.2 interrupt_fb

```
int interrupt_fb
```

Function block that had interrupt. The MSB indicates if this was a DMA interrupt or not. (0 = Not DMA, 1 = DMA)

Definition at line 260 of file dm35418_board_access_structs.h.

Referenced by ISR().

5.8.2.3 interrupts_remaining

```
int interrupts_remaining
```

Count of interrupts remaining in the driver queue.

Definition at line 244 of file dm35418_board_access_structs.h.

5.8.2.4 valid_interrupt

```
int valid_interrupt
```

Boolean of if interrupt is valid or not.

Definition at line 249 of file dm35418_board_access_structs.h.

Referenced by ISR().

The documentation for this struct was generated from the following file:

- [include/dm35418_board_access_structs.h](#)

5.9 dm35418_ioctl_region_modify Struct Reference

ioctl() request structure for PCI region read/modify/write

```
#include <dm35418_board_access_structs.h>
```

Data Fields

- struct [dm35418_pci_access_request](#) access
- union {
 - uint8_t [mask8](#)
 - uint16_t [mask16](#)
 - uint32_t [mask32](#)
- } [mask](#)

5.9.1 Detailed Description

ioctl() request structure for PCI region read/modify/write

Definition at line 189 of file dm35418_board_access_structs.h.

5.9.2 Field Documentation

5.9.2.1 access

```
struct dm35418_pci_access_request access
```

PCI region access request

Definition at line 164 of file dm35418_board_access_structs.h.

5.9.2.2

```
union { ... } mask
```

Bit mask that controls which bits can be modified. A zero in a bit position means that the corresponding register bit should not be modified. A one in a bit position means that the corresponding register bit should be modified.

Note that it's possible to set bits outside of the mask depending upon the register value before modification. When processing the associated request code, the driver will silently prevent this from happening but will not return an indication that the mask or new value was incorrect.

5.9.2.3 mask16

```
uint16_t mask16
```

Mask for 16-bit operations

Definition at line 220 of file dm35418_board_access_structs.h.

5.9.2.4 mask32

```
uint32_t mask32
```

Mask for 32-bit operations

Definition at line 226 of file dm35418_board_access_structs.h.

5.9.2.5 mask8

```
uint8_t mask8
```

Mask for 8-bit operations

Definition at line 214 of file dm35418_board_access_structs.h.

The documentation for this struct was generated from the following file:

- [include/dm35418_board_access_structs.h](#)

5.10 dm35418_ioctl_region_readwrite Struct Reference

ioctl() request structure for read from or write to PCI region

```
#include <dm35418_board_access_structs.h>
```

Data Fields

- struct [dm35418_pci_access_request](#) access

5.10.1 Detailed Description

ioctl() request structure for read from or write to PCI region

Definition at line 174 of file dm35418_board_access_structs.h.

5.10.2 Field Documentation

5.10.2.1 access

```
struct dm35418\_pci\_access\_request access
```

PCI region access request

Definition at line 164 of file dm35418_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/[dm35418_board_access_structs.h](#)

5.11 dm35418_pci_access_request Struct Reference

PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.

```
#include <dm35418_board_access_structs.h>
```

Data Fields

- enum [dm35418_pci_region_access_size](#) size
- enum [dm35418_pci_region_num](#) region
- uint16_t offset
- union {
 - uint8_t [data8](#)
 - uint16_t [data16](#)
 - uint32_t [data32](#)
- } [data](#)

5.11.1 Detailed Description

PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.

Definition at line 122 of file dm35418_board_access_structs.h.

5.11.2 Field Documentation

5.11.2.1

```
union { ... } data
```

Data to write or the data read

5.11.2.2 data16

```
uint16_t data16
```

16-bit value

Definition at line 158 of file dm35418_board_access_structs.h.

5.11.2.3 data32

```
uint32_t data32
```

32-bit value

Definition at line 164 of file dm35418_board_access_structs.h.

5.11.2.4 data8

```
uint8_t data8
```

8-bit value

Definition at line 152 of file dm35418_board_access_structs.h.

5.11.2.5 offset

```
uint16_t offset
```

Offset within region to access

Definition at line 140 of file dm35418_board_access_structs.h.

5.11.2.6 region

```
enum dm35418_pci_region_num region
```

The PCI region to access

Definition at line 1 of file dm35418_board_access_structs.h.

5.11.2.7 size

```
enum dm35418_pci_region_access_size size
```

Size of access in bits

Definition at line 1 of file dm35418_board_access_structs.h.

The documentation for this struct was generated from the following file:

- include/dm35418_board_access_structs.h

5.12 dm35418_pci_region Struct Reference

DM35418 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.

```
#include <dm35418_driver.h>
```

Data Fields

- unsigned long [io_addr](#)
- unsigned long [length](#)
- unsigned long [phys_addr](#)
- void __iomem * [virt_addr](#)
- uint8_t [allocated](#)

5.12.1 Detailed Description

DM35418 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.

Definition at line 129 of file dm35418_driver.h.

5.12.2 Field Documentation

5.12.2.1 allocated

```
uint8_t allocated
```

Flag indicating whether or not the I/O-mapped memory ranged was allocated. A value of zero means the memory range was not allocated. Any other value means the memory range was allocated.

Definition at line 163 of file dm35418_driver.h.

5.12.2.2 io_addr

```
unsigned long io_addr
```

I/O port number if I/O mapped

Definition at line 135 of file dm35418_driver.h.

5.12.2.3 length

```
unsigned long length
```

Length of region in bytes

Definition at line 141 of file dm35418_driver.h.

5.12.2.4 phys_addr

```
unsigned long phys_addr
```

Region's physical address if memory mapped or I/O port number if I/O mapped

Definition at line 148 of file dm35418_driver.h.

5.12.2.5 virt_addr

```
void __iomem* virt_addr
```

Address at which region is mapped in kernel virtual address space if memory mapped

Definition at line 155 of file dm35418_driver.h.

The documentation for this struct was generated from the following file:

- [include/dm35418_driver.h](#)

Chapter 6

File Documentation

6.1 examples/dm35418_adc.c File Reference

Example program which demonstrates the use of the ADC, setting and responding to interrupts.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35418_gbc_library.h"
#include "dm35418_adc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_dma_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_util_library.h"
#include "dm35418_board_access.h"
#include "dm35418_types.h"
#include "dm35418.h"
```

Macros

- `#define DEFAULT_RATE 6400`
- `#define DAC_RATE 25`
- `#define BUFFER_SIZE_SAMPLES 100`
- `#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))`
- `#define DEFAULT_RANGE DM35418_ADC_RNG_BIPOLAR_5V`

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- void `ISR` (struct `dm35418_ioctl_interrupt_info_request` int_info)
The interrupt subroutine that will execute when an interrupt occurs. It will simply increment a count, which the main program.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- volatile int `interrupt_count` = 0
- volatile int `exit_program` = 0

6.1.1 Detailed Description

Example program which demonstrates the use of the ADC, setting and responding to interrupts.

This example program uses an ADC to collect data. An interrupt is generated every time data is collected by the ADC. After acknowledging the interrupt, the program queries the value last taken by the ADC, and the sample counter, and prints them to the screen.

You can put any signal you want on the ADC input pins. However, for convenience, this example sets up the DACs to provide a signal for the ADC to measure. In order for that to work, you must loopback the DAC outputs to the ADC inputs (DAC0.0 to ADC0.0, DAC1.0 to ADC1.0, etc)

The program will continue to run until CTRL-C is pressed.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

`dm35418_adc.c` 141531 2024-03-06 21:05:25Z lfrankenfield

6.1.2 Macro Definition Documentation

6.1.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 82 of file dm35418_adc.c.

6.1.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to play out DAC pins

Definition at line 77 of file dm35418_adc.c.

6.1.2.3 DAC_RATE

```
#define DAC_RATE 25
```

DAC rate to use. (Hz)

Definition at line 72 of file dm35418_adc.c.

6.1.2.4 DEFAULT_RANGE

```
#define DEFAULT_RANGE DM35418_ADC_RNG_BIPOLAR_5V
```

Define a default range to use, if the user does not provide one.

Definition at line 88 of file dm35418_adc.c.

6.1.2.5 DEFAULT_RATE

```
#define DEFAULT_RATE 6400
```

Rate to run at, if the user does not provide one. (Hz)

Definition at line 67 of file dm35418_adc.c.

6.1.3 Function Documentation

6.1.3.1 main()

```
int main (  
    int argument_count,  
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure.

First, setup the DACS. They will produce a sine wave that needs to be looped back to the ADC inputs.

Definition at line 217 of file dm35418_adc.c.

References board, DM35418_NUM_ADC_ON_BOARD, and my_adc.

6.1.3.2 sigint_handler()

```
static void sigint_handler (  
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 184 of file dm35418_adc.c.

References exit_program.

6.1.4 Variable Documentation**6.1.4.1 exit_program**

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 103 of file dm35418_adc.c.

Referenced by sigint_handler().

6.1.4.2 interrupt_count

```
volatile int interrupt_count = 0
```

Count of interrupts that have happened.

Definition at line 98 of file dm35418_adc.c.

Referenced by ISR().

6.1.4.3 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 93 of file dm35418_adc.c.

Referenced by usage().

6.2 examples/dm35418_adc_basic_ints.c File Reference

Example program which demonstrates the use of the non DMA ADC Interrupts.

```

#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35418_gbc_library.h"
#include "dm35418_adc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_dma_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_util_library.h"
#include "dm35418_board_access.h"
#include "dm35418_types.h"
#include "dm35418.h"

```

Macros

- `#define` [DEFAULT_RATE](#) 1000
- `#define` [TOTAL_INT_SRC_MASK](#)
- `#define` [DAC_RATE](#) 25
- `#define` [BUFFER_SIZE_SAMPLES](#) 100
- `#define` [BUFFER_SIZE_BYTES](#) ([BUFFER_SIZE_SAMPLES](#) * sizeof(int))
- `#define` [DEFAULT_RANGE_DM35418_ADC_RNG_BIPOLAR_5V](#)

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- void `ISR` (struct `dm35418_ioctl_interrupt_info_request` int_info)
The interrupt subroutine that will execute when an interrupt occurs. It will simply increment a count, which the main program.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- static struct `DM35418_Board_Descriptor` * `board`
- static struct `DM35418_Function_Block` `my_adc`
- volatile int `exit_program` = 0
- volatile int `sample_count` = 0

6.2.1 Detailed Description

Example program which demonstrates the use of the non DMA ADC Interrupts.

This program will demonstrate the useage of various non-DMA interrupts and how to handle interrupts in the user ISR. The program initializes the board and ADC0 and waits for the input voltage to exceed one volt. Once that Interrupt occurs, the program fills and outputs the post trigger buffer as samples are taken. The program waits for the Post buffer to trigger an interrupt indicating that it is full and closes the program.

The program counts the number of interrutpts occuring between the trigger and the post buffer filled interrupts. Sometimes it counts the trigger interrupt and/or misses interrupts generated from a very fast sampleing adc. This shows the lag between moving data through kernel space to user space without usign a DMA.

The program will continue to run until the sampling threshold is crossed or until CTRL-C is pressed.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

`dm35418_adc_basic_ints.c` 141531 2024-03-06 21:05:25Z lfrankenfield

6.2.2 Macro Definition Documentation

6.2.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 91 of file dm35418_adc_basic_ints.c.

6.2.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to play out DAC pins

Definition at line 86 of file dm35418_adc_basic_ints.c.

6.2.2.3 DAC_RATE

```
#define DAC_RATE 25
```

DAC rate to use. (Hz)

Definition at line 81 of file dm35418_adc_basic_ints.c.

6.2.2.4 DEFAULT_RANGE

```
#define DEFAULT_RANGE DM35418_ADC_RNG_BIPOLAR_5V
```

Define a default range to use, if the user does not provide one.

Definition at line 97 of file dm35418_adc_basic_ints.c.

6.2.2.5 DEFAULT_RATE

```
#define DEFAULT_RATE 1000
```

Rate to run at, if the user does not provide one. (Hz)

Definition at line 68 of file dm35418_adc_basic_ints.c.

6.2.2.6 TOTAL_INT_SRC_MASK

```
#define TOTAL_INT_SRC_MASK
```

Value:

```
DM35418_ADC_INT_SAMPLE_TAKEN_MASK | \
DM35418_ADC_INT_POST_BUFF_FULL_MASK | \
DM35418_ADC_INT_START_TRIG_MASK
```

Total list of interrupt sources used

Definition at line 73 of file dm35418_adc_basic_ints.c.

6.2.3 Function Documentation

6.2.3.1 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

0	Success
Non-Zero	Failure.

Must have a pre trigger sample count > 0 in order to trigger a threshold trigger or interrupt otherwise it is impossible to detect a sample higher than a threshold

Definition at line 258 of file dm35418_adc_basic_ints.c.

References board, CHANNEL_0, DEFAULT_RANGE, DEFAULT_RATE, DM35418_ADC_CHAN_INTR_HIGH↵
_THRESHOLD_MASK, DM35418_Adc_Channel_Get_Thresholds(), DM35418_Adc_Channel_Interrupt_Get↵
_Config(), DM35418_Adc_Channel_Interrupt_Set_Config(), DM35418_Adc_Channel_Set_High_Threshold(),
DM35418_Adc_Channel_Setup(), DM35418_Adc_Get_Mode_Status(), DM35418_Adc_Initialize(), DM35418↵
_ADC_INPUT_SINGLE_ENDED, DM35418_Adc_Interrupt_Get_Config(), DM35418_Adc_Interrupt_Set_↵
Config(), DM35418_Adc_Open(), DM35418_ADC_RNG_BIPOLAR_10V, DM35418_ADC_RNG_BIPOLAR↵
_1_25V, DM35418_ADC_RNG_BIPOLAR_2_5V, DM35418_ADC_RNG_BIPOLAR_5V, DM35418_ADC_↵
RNG_BIPOLAR_625mV, DM35418_ADC_RNG_UNIPOLAR_10V, DM35418_ADC_RNG_UNIPOLAR_1_25V,
DM35418_ADC_RNG_UNIPOLAR_2_5V, DM35418_ADC_RNG_UNIPOLAR_5V, DM35418_Adc_Set_Clock↵
_Src(), DM35418_Adc_Set_Post_Stop_Samples(), DM35418_Adc_Set_Pre_Trigger_Samples(), DM35418_Adc↵
_Set_Sample_Rate(), DM35418_Adc_Set_Start_Trigger(), DM35418_Adc_Set_Stop_Trigger(), DM35418_Adc_↵
Start(), DM35418_ADC_STAT_WAITING_START_TRIG, DM35418_Adc_Volts_To_Sample(), DM35418_Board_↵
Open(), DM35418_Check_Result(), DM35418_CLK_SRC_CHAN_THRESH, DM35418_CLK_SRC_IMMEDIATE,

DM35418_Gbc_Board_Reset(), DM35418_General_InstallISR(), DM35418_Micro_Sleep(), exit_program, HELP↔_OPTION, INTERRUPT_ENABLE, ISR(), MINOR_OPTION, my_adc, DM35418_Function_Block::num_dma↔_buffers, DM35418_Function_Block::num_dma_channels, program_name, RANGE_OPTION, RATE_OPTION, sample_count, sigint_handler(), TOTAL_INT_SRC_MASK, and usage().

6.2.3.2 sigint_handler()

```
static void sigint_handler (
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 225 of file dm35418_adc_basic_ints.c.

References exit_program.

Referenced by main().

6.2.4 Variable Documentation

6.2.4.1 board

```
struct DM35418_Board_Descriptor* board [static]
```

Board Structure that holds the information of specific board config

Definition at line 106 of file dm35418_adc_basic_ints.c.

Referenced by ISR(), main(), and setup_syncbus().

6.2.4.2 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 116 of file dm35418_adc_basic_ints.c.

Referenced by main(), and sigint_handler().

6.2.4.3 my_adc

```
struct DM35418_Function_Block my_adc [static]
```

ADC structure that is used to hold data used by the ISR and user program

Definition at line 106 of file dm35418_adc_basic_ints.c.

Referenced by ISR(), and main().

6.2.4.4 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 102 of file dm35418_adc_basic_ints.c.

Referenced by main(), and usage().

6.2.4.5 sample_count

```
volatile int sample_count = 0
```

Int that tries to count the number of interrupts occurring.

Definition at line 120 of file dm35418_adc_basic_ints.c.

Referenced by main().

6.3 examples/dm35418_adc_continuous_dma.c File Reference

Example program which demonstrates the use of the ADC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include "dm35418_gbc_library.h"
#include "dm35418_adc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_dma_library.h"
#include "dm35418.h"
#include "dm35418_util_library.h"
```

Macros

- #define `DEFAULT_RATE` 10000
- #define `DEFAULT_NUM_ADC` 4
- #define `BUFFER_SIZE_SAMPLES` 1000000
- #define `BUFFER_SIZE_BYTES` (`BUFFER_SIZE_SAMPLES` * `sizeof(int)`)
- #define `DAC_RATE` (`BUFFER_SIZE_SAMPLES`)
- #define `ASCII_FILE_NAME` `"./adc_dma.txt"`
- #define `BIN_FILE_NAME` `"./adc_dma.bin"`

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void `output_channel_status` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel)
Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void `ISR` (struct `dm35418_ioctl_interrupt_info_request` int_info)
The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void `setup_dacs` (struct `DM35418_Function_Block` *my_dac)
Setup the DACs to produce a sine wave as a signal to sample.
- void `setup_adcs` (unsigned int num_adc_to_use)
Setup the ADCs to sample.
- void `setup_slave_adcs` (uint32_t rate, unsigned int num_adc_to_use)
Setup the ADCs that will be triggered (started) off of the first ADC. This causes all of the ADCs to start sampling at the same time.
- void `setup_ctrlc_handler` ()
Handler to detect when user hits Ctrl-C.
- void `convert_bin_to_txt` (unsigned int num_adc, unsigned int samples_in_buff)
Convert a binary data file to ASCII values. The format will be the same as the data file produced without the `-binary` argument. The example program will exit after finishing.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- static int `dma_has_error` = 0
- static struct `DM35418_Board_Descriptor` * `board`
- static struct `DM35418_Function_Block` `my_adc` [`DM35418_NUM_ADC_ON_BOARD`]
- static unsigned long `buffer_count` [`DM35418_NUM_ADC_ON_BOARD`]
- static int ** `local_buffer` [`DM35418_NUM_ADC_ON_BOARD`]
- static volatile int `exit_program` = 0
- static unsigned long `buffer_size_bytes` = 0
- static unsigned int `next_buffer` [`DM35418_NUM_ADC_ON_BOARD`]

6.3.1 Detailed Description

Example program which demonstrates the use of the ADC and DMA.

This example program will collect data from the ADC(s) specified by the user, at the rate specified by the user, and will write the data to a file. It will do this continuously until the user hits CTRL-C (or the filesystem becomes full).

You can put any signal you want on the ADC input pins. However, for convenience, this example sets up the DACs to provide a signal for the ADC to measure. In order for that to work, you must loopback the DAC outputs to the ADC inputs (DAC0.0 to ADC0.0, DAC1.0 to ADC1.0, etc). The DAC signal is a 1 Hz sine wave.

Maximum sustainable throughput is HIGHLY system dependent. Higher sample rates might be achievable through better buffer size selection or use of an operating system with realtime features.

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35418_adc_continuous_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.3.2 Macro Definition Documentation

6.3.2.1 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_dma.txt"
```

Name of file when saving as ASCII

Definition at line 90 of file dm35418_adc_continuous_dma.c.

6.3.2.2 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_dma.bin"
```

Name of file when saving as binary

Definition at line 95 of file dm35418_adc_continuous_dma.c.

6.3.2.3 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 80 of file dm35418_adc_continuous_dma.c.

6.3.2.4 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 1000000
```

Number of samples to play out DAC pins

Definition at line 75 of file dm35418_adc_continuous_dma.c.

6.3.2.5 DAC_RATE

```
#define DAC_RATE (BUFFER_SIZE_SAMPLES)
```

Rate for the DAC

Definition at line 85 of file dm35418_adc_continuous_dma.c.

6.3.2.6 DEFAULT_NUM_ADC

```
#define DEFAULT_NUM_ADC 4
```

Default number of ADC to use, if user does not provide a num

Definition at line 70 of file dm35418_adc_continuous_dma.c.

6.3.2.7 DEFAULT_RATE

```
#define DEFAULT_RATE 10000
```

Default rate to use, if user does not enter one. (Hz)

Definition at line 65 of file dm35418_adc_continuous_dma.c.

6.3.3 Function Documentation

6.3.3.1 convert_bin_to_txt()

```
void convert_bin_to_txt (
    unsigned int num_adc,
    unsigned int samples_in_buff )
```

Convert a binary data file to ASCII values. The format will be the same as the data file produced without the `-binary` argument. The example program will exit after finishing.

Return values

<i>None.</i>	
--------------	--

Definition at line 772 of file dm35418_adc_continuous_dma.c.

References ASCII_FILE_NAME, BIN_FILE_NAME, and DM35418_NUM_ADC_ON_BOARD.

6.3.3.2 ISR()

```
void ISR (
    struct dm35418_ioctl_interrupt_info_request int_info )
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

<i>int_info</i>	A structure containing information about the interrupt.
-----------------	---

Return values

<i>None.</i>	
--------------	--

Definition at line 291 of file dm35418_adc_continuous_dma.c.

References board, buffer_count, buffer_size_bytes, CHANNEL_0, CLEAR_INTERRUPT, DM35418_Check_↵
Result(), DM35418_Dma_Check_Buffer_Used(), DM35418_Dma_Check_For_Error(), DM35418_Dma_Clear_↵
Interrupt(), DM35418_Dma_Read(), DM35418_Dma_Reset_Buffer(), DM35418_Gbc_Ack_Interrupt(), DM35418_↵
_NUM_ADC_ON_BOARD, dma_has_error, exit_program, DM35418_Function_Block::fb_num, dm35418_ioctl_↵
_interrupt_info_request::interrupt_fb, local_buffer, my_adc, next_buffer, NO_CLEAR_INTERRUPT, DM35418_↵
Function_Block::num_dma_buffers, and dm35418_ioctl_interrupt_info_request::valid_interrupt.

6.3.3.3 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

0	Success
Non-Zero	Failure.

First, setup the DACs. They will produce a sine wave that needs to be looped back to the ADC inputs.

Definition at line 870 of file dm35418_adc_continuous_dma.c.

6.3.3.4 output_channel_status()

```
void output_channel_status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

<i>handle</i>	Pointer to the board handle.
<i>func_block</i>	Pointer to the function block containing the DMA channel
<i>channel</i>	The DMA channel we want the status of.

Return values

None	
------	--

Definition at line 240 of file dm35418_adc_continuous_dma.c.

References DM35418_Check_Result(), DM35418_Dma_Status(), and DM35418_Function_Block::fb_num.

6.3.3.5 setup_adcs()

```
void setup_adcs (
    unsigned int num_adc_to_use )
```

Setup the ADCs to sample.

Parameters

<i>num_adc_to_use</i>	The number of ADCs that will be setup for sampling.
-----------------------	---

Return values

None.	
-------	--

Definition at line 570 of file dm35418_adc_continuous_dma.c.

References board, buffer_size_bytes, CHANNEL_0, DM35418_Adc_Channel_Setup(), DM35418_ADC_INPUT_SINGLE_ENDED, DM35418_Adc_Open(), DM35418_ADC_RNG_BIPOLAR_5V, DM35418_Adc_Set_Clock_Src(), DM35418_Check_Result(), DM35418_CLK_SRC_IMMEDIATE, DM35418_DMA_BUFFER_CTRL_INTR, DM35418_DMA_BUFFER_CTRL_LOOP, DM35418_DMA_BUFFER_CTRL_VALID, DM35418_Dma_Buffer_Setup(), DM35418_Dma_Buffer_Status(), DM35418_Dma_Configure_Interrupts(), DM35418_Dma_Initialize(), DM35418_Dma_Setup(), DM35418_DMA_SETUP_DIRECTION_READ, DM35418_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, next_buffer, NOT_IGNORE_USED, DM35418_Function_Block::num_dma_buffers, and DM35418_Function_Block::num_dma_channels.

6.3.3.6 setup_ctrlc_handler()

```
void setup_ctrlc_handler ( )
```

Handler to detect when user hits Ctrl-C.

Return values

None.	
-------	--

Definition at line 745 of file dm35418_adc_continuous_dma.c.

References sigint_handler().

6.3.3.7 setup_dacs()

```
void setup_dacs (
    struct DM35418_Function_Block * my_dac )
```

Setup the DACs to produce a sine wave as a signal to sample.

Parameters

my_dac	Pointer to the DAC function block structure.
--------	--

Return values

None.	
-------	--

Definition at line 421 of file dm35418_adc_continuous_dma.c.

6.3.3.8 setup_slave_adcs()

```
void setup_slave_adcs (
    uint32_t rate,
    unsigned int num_adc_to_use )
```

Setup the ADCs that will be triggered (started) off of the first ADC. This causes all of the ADCs to start sampling at the same time.

Parameters

<i>rate</i>	Sampling rate for all ADCs
<i>num_adc_to_use</i>	Number of ADCs that will be used.

Return values

<i>None.</i>	
--------------	--

Definition at line 689 of file dm35418_adc_continuous_dma.c.

References board, CHANNEL_0, DM35418_Adc_Initialize(), DM35418_Adc_Set_Sample_Rate(), DM35418_Adc_Set_Start_Trigger(), DM35418_Adc_Set_Stop_Trigger(), DM35418_Adc_Start(), DM35418_Check_Result(), DM35418_CLK_SRC_BUS2, DM35418_CLK_SRC_NEVER, DM35418_Dma_Start(), and my_adc.

6.3.3.9 sigint_handler()

```
static void sigint_handler (
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 209 of file dm35418_adc_continuous_dma.c.

References exit_program.

Referenced by setup_ctrlc_handler().

6.3.4 Variable Documentation

6.3.4.1 board

```
struct DM35418_Board_Descriptor* board [static]
```

Pointer to board descriptor

Definition at line 111 of file dm35418_adc_continuous_dma.c.

Referenced by ISR(), setup_adcs(), and setup_slave_adcs().

6.3.4.2 buffer_count

```
unsigned long buffer_count[DM35418_NUM_ADC_ON_BOARD] [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 122 of file dm35418_adc_continuous_dma.c.

Referenced by ISR().

6.3.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0 [static]
```

Size of the buffer allocated, in bytes.

Definition at line 138 of file dm35418_adc_continuous_dma.c.

Referenced by ISR(), and setup_adcs().

6.3.4.4 dma_has_error

```
int dma_has_error = 0 [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 106 of file dm35418_adc_continuous_dma.c.

Referenced by ISR().

6.3.4.5 exit_program

```
volatile int exit_program = 0 [static]
```

Boolean indicating the program should exit.

Definition at line 133 of file dm35418_adc_continuous_dma.c.

Referenced by ISR(), and sigint_handler().

6.3.4.6 local_buffer

```
int** local_buffer[DM35418_NUM_ADC_ON_BOARD] [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 128 of file dm35418_adc_continuous_dma.c.

Referenced by ISR().

6.3.4.7 my_adc

```
struct DM35418_Function_Block my_adc[DM35418_NUM_ADC_ON_BOARD] [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 111 of file dm35418_adc_continuous_dma.c.

Referenced by ISR(), setup_adcs(), and setup_slave_adcs().

6.3.4.8 next_buffer

```
unsigned int next_buffer[DM35418_NUM_ADC_ON_BOARD] [static]
```

Which buffer is next to be copied from DMA

Definition at line 143 of file dm35418_adc_continuous_dma.c.

Referenced by ISR(), and setup_adcs().

6.3.4.9 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 101 of file dm35418_adc_continuous_dma.c.

Referenced by usage().

6.4 examples/dm35418_adc_sync_trig_dma.c File Reference

Example program which demonstrates using the Synctbus input as an external trigger to sample with the ADC.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include <time.h>
#include <sys/time.h>
#include "dm35418_gbc_library.h"
#include "dm35418_adc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_dio_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_dma_library.h"
#include "dm35418.h"
#include "dm35418_util_library.h"
#include "dm35418_synctbus_library.h"
```

Macros

- #define [DEFAULT_RATE](#) 2000
- #define [DAC_RATE](#) 1000
- #define [DAC_BUFF_NUM_SAMPLES](#) 1000
- #define [NUM_SAMPLES_PER_TRIGGER](#) 1000
- #define [SYNCTBUS_TRIGGER_ON_USEC](#) 30
- #define [SYNCTBUS_TRIGGER_OFF_USEC](#) 999970
- #define [DIO_TRIGGER_ON](#) 0x55555555
- #define [DIO_TRIGGER_OFF](#) 0xAAAAAAAA
- #define [DIO_DIR_OUTPUT](#) 0xFFFFFFFF
- #define [DAC_BUFF_SIZE_BYTES](#) (DAC_BUFF_NUM_SAMPLES * sizeof(int))
- #define [ASCII_FILE_NAME](#) "./adc_sync_dma.txt"
- #define [BIN_FILE_NAME](#) "./adc_sync_dma.bin"

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void `output_channel_status` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel)
Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void `ISR` (struct `dm35418_ioctl_interrupt_info_request` int_info)
The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void `setup_dac` (struct `DM35418_Function_Block` *my_dac)
Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.
- void `setup_syncbus` (struct `DM35418_Function_Block` *my_syncbus)
Setup the Syncbus to sense the trigger signal.
- void `setup_adc` (unsigned long samples_to_collect)
Setup ADC 0 to sample. It will start sampling based on the trigger from the syncbus, sample an arbitrary number of samples, and then reset to wait for the next trigger.
- void `setup_dio` (struct `DM35418_Function_Block` *my_dio)
Setup DIO to produce trigger signal.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- static int `dma_has_error` = 0
- static struct `DM35418_Board_Descriptor` * `board`
- static struct `DM35418_Function_Block` `my_adc`
- static unsigned long `buffer_count`
- static int ** `local_buffer`
- static volatile int `exit_program` = 0
- static unsigned long `buffer_size_bytes` = 0
- static unsigned int `next_buffer`

6.4.1 Detailed Description

Example program which demonstrates using the Syncbus input as an external trigger to sample with the ADC.

The Syncbus is not as flexible of an external clock input as other boards that have a dedicated external clocking function block (such as the DM35425), but it can still be used as a trigger for sampling. This example shows how, using only the single board.

The DIO will be used to produce the "external" signal. Once you've verified this works, you can substitute your own signal. Note that the Syncbus input is a differential input and will require a suitable signal to trigger correctly.

The input to the LVDS from the DIO then triggers sampling on ADC0. Once the specified number of samples have been acquired, the ADC0 stops and re-arms, preparing for the next trigger. It will continue doing this until you stop it with Ctrl-C.

To run this example, make these connections:

DIO 0.0 (CN3 Pin 1) to Sync0+ (CN5 Pin 1) with a 100 Ohm resistor
 DIO 0.1 (CN3 Pin 2) to Sync0- (CN5 Pin 2) with a 100 Ohm resistor
 Attach signal of interest to ADC 0.0 (CN4 Pin 1)

The 100 Ohm resistor is to protect the LVDS input from the LVTTTL output of the DIO.

For convenience, a sine wave signal is output on DAC 0.0. This can be connected to ADC 0.0 so that external equipment is not needed for this example.

Samples taken are output to a data file (adc_sync_dma.dat) and can be viewed with gnuplot using the plot_adc_sync_dma file:
 gnuplot -persist plot_adc_sync_dma

Hit Ctrl-C to exit.

 This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35418_adc_sync_trig_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.4.2 Macro Definition Documentation

6.4.2.1 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_sync_dma.txt"
```

Name of file when saving as ASCII

Definition at line 136 of file dm35418_adc_sync_trig_dma.c.

6.4.2.2 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_sync_dma.bin"
```

Name of file when saving as binary

Definition at line 141 of file dm35418_adc_sync_trig_dma.c.

6.4.2.3 DAC_BUFF_NUM_SAMPLES

```
#define DAC_BUFF_NUM_SAMPLES 1000
```

Number of samples to play out DAC pins

Definition at line 96 of file dm35418_adc_sync_trig_dma.c.

6.4.2.4 DAC_BUFF_SIZE_BYTES

```
#define DAC_BUFF_SIZE_BYTES (DAC_BUFF_NUM_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 131 of file dm35418_adc_sync_trig_dma.c.

6.4.2.5 DAC_RATE

```
#define DAC_RATE 1000
```

Rate of the DAC, for its sine wave

Definition at line 91 of file dm35418_adc_sync_trig_dma.c.

6.4.2.6 DEFAULT_RATE

```
#define DEFAULT_RATE 2000
```

Default rate to use (Hz)

Definition at line 86 of file dm35418_adc_sync_trig_dma.c.

6.4.2.7 DIO_DIR_OUTPUT

```
#define DIO_DIR_OUTPUT 0xFFFFFFFF
```

Define DIO direction for pins as Output

Definition at line 126 of file dm35418_adc_sync_trig_dma.c.

6.4.2.8 DIO_TRIGGER_OFF

```
#define DIO_TRIGGER_OFF 0xAAAAAAAA
```

Pattern to put on the DIO pins when trigger is off

Definition at line 121 of file dm35418_adc_sync_trig_dma.c.

6.4.2.9 DIO_TRIGGER_ON

```
#define DIO_TRIGGER_ON 0x55555555
```

Pattern to put on DIO pins when trigger is on

Definition at line 116 of file dm35418_adc_sync_trig_dma.c.

6.4.2.10 NUM_SAMPLES_PER_TRIGGER

```
#define NUM_SAMPLES_PER_TRIGGER 1000
```

Number of samples to take on the ADC when the trigger occurs.

Definition at line 101 of file dm35418_adc_sync_trig_dma.c.

6.4.2.11 SYNCBUS_TRIGGER_OFF_USEC

```
#define SYNCBUS_TRIGGER_OFF_USEC 999970
```

How long the "low" signal should show on the Syncbus input

Definition at line 111 of file dm35418_adc_sync_trig_dma.c.

6.4.2.12 SYNCBUS_TRIGGER_ON_USEC

```
#define SYNCBUS_TRIGGER_ON_USEC 30
```

How long the "high" signal should show on the Syncbus input

Definition at line 106 of file dm35418_adc_sync_trig_dma.c.

6.4.3 Function Documentation

6.4.3.1 ISR()

```
void ISR (
    struct dm35418_ioctl_interrupt_info_request int_info )
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

<i>int_info</i>	A structure containing information about the interrupt.
-----------------	---

Return values

<i>None.</i>	
--------------	--

Definition at line 323 of file dm35418_adc_sync_trig_dma.c.

References board, buffer_count, buffer_size_bytes, CHANNEL_0, CLEAR_INTERRUPT, DM35418_Check_↵ Result(), DM35418_Dma_Check_Buffer_Used(), DM35418_Dma_Check_For_Error(), DM35418_Dma_Clear_↵ _Interrupt(), DM35418_Dma_Read(), DM35418_Dma_Reset_Buffer(), DM35418_Gbc_Ack_Interrupt(), dma_↵ has_error, exit_program, DM35418_Function_Block::fb_num, dm35418_ioctl_interrupt_info_request::interrupt_fb, local_buffer, my_adc, next_buffer, NO_CLEAR_INTERRUPT, DM35418_Function_Block::num_dma_buffers, and dm35418_ioctl_interrupt_info_request::valid_interrupt.

6.4.3.2 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure.

Setup DAC 0 to produce a sine wave. This can be looped to the ADC input to have a signal to measure.

Definition at line 801 of file dm35418_adc_sync_trig_dma.c.

6.4.3.3 output_channel_status()

```
void output_channel_status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

<i>handle</i>	Pointer to the board handle.
<i>func_block</i>	Pointer to the function block containing the DMA channel
<i>channel</i>	The DMA channel we want the status of.

Return values

<i>None</i>	
-------------	--

Definition at line 271 of file dm35418_adc_sync_trig_dma.c.

References DM35418_Check_Result(), DM35418_Dma_Status(), and DM35418_Function_Block::fb_num.

6.4.3.4 setup_adc()

```
void setup_adc (
    unsigned long samples_to_collect )
```

Setup ADC 0 to sample. It will start sampling based on the trigger from the syncbus, sample an arbitrary number of samples, and then reset to wait for the next trigger.

Parameters

<i>samples_to_collect</i>	Number of samples to collect when triggered.
---------------------------	--

Return values

<i>None.</i>	
--------------	--

Definition at line 619 of file dm35418_adc_sync_trig_dma.c.

References ADC_0, board, buffer_size_bytes, CHANNEL_0, DEFAULT_RATE, DM35418_Adc_Channel_↵ Setup(), DM35418_ADC_INPUT_SINGLE_ENDED, DM35418_Adc_Open(), DM35418_ADC_RNG_BIPOLAR_5V, DM35418_Adc_Set_Clock_Src(), DM35418_Adc_Set_Post_Stop_Samples(), DM35418_Adc_Set_Pre_Trigger_↵ _Samples(), DM35418_Adc_Set_Sample_Rate(), DM35418_Adc_Set_Start_Trigger(), DM35418_Adc_Set_↵ Stop_Trigger(), DM35418_Check_Result(), DM35418_CLK_SRC_BUS2, DM35418_CLK_SRC_IMMEDIATE, DM35418_DMA_BUFFER_CTRL_INTR, DM35418_DMA_BUFFER_CTRL_LOOP, DM35418_DMA_BUFFER_↵ _CTRL_VALID, DM35418_Dma_Buffer_Setup(), DM35418_Dma_Buffer_Status(), DM35418_Dma_Configure_↵ Interrupts(), DM35418_Dma_Initialize(), DM35418_Dma_Setup(), DM35418_DMA_SETUP_DIRECTION_READ, DM35418_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, NOT_↵ IGNORE_USED, DM35418_Function_Block::num_dma_buffers, and DM35418_Function_Block::num_dma_↵ channels.

6.4.3.5 setup_dac()

```
void setup_dac (
    struct DM35418_Function_Block * my_dac )
```

Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.

Parameters

<i>my_dac</i>	Pointer to the DAC function block structure
---------------	---

Return values

<i>None.</i>	
--------------	--

Definition at line 432 of file dm35418_adc_sync_trig_dma.c.

6.4.3.6 setup_dio()

```
void setup_dio (
    struct DM35418_Function_Block * my_dio )
```

Setup DIO to produce trigger signal.

Parameters

<i>my_dio</i>	Pointer to DIO function block.
---------------	--------------------------------

Return values

<i>None.</i>	
--------------	--

Definition at line 750 of file dm35418_adc_sync_trig_dma.c.

6.4.3.7 setup_syncbus()

```
void setup_syncbus (
    struct DM35418_Function_Block * my_syncbus )
```

Setup the Syncbus to sense the trigger signal.

Parameters

<i>my_syncbus</i>	Pointer to the syncbus function block structure.
-------------------	--

Return values

<i>None.</i>	
--------------	--

Definition at line 564 of file dm35418_adc_sync_trig_dma.c.

References board, DM35418_Check_Result(), DM35418_CLK_SRC_BUS2, DM35418_SYNCBUS_CLK_BUS↵
_SRC_SYNCBUS_0, DM35418_SYNCBUS_CLK_SEL_SYNC, DM35418_Syncbus_Clock_Source_Global_↵
Set(), DM35418_SYNCBUS_DIR_INPUT, DM35418_SYNCBUS_DIR_OUTPUT, DM35418_Syncbus_Enable(),
DM35418_Syncbus_Open(), DM35418_Syncbus_Set_Clk_Sel(), DM35418_Syncbus_Set_Direction(), and
DM35418_Syncbus_Termination_Enable().

6.4.3.8 sigint_handler()

```
static void sigint_handler (
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 240 of file dm35418_adc_sync_trig_dma.c.

References exit_program.

6.4.4 Variable Documentation

6.4.4.1 board

```
struct DM35418_Board_Descriptor* board [static]
```

Pointer to board descriptor

Definition at line 156 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR(), setup_adc(), and setup_syncbus().

6.4.4.2 buffer_count

```
unsigned long buffer_count [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 167 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR().

6.4.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0 [static]
```

Size of the buffer allocated, in bytes.

Definition at line 183 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR(), and setup_adc().

6.4.4.4 dma_has_error

```
int dma_has_error = 0 [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 151 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR().

6.4.4.5 exit_program

```
volatile int exit_program = 0 [static]
```

Boolean indicating the program should exit.

Definition at line 178 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR(), and sigint_handler().

6.4.4.6 local_buffer

```
int** local_buffer [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 173 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR().

6.4.4.7 my_adc

```
struct DM35418_Function_Block my_adc [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 156 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR(), and setup_adc().

6.4.4.8 next_buffer

```
unsigned int next_buffer [static]
```

Which buffer is next to be copied from DMA

Definition at line 188 of file dm35418_adc_sync_trig_dma.c.

Referenced by ISR().

6.4.4.9 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 146 of file dm35418_adc_sync_trig_dma.c.

Referenced by usage().

6.5 examples/dm35418_adc_thresh_dma.c File Reference

Demonstrates use of threshold values to control when data is collected.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include <time.h>
#include <sys/time.h>
#include "dm35418_gbc_library.h"
#include "dm35418_adc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_dma_library.h"
#include "dm35418.h"
#include "dm35418_util_library.h"
```

Macros

- #define [DEFAULT_RATE](#) 2000
- #define [DAC_RATE](#) 1000
- #define [ADC_THRESH_HIGH_VOLTS](#) 3.0f
- #define [ADC_THRESH_LOW_VOLTS](#) -3.0f
- #define [DAC_BUFF_NUM_SAMPLES](#) 1000
- #define [NUM_SAMPLES_PER_TRIGGER](#) (DAC_BUFF_NUM_SAMPLES * 2)
- #define [DAC_BUFF_SIZE_BYTES](#) (DAC_BUFF_NUM_SAMPLES * sizeof(int))
- #define [ASCII_FILE_NAME](#) "./adc_thresh_dma.txt"
- #define [BIN_FILE_NAME](#) "./adc_thresh_dma.bin"

Functions

- static void [usage](#) (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void [sigint_handler](#) (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void [output_channel_status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)
Output the status of a DMA channel. This is a helper function to determine the cause of an error if it occurs.
- void [ISR](#) (struct [dm35418_ioctl_interrupt_info_request](#) int_info)
The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void [setup_dac](#) (struct [DM35418_Function_Block](#) *my_dac)
Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.
- void [setup_adc](#) ()
Setup ADC 0 to sample. It will start sampling based on the trigger from the threshold crossing.
- int [main](#) (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)
- static int [dma_has_error](#) = 0
- static struct [DM35418_Board_Descriptor](#) * [board](#)
- static struct [DM35418_Function_Block](#) [my_adc](#)
- static unsigned long [buffer_count](#)
- static int ** [local_buffer](#)
- static volatile int [exit_program](#) = 0
- static unsigned long [buffer_size_bytes](#) = 0
- static unsigned int [next_buffer](#)

6.5.1 Detailed Description

Demonstrates use of threshold values to control when data is collected.

This example samples data using the ADC Channel 0 input. It will start collecting data when the threshold is exceeded and then collect 10000 samples. By default, it will start when the high threshold is exceeded, but you can select the low threshold instead with a command line argument.

For convenience, a 5V to -5V sine wave is produced at the DAC Channel 0 output. Simply connect CN4 Pin 33 to CN4 Pin 1 to execute this example without the need for external equipment.

Samples taken are output to a data file ([adc_thresh_dma.dat](#)) and can be viewed with [gnuplot](#) using the [plot_adc_thresh_dma](#) file.

Hit Ctrl-C to exit.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35418_adc_thresh_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.5.2 Macro Definition Documentation

6.5.2.1 ADC_THRESH_HIGH_VOLTS

```
#define ADC_THRESH_HIGH_VOLTS 3.0f
```

Upper voltage threshold

Definition at line 76 of file [dm35418_adc_thresh_dma.c](#).

6.5.2.2 ADC_THRESH_LOW_VOLTS

```
#define ADC_THRESH_LOW_VOLTS -3.0f
```

Lower voltage threshold

Definition at line 81 of file dm35418_adc_thresh_dma.c.

6.5.2.3 ASCII_FILE_NAME

```
#define ASCII_FILE_NAME "./adc_thresh_dma.txt"
```

Name of file when saving as ASCII

Definition at line 101 of file dm35418_adc_thresh_dma.c.

6.5.2.4 BIN_FILE_NAME

```
#define BIN_FILE_NAME "./adc_thresh_dma.bin"
```

Name of file when saving as binary

Definition at line 106 of file dm35418_adc_thresh_dma.c.

6.5.2.5 DAC_BUFF_NUM_SAMPLES

```
#define DAC_BUFF_NUM_SAMPLES 1000
```

Number of samples to play out DAC pins

Definition at line 86 of file dm35418_adc_thresh_dma.c.

6.5.2.6 DAC_BUFF_SIZE_BYTES

```
#define DAC_BUFF_SIZE_BYTES (DAC_BUFF_NUM_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 96 of file dm35418_adc_thresh_dma.c.

6.5.2.7 DAC_RATE

```
#define DAC_RATE 1000
```

Rate of the DAC, for its sine wave

Definition at line 71 of file dm35418_adc_thresh_dma.c.

6.5.2.8 DEFAULT_RATE

```
#define DEFAULT_RATE 2000
```

Default rate to use (Hz)

Definition at line 66 of file dm35418_adc_thresh_dma.c.

6.5.2.9 NUM_SAMPLES_PER_TRIGGER

```
#define NUM_SAMPLES_PER_TRIGGER (DAC_BUFF_NUM_SAMPLES * 2)
```

Number of samples to take on the ADC when the trigger occurs.

Definition at line 91 of file dm35418_adc_thresh_dma.c.

6.5.3 Function Documentation

6.5.3.1 ISR()

```
void ISR (
    struct dm35418_ioctl_interrupt_info_request int_info )
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

<i>int_info</i>	A structure containing information about the interrupt.
-----------------	---

Return values

<i>None.</i>	
--------------	--

Definition at line 285 of file dm35418_adc_thresh_dma.c.

References board, buffer_count, buffer_size_bytes, CHANNEL_0, CLEAR_INTERRUPT, DM35418_Check_↵
Result(), DM35418_Dma_Check_Buffer_Used(), DM35418_Dma_Check_For_Error(), DM35418_Dma_Clear_↵
Interrupt(), DM35418_Dma_Read(), DM35418_Dma_Reset_Buffer(), dma_has_error, exit_program, DM35418_↵
_Function_Block::fb_num, dm35418_ioctl_interrupt_info_request::interrupt_fb, local_buffer, my_adc, next_buffer,
NO_CLEAR_INTERRUPT, DM35418_Function_Block::num_dma_buffers, and dm35418_ioctl_interrupt_info_↵
request::valid_interrupt.

6.5.3.2 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

0	Success
Non-Zero	Failure.

Setup DAC 0 to produce a sine wave. This can be looped to the ADC input to have a signal to measure.

Definition at line 697 of file dm35418_adc_thresh_dma.c.

6.5.3.3 output_channel_status()

```
void output_channel_status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error if it occurs.

Parameters

<i>handle</i>	Pointer to the board handle.
<i>func_block</i>	Pointer to the function block containing the DMA channel
<i>channel</i>	The DMA channel we want the status of.

Return values

<i>None</i>	
-------------	--

Definition at line 233 of file dm35418_adc_thresh_dma.c.

References DM35418_Check_Result(), DM35418_Dma_Status(), and DM35418_Function_Block::fb_num.

6.5.3.4 setup_adc()

```
void setup_adc ( )
```

Setup ADC 0 to sample. It will start sampling based on the trigger from the threshold crossing.

Return values

<i>None.</i>	
--------------	--

Definition at line 519 of file dm35418_adc_thresh_dma.c.

References ADC_0, ADC_THRESH_HIGH_VOLTS, ADC_THRESH_LOW_VOLTS, board, buffer_size_bytes, CHANNEL_0, DEFAULT_RATE, DM35418_Adc_Channel_Set_High_Threshold(), DM35418_Adc_Channel_Set_Low_Threshold(), DM35418_Adc_Channel_Setup(), DM35418_ADC_INPUT_SINGLE_ENDED, DM35418_Adc_Open(), DM35418_ADC_RNG_BIPOLAR_5V, DM35418_Adc_Set_Clock_Src(), DM35418_Adc_Set_Post_Stop_Samples(), DM35418_Adc_Set_Pre_Trigger_Samples(), DM35418_Adc_Set_Sample_Rate(), DM35418_Adc_Set_Start_Trigger(), DM35418_Adc_Set_Stop_Trigger(), DM35418_Adc_Volts_To_Sample(), DM35418_Check_Result(), DM35418_CLK_SRC_CHAN_THRESH, DM35418_CLK_SRC_IMMEDIATE, DM35418_DMA_BUFFER_CTRL_INTR, DM35418_DMA_BUFFER_CTRL_LOOP, DM35418_DMA_BUFFER_CTRL_VALID, DM35418_Dma_Buffer_Setup(), DM35418_Dma_Buffer_Status(), DM35418_Dma_Configure_Interrupts(), DM35418_Dma_Initialize(), DM35418_Dma_Setup(), DM35418_DMA_SETUP_DIRECTION_READ, DM35418_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, NOT_IGNORE_USED, DM35418_Function_Block::num_dma_buffers, DM35418_Function_Block::num_dma_channels, and NUM_SAMPLES_PER_TRIGGER.

6.5.3.5 setup_dac()

```
void setup_dac (
    struct DM35418_Function_Block * my_dac )
```

Setup DAC 0 to provide a sine wave signal. This is purely for convenience as a signal for the ADC to sample.

Parameters

<i>my_dac</i>	Pointer to the DAC function block structure
---------------	---

Return values

<i>None.</i>	
--------------	--

Definition at line 390 of file dm35418_adc_thresh_dma.c.

6.5.3.6 sigint_handler()

```
static void sigint_handler (  
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 202 of file dm35418_adc_thresh_dma.c.

References `exit_program`.

6.5.4 Variable Documentation

6.5.4.1 board

```
struct DM35418_Board_Descriptor* board [static]
```

Pointer to board descriptor

Definition at line 121 of file dm35418_adc_thresh_dma.c.

Referenced by `ISR()`, and `setup_adc()`.

6.5.4.2 buffer_count

```
unsigned long buffer_count [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 132 of file dm35418_adc_thresh_dma.c.

Referenced by `ISR()`.

6.5.4.3 buffer_size_bytes

```
unsigned long buffer_size_bytes = 0 [static]
```

Size of the buffer allocated, in bytes.

Definition at line 148 of file dm35418_adc_thresh_dma.c.

Referenced by ISR(), and setup_adc().

6.5.4.4 dma_has_error

```
int dma_has_error = 0 [static]
```

Boolean flag indicating if there was a DMA error.

Definition at line 116 of file dm35418_adc_thresh_dma.c.

Referenced by ISR().

6.5.4.5 exit_program

```
volatile int exit_program = 0 [static]
```

Boolean indicating the program should exit.

Definition at line 143 of file dm35418_adc_thresh_dma.c.

Referenced by ISR(), and sigint_handler().

6.5.4.6 local_buffer

```
int** local_buffer [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 138 of file dm35418_adc_thresh_dma.c.

Referenced by ISR().

6.5.4.7 my_adc

```
struct DM35418_Function_Block my_adc [static]
```

Pointer to array of function blocks that will hold the ADC descriptors

Definition at line 121 of file dm35418_adc_thresh_dma.c.

Referenced by ISR(), and setup_adc().

6.5.4.8 next_buffer

```
unsigned int next_buffer [static]
```

Which buffer is next to be copied from DMA

Definition at line 153 of file dm35418_adc_thresh_dma.c.

Referenced by ISR().

6.5.4.9 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 111 of file dm35418_adc_thresh_dma.c.

Referenced by usage().

6.6 examples/dm35418_adc_to_dac_dma.c File Reference

Demonstrates transferring data from ADC DMA to DAC DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <limits.h>
#include <getopt.h>
#include "dm35418_gbc_library.h"
#include "dm35418_adc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_dma_library.h"
#include "dm35418.h"
#include "dm35418_util_library.h"
```

Macros

- #define `DEFAULT_RATE` 100000
- #define `NUM_FB_TO_USE` 4
- #define `BUFFER_SIZE_SAMPLES` 5000
- #define `BUFFER_SIZE_BYTES` (`BUFFER_SIZE_SAMPLES` * `sizeof(int)`)

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void `output_channel_status` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel)
Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.
- void `ISR` (struct `dm35418_ioctl_interrupt_info_request` int_info)
The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.
- void `setup_dacs` (uint32_t rate)
Setup the DACs to output continuously via DMA.
- void `setup_adcs` ()
Set the ADCs to continuously sample data into the DMA buffer.
- void `setup_isr` ()
Install the interrupt subroutine.
- void `start_slave_adcs` (uint32_t rate)
Start all of the "slave" ADCs. These started ADCs will wait until the start trigger, which is the starting of ADC0. In this way, all ADCs will start together.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- static int `adc_dma_error` = 0
- static struct `DM35418_Board_Descriptor` * `board`
- static struct `DM35418_Function_Block` `my_adc` [`NUM_FB_TO_USE`]
- static unsigned long `buff_cnt` [`NUM_FB_TO_USE`]
- static int ** `local_buffer` [`NUM_FB_TO_USE`]
- static volatile int `exit_program` = 0
- static unsigned int `next_buffer` [`NUM_FB_TO_USE`]

6.6.1 Detailed Description

Demonstrates transferring data from ADC DMA to DAC DMA.

This example program will collect data from the ADCs at an arbitrary rate and be transferred via DMA. Once a buffer is completed, it will undergo a quick transformation and then be written to the corresponding DAC DMA buffer (ADC0 to DAC0, etc). The DAC will then write the data out at the same arbitrary rate.

In this way, a signal can be fed into ADC0 and it will come out of DAC0. DAC0 can be looped to ADC1 and the result will come out DAC1, etc.

The example will run continuously until the user hits CTRL-C.

Maximum sustainable throughput is system dependent. Higher sample rates might be achievable through better buffer size selection or use of an operating system with realtime features.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35418_adc_to_dac_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.6.2 Macro Definition Documentation

6.6.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Number of bytes in DAC sample buffer

Definition at line 80 of file dm35418_adc_to_dac_dma.c.

6.6.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 5000
```

Number of samples to play out DAC pins

Definition at line 75 of file dm35418_adc_to_dac_dma.c.

6.6.2.3 DEFAULT_RATE

```
#define DEFAULT_RATE 100000
```

Default rate to use, if user does not enter one. (Hz)

Definition at line 65 of file dm35418_adc_to_dac_dma.c.

6.6.2.4 NUM_FB_TO_USE

```
#define NUM_FB_TO_USE 4
```

Default number of ADC to use, if user does not provide a num

Definition at line 70 of file dm35418_adc_to_dac_dma.c.

6.6.3 Function Documentation

6.6.3.1 ISR()

```
void ISR (
    struct dm35418_ioctl_interrupt_info_request int_info )
```

The interrupt subroutine that will execute when a DMA interrupt occurs. This function will read from the DMA, copying data from the kernel buffers to the user buffers so that we can access the data.

Parameters

<i>int_info</i>	A structure containing information about the interrupt.
-----------------	---

Return values

<i>None.</i>	
--------------	--

Definition at line 250 of file dm35418_adc_to_dac_dma.c.

References `adc_dma_error`, `board`, `buff_cnt`, `BUFFER_SIZE_BYTES`, `CHANNEL_0`, `CLEAR_INTERRUPT`, `DM35418_Check_Result()`, `DM35418_Dma_Check_Buffer_Used()`, `DM35418_Dma_Check_For_Error()`, `DM35418_Dma_Clear_Interru↵t()`, `DM35418_Dma_Read()`, `DM35418_Dma_Reset_Buffer()`, `DM35418_Gbc_Ack_↵Interrupt()`, `exit_program`, `DM35418_Function_Block::fb_num`, `dm35418_ioctl_interrupt_info_request::interrupt_↵_fb`, `local_buffer`, `my_adc`, `next_buffer`, `NO_CLEAR_INTERRUPT`, `DM35418_Function_Block::num_dma_buffers`, `NUM_FB_TO_USE`, and `dm35418_ioctl_interrupt_info_request::valid_interrupt`.

Referenced by `setup_isr()`.

6.6.3.2 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

0	Success
Non-Zero	Failure.

Definition at line 740 of file dm35418_adc_to_dac_dma.c.

References ADC_0, board, buff_cnt, BUFFER_SIZE_BYTES, BUFFER_SIZE_SAMPLES, CHANNEL_0, DEFAULT_RATE, DM35418_ADC_CLK_BUS_SRC_START_TRIG, DM35418_Adc_Initialize(), DM35418_ADC_MAX_RATE, DM35418_Adc_Set_Clock_Source_Global(), DM35418_Adc_Set_Sample_Rate(), DM35418_Adc_Set_Start_Trigger(), DM35418_Adc_Set_Stop_Trigger(), DM35418_Adc_Start(), DM35418_Board_Open(), DM35418_Check_Result(), DM35418_CLK_SRC_BUS2, DM35418_CLK_SRC_IMMEDIATE, DM35418_CLK_SRC_NEVER, DM35418_Dma_Start(), DM35418_Dma_Write(), DM35418_Gbc_Board_Reset(), DM35418_Micro_Sleep(), exit_program, HELP_OPTION, local_buffer, MINOR_OPTION, my_adc, DM35418_Function_Block::num_dma_buffers, NUM_FB_TO_USE, program_name, RATE_OPTION, setup_adcs(), setup_dacs(), setup_isr(), sigint_handler(), start_slave_adcs(), and usage().

6.6.3.3 output_channel_status()

```
void output_channel_status (
    struct DM35418_Board_Descriptor * handle,
    const struct DM35418_Function_Block * func_block,
    unsigned int channel )
```

Output the status of a DMA channel. This is a helper function to determine the cause of an error when it occurs.

Parameters

<i>handle</i>	Pointer to the board handle.
<i>func_block</i>	Pointer to the function block containing the DMA channel
<i>channel</i>	The DMA channel we want the status of.

Return values

None	
------	--

Definition at line 199 of file dm35418_adc_to_dac_dma.c.

References DM35418_Check_Result(), DM35418_Dma_Status(), and DM35418_Function_Block::fb_num.

6.6.3.4 setup_adcs()

```
void setup_adcs ( )
```

Set the ADCs to continuously sample data into the DMA buffer.

Return values

None.	
-------	--

Definition at line 528 of file dm35418_adc_to_dac_dma.c.

References board, buff_cnt, BUFFER_SIZE_BYTES, CHANNEL_0, DM35418_Adc_Channel_Setup(), DM35418_ADC_INPUT_SINGLE_ENDED, DM35418_Adc_Open(), DM35418_ADC_RNG_BIPOLAR_5V, DM35418_Adc_Set_Clock_Src(), DM35418_Check_Result(), DM35418_CLK_SRC_IMMEDIATE, DM35418_DMA_BUFFER_CTRL_INTR, DM35418_DMA_BUFFER_CTRL_LOOP, DM35418_DMA_BUFFER_CTRL_VALID, DM35418_Dma_Buffer_Setup(), DM35418_Dma_Buffer_Status(), DM35418_Dma_Configure_Interrupts(), DM35418_Dma_Initialize(), DM35418_Dma_Setup(), DM35418_DMA_SETUP_DIRECTION_READ, DM35418_NUM_ADC_DMA_BUFFERS, ERROR_INTR_ENABLE, INTERRUPT_ENABLE, my_adc, next_buffer, NOT_IGNORE_USED, DM35418_Function_Block::num_dma_buffers, DM35418_Function_Block::num_dma_channels, and NUM_FB_TO_USE.

Referenced by main().

6.6.3.5 setup_dacs()

```
void setup_dacs (
    uint32_t rate )
```

Setup the DACs to output continuously via DMA.

Parameters

rate	Rate of data output. This should be the same as the rate of data input.
------	---

Return values

None.	
-------	--

Definition at line 382 of file dm35418_adc_to_dac_dma.c.

References board, DM35418_Dac_Open(), and NUM_FB_TO_USE.

Referenced by main().

6.6.3.6 setup_isr()

```
void setup_isr ( )
```

Install the interrupt subroutine.

Return values

None.

Definition at line 636 of file dm35418_adc_to_dac_dma.c.

References board, DM35418_Check_Result(), DM35418_General_InstallISR(), and ISR().

Referenced by main().

6.6.3.7 sigint_handler()

```
static void sigint_handler (
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 168 of file dm35418_adc_to_dac_dma.c.

References exit_program.

Referenced by main().

6.6.3.8 start_slave_adcs()

```
void start_slave_adcs (
    uint32_t rate )
```

Start all of the "slave" ADCs. These started ADCs will wait until the start trigger, which is the starting of ADC0. In this way, all ADCs will start together.

Parameters

<i>rate</i>	Rate at which to sample data.
-------------	-------------------------------

Return values

<i>None.</i>	
--------------	--

Definition at line 666 of file dm35418_adc_to_dac_dma.c.

References board, CHANNEL_0, DM35418_Adc_Initialize(), DM35418_Adc_Set_Sample_Rate(), DM35418_Adc_Set_Start_Trigger(), DM35418_Adc_Set_Stop_Trigger(), DM35418_Adc_Start(), DM35418_Check_Result(), DM35418_CLK_SRC_BUS2, DM35418_CLK_SRC_NEVER, DM35418_Dma_Start(), my_adc, and NUM_FB_TO_USE.

Referenced by main().

6.6.4 Variable Documentation

6.6.4.1 adc_dma_error

```
int adc_dma_error = 0 [static]
```

Boolean flag indicating if there was an ADC DMA error.

Definition at line 91 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR().

6.6.4.2 board

```
struct DM35418_Board_Descriptor* board [static]
```

Pointer to board descriptor

Definition at line 96 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR(), main(), setup_adcs(), setup_dacs(), setup_isr(), and start_slave_adcs().

6.6.4.3 buff_cnt

```
unsigned long buff_cnt[NUM\_FB\_TO\_USE] [static]
```

Array of buffer counts, used to track progress of each ADC as data is copied.

Definition at line 108 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR(), main(), and setup_adcs().

6.6.4.4 exit_program

```
volatile int exit_program = 0 [static]
```

Boolean indicating the program should exit.

Definition at line 119 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR(), main(), and sigint_handler().

6.6.4.5 local_buffer

```
int** local_buffer[NUM\_FB\_TO\_USE] [static]
```

Pointer to local memory buffer where data is copied from the kernel buffers when a DMA buffer becomes full.

Definition at line 114 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR(), and main().

6.6.4.6 my_adc

```
struct DM35418\_Function\_Block my_adc[NUM\_FB\_TO\_USE] [static]
```

Pointer to array of function blocks that will hold the descriptors

Definition at line 96 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR(), main(), setup_adcs(), and start_slave_adcs().

6.6.4.7 next_buffer

```
unsigned int next_buffer[NUM_FB_TO_USE] [static]
```

Which buffer is next to be copied from DMA

Definition at line 124 of file dm35418_adc_to_dac_dma.c.

Referenced by ISR(), and setup_adcs().

6.6.4.8 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 86 of file dm35418_adc_to_dac_dma.c.

Referenced by main(), and usage().

6.7 examples/dm35418_dac.c File Reference

Example program which demonstrates the use of the DAC.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <termios.h>
#include <time.h>
#include <sys/time.h>
#include "dm35418_gbc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418.h"
#include "dm35418_util_library.h"
```

Macros

- #define `DEFAULT_DAC_NUM` 0

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`

6.7.1 Detailed Description

Example program which demonstrates the use of the DAC.

This example program sends data to the DAC for instant conversion. To see the output data, connect an oscilloscope to the DACx.0 pin, where x is the DAC you select.

The user can control what value goes out the DAC by using keys to increase or decrease the desired voltage, up to 5 V and down to -5 V. Follow the on-screen instructions for adjusting the voltage.

Press 'q' to quit the program.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

`dm35418_dac.c` 141531 2024-03-06 21:05:25Z lfrankfield

6.7.2 Macro Definition Documentation

6.7.2.1 DEFAULT_DAC_NUM

```
#define DEFAULT_DAC_NUM 0
```

DAC to use, if user does not choose one.

Definition at line 59 of file dm35418_dac.c.

6.7.3 Function Documentation

6.7.3.1 `main()`

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure.

The DAC cannot achieve +5.0 volts, but for purposes of the example, we allow them to select 5 as a value. However, we'll have to request the actual max value from the library function.

Definition at line 121 of file dm35418_dac.c.

References board.

6.7.4 Variable Documentation

6.7.4.1 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 64 of file dm35418_dac.c.

Referenced by usage().

6.8 examples/dm35418_dac_dma.c File Reference

Example program which demonstrates the use of the DAC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>
#include "dm35418_gbc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_dma_library.h"
#include "dm35418_util_library.h"
#include "dm35418.h"
```

Macros

- `#define NUM_BUFFERS_TO_USE 1`
- `#define DEFAULT_DAC_TO_USE 0`
- `#define DEFAULT_RATE 100`
- `#define BUFFER_SIZE_SAMPLES 100`
- `#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))`

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- volatile int `exit_program` = 0

6.8.1 Detailed Description

Example program which demonstrates the use of the DAC and DMA.

This example program generates wave form data and "plays" it out the specified DAC channel. To see the output data, connect an oscilloscope to the DACx.0 pin, where x is the DAC number chosen.

After the program is running, you can alter the rate of DAC output by entering a new frequency and hitting Enter. Note that the frequency of the waveform seen on an oscilloscope will be different than the frequency of the DAC, depending on the number of samples used in creating the wave.

Use the --help command line option to see all possible input values.

Hit Ctrl-C to exit the example.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35418_dac_dma.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.8.2 Macro Definition Documentation

6.8.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Buffer size to allocate in bytes

Definition at line 82 of file dm35418_dac_dma.c.

6.8.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to create. Increase this number for a "finer" waveform.

Definition at line 77 of file dm35418_dac_dma.c.

6.8.2.3 DEFAULT_DAC_TO_USE

```
#define DEFAULT_DAC_TO_USE 0
```

DAC to use if user does not enter one on the command line

Definition at line 66 of file dm35418_dac_dma.c.

6.8.2.4 DEFAULT_RATE

```
#define DEFAULT_RATE 100
```

Rate to use if user does not enter one on the command line (Hz)

Definition at line 71 of file dm35418_dac_dma.c.

6.8.2.5 NUM_BUFFERS_TO_USE

```
#define NUM_BUFFERS_TO_USE 1
```

We will only use one buffer in this example, and loop it

Definition at line 61 of file dm35418_dac_dma.c.

6.8.3 Function Documentation

6.8.3.1 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure.

Definition at line 178 of file dm35418_dac_dma.c.

References board.

6.8.3.2 sigint_handler()

```
static void sigint_handler (  
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 146 of file dm35418_dac_dma.c.

References exit_program.

6.8.4 Variable Documentation**6.8.4.1 exit_program**

```
volatile int exit_program = 0
```

Boolean indicating the program should be exited.

Definition at line 93 of file dm35418_dac_dma.c.

Referenced by sigint_handler().

6.8.4.2 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 88 of file dm35418_dac_dma.c.

Referenced by usage().

6.9 examples/dm35418_dac_syncbus.c File Reference

Example program which demonstrates the use of Syncbus with the DAC and DMA.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include <signal.h>
#include <string.h>
#include "dm35418_gbc_library.h"
#include "dm35418_dac_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_dma_library.h"
#include "dm35418_syncbus_library.h"
#include "dm35418_util_library.h"
#include "dm35418.h"
```

Macros

- `#define BUFFER_SIZE_SAMPLES 100`
- `#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))`
- `#define NUM_BUFFERS_TO_USE 1`
- `#define DEFAULT_RATE 100`
- `#define DEFAULT_DAC_TO_USE 0`

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void `sigint_handler` (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- void `setup_syncbus` (struct `DM35418_Board_Descriptor` *board, struct `DM35418_Function_Block` *my_syncbus, int syncbus_mode, int syncbus_term)
Setup the Syncbus function block.
- void `setup_dac` (struct `DM35418_Board_Descriptor` *board, struct `DM35418_Function_Block` *my_dac, unsigned int dac_num, int syncbus_mode, uint32_t rate, uint32_t *actual_rate)
Setup the DAC.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`
- volatile int `exit_program` = 0

6.9.1 Detailed Description

Example program which demonstrates the use of Syncbus with the DAC and DMA.

This example program generates wave form data and "plays" it out the specified DAC channel. It uses the SyncBus to ensure the output of DACs on separate boards stay perfectly synchronized.

You can alter the rate of DAC output by entering a new frequency and hitting enter. Note that the frequency of the waveform seen on an oscilloscope will be different, depending on the number of samples used in creating the wave.

In order to use the SyncBus in this example, you will need at least 2 DM35418 in the stack, with connected SyncBus. If the board features a right-angle syncbus connector, then the syncbus connectors must be connected pin-to-pin with a straight ribbon cable. Connect all boards that you intend to use SyncBus with. If the boards have stack through syncbus connectors, then no other connection is needed.

You must start a separate instance of this example program for each board to be used, and use the command line arguments to designate one board to be the master and the other boards to be the slaves. Refer to the help for the correct commands to do this by using the --help command line argument.

In syncbus master mode the board drives the syncbus clock line with a 10MHz reference clock, and syncbus #2 line with the sample enable of the DAC. The master syncbus example will allow you to change the frequency of the master clock.

In syncbus slave mode the FPGA master clock is synthesized from the syncbus clock line. The DAC sample enable frequency is the same frequency as the syncbus 2 input pulses (divider=0). The slave syncbus example will allow you to change the divider.

The syncbus differential termination must be enabled or disabled based on the board position in the stack: the boards on the top and bottom of the stack must have termination enabled, while the boards in between must have termination disabled.

Hit Ctrl-C to exit.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

ld

`dm35418_dac_syncbus.c` 141531 2024-03-06 21:05:25Z lfrankenfield

6.9.2 Macro Definition Documentation

6.9.2.1 BUFFER_SIZE_BYTES

```
#define BUFFER_SIZE_BYTES (BUFFER_SIZE_SAMPLES * sizeof(int))
```

Buffer size to hold waveform to output, in bytes

Definition at line 94 of file dm35418_dac_syncbus.c.

6.9.2.2 BUFFER_SIZE_SAMPLES

```
#define BUFFER_SIZE_SAMPLES 100
```

Number of samples to generate waveform with.

Definition at line 89 of file dm35418_dac_syncbus.c.

6.9.2.3 DEFAULT_DAC_TO_USE

```
#define DEFAULT_DAC_TO_USE 0
```

DAC to use if user does not provide one.

Definition at line 111 of file dm35418_dac_syncbus.c.

6.9.2.4 DEFAULT_RATE

```
#define DEFAULT_RATE 100
```

Rate to use if user does not provide one. (Hz)

Definition at line 106 of file dm35418_dac_syncbus.c.

6.9.2.5 NUM_BUFFERS_TO_USE

```
#define NUM_BUFFERS_TO_USE 1
```

Number of buffers to use. Note that this is just a readability constant...changing this will not affect the number of buffers the waveform resides in.

Definition at line 101 of file dm35418_dac_syncbus.c.

6.9.3 Function Documentation

6.9.3.1 `main()`

```
int main (  
    int argument_count,  
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure.

Definition at line 414 of file dm35418_dac_syncbus.c.

References board.

6.9.3.2 setup_dac()

```
void setup_dac (
    struct DM35418_Board_Descriptor * board,
    struct DM35418_Function_Block * my_dac,
    unsigned int dac_num,
    int syncbus_mode,
    uint32_t rate,
    uint32_t * actual_rate )
```

Setup the DAC.

Parameters

<i>board</i>	Pointer to the board descriptor
<i>my_dac</i>	Pointer to the DAC function block we care about
<i>dac_num</i>	Which DAC on the board to open
<i>syncbus_mode</i>	Is the syncbus in master or slave?
<i>rate</i>	Rate of conversion for the DAC
<i>actual_rate</i>	Pointer to the returned actual rate of the DAC

Return values

<i>None</i>	
-------------	--

Definition at line 327 of file dm35418_dac_syncbus.c.

6.9.3.3 setup_syncbus()

```
void setup_syncbus (
    struct DM35418_Board_Descriptor * board,
```

```

struct DM35418_Function_Block * my_syncbus,
int syncbus_mode,
int syncbus_term )

```

Setup the Syncbus function block.

Parameters

<i>board</i>	Pointer to the board descriptor
<i>my_syncbus</i>	Pointer to the syncbus function block we care about
<i>syncbus_mode</i>	Is board master or slave in this configuration?
<i>syncbus_term</i>	Boolean value to enable or disable termination

Return values

<i>None</i>	
-------------	--

Definition at line 214 of file dm35418_dac_syncbus.c.

References board, DM35418_Check_Result(), DM35418_CLK_SRC_BUS2, DM35418_SYNCBUS_2_CLK_SRC, DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2, DM35418_SYNCBUS_CLK_SEL_LOCAL, DM35418_SYNCBUS_CLK_SEL_SYNC, DM35418_Syncbus_Clock_Source_Global_Set(), DM35418_SYNCBUS_DIR_INPUT, DM35418_SYNCBUS_DIR_OUTPUT, DM35418_Syncbus_Drive_Source_Set(), DM35418_Syncbus_Enable(), DM35418_Syncbus_Open(), DM35418_Syncbus_Set_Clk_Sel(), DM35418_Syncbus_Set_Direction(), DM35418_Syncbus_Termination_Disable(), DM35418_Syncbus_Termination_Enable(), and SYNCBUS_MASTER.

6.9.3.4 sigint_handler()

```

static void sigint_handler (
    int signal_number ) [static]

```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 179 of file dm35418_dac_syncbus.c.

References exit_program.

6.9.4 Variable Documentation

6.9.4.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating program should be exited

Definition at line 121 of file dm35418_dac_syncbus.c.

Referenced by sigint_handler().

6.9.4.2 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 116 of file dm35418_dac_syncbus.c.

Referenced by usage().

6.10 examples/dm35418_dio.c File Reference

Example program which demonstrates the use of the DIO.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include "dm35418_gbc_library.h"
#include "dm35418_dio_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_util_library.h"
#include "dm35418_examples.h"
```

Functions

- static void [usage](#) (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int [main](#) (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)

6.10.1 Detailed Description

Example program which demonstrates the use of the DIO.

This example program sets the lower 16-bits of DIO to output, and the upper 16-bits to input. We'll then write every possible 16-bit value to the output and verify the same value on the input pins.

This example requires a loopback from DIO Pins 0-15 to DIO Pins 16-31. Check the manual for which pins on connector CN3 correspond to those.

Port 0, Pins 0-15: Output
Port 0, Pins 16-31: Input

This file and its contents are copyright (C) RTD Embedded Technologies, Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License Agreement. For a copy of this agreement, refer to the file LICENSE.TXT (which should be included with this software) or contact RTD Embedded Technologies, Inc.

Id

[dm35418_dio.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.10.2 Function Documentation

6.10.2.1 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

0	Success
Non-Zero	Failure.

Definition at line 116 of file dm35418_dio.c.

References `DM35418_Board_Open()`, `HELP_OPTION`, `MINOR_OPTION`, `program_name`, and `usage()`.

6.10.3 Variable Documentation

6.10.3.1 `program_name`

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 59 of file `dm35418_dio.c`.

Referenced by `main()`, and `usage()`.

6.11 examples/dm35418_list_fb.c File Reference

Example program which demonstrates use of the library to open a function block for use.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <getopt.h>
#include "dm35418_gbc_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_util_library.h"
```

Functions

- static void `usage` (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- int `main` (int argument_count, char **arguments)
The main program.

Variables

- static char * `program_name`

6.11.1 Detailed Description

Example program which demonstrates use of the library to open a function block for use.

This example program uses the board library to query all function blocks on the board. When a function block is opened that has a valid function type, then the number of DMA channels and buffers is printed to the screen. In this way, the example program shows an inventory of the function blocks on a given board.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35418_list_fb.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.11.2 Function Documentation

6.11.2.1 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

0	Success
Non-Zero	Failure.

Definition at line 105 of file dm35418_list_fb.c.

References [board](#), [DM35418_Board_Close\(\)](#), [DM35418_Board_Open\(\)](#), [DM35418_Check_Result\(\)](#), [DM35418_FUNC_BLOCK_ADC](#), [DM35418_FUNC_BLOCK_DAC](#), [DM35418_FUNC_BLOCK_DIO](#), [DM35418_FUNC_BLOCK_INVALID](#), [DM35418_FUNC_BLOCK_SYNCBUS](#), [DM35418_FUNC_BLOCK_TEMPERATURE_SENSOR](#), [DM35418_Function_Block_Open\(\)](#), [DM35418_Gbc_Board_Reset\(\)](#), [DM35418_Gbc_Get_Fpga_Build\(\)](#),

DM35418_Gbc_Get_Pdp_Number(), DM35418_Gbc_Get_Revision(), DM35418_MAX_FB, DM35418_Function↵
_Block::fb_num, HELP_OPTION, MINOR_OPTION, DM35418_Function_Block::num_dma_buffers, DM35418↵
_Function_Block::num_dma_channels, program_name, DM35418_Function_Block::sub_type, DM35418_↵
Function_Block::type, and usage().

6.11.3 Variable Documentation

6.11.3.1 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 51 of file dm35418_list_fb.c.

Referenced by main(), and usage().

6.12 examples/dm35418_temperature.c File Reference

Example program for read the temperature sensor.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <errno.h>
#include <error.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <getopt.h>
#include <string.h>
#include "dm35418_gbc_library.h"
#include "dm35418_temperature_library.h"
#include "dm35418_ioctl.h"
#include "dm35418_examples.h"
#include "dm35418_util_library.h"
```

Functions

- static void [usage](#) (void)
Print information on stderr about how the program is to be used. After doing so, the program is exited.
- static void [sigint_handler](#) (int signal_number)
Signal handler for SIGINT Control-C keyboard interrupt.
- int [main](#) (int argument_count, char **arguments)
The main program.

Variables

- static char * [program_name](#)
- volatile int [exit_program](#) = 0

6.12.1 Detailed Description

Example program for read the temperature sensor.

The program will read the temperature value from the temperature function block and print it to the screen. It will also print the maximum and minimum temperature received from the board for the time that the example program has been running.

The program will continue to run until CTRL-C is pressed.

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.

This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35418_temperature.c](#) 141531 2024-03-06 21:05:25Z lfrankenfield

6.12.2 Function Documentation

6.12.2.1 main()

```
int main (
    int argument_count,
    char ** arguments )
```

The main program.

Parameters

<i>argument_count</i>	Number of args passed on the command line, including the executable name
<i>arguments</i>	Pointer to array of character strings, which are the args themselves.

Return values

<i>0</i>	Success
<i>Non-Zero</i>	Failure.

Definition at line 134 of file dm35418_temperature.c.

References board, DM35418_Board_Close(), DM35418_Board_Open(), DM35418_Check_Result(), DM35418_Gbc_Board_Reset(), DM35418_Micro_Sleep(), DM35418_Temperature_Open(), DM35418_Temperature_Read(), exit_program, HELP_OPTION, MINOR_OPTION, program_name, sigint_handler(), and usage().

6.12.2.2 sigint_handler()

```
static void sigint_handler (
    int signal_number ) [static]
```

Signal handler for SIGINT Control-C keyboard interrupt.

Parameters

<i>signal_number</i>	Signal number passed in from the kernel.
----------------------	--

Warning

One must be extremely careful about what functions are called from a signal handler.

Definition at line 101 of file dm35418_temperature.c.

References exit_program.

Referenced by main().

6.12.3 Variable Documentation

6.12.3.1 exit_program

```
volatile int exit_program = 0
```

Boolean indicating whether or not to exit the program.

Definition at line 60 of file dm35418_temperature.c.

Referenced by main(), and sigint_handler().

6.12.3.2 program_name

```
char* program_name [static]
```

Name of the program as invoked on the command line

Definition at line 55 of file dm35418_temperature.c.

Referenced by main(), and usage().

6.13 include/dm35418.h File Reference

Defines for the DM35418 (Device-specific values)

Macros

- #define [DM35418_PCI_VENDOR_ID](#) 0x1435
DM35418 PCI vendor ID.
- #define [DM35418_PCI_DEVICE_ID](#) 0x5418
DM35418 PCI device ID.
- #define [DM35418_NUM_ADC_ON_BOARD](#) 8
Number of ADC on the DM35418.
- #define [DM35418_NUM_DAC_ON_BOARD](#) 8
Number of DAC on the DM35418.
- #define [DM35418_NUM_ADC_DMA_CHANNELS](#) 1
Number of channels per ADC.
- #define [DM35418_NUM_ADC_DMA_BUFFERS](#) 7
Number of buffers per ADC DMA channel.
- #define [DM35418_NUM_DAC_DMA_CHANNELS](#) 1
Number of channels per DAC.
- #define [DM35418_NUM_DAC_DMA_BUFFERS](#) 7
Number of buffers per DAC DMA channel.
- #define [DM35418_DAC_FE_CONFIG_GAIN_MASK](#) 0x03
Bit Mask for the gain bits of the FE Config.
- #define [DM35418_FIFO_SAMPLE_SIZE](#) 511
Sample size of the FIFO.
- #define [DM35418_DAC_MAX_VALUE](#) 32767
Max value of the DAC.
- #define [DM35418_DAC_MIN_VALUE](#) -32768
Min value of the DAC.

6.13.1 Detailed Description

Defines for the DM35418 (Device-specific values)

```
-----
This file and its contents are copyright (C) RTD Embedded Technologies,
Inc. All Rights Reserved.
```

```
This software is licensed as described in the RTD End-User Software License
Agreement. For a copy of this agreement, refer to the file LICENSE.TXT
(which should be included with this software) or contact RTD Embedded
Technologies, Inc.
-----
```

Id

[dm35418.h](#) 96149 2016-01-20 21:01:00Z rgroner

6.14 include/dm35418_adc_library.h File Reference

Definitions for the DM35418 ADC Library.

```
#include "dm35418_gbc_library.h"
```

Macros

- `#define DM35418_ADC_MODE_RESET 0x00`
Register value for ADC Mode Reset.
- `#define DM35418_ADC_MODE_PAUSE 0x01`
Register value for ADC Mode Pause.
- `#define DM35418_ADC_MODE_GO_SINGLE_SHOT 0x02`
Register value for ADC Mode Go (Single Shot)
- `#define DM35418_ADC_MODE_GO_REARM 0x03`
Register value for ADC Mode Go (Rearm after Stop)
- `#define DM35418_ADC_MODE_UNINITIALIZED 0x04`
Register value for ADC Mode Uninitialized.
- `#define DM35418_ADC_STAT_STOPPED 0x00`
Register value for ADC Status - Stopped.
- `#define DM35418_ADC_STAT_FILLING_PRE_TRIG_BUFF 0x01`
Register value for ADC Status - Filling Pre-Start Buffer.
- `#define DM35418_ADC_STAT_WAITING_START_TRIG 0x02`
Register value for ADC Status - Waiting for Start Trigger.
- `#define DM35418_ADC_STAT_SAMPLING 0x03`
Register value for ADC Status - Sampling Data.
- `#define DM35418_ADC_STAT_FILLING_POST_TRIG_BUFF 0x04`
Register value for ADC Status - Filling Post-Stop Buffer.
- `#define DM35418_ADC_STAT_WAIT_REARM 0x05`
Register value for ADC Status - Wait for Rearm.
- `#define DM35418_ADC_STAT_DONE 0x07`
Register value for ADC Status - Done.
- `#define DM35418_ADC_STAT_UNINITIALIZED 0x08`
Register value for ADC Status - Uninitialized.
- `#define DM35418_ADC_STAT_INITIALIZING 0x09`
Register value for ADC Status - Initializing.
- `#define DM35418_ADC_INT_SAMPLE_TAKEN_MASK 0x01`
Register value for Interrupt Mask - Sample Taken.
- `#define DM35418_ADC_INT_CHAN_THRESHOLD_MASK 0x02`
Register value for Interrupt Mask - Channel Threshold Exceeded.
- `#define DM35418_ADC_INT_PRE_BUFF_FULL_MASK 0x04`
Register value for Interrupt Mask - Pre-Start Buffer Filled.
- `#define DM35418_ADC_INT_START_TRIG_MASK 0x08`
Register value for Interrupt Mask - Start Trigger Occurred.
- `#define DM35418_ADC_INT_STOP_TRIG_MASK 0x10`
Register value for Interrupt Mask - Stop Trigger Occurred.
- `#define DM35418_ADC_INT_POST_BUFF_FULL_MASK 0x20`
Register value for Interrupt Mask - Post-Stop Buffer Filled.
- `#define DM35418_ADC_INT_SAMP_COMPL_MASK 0x40`

- Register value for Interrupt Mask - Sampling Complete.*

 - #define [DM35418_ADC_INT_PACER_TICK_MASK](#) 0x80
- Register value for Interrupt Mask - Pacer Clock Tick Occurred.*

 - #define [DM35418_ADC_INT_ALL_MASK](#) 0xFF
- Register value for Interrupt Mask - All Bits.*

 - #define [DM35418_ADC_CHAN_INTR_LOW_THRESHOLD_MASK](#) 0x01
- Register value for Channel Low Threshold Interrupt.*

 - #define [DM35418_ADC_CHAN_INTR_HIGH_THRESHOLD_MASK](#) 0x02
- Register value for Channel High Threshold Interrupt.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER0](#) 0x0
- Register value for Channel Filter Order 0.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER1](#) 0x1
- Register value for Channel Filter Order 1.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER2](#) 0x2
- Register value for Channel Filter Order 2.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER3](#) 0x3
- Register value for Channel Filter Order 3.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER4](#) 0x4
- Register value for Channel Filter Order 4.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER5](#) 0x5
- Register value for Channel Filter Order 5.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER6](#) 0x6
- Register value for Channel Filter Order 6.*

 - #define [DM35418_ADC_CHAN_FILTER_ORDER7](#) 0x7
- Register value for Channel Filter Order 7.*

 - #define [DM35418_ADC_FE_CONFIG_GAIN_05](#) 0x00
- Register value for setting Half-Gain.*

 - #define [DM35418_ADC_FE_CONFIG_GAIN_1](#) 0x20
- Register value for setting a Gain of 1.*

 - #define [DM35418_ADC_FE_CONFIG_GAIN_2](#) 0x28
- Register value for setting a Gain of 2.*

 - #define [DM35418_ADC_FE_CONFIG_GAIN_4](#) 0x30
- Register value for setting a Gain of 4.*

 - #define [DM35418_ADC_FE_CONFIG_GAIN_8](#) 0x38
- Register value for setting a Gain of 8.*

 - #define [DM35418_ADC_FE_CONFIG_BIPOLAR](#) 0x00
- Register value for setting input to Bi-Polar.*

 - #define [DM35418_ADC_FE_CONFIG_UNIPOLAR](#) 0x04
- Register value for setting input to Uni-Polar.*

 - #define [DM35418_ADC_FE_CONFIG_SINGLE_ENDED](#) 0x01
- Register value for setting input to Single-Ended.*

 - #define [DM35418_ADC_FE_CONFIG_DIFFERENTIAL](#) 0x03
- Register value for setting input to Differential.*

 - #define [DM35418_ADC_FE_CONFIG_INPUT_VREF_5V](#) 0x00
- Register value for setting input to VREF (5V)*

 - #define [DM35418_ADC_FE_CONFIG_GAIN_MASK](#) 0x38
- Bit mask for the channel gain bits of the FE Config.*

 - #define [DM35418_ADC_FE_CONFIG_POLARITY_MASK](#) 0x04
- Bit mask for the polarity bit of the FE Config.*

 - #define [DM35418_ADC_FE_CONFIG_MODE_MASK](#) 0x03
- Bit mask for the input mode bits of the FE Config.*

- #define `DM35418_ADC_MAX_RATE` 1538461
Max allowable rate for the ADC (Hz)
- #define `DM35418_ADC_UNIPOLAR_THRESHOLD_MAX` 262144L
Maximum allowable value to write to the threshold register in unipolar mode.
- #define `DM35418_ADC_UNIPOLAR_THRESHOLD_MIN` 0L
Minimum allowable value to write to the threshold register in unipolar mode.
- #define `DM35418_ADC_BIPOLAR_THRESHOLD_MAX` 131071L
Maximum allowable value to write to the threshold register in unipolar mode.
- #define `DM35418_ADC_BIPOLAR_THRESHOLD_MIN` -131072L
Minimum allowable value to write to the threshold register in unipolar mode.
- #define `DM35418_ADC_BIT_WIDTH_MAX` 262144L
The maximum value represented by the bit width of the ADC.
- #define `DM35418_ADC_BIT_WIDTH_MAX_FLT` ((float) `DM35418_ADC_BIT_WIDTH_MAX`)
The max value of the ADC as a float.
- #define `DM35418_ADC_RNG_1_25_LSB` 0.00000476837158203125
What each bit is worth at a range of 1.25.
- #define `DM35418_ADC_RNG_2_5_LSB` 0.0000095367431640625
What each bit is worth at a range of 2.5.
- #define `DM35418_ADC_RNG_5_LSB` 0.000019073486328125
What each bit is worth at a range of 5.
- #define `DM35418_ADC_RNG_10_LSB` 0.00003814697265625
What each bit is worth at a range of 10.
- #define `DM35418_ADC_RNG_20_LSB` 0.0000762939453125
What each bit is worth at a range of 20.
- #define `DM35418_ADC_UNIPOLAR_MAX` 262143L
Max possible ADC value when in Unipolar.
- #define `DM35418_ADC_UNIPOLAR_MIN` 0
Min possible ADC value when in Unipolar.
- #define `DM35418_ADC_BIPOLAR_MAX` 131071L
Max possible ADC value when in Bipolar.
- #define `DM35418_ADC_BIPOLAR_MIN` -131072L
Min possible ADC value when in Bipolar.

Enumerations

- enum `DM35418_Adc_Clock_Events` {
`DM35418_ADC_CLK_BUS_SRC_DISABLE` = 0x00 , `DM35418_ADC_CLK_BUS_SRC_SAMPLE_TAKEN` = 0x80 , `DM35418_ADC_CLK_BUS_SRC_CHAN_THRESH` = 0x81 , `DM35418_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL` = 0x82 ,
`DM35418_ADC_CLK_BUS_SRC_START_TRIG` = 0x83 , `DM35418_ADC_CLK_BUS_SRC_STOP_TRIG` = 0x84 , `DM35418_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL` = 0x85 , `DM35418_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE` = 0x86 ,
`DM35418_ADC_CLK_BUS_SRC_PACER_TICK` = 0x87 }
Clock events for the global source clocks.
- enum `DM35418_Input_Ranges` {
`DM35418_ADC_RNG_BIPOLAR_10V` , `DM35418_ADC_RNG_BIPOLAR_5V` , `DM35418_ADC_RNG_BIPOLAR_2_5V` ,
`DM35418_ADC_RNG_BIPOLAR_1_25V` ,
`DM35418_ADC_RNG_BIPOLAR_625mV` , `DM35418_ADC_RNG_UNIPOLAR_5V` , `DM35418_ADC_RNG_UNIPOLAR_10V` ,
`DM35418_ADC_RNG_UNIPOLAR_2_5V` ,
`DM35418_ADC_RNG_UNIPOLAR_1_25V` }
Input range of the ADC input pin. This combines polarity and gain into a single enumeration, and is the preferred way of setting polarity and gain.

- enum `DM35418_Input_Mode` { `DM35418_ADC_INPUT_SINGLE_ENDED` , `DM35418_ADC_INPUT_DIFFERENTIAL` , `DM35418_ADC_INPUT_VREF_5V` }
Input mode of the ADC pin.
- enum `DM35418_Gains` {
 `DM35418_ADC_GAIN_05` , `DM35418_ADC_GAIN_1` , `DM35418_ADC_GAIN_2` , `DM35418_ADC_GAIN_4` ,
 `DM35418_ADC_GAIN_8` , `DM35418_ADC_GAIN_16` , `DM35418_ADC_GAIN_32` , `DM35418_ADC_GAIN_64`
 ,
 `DM35418_ADC_GAIN_128` }
Input gain to apply to the incoming signal. Note that the preferred method of setting the gain is through the input range enumeration.

Functions

- `DM35418LIB_API int DM35418_Adc_Open` (struct `DM35418_Board_Descriptor` *handle, unsigned int number_of_type, struct `DM35418_Function_Block` *func_block)
Open the ADC indicated, and determine register locations of control blocks needed to control it.
- `DM35418LIB_API int DM35418_Adc_Get_Start_Trigger` (struct `DM35418_Board_Descriptor` *handle, struct `DM35418_Function_Block` *func_block, uint8_t *start_trigger)
Get the start trigger for data collection.
- `DM35418LIB_API int DM35418_Adc_Set_Start_Trigger` (struct `DM35418_Board_Descriptor` *handle, struct `DM35418_Function_Block` *func_block, uint8_t start_trigger)
Set the start trigger for data collection.
- `DM35418LIB_API int DM35418_Adc_Get_Stop_Trigger` (struct `DM35418_Board_Descriptor` *handle, struct `DM35418_Function_Block` *func_block, uint8_t *stop_trigger)
Get the stop trigger for data collection.
- `DM35418LIB_API int DM35418_Adc_Set_Stop_Trigger` (struct `DM35418_Board_Descriptor` *handle, struct `DM35418_Function_Block` *func_block, uint8_t stop_trigger)
Set the stop trigger for data collection.
- `DM35418LIB_API int DM35418_Adc_Get_Pre_Trigger_Samples` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, uint32_t *count)
Get the amount of data to capture prior to start trigger.
- `DM35418LIB_API int DM35418_Adc_Set_Pre_Trigger_Samples` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, uint32_t count)
Set the amount of data to capture prior to start trigger.
- `DM35418LIB_API int DM35418_Adc_Get_Post_Stop_Samples` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, uint32_t *count)
Get the amount of data to capture after stop trigger.
- `DM35418LIB_API int DM35418_Adc_Set_Post_Stop_Samples` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, uint32_t count)
Set the amount of data to capture after stop trigger.
- `DM35418LIB_API int DM35418_Adc_Get_Clock_Src` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, enum `DM35418_Clock_Sources` *source)
Get the clock source for the ADC.
- `DM35418LIB_API int DM35418_Adc_Set_Clock_Src` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, enum `DM35418_Clock_Sources` source)
Set the clock source for the ADC.
- `DM35418LIB_API int DM35418_Adc_Initialize` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Prepare the ADC for actual data collection. Moves the ADC from uninitialized to stopped.
- `DM35418LIB_API int DM35418_Adc_Set_Clk_Divider` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, uint32_t divider)
Set the Clock Divider for the ADC function block.

- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Set_Sample_Rate](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t rate, uint32_t *actual_rate)
Set the sampling rate for the ADC.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Channel_Get_Front_End_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint16_t *fe_config)
Get the front-end config register contents.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Interrupt_Set_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t int_source, int enable)
Configure the interrupts for the ADC.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Interrupt_Get_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for the ADC.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Start](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Start.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Start_Rearm](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Start-Rearm.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Reset.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Pause](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Pause.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Uninitialize](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the ADC mode to Uninitialized.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Get_Mode_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *mode_status)
Get the ADC mode-status value.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Channel_Get_Last_Sample](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t *value)
Get the last sample taken from the ADC.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Get_Sample_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get the count of number of samples taken.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Interrupt_Get_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *value)
Get the interrupt status register.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Interrupt_Clear_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t value)
Clear the interrupt status register.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Channel_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, enum [DM35418_Input_Ranges](#) input_range, enum [DM35418_Input_Mode](#) input_mode)
Setup the channel input for the ADC.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Channel_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)
Reset the channel front-end config.
- [DM35418LIB_API](#) [int](#) [DM35418_Adc_Channel_Interrupt_Set_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t interrupts_to_← set, int enable)
Setup the channel interrupts.

- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Interrupt_Get_Config` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_intr_↔ enable)
Get the channel interrupt configuration.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Interrupt_Get_Status` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_intr_↔ status)
Get the channel interrupt status.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Interrupt_Clear_Status` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t chan_intr_status)
Clear the interrupt status for this channel.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Find_Interrupt` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int *channel_with_interrupt, int *channel_↔ has_interrupt, uint8_t *channel_intr_status, uint8_t *channel_intr_enable)
Find the first channel with an interrupt. Note that this is only useful when looking for a threshold interrupt.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Set_Filter` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t chan_filter)
Set the filter value for the channel.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Get_Filter` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *chan_filter)
Get the filter value for the channel.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Set_Low_Threshold` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t threshold)
Set the lower threshold for this channel.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Set_High_Threshold` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t threshold)
Set the high threshold for this channel.
- [DM35418LIB_API](#) `int DM35418_Adc_Channel_Get_Thresholds` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t *low_threshold, int32_t *high_threshold)
Get both thresholds for this channel.
- [DM35418LIB_API](#) `int DM35418_Adc_Fifo_Channel_Read` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t *value)
Read an ADC sample stored in the onboard FIFO.
- [DM35418LIB_API](#) `int DM35418_Adc_Set_Clock_Source_Global` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) clock_↔ select, enum [DM35418_Adc_Clock_Events](#) clock_driver)
Set the global clock source for the ADC.
- [DM35418LIB_API](#) `int DM35418_Adc_Get_Clock_Source_Global` (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clock_select, int *clock_source)
Get the global clock source for the selected clock.
- [DM35418LIB_API](#) `int DM35418_Adc_Sample_To_Volts` (enum [DM35418_Input_Ranges](#) input_range, int32_t adc_sample, float *volts)
Convert an ADC sample to a volts value.
- [DM35418LIB_API](#) `int DM35418_Adc_Volts_To_Sample` (enum [DM35418_Input_Ranges](#) input_range, float volts, int32_t *adc_sample)
Convert volts to an ADC value.

6.14.1 Detailed Description

Definitions for the DM35418 ADC Library.

Id

[dm35418_adc_library.h](#) 106898 2017-03-08 13:44:23Z rgroner

6.15 include/dm35418_board_access.h File Reference

Structures for the DM35418 Board Access Library.

```
#include <stdint.h>
#include "dm35418_board_access_structs.h"
#include "dm35418_types.h"
#include "dm35418_os.h"
```

Data Structures

- struct [DM35418_DMA_Descriptor](#)
Descriptor for the DMA on this board.
- struct [DM35418_Function_Block](#)
DM35418 function block descriptor. This structure holds information about a function block, including type, number of DMA channels and buffers, descriptors for each DMA channel, and memory offsets to various control locations.

Macros

- `#define` [DM35418LIB_API](#)

Functions

- [DM35418LIB_API](#) int [DM35418_Board_Open](#) (uint8_t dev_num, struct [DM35418_Board_Descriptor](#) **handle)
Open the board, providing the file descriptor that all future operations will reference. Also allocate memory for the device descriptor.
- [DM35418LIB_API](#) int [DM35418_Board_Close](#) (struct [DM35418_Board_Descriptor](#) *handle)
Close the board, closing the open handle for the device file, and freeing the memory allocated for the descriptor.
- [DM35418LIB_API](#) int [DM35418_Read](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Read from the board.
- [DM35418LIB_API](#) int [DM35418_Write](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Write to the board.
- [DM35418LIB_API](#) int [DM35418_Modify](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Read/Modify/Write to the board.
- int [DM35418_Dma](#) (struct [DM35418_Board_Descriptor](#) *handle, union [dm35418_ioctl_argument](#) *ioctl_request)
Perform a DMA operation.

6.15.1 Detailed Description

Structures for the DM35418 Board Access Library.

Id

[dm35418_board_access.h](#) 104840 2016-11-30 19:20:54Z rgroner

6.15.2 Macro Definition Documentation

6.15.2.1 DM35418LIB_API

```
#define DM35418LIB_API
```

Conditionally set up the library export symbol for the Windows DLL. This will expand to nothing when compiled for Linux.

Definition at line 47 of file dm35418_board_access.h.

6.16 include/dm35418_board_access_structs.h File Reference

Structures for the DM35418 Board Access Library.

Data Structures

- struct [dm35418_pci_access_request](#)
PCI region access request descriptor. This structure holds information about a request to read data from or write data to one of a device's PCI regions.
- struct [dm35418_ioctl_region_readwrite](#)
ioctl() request structure for read from or write to PCI region
- struct [dm35418_ioctl_region_modify](#)
ioctl() request structure for PCI region read/modify/write
- struct [dm35418_ioctl_interrupt_info_request](#)
ioctl() request structure for interrupt
- struct [dm35418_ioctl_dma](#)
ioctl() request structure for DMA
- union [dm35418_ioctl_argument](#)
ioctl() request structure encapsulating all possible requests. This is what gets passed into the kernel from user space on the ioctl() call.

Enumerations

- enum [dm35418_pci_region_num](#) { [DM35418_PCI_REGION_GBC](#) = 0 , [DM35418_PCI_REGION_GBC2](#) , [DM35418_PCI_REGION_FB](#) }
Standard PCI region number.
- enum [dm35418_pci_region_access_size](#) { [DM35418_PCI_REGION_ACCESS_8](#) = 0 , [DM35418_PCI_REGION_ACCESS_16](#) , [DM35418_PCI_REGION_ACCESS_32](#) }
Desired size in bits of access to standard PCI region.
- enum [DM35418_DMA_FUNCTIONS](#) { [DM35418_DMA_INITIALIZE](#) , [DM35418_DMA_READ](#) , [DM35418_DMA_WRITE](#) }

6.16.1 Detailed Description

Structures for the DM35418 Board Access Library.

Id

[dm35418_board_access_structs.h](#) 80523 2014-07-17 18:38:59Z rgroner

6.17 include/dm35418_dac_library.h File Reference

Definitions for the DM35418 DAC Library.

Macros

- #define [DM35418_DAC_INT_CONVERSION_SENT_MASK](#) 0x01
Register value for Interrupt Mask - Conversion Sent.
- #define [DM35418_DAC_INT_CHAN_MARKER_MASK](#) 0x02
Register value for Interrupt Mask - Channel has enabled marker.
- #define [DM35418_DAC_INT_START_TRIG_MASK](#) 0x08
Register value for Interrupt Mask - Start Trigger Occurred.
- #define [DM35418_DAC_INT_STOP_TRIG_MASK](#) 0x10
Register value for Interrupt Mask - Stop Trigger Occurred.
- #define [DM35418_DAC_INT_POST_STOP_DONE_MASK](#) 0x20
Register value for Interrupt Mask - Post-Stop Conversions Completed.
- #define [DM35418_DAC_INT_PACER_TICK_MASK](#) 0x80
Register value for Interrupt Mask - Pacer Clock Tick.
- #define [DM35418_DAC_INT_ALL_MASK](#) 0xBB
Register value for Interrupt Mask - All Bits.
- #define [DM35418_DAC_MODE_RESET](#) 0x00
Register value for Mode - Reset.
- #define [DM35418_DAC_MODE_PAUSE](#) 0x01
Register value for Mode - Pause.
- #define [DM35418_DAC_MODE_GO_SINGLE_SHOT](#) 0x02
Register value for Mode - Go (Single Shot)
- #define [DM35418_DAC_MODE_GO_REARM](#) 0x03
Register value for Mode - Go (Re-arm)
- #define [DM35418_DAC_STATUS_STOPPED](#) 0x00
Register value for DAC Status - Stopped.
- #define [DM35418_DAC_STATUS_WAITING_START_TRIG](#) 0x02
Register value for DAC Status - Waiting for Start Trigger.
- #define [DM35418_DAC_STATUS_CONVERTING](#) 0x03
Register value for DAC Status - Converting Data.
- #define [DM35418_DAC_STATUS_OUTPUT_POST](#) 0x04
Register value for DAC Status - Outputting Post-Stop conversions.
- #define [DM35418_DAC_STATUS_WAITING_REARM](#) 0x05
Register value for DAC Status - Waiting for Re-Arm.
- #define [DM35418_DAC_STATUS_DONE](#) 0x07
Register value for DAC Status - Done.
- #define [DM35418_DAC_FE_CONFIG_GAIN_1](#) 0x00

- Register value for setting a Gain of 1.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_2` 0x01
- Register value for setting a Gain of 2.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_4` 0x02
- Register value for setting a Gain of 4.*
 - #define `DM35418_DAC_FE_CONFIG_GAIN_MASK` 0x03
- Bit Mask for the gain bits of the FE Config.*
 - #define `DM35418_DAC_MAX_RATE` 1000000
- Max allowable rate for the DAC (Hz)*
 - #define `DM35418_DAC_BIPOLAR_MIN` -32768
- DAC Min value in Bipolar mode.*
 - #define `DM35418_DAC_BIPOLAR_MAX` 32767
- DAC Max value in Bipolar mode.*
 - #define `DM35418_DAC_RNG_5_LSB` 0.0000762939453125f
- What each bit is worth at a range of 5.*
 - #define `DM35418_DAC_RNG_10_LSB` 0.000152587890625f
- What each bit is worth at a range of 10.*
 - #define `DM35418_DAC_RNG_20_LSB` 0.00030517578125f
- What each bit is worth at a range of 20.*

Enumerations

- enum `DM35418_Dac_Clock_Events` {
`DM35418_DAC_CLK_BUS_SRC_DISABLE` = 0x00 , `DM35418_DAC_CLK_BUS_SRC_CONVERSION_SENT`
 = 0x80 , `DM35418_DAC_CLK_BUS_SRC_CHAN_MARKER` = 0x81 , `DM35418_DAC_CLK_BUS_SRC_START_TRIG`
 = 0x83 ,
`DM35418_DAC_CLK_BUS_SRC_STOP_TRIG` = 0x84 , `DM35418_DAC_CLK_BUS_SRC_CONV_COMPL`
 = 0x85 }
Clocking events that can be used as the global clock sources.
- enum `DM35418_Output_Ranges` { `DM35418_DAC_RNG_BIPOLAR_2_5V` , `DM35418_DAC_RNG_BIPOLAR_5V`
 , `DM35418_DAC_RNG_BIPOLAR_10V` }
Output range of the DAC pin.

Functions

- `DM35418LIB_API int DM35418_Dac_Open` (struct `DM35418_Board_Descriptor` *handle, unsigned int
 number_of_type, struct `DM35418_Function_Block` *func_block)
Open the DAC indicated, and determine register locations of control blocks needed to control it.
- `DM35418LIB_API int DM35418_Dac_Set_Clock_Src` (struct `DM35418_Board_Descriptor` *handle, const
 struct `DM35418_Function_Block` *func_block, enum `DM35418_Clock_Sources` source)
Set the clock source of the DAC.
- `DM35418LIB_API int DM35418_Dac_Get_Clock_Src` (struct `DM35418_Board_Descriptor` *handle, const
 struct `DM35418_Function_Block` *func_block, enum `DM35418_Clock_Sources` *source)
Get the clock source of the DAC.
- `DM35418LIB_API int DM35418_Dac_Get_Clock_Div` (struct `DM35418_Board_Descriptor` *handle, const
 struct `DM35418_Function_Block` *func_block, uint32_t *divider)
Get the clock divider value.
- `DM35418LIB_API int DM35418_Dac_Set_Clock_Div` (struct `DM35418_Board_Descriptor` *handle, const
 struct `DM35418_Function_Block` *func_block, uint32_t divider)
Set the clock divider value.

- [DM35418LIB_API](#) int [DM35418_Dac_Set_Conversion_Rate](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t requested_rate, uint32_t *actual_rate)
Set the conversion rate of this DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Interrupt_Set_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t interrupt_src, int enable)
Set the interrupt configuration for this DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Interrupt_Get_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *interrupt_ena)
Get the interrupt configuration for this DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Start_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t trigger_value)
Set the start trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Stop_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t trigger_value)
Set the stop trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Start_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *trigger_value)
Get the start trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Stop_Trigger](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *trigger_value)
Get the stop trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Start](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the DAC Mode to Start.
- [DM35418LIB_API](#) int [DM35418_Dac_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the DAC Mode to Reset.
- [DM35418LIB_API](#) int [DM35418_Dac_Pause](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block)
Set the DAC Mode to Pause.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Mode_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint8_t *mode_status)
Get the Mode and Status of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Last_Conversion](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker, int16_t *value)
Get the value of the last conversion of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Last_Conversion](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t marker, int16_t value)
Set a value to be converted by the DAC immediately.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Conversion_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get a count of the number of conversions that DAC has executed.
- [DM35418LIB_API](#) int [DM35418_Dac_Interrupt_Get_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t *value)
Get a interrupt status register of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Interrupt_Clear_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint16_t value)
Clear the interrupt status register of the DAC.
- [DM35418LIB_API](#) int [DM35418_Dac_Set_Post_Stop_Conversion_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t value)
Set the number of conversions the DAC will make after a stop trigger.
- [DM35418LIB_API](#) int [DM35418_Dac_Get_Post_Stop_Conversion_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)

Get the number of conversions the DAC will make after a stop trigger.

- [DM35418LIB_API](#) int [DM35418_Dac_Set_Clock_Source_Global](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, enum [DM35418_Clock_Sources](#) clock, enum [DM35418_Dac_Clock_Events](#) clock_driver)

Set the source that will drive the global clock.

- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, enum [DM35418_Output_Ranges](#) output_range)

Setup the selected DAC channel.

- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Reset](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel)

Reset the DAC channel, by writing all zeros to the front-end config.

- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Set_Marker_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_enable)

Set the configuration of the marker interrupts for this channel.

- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Get_Marker_Config](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_enable)

Get the configuration of the marker interrupts for this channel.

- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Get_Marker_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t *marker_status)

Get the status of the marker interrupts for this channel.

- [DM35418LIB_API](#) int [DM35418_Dac_Channel_Clear_Marker_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint8_t marker_to_clear)

Clear the marker interrupts for this channel.

- [DM35418LIB_API](#) int [DM35418_Dac_Fifo_Channel_Write](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int32_t value)

Write a value to the onboard FIFO.

- [DM35418LIB_API](#) int [DM35418_Dac_Volts_To_Conv](#) (enum [DM35418_Output_Ranges](#) output_range, float volts, int16_t *dac_conversion)

Convert a value in volts to a DAC equivalent signed value.

- [DM35418LIB_API](#) int [DM35418_Dac_Conv_To_Volts](#) (enum [DM35418_Output_Ranges](#) output_range, int16_t conversion, float *volts)

Convert a DAC conversion value to volts.

6.17.1 Detailed Description

Definitions for the DM35418 DAC Library.

Id

[dm35418_dac_library.h](#) 106898 2017-03-08 13:44:23Z rgroner

6.18 include/dm35418_dio_library.h File Reference

Definitions for the DM35418 DIO Library.

```
#include "dm35418_gbc_library.h"
```

Functions

- [DM35418LIB_API](#) int [DM35418_Dio_Open](#) (struct [DM35418_Board_Descriptor](#) *handle, unsigned int number_of_type, struct [DM35418_Function_Block](#) *func_block)
Open the DIO indicated, and determine register locations of control blocks needed to control it.
- [DM35418LIB_API](#) int [DM35418_Dio_Set_Direction](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t direction)
Set the direction of the DIO pins.
- [DM35418LIB_API](#) int [DM35418_Dio_Get_Direction](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *direction)
Get the direction of the DIO pins.
- [DM35418LIB_API](#) int [DM35418_Dio_Get_Input_Value](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get the input value of the DIO.
- [DM35418LIB_API](#) int [DM35418_Dio_Get_Output_Value](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t *value)
Get the current value of the output register.
- [DM35418LIB_API](#) int [DM35418_Dio_Set_Output_Value](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, uint32_t value)
Set the value to be put on output pins.

6.18.1 Detailed Description

Definitions for the DM35418 DIO Library.

Id

[dm35418_dio_library.h](#) 84663 2014-12-19 22:03:35Z mmcintire

6.19 include/dm35418_dma_library.h File Reference

Definitions for the DM35418 DMA Library.

```
#include <stdint.h>
```

Macros

- [#define](#) [DM35418_DMA_ACTION_CLEAR](#) 0x00
Register value for DMA clear action.
- [#define](#) [DM35418_DMA_ACTION_GO](#) 0x01
Register value for DMA go action.
- [#define](#) [DM35418_DMA_ACTION_PAUSE](#) 0x02
Register value for DMA pause action.
- [#define](#) [DM35418_DMA_ACTION_HALT](#) 0x03
Register value for DMA halt action.
- [#define](#) [DM35418_DMA_SETUP_DIRECTION_READ](#) 0x04
Register value to set DMA to READ direction.
- [#define](#) [DM35418_DMA_SETUP_DIRECTION_WRITE](#) 0x00

- Register value to set DMA to WRITE direction.*

 - #define [DM35418_DMA_SETUP_DIRECTION_MASK](#) 0x04
- Register value to set DMA to READ direction.*

 - #define [DM35418_DMA_SETUP_IGNORE_USED](#) 0x08
- Register value to tell DMA to ignore used buffers.*

 - #define [DM35418_DMA_SETUP_NOT_IGNORE_USED](#) 0x00
- Register value to tell DMA to not ignore used buffers.*

 - #define [DM35418_DMA_SETUP_IGNORE_USED_MASK](#) 0x08
- Bit mask for Ignore Used bit in setup register.*

 - #define [DM35418_DMA_SETUP_INT_ENABLE](#) 0x01
- Register value to enabled interrupts in the setup register.*

 - #define [DM35418_DMA_SETUP_INT_DISABLE](#) 0x00
- Register value to disable interrupts in the setup register.*

 - #define [DM35418_DMA_SETUP_INT_MASK](#) 0x01
- Bit mask for the interrupt bit in the setup register.*

 - #define [DM35418_DMA_SETUP_ERR_INT_ENABLE](#) 0x02
- Register value to enable the error interrupt.*

 - #define [DM35418_DMA_SETUP_ERR_INT_DISABLE](#) 0x00
- Register value to disable the error interrupt.*

 - #define [DM35418_DMA_SETUP_ERR_INT_MASK](#) 0x02
- Bit mask for the error interrupt bit in the setup register.*

 - #define [DM35418_DMA_STATUS_CLEAR](#) 0x00
- Register value to write to status registers to clear them.*

 - #define [DM35418_DMA_CTRL_CLEAR](#) 0x00
- Register value to write to control register to clear it.*

 - #define [DM35418_DMA_BUFFER_STATUS_CLEAR](#) 0x00
- Register value to write to the buffer status register to clear it.*

 - #define [DM35418_DMA_BUFFER_CTRL_CLEAR](#) 0x00
- Register value to write to the buffer control register to clear it.*

 - #define [DM35418_DMA_BUFFER_STATUS_USED_MASK](#) 0x01
- Bit mask for the used buffer bit in the buffer status register.*

 - #define [DM35418_DMA_BUFFER_STATUS_TERM_MASK](#) 0x02
- Bit mask for the terminated buffer bit in the buffer status register.*

 - #define [DM35418_DMA_BUFFER_CTRL_VALID](#) 0x01
- Register value to write to buffer control register to mark it as valid.*

 - #define [DM35418_DMA_BUFFER_CTRL_HALT](#) 0x02
- Register value to write to buffer control register to tell DMA to halt after processing this buffer.*

 - #define [DM35418_DMA_BUFFER_CTRL_LOOP](#) 0x04
- Register value to write to buffer control register to tell DMA to loop back to buffer 0 after using this buffer.*

 - #define [DM35418_DMA_BUFFER_CTRL_INTR](#) 0x08
- Register value to write to buffer control register to tell DMA to issue an interrupt after using this buffer.*

 - #define [DM35418_DMA_BUFFER_CTRL_PAUSE](#) 0x10
- Register value to write to buffer control register to tell DMA to pause after processing this buffer.*

 - #define [DM35418_DMA_CTRL_BLOCK_SIZE](#) 0x10
- Constant value indicating DMA control block size.*

 - #define [DM35418_DMA_BUFFER_CTRL_BLOCK_SIZE](#) 0x10
- Constant value indicating DMA buffer control block size.*

 - #define [DM35418_BIT_MASK_DMA_BUFFER_SIZE](#) 0xFFFFFFFF
- Bit mask for the DMA buffer size, since it is 24-bits of a 32-bit register.*

Enumerations

- enum [DM35418_Fifo_States](#) { [DM35418_FIFO_UNKNOWN](#) , [DM35418_FIFO_EMPTY](#) , [DM35418_FIFO_FULL](#) , [DM35418_FIFO_HAS_DATA](#) }

Descriptions of the possible states the FIFO might be in.

Functions

- [DM35418LIB_API](#) int [DM35418_Dma_Start](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)

Start the DMA.

- [DM35418LIB_API](#) int [DM35418_Dma_Stop](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)

Stop the DMA.

- [DM35418LIB_API](#) int [DM35418_Dma_Pause](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)

Pause the DMA.

- [DM35418LIB_API](#) int [DM35418_Dma_Clear](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel)

Clear the DMA.

- [DM35418LIB_API](#) int [DM35418_Dma_Get_Fifo_Counts](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint16_t *write_count, uint16_t *read_count)

Get the Read and Write FIFO count values.

- [DM35418LIB_API](#) int [DM35418_Dma_Get_Fifo_State](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, enum [DM35418_Fifo_States](#) *state)

Get the state of the FIFO.

- [DM35418LIB_API](#) int [DM35418_Dma_Configure_Interrupts](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int enable, int error_enable)

Configure the interrupts for the DMA channel.

- [DM35418LIB_API](#) int [DM35418_Dma_Get_Interrupt_Configuration](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int *enable, int *error_enable)

Get the configuration of the interrupts for the DMA channel.

- [DM35418LIB_API](#) int [DM35418_Dma_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int direction, int ignore_used)

Setup the DMA channel, specifically the direction and if used buffers are ignored.

- [DM35418LIB_API](#) int [DM35418_Dma_Setup_Set_Direction](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int direction)

Set the direction of the DMA, read or write.

- [DM35418LIB_API](#) int [DM35418_Dma_Setup_Set_Used](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int ignore_used)

Set the DMA channel to ignore or not ignore a used buffer. Ignoring used buffers is mostly useful when outputting a repeating data cycle.

- [DM35418LIB_API](#) int [DM35418_Dma_Get_Errors](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid)

Get the current value of the DMA channel error registers.

- [DM35418LIB_API](#) int [DM35418_Dma_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count, int *current_action, int *stat_overflow, int *stat_underflow, int *stat_used, int *stat_invalid, int *stat_complete)

Get the current status of the DMA channel. Determine which buffer it is using, what its current action is, and the state of all error conditions and normal interrupt conditions.

- [DM35418LIB_API](#) int [DM35418_Dma_Get_Current_Buffer_Count](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, uint32_t *current_buffer, uint32_t *current_count)

Get the current buffer and buffer count in use by the DMA.

- [DM35418LIB_API](#) int [DM35418_Dma_Check_For_Error](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int *has_error)

Check the DMA channel for any error conditions. This just returns a simple boolean as quickly as possible. If there is an error condition, you will have to query the DMA again to determine what the error is.

- [DM35418LIB_API](#) int [DM35418_Dma_Buffer_Setup](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t ctrl)

Setup the DMA buffer for use.

- [DM35418LIB_API](#) int [DM35418_Dma_Buffer_Status](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer, uint8_t *status, uint8_t *control, uint32_t *size)

Get the status of the buffer. This gets the status, control, and size registers.

- [DM35418LIB_API](#) int [DM35418_Dma_Check_Buffer_Used](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer_num, int *is_used)

Check if the indicated buffer has the "Used" flag set.

- int [DM35418_Dma_Find_Interrupt](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int *channel, int *channel_complete, int *channel_error)

Find which DMA channel has an interrupt condition, whether from using a buffer with interrupt set, or from an error. DMA channels are evaluated starting at Channel 0.

- int [DM35418_Dma_Clear_Interrupt](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, int clear_overflow, int clear_underflow, int clear_used, int clear_invalid, int clear_complete)

Clear the interrupt flag from a DMA channel. Clearing the flags will allow another interrupt of the same type to occur again, and is the normal operation after handling the interrupt itself.

- int [DM35418_Dma_Reset_Buffer](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, unsigned int channel, unsigned int buffer)

Reset the DMA buffer, preparing it to be used again by the DMA engine.

6.19.1 Detailed Description

Definitions for the DM35418 DMA Library.

Id

[dm35418_dma_library.h](#) 105018 2016-12-07 15:47:31Z rgroner

6.20 include/dm35418_driver.h File Reference

Structures and defines for the DM35418 driver module.

```
#include <linux/pci.h>
#include <linux/spinlock.h>
#include <linux/types.h>
```

Data Structures

- struct [dm35418_pci_region](#)
DM35418 PCI region descriptor. This structure holds information about one of a device's PCI memory regions.
- struct [dm35418_dma_descriptor](#)
DM35418 DMA descriptor. This structure holds information about a single DMA buffer.
- struct [dm35418_device_descriptor](#)
DM35418 Device Descriptor. The identifying info for this particular board.

Macros

- #define [DM35418_MAX_NUM_DEVICES](#) 8
DM35418 Maximum number of devices.
- #define [DM35418_NUM_MINORS_PER_DEVICE](#) 1
DM35418 Number of minors per device.
- #define [DM35418_NAME_LENGTH](#) 200
DM35418 Max possible board name length.
- #define [DM35418_PCI_NUM_REGIONS](#) PCI_ROM_RESOURCE
Number of standard PCI regions.
- #define [DM35418_INT_QUEUE_SIZE](#) 256
Number of interrupts to hold in a queue for processing.

Enumerations

- enum [dm35418_pci_region_access_dir](#) { [DM35418_PCI_REGION_ACCESS_READ](#) = 0, [DM35418_PCI_REGION_ACCESS_WRITE](#) }
Direction of access to standard PCI region.

Variables

- static struct file_operations [dm35418_file_ops](#)
Placeholder prototype for file ops struct.

6.20.1 Detailed Description

Structures and defines for the DM35418 driver module.

Id

[dm35418_driver.h](#) 141380 2024-02-29 20:50:59Z lfrankenfield

6.21 include/dm35418_examples.h File Reference

Defines for the DM35418 Example programs. Commonly used constants for the example programs included with the software package.

Macros

- `#define BUFFER_VALID 1`
Boolean indicating buffer valid.
- `#define BUFFER_NO_VALID 0`
Boolean indicating buffer not valid.
- `#define BUFFER_HALT 1`
Boolean indicating buffer halt set.
- `#define BUFFER_NO_HALT 0`
Boolean indicating buffer halt not set.
- `#define BUFFER_LOOP 1`
Boolean indicating buffer loop set.
- `#define BUFFER_NO_LOOP 0`
Boolean indicating buffer loop not set.
- `#define BUFFER_INTERRUPT 1`
Boolean indicating buffer interrupt.
- `#define BUFFER_NO_INTERRUPT 0`
Boolean indicating no buffer interrupt.
- `#define BUFFER_PAUSE 1`
Boolean indicating buffer should pause when filled.
- `#define BUFFER_NO_PAUSE 0`
Boolean indicating buffer should not pause when filled.
- `#define IGNORE_USED 1`
Boolean indicating ignore used buffers.
- `#define NOT_IGNORE_USED 0`
Boolean indicating not ignore used buffers.
- `#define CLEAR_INTERRUPT 1`
Boolean indicating to clear an interrupt.
- `#define NO_CLEAR_INTERRUPT 0`
Boolean indicating to not clear an interrupt.
- `#define INTERRUPT_ENABLE 1`
Boolean indicating interrupt enable.
- `#define INTERRUPT_DISABLE 0`
Boolean indicating interrupt disable.
- `#define ERROR_INTR_ENABLE 1`
Boolean indicating error interrupt enable.
- `#define ERROR_INTR_DISABLE 0`
Boolean indicating error interrupt disable.
- `#define SYNCBUS_NONE 0`
Value indicating no Syncbus option was chosen.
- `#define SYNCBUS_MASTER 1`
Value indicating Syncbus Master was chosen.
- `#define SYNCBUS_SLAVE 2`
Value indicating Syncbus Slave was chosen.
- `#define CHANNEL_0 0`
Constant for selecting Channel 0.
- `#define CHANNEL_1 1`
Constant for selecting Channel 1.
- `#define CHANNEL_2 2`
Constant for selecting Channel 2.
- `#define CHANNEL_3 3`

- Constant for selecting Channel 3.*
- #define [BUFFER_0](#) 0
 - Constant for selecting Buffer 0.*
- #define [BUFFER_1](#) 1
 - Constant for selecting Buffer 1.*
- #define [ADC_0](#) 0
 - Constant for selecting ADC 0.*
- #define [ADC_1](#) 1
 - Constant for selecting ADC 1.*
- #define [DAC_0](#) 0
 - Constant for selecting DAC 0.*
- #define [DAC_1](#) 1
 - Constant for selecting DAC 1.*
- #define [DAC_2](#) 2
 - Constant for selecting DAC 2.*
- #define [DAC_3](#) 3
 - Constant for selecting DAC 3.*
- #define [REF_0](#) 0
 - Constant for selecting REF 0.*
- #define [REF_1](#) 1
 - Constant for selecting REF 1.*
- #define [DIO_0](#) 0
 - Constant for selecting DIO 0.*
- #define [ADIO_0](#) 0
 - Constant for selecting ADIO 0.*
- #define [ENABLED](#) 1
 - Constant to indicate an Enabled value.*
- #define [DISABLED](#) 0
 - Constant to indicate a Disabled value.*

Enumerations

- enum [Help_Options](#) {
 - [HELP_OPTION](#) = 1 , [MINOR_OPTION](#) , [RATE_OPTION](#) , [CHANNELS_OPTION](#) ,
 - [FILE_OPTION](#) , [START_OPTION](#) , [WAVE_OPTION](#) , [TEST_OPTION](#) ,
 - [NOSTOP_OPTION](#) , [SYNCBUS_OPTION](#) , [DUMP_OPTION](#) , [HOURS_OPTION](#) ,
 - [OUTPUT_RMS_OPTION](#) , [OUTPUT_ADC_OPTION](#) , [ADC_NUM_OPTION](#) , [DAC_NUM_OPTION](#) ,
 - [ADC_OPTION](#) , [DAC_OPTION](#) , [PATTERN_OPTION](#) , [SAMPLES_OPTION](#) ,
 - [MODE_OPTION](#) , [AD_MODE_OPTION](#) , [REF_NUM_OPTION](#) , [BINARY_OPTION](#) ,
 - [SENDER_OPTION](#) , [RECEIVER_OPTION](#) , [RANGE_OPTION](#) , [REFILL_FIFO_OPTION](#) ,
 - [LOW_THRESHOLD_OPTION](#) , [PORT_OPTION](#) , [BAUD_OPTION](#) , [EXTERNAL_OPTION](#) ,
 - [SIZE_OPTION](#) , [VERBOSE_OPTION](#) , [USER_ID_OPTION](#) , [COUNT_OPTION](#) ,
 - [NUM_OPTION](#) , [SYNC_TERM_OPTION](#) , [BIN2TXT_OPTION](#) , [STORE_OPTION](#) ,
 - [TERM_OPTION](#) , [REFCLK_OPTION](#) , [OFILE_OPTION](#) , [PACKED_OPTION](#) ,
 - [MASTER_OPTION](#) , [SLAVE_OPTION](#) , [SYNC_CONN_OPTION](#) }
 - Constants used for parsing command line parameters of example programs.*

6.21.1 Detailed Description

Defines for the DM35418 Example programs. Commonly used constants for the example programs included with the software package.

Id

[dm35418_examples.h](#) 114740 2018-07-12 14:41:17Z prucz

6.22 include/dm35418_gbc_library.h File Reference

Definitions for the DM35418 Board Library, a library for accessing the board registers.

```
#include <stdint.h>
#include <time.h>
#include "dm35418_board_access.h"
```

Macros

- `#define CLK_40MHZ 40000000`

Functions

- [DM35418LIB_API](#) `int DM35418_Gbc_Board_Reset` (struct [DM35418_Board_Descriptor](#) *handle)
Write the reset value to the correct register to initiate a board-level reset.
- [DM35418LIB_API](#) `int DM35418_Gbc_Ack_Interrupt` (struct [DM35418_Board_Descriptor](#) *handle)
Send an End-Of-Interrupt acknowledgement to the board. This will cause any pending interrupts to re-issue. This is a protection against missing interrupts while in the interrupt handler.
- [DM35418LIB_API](#) `int DM35418_Function_Block_Open` (struct [DM35418_Board_Descriptor](#) *handle, unsigned int number, struct [DM35418_Function_Block](#) *func_block)
Open a specific function block. Nothing is opened in a file sense, but the memory location for the function block is read and certain important values are read. A function block descriptor is allocated to hold the data that will be used every time this function block is accessed.
- [DM35418LIB_API](#) `int DM35418_Function_Block_Open_Module` (struct [DM35418_Board_Descriptor](#) *handle, uint32_t fb_type, unsigned int number_of_type, struct [DM35418_Function_Block](#) *func_block)
Open a specific function block module. This is the same as opening a function block, except we are looking for a function block with a specific type. This is the method you would use to open the 2nd ADC, for example.
- [DM35418LIB_API](#) `int DM35418_Gbc_Get_Format` (struct [DM35418_Board_Descriptor](#) *handle, uint8_t *format_id)
Get the format ID of the board.
- [DM35418LIB_API](#) `int DM35418_Gbc_Get_Revision` (struct [DM35418_Board_Descriptor](#) *handle, uint8_t *rev)
Get the PDP revision number of the board.
- [DM35418LIB_API](#) `int DM35418_Gbc_Get_Pdp_Number` (struct [DM35418_Board_Descriptor](#) *handle, uint32_t *pdp_num)
Get PDP Number of the board.
- [DM35418LIB_API](#) `int DM35418_Gbc_Get_Fpga_Build` (struct [DM35418_Board_Descriptor](#) *handle, uint32_t *fpga_build)
Get the FPGA Build number of the board.
- [DM35418LIB_API](#) `int DM35418_Gbc_Get_Sys_Clock_Freq` (struct [DM35418_Board_Descriptor](#) *handle, uint32_t *clock_freq, int *is_std_clk)
Get the measured frequency of the system clock of the board.

6.22.1 Detailed Description

Definitions for the DM35418 Board Library, a library for accessing the board registers.

Id

[dm35418_gbc_library.h](#) 103741 2016-10-17 20:35:58Z rgroner

6.23 include/dm35418_ioctl.h File Reference

DM35418 Low level ioctl() request descriptor structure and request code definitions.

```
#include <linux/types.h>
#include <linux/ioctl.h>
```

Macros

- `#define DM35418_IOCTL_MAGIC 'D'`
Unique 8-bit value used to generate unique ioctl() request codes.
- `#define DM35418_IOCTL_REQUEST_BASE 0x00`
First ioctl() request number.
- `#define DM35418_IOCTL_REGION_READ`
ioctl() request code for reading from a PCI region
- `#define DM35418_IOCTL_REGION_WRITE`
ioctl() request code for writing to a PCI region
- `#define DM35418_IOCTL_REGION_MODIFY`
ioctl() request code for PCI region read/modify/write
- `#define DM35418_IOCTL_DMA_FUNCTION`
ioctl() request code for DMA function
- `#define DM35418_IOCTL_WAKEUP`
ioctl() request code for User ISR thread wake up
- `#define DM35418_IOCTL_INTERRUPT_GET`
ioctl() request code to retrieve interrupt status information

6.23.1 Detailed Description

DM35418 Low level ioctl() request descriptor structure and request code definitions.

Id

[dm35418_ioctl.h](#) 80523 2014-07-17 18:38:59Z rgroner

6.24 include/dm35418_os.h File Reference

Function declarations for the DM35418 that are Linux specific.

```
#include <pthread.h>
```

Data Structures

- struct [DM35418_Board_Descriptor](#)
DM35418 board descriptor. This structure holds information about the board as a whole. It holds the file descriptor and ISR callback function, if applicable.

Functions

- `int DM35418_Dma_Initialize` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel, unsigned int num_buffers, uint32_t buffer_size)
Initialize the DMA channel and prepare it for data. Interrupts are disabled, error conditions are cleared, buffers are allocated in kernel space and their status and controls are cleared.
- `int DM35418_Dma_Read` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel, unsigned int buffer_to_read_from, uint32_t buffer_size, void *local_buffer_ptr)
Read data from the DMA buffer. Data is copied from kernel buffers to local user-space buffers.
- `int DM35418_Dma_Write` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, unsigned int channel, unsigned int buffer_to_write_to, uint32_t buffer_size, void *local_buffer_ptr)
Write data to the DMA buffer. Data is copied from local user buffers to kernel buffers.
- `int DM35418_General_RemoveISR` (struct `DM35418_Board_Descriptor` *handle)
Remove the ISR from the system interrupt.
- `void * DM35418_General_WaitForInterrupt` (void *ptr)
Loop/Poll and wait for an interrupt to happen, then take action.
- `int DM35418_General_InstallISR` (struct `DM35418_Board_Descriptor` *handle, void(*isr_fnct))
Start a thread that will sit and wait for an interrupt from the board, and call the user ISR when it happens.
- `int DM35418_General_SetISRPriority` (struct `DM35418_Board_Descriptor` *handle, int priority)
Set the priority of the user ISR thread.

6.24.1 Detailed Description

Function declarations for the DM35418 that are Linux specific.

Id

[dm35418_os.h](#) 109114 2017-06-09 07:13:20Z prucz

6.25 include/dm35418_registers.h File Reference

Defines for the DM35418 Registers (Offsets)

Macros

- `#define DM35418_OFFSET_GBC_FORMAT` 0x00
Offset to General Board Control (BAR0) Format ID register.
- `#define DM35418_OFFSET_GBC_REV` 0x01
Offset to General Board Control (BAR0) Format ID register.
- `#define DM35418_OFFSET_GBC_END_INTERRUPT` 0x02
Offset to General Board Control (BAR0) EOI (End of Interrupt) register.
- `#define DM35418_OFFSET_GBC_BOARD_RESET` 0x03
Offset to General Board Control (BAR0) Board Reset register.
- `#define DM35418_OFFSET_GBC_PDP_NUMBER` 0x04
Offset to General Board Control (BAR0) PDP Number register.
- `#define DM35418_OFFSET_GBC_FPGA_BUILD` 0x08
Offset to General Board Control (BAR0) FPGA Build register.

- `#define DM35418_OFFSET_GBC_SYS_CLK_FREQ 0x0c`
Offset to General Board Control (BAR0) System Clock register.
- `#define DM35418_OFFSET_GBC_IRQ_STATUS 0x10`
Offset to General Board Control (BAR0) IRQ Status register. Each bit corresponds to a function block.
- `#define DM35418_OFFSET_GBC_DMA_IRQ_STATUS 0x18`
Offset to General Board Control (BAR0) DMA IRQ Status register. Each bit corresponds to a function block.
- `#define DM35418_OFFSET_GBC_FB_START 0x20`
Offset to the beginning of the Function Blocks section of the GBC.
- `#define DM35418_GBC_FB_BLK_SIZE 0x10`
Size of the function block entries in the GBC.
- `#define DM35418_OFFSET_GBC_FB_ID 0x00`
Offset to Function Block ID, from the start of the function block section.
- `#define DM35418_FB_ID_TYPE_MASK 0x0000FFFF`
Bit mask for TYPE portion of FB ID.
- `#define DM35418_FB_ID_SUBTYPE_MASK 0x00FF0000`
Bit mask for SUBTYPE portion of FB ID.
- `#define DM35418_FB_ID_TYPE_REV_MASK 0xFF000000`
Bit mask for TYPE REV portion of FB ID.
- `#define DM35418_OFFSET_GBC_FB_OFFSET 0x04`
Offset to the FB Offset in the GBC, from the start of the FB data block.
- `#define DM35418_OFFSET_GBC_FB_DMA_OFFSET 0x08`
Offset to the FB DMA Offset in the GBC, from the start of the FB data block.
- `#define DM35418_OFFSET_DMA_ACTION 0x00`
Offset to the DMA Action Register (BAR2)
- `#define DM35418_OFFSET_DMA_SETUP 0x01`
Offset to the DMA Setup Register (BAR2)
- `#define DM35418_OFFSET_DMA_STAT_OVERFLOW 0x02`
Offset to the DMA Status (Overflow) Register (BAR2)
- `#define DM35418_OFFSET_DMA_STAT_UNDERFLOW 0x03`
Offset to the DMA Status (Underflow) Register (BAR2)
- `#define DM35418_OFFSET_DMA_CURRENT_COUNT 0x04`
Offset to the DMA Current Count Register (BAR2)
- `#define DM35418_OFFSET_DMA_CURRENT_BUFFER 0x07`
Offset to the DMA Current Buffer Register (BAR2)
- `#define DM35418_OFFSET_DMA_WR_FIFO_CNT 0x08`
Offset to the DMA Write FIFO Count Register (BAR2)
- `#define DM35418_OFFSET_DMA_RD_FIFO_CNT 0x0A`
Offset to the DMA Read FIFO Count Register (BAR2)
- `#define DM35418_OFFSET_DMA_STAT_USED 0x0C`
Offset to the DMA Status (Used) Register (BAR2)
- `#define DM35418_OFFSET_DMA_STAT_INVALID 0x0D`
Offset to the DMA Status (Invalid) Register (BAR2)
- `#define DM35418_OFFSET_DMA_STAT_COMPLETE 0x0E`
Offset to the DMA Status (Complete) Register (BAR2)
- `#define DM35418_OFFSET_DMA_LAST_ACTION 0x0F`
Offset to the DMA Last Action Register (BAR2)
- `#define DM35418_OFFSET_DMA_BUFF_START 0x10`
Offset to the start of the buffer control section (BAR2)
- `#define DM35418_OFFSET_DMA_BUFFER_STAT 0x02`
Offset to the buffer status register, from the start of the buffer control section (BAR2)
- `#define DM35418_OFFSET_DMA_BUFFER_CTRL 0x03`

- Offset to the buffer control register, from the start of the buffer control section (BAR2)*

 - #define [DM35418_OFFSET_DMA_BUFFER_SIZE](#) 0x04
- Offset to the buffer size register, from the start of the buffer control section (BAR2)*

 - #define [DM35418_OFFSET_DMA_BUFFER_ADDRESS](#) 0x08
- Offset to the buffer address register, from the start of the buffer control section (BAR2)*

 - #define [DM35418_OFFSET_FB_DMA_CHANNELS](#) 0x06
- Offset to the DMA Channels count of the function block (BAR2)*

 - #define [DM35418_OFFSET_FB_DMA_BUFFERS](#) 0x07
- Offset to the DMA buffers count of the function block (BAR2)*

 - #define [DM35418_OFFSET_FB_CTRL_START](#) 0x08
- Offset to the beginning of the Function Block control section in BAR2.*

 - #define [DM35418_OFFSET_ADC_MODE_STATUS](#) 0x00
- Offset to the ADC Mode-Status register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_SRC](#) 0x01
- Offset to the ADC Clock Source register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_START_TRIG](#) 0x02
- Offset to the ADC Start Trigger register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_STOP_TRIG](#) 0x03
- Offset to the ADC Stop Trigger register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_DIV](#) 0x04
- Offset to the ADC Clock Divider register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_DIV_COUNTER](#) 0x08
- Offset to the ADC Clock Divider Counter register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_PRE_CAPT_COUNT](#) 0x0c
- Offset to the ADC Pre-Start Capture Count register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_POST_CAPT_COUNT](#) 0x10
- Offset to the ADC Post-Stop Capture Count register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_SAMPLE_COUNT](#) 0x14
- Offset to the ADC Sample Count register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_INT_ENABLE](#) 0x18
- Offset to the ADC Interrupt Enable register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_INT_STAT](#) 0x1e
- Offset to the ADC Interrupt Status register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_BUS2](#) 0x22
- Offset to the ADC Clock Bus 2, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_BUS3](#) 0x23
- Offset to the ADC Clock Bus 3 register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_BUS4](#) 0x24
- Offset to the ADC Clock Bus 4 register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_BUS5](#) 0x25
- Offset to the ADC Clock Bus 5 register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_BUS6](#) 0x26
- Offset to the ADC Clock Bus 6 register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CLK_BUS7](#) 0x27
- Offset to the ADC Clock Bus 7 register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_AD_CONFIG](#) 0x28
- Offset to the ADC AD Config register, from the start of the ADC control section.*

 - #define [DM35418_OFFSET_ADC_CHAN_CTRL_BLK_START](#) 0x2c
- Offset to the start of the Channel Control Section, from the start of the ADC control section.*

 - #define [DM35418_ADC_CHAN_CTRL_BLK_SIZE](#) 0x18
- Constant size of ADC channel section in function block.*

- `#define DM35418_OFFSET_ADC_CHAN_FRONT_END_CONFIG 0x00`
Offset to the Channel Front End Config register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_DATA_COUNT 0x04`
Offset to the Channel FIFO Data count register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_FILTER 0x09`
Offset to the Channel Filter register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_INTR_STAT 0x0a`
Offset to the Channel Interrupt Status register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_INTR_ENABLE 0x0b`
Offset to the Channel Interrupt Enable register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_LOW_THRESHOLD 0x0c`
Offset to the Channel Low Threshold register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_HIGH_THRESHOLD 0x10`
Offset to the Channel High Threshold register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_CHAN_LAST_SAMPLE 0x14`
Offset to the Channel Last Sample register, from the start of the ADC channel control section.
- `#define DM35418_OFFSET_ADC_FIFO_CTRL_BLK_START 0x334`
Offset to the start of the FIFO Control Section, from the start of the ADC control section.
- `#define DM35418_ADC_FIFO_CTRL_BLK_SIZE 0x4`
Constant size of ADC FIFO section in function block.
- `#define DM35418_OFFSET_FB_ADC_FIFO 0x0334`
Offset to the FIFO for non-DMA read and write operations.
- `#define DM35418_OFFSET_DAC_MODE_STATUS 0x00`
Offset to the Mode/Status register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_SRC 0x01`
Offset to the Clock Source register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_START_TRIG 0x02`
Offset to the Start Trigger register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_STOP_TRIG 0x03`
Offset to the Stop Trigger register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_DIV 0x04`
Offset to the Clock Divider register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_DIV_COUNT 0x08`
Offset to the Clock Divider Counter register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_POST_STOP_CONV 0x10`
Offset to the Post-Stop Conversion Count register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CONV_COUNT 0x14`
Offset to the Conversion Count register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_INT_ENABLE 0x18`
Offset to the Interrupt Enable register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_INT_STAT 0x1e`
Offset to the Interrupt Status register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_BUS2 0x22`
Offset to the Clock Bus 2 register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_BUS3 0x23`
Offset to the Clock Bus 3 register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_BUS4 0x24`
Offset to the Clock Bus 4 register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_BUS5 0x25`
Offset to the Clock Bus 5 register, from the start of the DAC control section.
- `#define DM35418_OFFSET_DAC_CLK_BUS6 0x26`

- Offset to the Clock Bus 6 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CLK_BUS7](#) 0x27
- Offset to the Clock Bus 7 register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_DA_CONFIG](#) 0x28
- Offset to the DA Config register, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_CHAN_CTRL_BLK_START](#) 0x2c
- Offset to the start of the DAC channel control section, from the start of the DAC control section.*

 - #define [DM35418_DAC_CHAN_CTRL_BLK_SIZE](#) 0x14
- Constant size of channel control section in function block.*

 - #define [DM35418_OFFSET_DAC_CHAN_FRONT_END_CONFIG](#) 0x00
- Offset to the Front-End Config register, from the start of the DAC channel control section.*

 - #define [DM35418_OFFSET_DAC_CHAN_MARKER_STATUS](#) 0x0a
- Offset to the Channel marker Interrupt Status register, from the start of the DAC channel control section.*

 - #define [DM35418_OFFSET_DAC_CHAN_MARKER_ENABLE](#) 0x0b
- Offset to the Channel marker Interrupt Enable register, from the start of the DAC channel control section.*

 - #define [DM35418_OFFSET_DAC_CHAN_LAST_CONVERSION](#) 0x10
- Offset to the Channel Last Conversion register, from the start of the DAC channel control section.*

 - #define [DM35418_OFFSET_DAC_FIFO_CTRL_BLK_START](#) 0x84
- Offset to the start of the DAC FIFO control section, from the start of the DAC control section.*

 - #define [DM35418_OFFSET_DAC_FIFO_CTRL_BLK_SIZE](#) 0x4
- Constant size of FIFO control section in function block.*

 - #define [DM35418_OFFSET_DIO_INPUT_VAL](#) 0x00
- Offset to the Input Value register, from the start of the DIO control section.*

 - #define [DM35418_OFFSET_DIO_OUTPUT_VAL](#) 0x04
- Offset to the Output Value register, from the start of the DIO control section.*

 - #define [DM35418_OFFSET_DIO_DIRECTION](#) 0x08
- Offset to the Direction register, from the start of the DIO control section.*

 - #define [DM35418_OFFSET_TEMPERATURE](#) 0x00
- Offset to the Temperature register, from the start of the Temperature control section.*

 - #define [DM35418_OFFSET_SYNCBUS_CLK_SEL](#) 0x00
- Offset to the Clock Select register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_PLL_LOCKED](#) 0x01
- Offset to the PLL Locked register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_TERM_ENABLE](#) 0x02
- Offset to the Termination Enable register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_ENABLE](#) 0x03
- Offset to the Enable register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_0_SRC](#) 0x04
- Offset to the Line 0 Source register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_1_SRC](#) 0x05
- Offset to the Line 1 Source register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_2_SRC](#) 0x06
- Offset to the Line 2 Source register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_DIRECTION](#) 0x07
- Offset to the Direction register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_CLK_BUS2](#) 0x0a
- Offset to the Clock Bus 2 register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_CLK_BUS3](#) 0x0b
- Offset to the Clock Bus 3 register, from the start of the SyncBus control section.*

 - #define [DM35418_OFFSET_SYNCBUS_CLK_BUS4](#) 0x0c
- Offset to the Clock Bus 4 register, from the start of the SyncBus control section.*

- `#define DM35418_OFFSET_SYNCBUS_CLK_BUS5 0x0d`
Offset to the Clock Bus 5 register, from the start of the SyncBus control section.
- `#define DM35418_OFFSET_SYNCBUS_CLK_BUS6 0x0e`
Offset to the Clock Bus 6 register, from the start of the SyncBus control section.
- `#define DM35418_OFFSET_SYNCBUS_CLK_BUS7 0x0f`
Offset to the Clock Bus 7 register, from the start of the SyncBus control section.

6.25.1 Detailed Description

Defines for the DM35418 Registers (Offsets)

Id

[dm35418_registers.h](#) 139902 2023-10-13 18:04:34Z lfrankenfield

6.26 include/dm35418_syncbus_library.h File Reference

Definitions for the DM35418 Sync Bus Library.

```
#include "dm35418_gbc_library.h"
```

Macros

- `#define DM35418_SYNCBUS_CLK_BUS_SRC_DISABLE 0x00`
Register value for Clock Event - Disable.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_0 0x80`
Register value for Clock Event - SyncBus 0.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_1 0x81`
Register value for Clock Event - SyncBus 1.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_SYNCBUS_2 0x82`
Register value for Clock Event - Syncbus 2.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_IMMEDIATE 0x00`
Register value for Clock Source - Immediate.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_NEVER 0x01`
Register value for Clock Source - Never.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL2 0x02`
Register value for Clock Source - Global 2.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL3 0x03`
Register value for Clock Source - Global 3.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL4 0x04`
Register value for Clock Source - Global 4.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL5 0x05`
Register value for Clock Source - Global 5.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL6 0x06`
Register value for Clock Source - Global 6.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL7 0x07`
Register value for Clock Source - Global 7.

- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL2_INV 0x0a`
Register value for Clock Source - Global 2 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL3_INV 0x0b`
Register value for Clock Source - Global 3 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL4_INV 0x0c`
Register value for Clock Source - Global 4 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL5_INV 0x0d`
Register value for Clock Source - Global 5 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL6_INV 0x0e`
Register value for Clock Source - Global 6 Inverted.
- `#define DM35418_SYNCBUS_CLK_BUS_SRC_GBL7_INV 0x0f`
Register value for Clock Source - Global 7 Inverted.
- `#define DM35418_SYNCBUS_0_CLK_SRC 0x00`
Value for selecting SyncBus 0 Clock Source.
- `#define DM35418_SYNCBUS_1_CLK_SRC 0x01`
Value for selecting SyncBus 1 Clock Source.
- `#define DM35418_SYNCBUS_2_CLK_SRC 0x02`
Value for selecting SyncBus 2 Clock Source.
- `#define DM35418_SYNCBUS_DIR_OUTPUT 1`
Value for setting syncbus line to output.
- `#define DM35418_SYNCBUS_DIR_INPUT 0`
Value for setting syncbus line to input.
- `#define DM35418_SYNCBUS_CLK_SEL_LOCAL 0`
Value for setting the clock select to local clock.
- `#define DM35418_SYNCBUS_CLK_SEL_SYNC 1`
Value for setting the clock select to syncbus clock.

Functions

- `DM35418LIB_API int DM35418_Syncbus_Open` (struct `DM35418_Board_Descriptor` *handle, unsigned int number, struct `DM35418_Function_Block` *func_block)
Open the Syncbus functional block, making it available for operations.
- `DM35418LIB_API int DM35418_Syncbus_Enable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Enable the Syncbus drivers.
- `DM35418LIB_API int DM35418_Syncbus_Disable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Disable the Syncbus drivers.
- `DM35418LIB_API int DM35418_Syncbus_Termination_Enable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Enable the 100 Ohm differential termination on the Syncbus.
- `DM35418LIB_API int DM35418_Syncbus_Termination_Disable` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block)
Disable the 100 Ohm differential termination on the Syncbus.
- `DM35418LIB_API int DM35418_Syncbus_Set_Direction` (struct `DM35418_Board_Descriptor` *handle, const struct `DM35418_Function_Block` *func_block, int dir_0, int dir_1, int dir_2, int dir_3)
Set the direction of the individual Syncbus lines. There must be only one board that has output direction enabled (the master), the other board should be configured as input (slave). The syncbus0..syncbus2 lines are data (trigger) lines, which can be connected to the internal clock bus. The syncbus3 is the 10MHz clock line. One of the boards must be the master, driving that line, the others are slave, synchronizing their PLL from the Syncbus clock.

- [DM35418LIB_API](#) int [DM35418_Syncbus_Set_Clk_Sel](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clkssel)
Set master clock source of the board. It can be either the local onboard oscillator (clkssel=0) or the syncbus clock line (clkssel=1, provided that the PLL is locked).
- [DM35418LIB_API](#) int [DM35418_Syncbus_Get_Clk_Sel](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int *clkssel)
Get master clock source of the board. It can be either the local onboard oscillator (clkssel=0) or the syncbus clock line (clkssel=1, provided that the PLL is locked).
- [DM35418LIB_API](#) int [DM35418_Syncbus_Get_PLL_Locked](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int *pll_locked)
Get the syncbus clock PLL lock status.
- [DM35418LIB_API](#) int [DM35418_Syncbus_Clock_Source_Global_Set](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clock_select, int clock_source)
Set a syncbus line as a source (input) to one of the internal global clock bus lines. The specified syncbus line must be set as input.
- [DM35418LIB_API](#) int [DM35418_Syncbus_Clock_Source_Global_Get](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int clock_select, int *clock_source)
Get the source syncbus line of the specified global clock bus line.
- [DM35418LIB_API](#) int [DM35418_Syncbus_Drive_Source_Set](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int syncbus_select, int clock_source)
Set the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.
- [DM35418LIB_API](#) int [DM35418_Syncbus_Drive_Source_Get](#) (struct [DM35418_Board_Descriptor](#) *handle, const struct [DM35418_Function_Block](#) *func_block, int syncbus_select, int *clock_source)
Get the source global clock bus line used to drive the specified syncbus line. The specified syncbus line must be set as output.

6.26.1 Detailed Description

Definitions for the DM35418 Sync Bus Library.

Id

[dm35418_syncbus_library.h](#) 60276 2012-06-05 16:04:15Z rgroner

6.27 include/dm35418_temperature_library.h File Reference

Definitions for the DM35418 Temperature Library.

```
#include "dm35418_gbc_library.h"
```

Functions

- [DM35418LIB_API](#) int [DM35418_Temperature_Open](#) (struct [DM35418_Board_Descriptor](#) *handle, unsigned int ordinal_to_open, struct [DM35418_Function_Block](#) *fb_temp)
Open the temperature function block, getting address values that will be used later by other library functions.
- [DM35418LIB_API](#) int [DM35418_Temperature_Read](#) (struct [DM35418_Board_Descriptor](#) *handle, struct [DM35418_Function_Block](#) *temp_fb, float *temperature)
Read the temperature of the board, in Celsius degrees.

6.27.1 Detailed Description

Definitions for the DM35418 Temperature Library.

Id

[dm35418_temperature_library.h](#) 60276 2012-06-05 16:04:15Z rgroner

6.28 include/dm35418_types.h File Reference

Defines for the DM35418. Values for the general board, not specific to a particular function block.

Macros

- `#define DM35418_SUBTYPE_00 0`
Constant for FB subtype 0.
- `#define DM35418_SUBTYPE_01 1`
Constant for FB subtype 1.
- `#define DM35418_SUBTYPE_02 2`
Constant for FB subtype 2.
- `#define DM35418_SUBTYPE_03 3`
Constant for FB subtype 3.
- `#define DM35418_SUBTYPE_INVALID 0xFF`
Constant value indicating an invalid subtype.
- `#define DM35418_FUNC_BLOCK_INVALID 0x0000`
Constant value indicating an invalid function block.
- `#define DM35418_FUNC_BLOCK_INVALID2 0xFFFF`
Constant value indicating an invalid function block.
- `#define DM35418_FUNC_BLOCK_SYNCBUS 0x0001`
Function Block Constant for SyncBus.
- `#define DM35418_FUNC_BLOCK_EXT_CLOCKING 0x0002`
Function Block Constant for Global Clocking.
- `#define DM35418_FUNC_BLOCK_CLK0003 0x0003`
Function Block Constant for External Clocking (0003)
- `#define DM35418_FUNC_BLOCK_CAPTWIN 0x0005`
Function Block Constant for Capture Window.
- `#define DM35418_FUNC_BLOCK_ADC 0x1000`
Function Block Constant for ADC.
- `#define DM35418_FUNC_BLOCK_ADC1001 0x1001`
Function Block Constant for 10 MHz ADC (1001)
- `#define DM35418_FUNC_BLOCK_DAC 0x2000`
Function Block Constant for DAC.
- `#define DM35418_FUNC_BLOCK_DAC2001 0x2001`
Function Block Constant for High Speed DAC (2001)
- `#define DM35418_FUNC_BLOCK_DIO 0x3000`
Function Block Constant for DIO.
- `#define DM35418_FUNC_BLOCK_ADIO 0x3001`
Function Block Constant for ADIO.
- `#define DM35418_FUNC_BLOCK_ADIO3010 0x3010`

- Function Block Constant for ADIO3010.*

 - #define `DM35418_FUNC_BLOCK_USART` 0x4000
- Function Block Constant for Synchronous/Asynchronous Serial Port.*

 - #define `DM35418_FUNC_BLOCK_REF_ADJUST` 0xF000
- Function Block Constant for Reference Adjustment.*

 - #define `DM35418_FUNC_BLOCK_TEMPERATURE_SENSOR` 0xF001
- Function Block Constant for Temperature Sensor.*

 - #define `DM35418_FUNC_BLOCK_FLASH_PROGRAMMER` 0xF002
- Function Block Constant for Module Firmware Programmer.*

 - #define `DM35418_FUNC_BLOCK_CLK_GEN` 0xF003
- Function Block Constant for Clock Generator.*

 - #define `DM35418_FUNC_BLOCK_DIN3011` 0x3011
- Function Block Constant for Digital Input (3011)*

 - #define `DM35418_FUNC_BLOCK_DOT3012` 0x3012
- Function Block Constant for Digital Output (3012)*

 - #define `DM35418_FUNC_BLOCK_INC3200` 0x3200
- Function Block Constant for Incremental Encoder (3200)*

 - #define `DM35418_FUNC_BLOCK_PWM3100` 0x3100
- Function Block Constant for PWM (3100)*

 - #define `DM35418_FUNC_BLOCK_CLK0004` 0x0004
- Function Block Constant for Programmable Clock (0004)*

 - #define `DM35418_FUNC_BLOCK_I2C3300` 0x3300
- Function Block Constant for I2C Bus Master (3300)*

 - #define `DM35418_MAX_FB` 62
- Maximum possible number of function blocks on a board.*

 - #define `MAX_DMA_BUFFERS` 16
- Maximum possible number of DMA buffers for any function block.*

 - #define `MAX_DMA_CHANNELS` 32
- Maximum possible number of DMA channels for any function block.*

 - #define `DM35418_DMA_MAX_BUFFER_SIZE` 0xFFFFFC
- Maximum possible DMA buffer size.*

 - #define `DM35418_BOARD_ACK_INTERRUPT` 0x1
- Value to write to the EOI register to acknowledge interrupts.*

 - #define `DM35418_BOARD_RESET_VALUE` 0xAA
- Value to write to the Reset register in order to reset the board.*

 - #define `DM35418_FIFO_ACCESS_FB_REVISION` 0x01
- Minimum function block revision that supports direct FIFO read/write access.*

Enumerations

- enum `DM35418_Clock_Sources` {
`DM35418_CLK_SRC_IMMEDIATE` , `DM35418_CLK_SRC_NEVER` , `DM35418_CLK_SRC_BUS2` ,
`DM35418_CLK_SRC_BUS3` ,
`DM35418_CLK_SRC_BUS4` , `DM35418_CLK_SRC_BUS5` , `DM35418_CLK_SRC_BUS6` , `DM35418_CLK_SRC_BUS7`
 ,
`DM35418_CLK_SRC_CHAN_THRESH` = 0x08 , `DM35418_CLK_SRC_CHAN_THRESH_INV` = 0x09 ,
`DM35418_CLK_SRC_BUS2_INV` = 0x0A , `DM35418_CLK_SRC_BUS3_INV` ,
`DM35418_CLK_SRC_BUS4_INV` , `DM35418_CLK_SRC_BUS5_INV` , `DM35418_CLK_SRC_BUS6_INV` ,
`DM35418_CLK_SRC_BUS7_INV` }

Possible clock sources used by function blocks. Note that some clock sources may not be available on your particular board. Check the hardware manual to verify which clock sources can be used.
- enum `DM35418_Clock_Buses` {
`DM35418_CLK_BUS2` = 2 , `DM35418_CLK_BUS3` , `DM35418_CLK_BUS4` , `DM35418_CLK_BUS5` ,
`DM35418_CLK_BUS6` , `DM35418_CLK_BUS7` }

Clock buses available to the function block.

6.28.1 Detailed Description

Defines for the DM35418. Values for the general board, not specific to a particular function block.

Id

[dm35418_types.h](#) 138274 2023-07-06 20:25:13Z lfrankenfield

6.28.2 Enumeration Type Documentation

6.28.2.1 DM35418_Clock_Buses

enum [DM35418_Clock_Buses](#)

Clock buses available to the function block.

Enumerator

DM35418_CLK_BUS2	Clock Bus 2.
DM35418_CLK_BUS3	Clock Bus 3.
DM35418_CLK_BUS4	Clock Bus 4.
DM35418_CLK_BUS5	Clock Bus 5.
DM35418_CLK_BUS6	Clock Bus 6.
DM35418_CLK_BUS7	Clock Bus 7.

Definition at line 353 of file dm35418_types.h.

6.28.2.2 DM35418_Clock_Sources

enum [DM35418_Clock_Sources](#)

Possible clock sources used by function blocks. Note that some clock sources may not be available on your particular board. Check the hardware manual to verify which clock sources can be used.

DM35418_General_Definitions

Enumerator

DM35418_CLK_SRC_IMMEDIATE	Clock Source - Immediate (0x00)
DM35418_CLK_SRC_NEVER	Clock Source - Never (0x01)
DM35418_CLK_SRC_BUS2	Clock Source - Bus 2 (0x02)
DM35418_CLK_SRC_BUS3	Clock Source - Bus 3 (0x03)
DM35418_CLK_SRC_BUS4	Clock Source - Bus 4 (0x04)
DM35418_CLK_SRC_BUS5	Clock Source - Bus 5 (0x05)
DM35418_CLK_SRC_BUS6	Clock Source - Bus 6 (0x06)

Enumerator

DM35418_CLK_SRC_BUS7	Clock Source - Bus 7 (0x07)
DM35418_CLK_SRC_CHAN_THRESH	Clock Source - Threshold Exceeded (0x08)
DM35418_CLK_SRC_CHAN_THRESH_INV	Clock Source - Threshold Inverse (None Exceeded) (0x09)
DM35418_CLK_SRC_BUS2_INV	Clock Source - Bus 2 Inverse (0x0A)
DM35418_CLK_SRC_BUS3_INV	Clock Source - Bus 3 Inverse (0x0B)
DM35418_CLK_SRC_BUS4_INV	Clock Source - Bus 4 Inverse (0x0C)
DM35418_CLK_SRC_BUS5_INV	Clock Source - Bus 5 Inverse (0x0D)
DM35418_CLK_SRC_BUS6_INV	Clock Source - Bus 6 Inverse (0x0E)
DM35418_CLK_SRC_BUS7_INV	Clock Source - Bus 7 Inverse (0x0F)

Definition at line 265 of file dm35418_types.h.

6.29 include/dm35418_util_library.h File Reference

Definitions for the DM35418 Utilities library, various helper functions.

```
#include <time.h>
#include <sys/time.h>
```

Enumerations

- enum [DM35418_Waveforms](#) { [DM35418_SINE_WAVE](#) , [DM35418_SQUARE_WAVE](#) , [DM35418_SAWTOOTH_WAVE](#) }

List of possible waveforms that can be generated for DAC purposes.

Functions

- uint32_t [DM35418_Get_Maskable](#) (uint16_t data, uint16_t mask)
Return a 32-bit maskable register value from the data and mask.
- void [DM35418_Micro_Sleep](#) (unsigned long microseconds)
Sleep for a specified number of microseconds.
- long [DM35418_Get_Time_Diff](#) (struct timeval last, struct timeval first)
Calculate the time difference between the two timeval structs, in microseconds.
- int [DM35418_Generate_Signal_Data](#) (enum [DM35418_Waveforms](#) waveform, int32_t *data, uint32_t data←_count, int32_t max, int32_t minimum, int32_t offset, uint32_t mask)
Generate data with a specific wave pattern. This is useful for producing recognizable waves for DAC output.
- void [DM35418_Check_Result](#) (int return_val, char *message)
Check the result of an operation, usually a library call. If the result is non-zero, then it is an error and output the passed message.

6.29.1 Detailed Description

Definitions for the DM35418 Utilities library, various helper functions.

Id

[dm35418_util_library.h](#) 139458 2023-09-20 18:22:05Z lfrankfield

Index

- access
 - dm35418_ioctl_region_modify, [143](#)
 - dm35418_ioctl_region_readwrite, [145](#)
- AD_MODE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- adc_dma_error
 - dm35418_adc_to_dac_dma.c, [194](#)
- ADC_NUM_OPTION
 - DM35418 Example Programs Constants, [94](#)
- ADC_OPTION
 - DM35418 Example Programs Constants, [94](#)
- ADC_THRESH_HIGH_VOLTS
 - dm35418_adc_thresh_dma.c, [180](#)
- ADC_THRESH_LOW_VOLTS
 - dm35418_adc_thresh_dma.c, [180](#)
- allocated
 - dm35418_pci_region, [148](#)
- ASCII_FILE_NAME
 - dm35418_adc_continuous_dma.c, [160](#)
 - dm35418_adc_sync_trig_dma.c, [170](#)
 - dm35418_adc_thresh_dma.c, [181](#)
- BAUD_OPTION
 - DM35418 Example Programs Constants, [94](#)
- BIN2TXT_OPTION
 - DM35418 Example Programs Constants, [94](#)
- BIN_FILE_NAME
 - dm35418_adc_continuous_dma.c, [160](#)
 - dm35418_adc_sync_trig_dma.c, [170](#)
 - dm35418_adc_thresh_dma.c, [181](#)
- BINARY_OPTION
 - DM35418 Example Programs Constants, [94](#)
- board
 - dm35418_adc_basic_ints.c, [157](#)
 - dm35418_adc_continuous_dma.c, [166](#)
 - dm35418_adc_sync_trig_dma.c, [176](#)
 - dm35418_adc_thresh_dma.c, [185](#)
 - dm35418_adc_to_dac_dma.c, [194](#)
- buff_cnt
 - dm35418_adc_to_dac_dma.c, [194](#)
- buffer
 - dm35418_dma_descriptor, [134](#)
 - dm35418_ioctl_dma, [140](#)
- buffer_count
 - dm35418_adc_continuous_dma.c, [166](#)
 - dm35418_adc_sync_trig_dma.c, [176](#)
 - dm35418_adc_thresh_dma.c, [185](#)
- buffer_ptr
 - dm35418_ioctl_dma, [140](#)
- buffer_size
 - dm35418_dma_descriptor, [134](#)
 - dm35418_ioctl_dma, [140](#)
- BUFFER_SIZE_BYTES
 - dm35418_adc.c, [150](#)
 - dm35418_adc_basic_ints.c, [155](#)
 - dm35418_adc_continuous_dma.c, [160](#)
 - dm35418_adc_to_dac_dma.c, [189](#)
 - dm35418_dac_dma.c, [201](#)
 - dm35418_dac_syncbus.c, [205](#)
- buffer_size_bytes
 - dm35418_adc_continuous_dma.c, [166](#)
 - dm35418_adc_sync_trig_dma.c, [177](#)
 - dm35418_adc_thresh_dma.c, [185](#)
- BUFFER_SIZE_SAMPLES
 - dm35418_adc.c, [151](#)
 - dm35418_adc_basic_ints.c, [155](#)
 - dm35418_adc_continuous_dma.c, [161](#)
 - dm35418_adc_to_dac_dma.c, [189](#)
 - dm35418_dac_dma.c, [201](#)
 - dm35418_dac_syncbus.c, [205](#)
- buffer_start_offset
 - DM35418_DMA_Descriptor, [133](#)
- bus_addr
 - dm35418_dma_descriptor, [134](#)
- cdev
 - dm35418_device_descriptor, [129](#)
- channel
 - dm35418_dma_descriptor, [134](#)
 - dm35418_ioctl_dma, [140](#)
- CHANNELS_OPTION
 - DM35418 Example Programs Constants, [93](#)
- CLK_40MHZ
 - DM35418 Board Macros, [95](#)
- control_offset
 - DM35418_DMA_Descriptor, [133](#)
 - DM35418_Function_Block, [136](#)
- convert_bin_to_txt
 - dm35418_adc_continuous_dma.c, [161](#)
- COUNT_OPTION
 - DM35418 Example Programs Constants, [94](#)
- DAC_BUFF_NUM_SAMPLES
 - dm35418_adc_sync_trig_dma.c, [170](#)
 - dm35418_adc_thresh_dma.c, [181](#)
- DAC_BUFF_SIZE_BYTES
 - dm35418_adc_sync_trig_dma.c, [171](#)
 - dm35418_adc_thresh_dma.c, [181](#)
- DAC_NUM_OPTION
 - DM35418 Example Programs Constants, [94](#)

- DAC_OPTION
 - DM35418 Example Programs Constants, [94](#)
- DAC_RATE
 - dm35418_adc.c, [151](#)
 - dm35418_adc_basic_ints.c, [155](#)
 - dm35418_adc_continuous_dma.c, [161](#)
 - dm35418_adc_sync_trig_dma.c, [171](#)
 - dm35418_adc_thresh_dma.c, [181](#)
- data
 - dm35418_pci_access_request, [146](#)
- data16
 - dm35418_pci_access_request, [146](#)
- data32
 - dm35418_pci_access_request, [146](#)
- data8
 - dm35418_pci_access_request, [146](#)
- DEFAULT_DAC_NUM
 - dm35418_dac.c, [197](#)
- DEFAULT_DAC_TO_USE
 - dm35418_dac_dma.c, [201](#)
 - dm35418_dac_syncbus.c, [205](#)
- DEFAULT_NUM_ADC
 - dm35418_adc_continuous_dma.c, [161](#)
- DEFAULT_RANGE
 - dm35418_adc.c, [151](#)
 - dm35418_adc_basic_ints.c, [155](#)
- DEFAULT_RATE
 - dm35418_adc.c, [151](#)
 - dm35418_adc_basic_ints.c, [155](#)
 - dm35418_adc_continuous_dma.c, [161](#)
 - dm35418_adc_sync_trig_dma.c, [171](#)
 - dm35418_adc_thresh_dma.c, [182](#)
 - dm35418_adc_to_dac_dma.c, [189](#)
 - dm35418_dac_dma.c, [201](#)
 - dm35418_dac_syncbus.c, [205](#)
- device_index
 - dm35418_device_descriptor, [129](#)
- device_lock
 - dm35418_device_descriptor, [129](#)
- DIO_DIR_OUTPUT
 - dm35418_adc_sync_trig_dma.c, [171](#)
- DIO_TRIGGER_OFF
 - dm35418_adc_sync_trig_dma.c, [171](#)
- DIO_TRIGGER_ON
 - dm35418_adc_sync_trig_dma.c, [172](#)
- DM35418 ADC Library Constants, [7](#)
 - DM35418_ADC_CLK_BUS_SRC_CHAN_THRESH, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_DISABLE, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_PACER_TICK, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_SAMPLE_TAKEN, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_START_TRIG, [10](#)
 - DM35418_ADC_CLK_BUS_SRC_STOP_TRIG, [10](#)
 - DM35418_Adc_Clock_Events, [10](#)
 - DM35418_ADC_GAIN_05, [11](#)
 - DM35418_ADC_GAIN_1, [11](#)
 - DM35418_ADC_GAIN_128, [11](#)
 - DM35418_ADC_GAIN_16, [11](#)
 - DM35418_ADC_GAIN_2, [11](#)
 - DM35418_ADC_GAIN_32, [11](#)
 - DM35418_ADC_GAIN_4, [11](#)
 - DM35418_ADC_GAIN_64, [11](#)
 - DM35418_ADC_GAIN_8, [11](#)
 - DM35418_ADC_INPUT_DIFFERENTIAL, [11](#)
 - DM35418_ADC_INPUT_SINGLE_ENDED, [11](#)
 - DM35418_ADC_INPUT_VREF_5V, [11](#)
 - DM35418_ADC_RNG_BIPOLAR_10V, [12](#)
 - DM35418_ADC_RNG_BIPOLAR_1_25V, [12](#)
 - DM35418_ADC_RNG_BIPOLAR_2_5V, [12](#)
 - DM35418_ADC_RNG_BIPOLAR_5V, [12](#)
 - DM35418_ADC_RNG_BIPOLAR_625mV, [12](#)
 - DM35418_ADC_RNG_UNIPOLAR_10V, [12](#)
 - DM35418_ADC_RNG_UNIPOLAR_1_25V, [12](#)
 - DM35418_ADC_RNG_UNIPOLAR_2_5V, [12](#)
 - DM35418_ADC_RNG_UNIPOLAR_5V, [12](#)
 - DM35418_Gains, [11](#)
 - DM35418_Input_Mode, [11](#)
 - DM35418_Input_Ranges, [12](#)
- DM35418 ADC Public Library Functions, [12](#)
 - DM35418_Adc_Channel_Find_Interrupt, [15](#)
 - DM35418_Adc_Channel_Get_Filter, [16](#)
 - DM35418_Adc_Channel_Get_Front_End_Config, [16](#)
 - DM35418_Adc_Channel_Get_Last_Sample, [17](#)
 - DM35418_Adc_Channel_Get_Thresholds, [17](#)
 - DM35418_Adc_Channel_Interrupt_Clear_Status, [18](#)
 - DM35418_Adc_Channel_Interrupt_Get_Config, [19](#)
 - DM35418_Adc_Channel_Interrupt_Get_Status, [19](#)
 - DM35418_Adc_Channel_Interrupt_Set_Config, [20](#)
 - DM35418_Adc_Channel_Reset, [21](#)
 - DM35418_Adc_Channel_Set_Filter, [21](#)
 - DM35418_Adc_Channel_Set_High_Threshold, [22](#)
 - DM35418_Adc_Channel_Set_Low_Threshold, [22](#)
 - DM35418_Adc_Channel_Setup, [23](#)
 - DM35418_Adc_Fifo_Channel_Read, [24](#)
 - DM35418_Adc_Get_Clock_Source_Global, [24](#)
 - DM35418_Adc_Get_Clock_Src, [25](#)
 - DM35418_Adc_Get_Mode_Status, [25](#)
 - DM35418_Adc_Get_Post_Stop_Samples, [26](#)
 - DM35418_Adc_Get_Pre_Trigger_Samples, [26](#)
 - DM35418_Adc_Get_Sample_Count, [27](#)
 - DM35418_Adc_Get_Start_Trigger, [27](#)
 - DM35418_Adc_Get_Stop_Trigger, [28](#)
 - DM35418_Adc_Initialize, [28](#)

- DM35418_Adc_Interrupt_Clear_Status, 29
- DM35418_Adc_Interrupt_Get_Config, 30
- DM35418_Adc_Interrupt_Get_Status, 30
- DM35418_Adc_Interrupt_Set_Config, 31
- DM35418_Adc_Open, 31
- DM35418_Adc_Pause, 32
- DM35418_Adc_Reset, 32
- DM35418_Adc_Sample_To_Volts, 33
- DM35418_Adc_Set_Clk_Divider, 33
- DM35418_Adc_Set_Clock_Source_Global, 34
- DM35418_Adc_Set_Clock_Src, 34
- DM35418_Adc_Set_Post_Stop_Samples, 35
- DM35418_Adc_Set_Pre_Trigger_Samples, 35
- DM35418_Adc_Set_Sample_Rate, 36
- DM35418_Adc_Set_Start_Trigger, 37
- DM35418_Adc_Set_Stop_Trigger, 37
- DM35418_Adc_Start, 38
- DM35418_Adc_Start_Rearm, 38
- DM35418_Adc_Uninitialize, 39
- DM35418_Adc_Volts_To_Sample, 39
- DM35418 Board Access Public Library Functions, 101
 - DM35418_Dma_Initialize, 101
 - DM35418_Dma_Read, 102
 - DM35418_Dma_Write, 103
 - DM35418_General_InstallISR, 103
 - DM35418_General_RemoveISR, 104
 - DM35418_General_SetISRPriority, 104
 - DM35418_General_WaitForInterrupt, 105
- DM35418 Board Access Structures, 40
 - DM35418_Board_Close, 40
 - DM35418_Board_Open, 41
 - DM35418_Dma, 41
 - DM35418_Modify, 42
 - DM35418_Read, 42
 - DM35418_Write, 43
- DM35418 Board Library Public Functions, 95
 - DM35418_Function_Block_Open, 96
 - DM35418_Function_Block_Open_Module, 96
 - DM35418_Gbc_Ack_Interrupt, 97
 - DM35418_Gbc_Board_Reset, 97
 - DM35418_Gbc_Get_Format, 98
 - DM35418_Gbc_Get_Fpga_Build, 98
 - DM35418_Gbc_Get_Pdp_Number, 99
 - DM35418_Gbc_Get_Revision, 99
 - DM35418_Gbc_Get_Sys_Clock_Freq, 99
- DM35418 Board Macros, 95
 - CLK_40MHZ, 95
- DM35418 Board Types, 121
- DM35418 DAC Library Constants, 45
 - DM35418_DAC_CLK_BUS_SRC_CHAN_MARKER, 47
 - DM35418_DAC_CLK_BUS_SRC_CONV_COMPL, 47
 - DM35418_DAC_CLK_BUS_SRC_CONVERSION_SENT, 47
 - DM35418_DAC_CLK_BUS_SRC_DISABLE, 47
 - DM35418_DAC_CLK_BUS_SRC_START_TRIG, 47
 - DM35418_DAC_CLK_BUS_SRC_STOP_TRIG, 47
 - DM35418_Dac_Clock_Events, 47
 - DM35418_DAC_RNG_BIPOLAR_10V, 47
 - DM35418_DAC_RNG_BIPOLAR_2_5V, 47
 - DM35418_DAC_RNG_BIPOLAR_5V, 47
 - DM35418_Output_Ranges, 47
- DM35418 DAC Library Public Functions, 48
 - DM35418_Dac_Channel_Clear_Marker_Status, 50
 - DM35418_Dac_Channel_Get_Marker_Config, 50
 - DM35418_Dac_Channel_Get_Marker_Status, 51
 - DM35418_Dac_Channel_Reset, 51
 - DM35418_Dac_Channel_Set_Marker_Config, 52
 - DM35418_Dac_Channel_Setup, 52
 - DM35418_Dac_Conv_To_Volts, 53
 - DM35418_Dac_Fifo_Channel_Write, 53
 - DM35418_Dac_Get_Clock_Div, 54
 - DM35418_Dac_Get_Clock_Src, 54
 - DM35418_Dac_Get_Conversion_Count, 55
 - DM35418_Dac_Get_Last_Conversion, 55
 - DM35418_Dac_Get_Mode_Status, 57
 - DM35418_Dac_Get_Post_Stop_Conversion_Count, 57
 - DM35418_Dac_Get_Start_Trigger, 58
 - DM35418_Dac_Get_Stop_Trigger, 58
 - DM35418_Dac_Interrupt_Clear_Status, 59
 - DM35418_Dac_Interrupt_Get_Config, 59
 - DM35418_Dac_Interrupt_Get_Status, 60
 - DM35418_Dac_Interrupt_Set_Config, 60
 - DM35418_Dac_Open, 61
 - DM35418_Dac_Pause, 61
 - DM35418_Dac_Reset, 62
 - DM35418_Dac_Set_Clock_Div, 62
 - DM35418_Dac_Set_Clock_Source_Global, 63
 - DM35418_Dac_Set_Clock_Src, 63
 - DM35418_Dac_Set_Conversion_Rate, 64
 - DM35418_Dac_Set_Last_Conversion, 64
 - DM35418_Dac_Set_Post_Stop_Conversion_Count, 65
 - DM35418_Dac_Set_Start_Trigger, 65
 - DM35418_Dac_Set_Stop_Trigger, 66
 - DM35418_Dac_Start, 66
 - DM35418_Dac_Volts_To_Conv, 67
- DM35418 DIO Public, 67
 - DM35418_Dio_Get_Direction, 68
 - DM35418_Dio_Get_Input_Value, 68
 - DM35418_Dio_Get_Output_Value, 69
 - DM35418_Dio_Open, 69
 - DM35418_Dio_Set_Direction, 70
 - DM35418_Dio_Set_Output_Value, 70
- DM35418 DMA Public Library Constants, 71
 - DM35418_FIFO_EMPTY, 73
 - DM35418_FIFO_FULL, 73
 - DM35418_FIFO_HAS_DATA, 73
 - DM35418_Fifo_States, 72
 - DM35418_FIFO_UNKNOWN, 73
- DM35418 DMA Public Library Functions, 73
 - DM35418_Dma_Buffer_Setup, 74

- DM35418_Dma_Buffer_Status, [75](#)
- DM35418_Dma_Check_Buffer_Used, [76](#)
- DM35418_Dma_Check_For_Error, [76](#)
- DM35418_Dma_Clear, [78](#)
- DM35418_Dma_Clear_Interrupt, [78](#)
- DM35418_Dma_Configure_Interrupts, [79](#)
- DM35418_Dma_Find_Interrupt, [80](#)
- DM35418_Dma_Get_Current_Buffer_Count, [81](#)
- DM35418_Dma_Get_Errors, [81](#)
- DM35418_Dma_Get_Fifo_Counts, [82](#)
- DM35418_Dma_Get_Fifo_State, [83](#)
- DM35418_Dma_Get_Interrupt_Configuration, [83](#)
- DM35418_Dma_Pause, [84](#)
- DM35418_Dma_Reset_Buffer, [84](#)
- DM35418_Dma_Setup, [85](#)
- DM35418_Dma_Setup_Set_Direction, [86](#)
- DM35418_Dma_Setup_Set_Used, [86](#)
- DM35418_Dma_Start, [87](#)
- DM35418_Dma_Status, [87](#)
- DM35418_Dma_Stop, [89](#)
- DM35418_Driver_Constants, [90](#)
- DM35418_Driver_Enumerations, [90](#)
 - dm35418_pci_region_access_dir, [90](#)
 - DM35418_PCI_REGION_ACCESS_READ, [90](#)
 - DM35418_PCI_REGION_ACCESS_WRITE, [90](#)
- DM35418_Driver_Structures, [91](#)
- DM35418_Example_Programs_Constants, [91](#)
 - AD_MODE_OPTION, [94](#)
 - ADC_NUM_OPTION, [94](#)
 - ADC_OPTION, [94](#)
 - BAUD_OPTION, [94](#)
 - BIN2TXT_OPTION, [94](#)
 - BINARY_OPTION, [94](#)
 - CHANNELS_OPTION, [93](#)
 - COUNT_OPTION, [94](#)
 - DAC_NUM_OPTION, [94](#)
 - DAC_OPTION, [94](#)
 - DUMP_OPTION, [94](#)
 - EXTERNAL_OPTION, [94](#)
 - FILE_OPTION, [93](#)
 - HELP_OPTION, [93](#)
 - Help_Options, [93](#)
 - HOURS_OPTION, [94](#)
 - LOW_THRESHOLD_OPTION, [94](#)
 - MASTER_OPTION, [94](#)
 - MINOR_OPTION, [93](#)
 - MODE_OPTION, [94](#)
 - NOSTOP_OPTION, [94](#)
 - NUM_OPTION, [94](#)
 - OFFILE_OPTION, [94](#)
 - OUTPUT_ADC_OPTION, [94](#)
 - OUTPUT_RMS_OPTION, [94](#)
 - PACKED_OPTION, [94](#)
 - PATTERN_OPTION, [94](#)
 - PORT_OPTION, [94](#)
 - RANGE_OPTION, [94](#)
 - RATE_OPTION, [93](#)
 - RECEIVER_OPTION, [94](#)
 - REF_NUM_OPTION, [94](#)
 - REFCLK_OPTION, [94](#)
 - REFILL_FIFO_OPTION, [94](#)
 - SAMPLES_OPTION, [94](#)
 - SENDER_OPTION, [94](#)
 - SIZE_OPTION, [94](#)
 - SLAVE_OPTION, [94](#)
 - START_OPTION, [93](#)
 - STORE_OPTION, [94](#)
 - SYNC_CONN_OPTION, [94](#)
 - SYNC_TERM_OPTION, [94](#)
 - SYNCBUS_OPTION, [94](#)
 - TERM_OPTION, [94](#)
 - TEST_OPTION, [94](#)
 - USER_ID_OPTION, [94](#)
 - VERBOSE_OPTION, [94](#)
 - WAVE_OPTION, [94](#)
- DM35418_ioctl macros, [100](#)
- DM35418_PCI_Region_Structures, [43](#)
 - DM35418_DMA_FUNCTIONS, [44](#)
 - DM35418_DMA_INITIALIZE, [44](#)
 - DM35418_DMA_READ, [44](#)
 - DM35418_DMA_WRITE, [44](#)
 - DM35418_PCI_REGION_ACCESS_16, [45](#)
 - DM35418_PCI_REGION_ACCESS_32, [45](#)
 - DM35418_PCI_REGION_ACCESS_8, [45](#)
 - dm35418_pci_region_access_size, [44](#)
 - DM35418_PCI_REGION_FB, [45](#)
 - DM35418_PCI_REGION_GBC, [45](#)
 - DM35418_PCI_REGION_GBC2, [45](#)
 - dm35418_pci_region_num, [45](#)
- DM35418_Register_Offsets, [105](#)
 - DM35418_OFFSET_FB_ADC_FIFO, [110](#)
- DM35418_Sync_Bus_Library_Public_Functions, [111](#)
 - DM35418_Syncbus_Clock_Source_Global_Get, [112](#)
 - DM35418_Syncbus_Clock_Source_Global_Set, [113](#)
 - DM35418_Syncbus_Disable, [113](#)
 - DM35418_Syncbus_Drive_Source_Get, [114](#)
 - DM35418_Syncbus_Drive_Source_Set, [114](#)
 - DM35418_Syncbus_Enable, [115](#)
 - DM35418_Syncbus_Get_Clk_Sel, [116](#)
 - DM35418_Syncbus_Get_PLL_Locked, [116](#)
 - DM35418_Syncbus_Open, [116](#)
 - DM35418_Syncbus_Set_Clk_Sel, [117](#)
 - DM35418_Syncbus_Set_Direction, [117](#)
 - DM35418_Syncbus_Termination_Disable, [118](#)
 - DM35418_Syncbus_Termination_Enable, [119](#)
- DM35418_SyncBus_Library_Constants, [110](#)
- DM35418_Temperature, [119](#)
 - DM35418_Temperature_Open, [120](#)
 - DM35418_Temperature_Read, [120](#)
- DM35418_Utility_Library_Functions, [122](#)
 - DM35418_Check_Result, [123](#)
 - DM35418_Generate_Signal_Data, [124](#)
 - DM35418_Get_Maskable, [124](#)
 - DM35418_Get_Time_Diff, [125](#)

- DM35418_Micro_Sleep, [125](#)
- DM35418_SAWTOOTH_WAVE, [123](#)
- DM35418_SINE_WAVE, [123](#)
- DM35418_SQUARE_WAVE, [123](#)
- DM35418_Waveforms, [123](#)
- dm35418_adc.c
 - BUFFER_SIZE_BYTES, [150](#)
 - BUFFER_SIZE_SAMPLES, [151](#)
 - DAC_RATE, [151](#)
 - DEFAULT_RANGE, [151](#)
 - DEFAULT_RATE, [151](#)
 - exit_program, [152](#)
 - interrupt_count, [152](#)
 - main, [151](#)
 - program_name, [153](#)
 - sigint_handler, [152](#)
- dm35418_adc_basic_ints.c
 - board, [157](#)
 - BUFFER_SIZE_BYTES, [155](#)
 - BUFFER_SIZE_SAMPLES, [155](#)
 - DAC_RATE, [155](#)
 - DEFAULT_RANGE, [155](#)
 - DEFAULT_RATE, [155](#)
 - exit_program, [157](#)
 - main, [156](#)
 - my_adc, [157](#)
 - program_name, [158](#)
 - sample_count, [158](#)
 - sigint_handler, [157](#)
 - TOTAL_INT_SRC_MASK, [155](#)
- DM35418_Adc_Channel_Find_Interrupt
 - DM35418 ADC Public Library Functions, [15](#)
- DM35418_Adc_Channel_Get_Filter
 - DM35418 ADC Public Library Functions, [16](#)
- DM35418_Adc_Channel_Get_Front_End_Config
 - DM35418 ADC Public Library Functions, [16](#)
- DM35418_Adc_Channel_Get_Last_Sample
 - DM35418 ADC Public Library Functions, [17](#)
- DM35418_Adc_Channel_Get_Thresholds
 - DM35418 ADC Public Library Functions, [17](#)
- DM35418_Adc_Channel_Interrupt_Clear_Status
 - DM35418 ADC Public Library Functions, [18](#)
- DM35418_Adc_Channel_Interrupt_Get_Config
 - DM35418 ADC Public Library Functions, [19](#)
- DM35418_Adc_Channel_Interrupt_Get_Status
 - DM35418 ADC Public Library Functions, [19](#)
- DM35418_Adc_Channel_Interrupt_Set_Config
 - DM35418 ADC Public Library Functions, [20](#)
- DM35418_Adc_Channel_Reset
 - DM35418 ADC Public Library Functions, [21](#)
- DM35418_Adc_Channel_Set_Filter
 - DM35418 ADC Public Library Functions, [21](#)
- DM35418_Adc_Channel_Set_High_Threshold
 - DM35418 ADC Public Library Functions, [22](#)
- DM35418_Adc_Channel_Set_Low_Threshold
 - DM35418 ADC Public Library Functions, [22](#)
- DM35418_Adc_Channel_Setup
 - DM35418 ADC Public Library Functions, [23](#)
- DM35418_ADC_CLK_BUS_SRC_CHAN_THRESH
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_DISABLE
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_PACER_TICK
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_POST_STOP_BUFF_FULL
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_PRE_START_BUFF_FULL
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_SAMPLE_TAKEN
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_SAMPLING_COMPLETE
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_START_TRIG
 - DM35418 ADC Library Constants, [10](#)
- DM35418_ADC_CLK_BUS_SRC_STOP_TRIG
 - DM35418 ADC Library Constants, [10](#)
- DM35418_Adc_Clock_Events
 - DM35418 ADC Library Constants, [10](#)
- dm35418_adc_continuous_dma.c
 - ASCII_FILE_NAME, [160](#)
 - BIN_FILE_NAME, [160](#)
 - board, [166](#)
 - buffer_count, [166](#)
 - BUFFER_SIZE_BYTES, [160](#)
 - buffer_size_bytes, [166](#)
 - BUFFER_SIZE_SAMPLES, [161](#)
 - convert_bin_to_txt, [161](#)
 - DAC_RATE, [161](#)
 - DEFAULT_NUM_ADC, [161](#)
 - DEFAULT_RATE, [161](#)
 - dma_has_error, [166](#)
 - exit_program, [166](#)
 - ISR, [162](#)
 - local_buffer, [167](#)
 - main, [162](#)
 - my_adc, [167](#)
 - next_buffer, [167](#)
 - output_channel_status, [163](#)
 - program_name, [167](#)
 - setup_adcs, [163](#)
 - setup_ctrlc_handler, [164](#)
 - setup_dacs, [164](#)
 - setup_slave_adcs, [165](#)
 - sigint_handler, [165](#)
- DM35418_Adc_Fifo_Channel_Read
 - DM35418 ADC Public Library Functions, [24](#)
- DM35418_ADC_GAIN_05
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_1
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_128
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_16
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_2
 - DM35418 ADC Library Constants, [11](#)

- DM35418_ADC_GAIN_32
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_4
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_64
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_GAIN_8
 - DM35418 ADC Library Constants, [11](#)
- DM35418_Adc_Get_Clock_Source_Global
 - DM35418 ADC Public Library Functions, [24](#)
- DM35418_Adc_Get_Clock_Src
 - DM35418 ADC Public Library Functions, [25](#)
- DM35418_Adc_Get_Mode_Status
 - DM35418 ADC Public Library Functions, [25](#)
- DM35418_Adc_Get_Post_Stop_Samples
 - DM35418 ADC Public Library Functions, [26](#)
- DM35418_Adc_Get_Pre_Trigger_Samples
 - DM35418 ADC Public Library Functions, [26](#)
- DM35418_Adc_Get_Sample_Count
 - DM35418 ADC Public Library Functions, [27](#)
- DM35418_Adc_Get_Start_Trigger
 - DM35418 ADC Public Library Functions, [27](#)
- DM35418_Adc_Get_Stop_Trigger
 - DM35418 ADC Public Library Functions, [28](#)
- DM35418_Adc_Initialize
 - DM35418 ADC Public Library Functions, [28](#)
- DM35418_ADC_INPUT_DIFFERENTIAL
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_INPUT_SINGLE_ENDED
 - DM35418 ADC Library Constants, [11](#)
- DM35418_ADC_INPUT_VREF_5V
 - DM35418 ADC Library Constants, [11](#)
- DM35418_Adc_Interrupt_Clear_Status
 - DM35418 ADC Public Library Functions, [29](#)
- DM35418_Adc_Interrupt_Get_Config
 - DM35418 ADC Public Library Functions, [30](#)
- DM35418_Adc_Interrupt_Get_Status
 - DM35418 ADC Public Library Functions, [30](#)
- DM35418_Adc_Interrupt_Set_Config
 - DM35418 ADC Public Library Functions, [31](#)
- DM35418_Adc_Open
 - DM35418 ADC Public Library Functions, [31](#)
- DM35418_Adc_Pause
 - DM35418 ADC Public Library Functions, [32](#)
- DM35418_Adc_Reset
 - DM35418 ADC Public Library Functions, [32](#)
- DM35418_ADC_RNG_BIPOLAR_10V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_BIPOLAR_1_25V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_BIPOLAR_2_5V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_BIPOLAR_5V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_BIPOLAR_625mV
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_UNIPOLAR_10V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_UNIPOLAR_1_25V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_UNIPOLAR_2_5V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_ADC_RNG_UNIPOLAR_5V
 - DM35418 ADC Library Constants, [12](#)
- DM35418_Adc_Sample_To_Volts
 - DM35418 ADC Public Library Functions, [33](#)
- DM35418_Adc_Set_Clk_Divider
 - DM35418 ADC Public Library Functions, [33](#)
- DM35418_Adc_Set_Clock_Source_Global
 - DM35418 ADC Public Library Functions, [34](#)
- DM35418_Adc_Set_Clock_Src
 - DM35418 ADC Public Library Functions, [34](#)
- DM35418_Adc_Set_Post_Stop_Samples
 - DM35418 ADC Public Library Functions, [35](#)
- DM35418_Adc_Set_Pre_Trigger_Samples
 - DM35418 ADC Public Library Functions, [35](#)
- DM35418_Adc_Set_Sample_Rate
 - DM35418 ADC Public Library Functions, [36](#)
- DM35418_Adc_Set_Start_Trigger
 - DM35418 ADC Public Library Functions, [37](#)
- DM35418_Adc_Set_Stop_Trigger
 - DM35418 ADC Public Library Functions, [37](#)
- DM35418_Adc_Start
 - DM35418 ADC Public Library Functions, [38](#)
- DM35418_Adc_Start_Rearm
 - DM35418 ADC Public Library Functions, [38](#)
- dm35418_adc_sync_trig_dma.c
 - ASCII_FILE_NAME, [170](#)
 - BIN_FILE_NAME, [170](#)
 - board, [176](#)
 - buffer_count, [176](#)
 - buffer_size_bytes, [177](#)
 - DAC_BUFF_NUM_SAMPLES, [170](#)
 - DAC_BUFF_SIZE_BYTES, [171](#)
 - DAC_RATE, [171](#)
 - DEFAULT_RATE, [171](#)
 - DIO_DIR_OUTPUT, [171](#)
 - DIO_TRIGGER_OFF, [171](#)
 - DIO_TRIGGER_ON, [172](#)
 - dma_has_error, [177](#)
 - exit_program, [177](#)
 - ISR, [172](#)
 - local_buffer, [177](#)
 - main, [173](#)
 - my_adc, [178](#)
 - next_buffer, [178](#)
 - NUM_SAMPLES_PER_TRIGGER, [172](#)
 - output_channel_status, [173](#)
 - program_name, [178](#)
 - setup_adc, [174](#)
 - setup_dac, [174](#)
 - setup_dio, [175](#)
 - setup_synbus, [175](#)
 - sigint_handler, [176](#)
 - SYNCBUS_TRIGGER_OFF_USEC, [172](#)
 - SYNCBUS_TRIGGER_ON_USEC, [172](#)

- dm35418_adc_thresh_dma.c
 - ADC_THRESH_HIGH_VOLTS, [180](#)
 - ADC_THRESH_LOW_VOLTS, [180](#)
 - ASCII_FILE_NAME, [181](#)
 - BIN_FILE_NAME, [181](#)
 - board, [185](#)
 - buffer_count, [185](#)
 - buffer_size_bytes, [185](#)
 - DAC_BUFF_NUM_SAMPLES, [181](#)
 - DAC_BUFF_SIZE_BYTES, [181](#)
 - DAC_RATE, [181](#)
 - DEFAULT_RATE, [182](#)
 - dma_has_error, [186](#)
 - exit_program, [186](#)
 - ISR, [182](#)
 - local_buffer, [186](#)
 - main, [183](#)
 - my_adc, [186](#)
 - next_buffer, [187](#)
 - NUM_SAMPLES_PER_TRIGGER, [182](#)
 - output_channel_status, [183](#)
 - program_name, [187](#)
 - setup_adc, [184](#)
 - setup_dac, [184](#)
 - sigint_handler, [185](#)
- dm35418_adc_to_dac_dma.c
 - adc_dma_error, [194](#)
 - board, [194](#)
 - buff_cnt, [194](#)
 - BUFFER_SIZE_BYTES, [189](#)
 - BUFFER_SIZE_SAMPLES, [189](#)
 - DEFAULT_RATE, [189](#)
 - exit_program, [195](#)
 - ISR, [190](#)
 - local_buffer, [195](#)
 - main, [190](#)
 - my_adc, [195](#)
 - next_buffer, [195](#)
 - NUM_FB_TO_USE, [190](#)
 - output_channel_status, [191](#)
 - program_name, [196](#)
 - setup_adcs, [192](#)
 - setup_dacs, [192](#)
 - setup_isr, [193](#)
 - sigint_handler, [193](#)
 - start_slave_adcs, [193](#)
- DM35418_Adc_Uninitialize
 - DM35418 ADC Public Library Functions, [39](#)
- DM35418_Adc_Volts_To_Sample
 - DM35418 ADC Public Library Functions, [39](#)
- dm35418_board_access.h
 - DM35418LIB_API, [224](#)
- DM35418_Board_Close
 - DM35418 Board Access Structures, [40](#)
- DM35418_Board_Descriptor, [127](#)
 - file_descriptor, [127](#)
 - isr, [127](#)
 - pid, [128](#)
- DM35418_Board_Open
 - DM35418 Board Access Structures, [41](#)
- DM35418_Check_Result
 - DM35418 Utility Library Functions, [123](#)
- DM35418_CLK_BUS2
 - dm35418_types.h, [248](#)
- DM35418_CLK_BUS3
 - dm35418_types.h, [248](#)
- DM35418_CLK_BUS4
 - dm35418_types.h, [248](#)
- DM35418_CLK_BUS5
 - dm35418_types.h, [248](#)
- DM35418_CLK_BUS6
 - dm35418_types.h, [248](#)
- DM35418_CLK_BUS7
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_BUS2
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_BUS2_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_BUS3
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_BUS3_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_BUS4
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_BUS4_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_BUS5
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_BUS5_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_BUS6
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_BUS6_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_BUS7
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_BUS7_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_CHAN_THRESH
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_CHAN_THRESH_INV
 - dm35418_types.h, [249](#)
- DM35418_CLK_SRC_IMMEDIATE
 - dm35418_types.h, [248](#)
- DM35418_CLK_SRC_NEVER
 - dm35418_types.h, [248](#)
- DM35418_Clock_Buses
 - dm35418_types.h, [248](#)
- DM35418_Clock_Sources
 - dm35418_types.h, [248](#)
- dm35418_dac.c
 - DEFAULT_DAC_NUM, [197](#)
 - main, [198](#)
 - program_name, [199](#)
- DM35418_Dac_Channel_Clear_Marker_Status
 - DM35418 DAC Library Public Functions, [50](#)

- DM35418_Dac_Channel_Get_Marker_Config
 - DM35418 DAC Library Public Functions, [50](#)
- DM35418_Dac_Channel_Get_Marker_Status
 - DM35418 DAC Library Public Functions, [51](#)
- DM35418_Dac_Channel_Reset
 - DM35418 DAC Library Public Functions, [51](#)
- DM35418_Dac_Channel_Set_Marker_Config
 - DM35418 DAC Library Public Functions, [52](#)
- DM35418_Dac_Channel_Setup
 - DM35418 DAC Library Public Functions, [52](#)
- DM35418_DAC_CLK_BUS_SRC_CHAN_MARKER
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_CLK_BUS_SRC_CONV_COMPL
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_CLK_BUS_SRC_CONVERSION_SENT
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_CLK_BUS_SRC_DISABLE
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_CLK_BUS_SRC_START_TRIG
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_CLK_BUS_SRC_STOP_TRIG
 - DM35418 DAC Library Constants, [47](#)
- DM35418_Dac_Clock_Events
 - DM35418 DAC Library Constants, [47](#)
- DM35418_Dac_Conv_To_Volts
 - DM35418 DAC Library Public Functions, [53](#)
- dm35418_dac_dma.c
 - BUFFER_SIZE_BYTES, [201](#)
 - BUFFER_SIZE_SAMPLES, [201](#)
 - DEFAULT_DAC_TO_USE, [201](#)
 - DEFAULT_RATE, [201](#)
 - exit_program, [202](#)
 - main, [201](#)
 - NUM_BUFFERS_TO_USE, [201](#)
 - program_name, [202](#)
 - sigint_handler, [202](#)
- DM35418_Dac_Fifo_Channel_Write
 - DM35418 DAC Library Public Functions, [53](#)
- DM35418_Dac_Get_Clock_Div
 - DM35418 DAC Library Public Functions, [54](#)
- DM35418_Dac_Get_Clock_Src
 - DM35418 DAC Library Public Functions, [54](#)
- DM35418_Dac_Get_Conversion_Count
 - DM35418 DAC Library Public Functions, [55](#)
- DM35418_Dac_Get_Last_Conversion
 - DM35418 DAC Library Public Functions, [55](#)
- DM35418_Dac_Get_Mode_Status
 - DM35418 DAC Library Public Functions, [57](#)
- DM35418_Dac_Get_Post_Stop_Conversion_Count
 - DM35418 DAC Library Public Functions, [57](#)
- DM35418_Dac_Get_Start_Trigger
 - DM35418 DAC Library Public Functions, [58](#)
- DM35418_Dac_Get_Stop_Trigger
 - DM35418 DAC Library Public Functions, [58](#)
- DM35418_Dac_Interrupt_Clear_Status
 - DM35418 DAC Library Public Functions, [59](#)
- DM35418_Dac_Interrupt_Get_Config
 - DM35418 DAC Library Public Functions, [59](#)
- DM35418_Dac_Interrupt_Get_Status
 - DM35418 DAC Library Public Functions, [60](#)
- DM35418_Dac_Interrupt_Set_Config
 - DM35418 DAC Library Public Functions, [60](#)
- DM35418_Dac_Open
 - DM35418 DAC Library Public Functions, [61](#)
- DM35418_Dac_Pause
 - DM35418 DAC Library Public Functions, [61](#)
- DM35418_Dac_Reset
 - DM35418 DAC Library Public Functions, [62](#)
- DM35418_DAC_RNG_BIPOLAR_10V
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_RNG_BIPOLAR_2_5V
 - DM35418 DAC Library Constants, [47](#)
- DM35418_DAC_RNG_BIPOLAR_5V
 - DM35418 DAC Library Constants, [47](#)
- DM35418_Dac_Set_Clock_Div
 - DM35418 DAC Library Public Functions, [62](#)
- DM35418_Dac_Set_Clock_Source_Global
 - DM35418 DAC Library Public Functions, [63](#)
- DM35418_Dac_Set_Clock_Src
 - DM35418 DAC Library Public Functions, [63](#)
- DM35418_Dac_Set_Conversion_Rate
 - DM35418 DAC Library Public Functions, [64](#)
- DM35418_Dac_Set_Last_Conversion
 - DM35418 DAC Library Public Functions, [64](#)
- DM35418_Dac_Set_Post_Stop_Conversion_Count
 - DM35418 DAC Library Public Functions, [65](#)
- DM35418_Dac_Set_Start_Trigger
 - DM35418 DAC Library Public Functions, [65](#)
- DM35418_Dac_Set_Stop_Trigger
 - DM35418 DAC Library Public Functions, [66](#)
- DM35418_Dac_Start
 - DM35418 DAC Library Public Functions, [66](#)
- dm35418_dac_synbus.c
 - BUFFER_SIZE_BYTES, [205](#)
 - BUFFER_SIZE_SAMPLES, [205](#)
 - DEFAULT_DAC_TO_USE, [205](#)
 - DEFAULT_RATE, [205](#)
 - exit_program, [208](#)
 - main, [206](#)
 - NUM_BUFFERS_TO_USE, [205](#)
 - program_name, [209](#)
 - setup_dac, [207](#)
 - setup_synbus, [207](#)
 - sigint_handler, [208](#)
- DM35418_Dac_Volts_To_Conv
 - DM35418 DAC Library Public Functions, [67](#)
- dm35418_device_descriptor, [128](#)
 - cdev, [129](#)
 - device_index, [129](#)
 - device_lock, [129](#)
 - dma_descr_list, [129](#)
 - dma_wait_queue, [129](#)
 - int_queue_count, [130](#)
 - int_queue_in_marker, [130](#)
 - int_queue_missed, [130](#)
 - int_queue_out_marker, [130](#)

- int_wait_queue, [130](#)
- interrupt_fb, [131](#)
- irq_number, [131](#)
- name, [131](#)
- pci, [131](#)
- pdev, [131](#)
- reference_count, [132](#)
- remove_isr_flag, [132](#)
- dm35418_dio.c
 - main, [210](#)
 - program_name, [211](#)
- DM35418_Dio_Get_Direction
 - DM35418 DIO Public, [68](#)
- DM35418_Dio_Get_Input_Value
 - DM35418 DIO Public, [68](#)
- DM35418_Dio_Get_Output_Value
 - DM35418 DIO Public, [69](#)
- DM35418_Dio_Open
 - DM35418 DIO Public, [69](#)
- DM35418_Dio_Set_Direction
 - DM35418 DIO Public, [70](#)
- DM35418_Dio_Set_Output_Value
 - DM35418 DIO Public, [70](#)
- DM35418_Dma
 - DM35418 Board Access Structures, [41](#)
- DM35418_Dma_Buffer_Setup
 - DM35418 DMA Public Library Functions, [74](#)
- DM35418_Dma_Buffer_Status
 - DM35418 DMA Public Library Functions, [75](#)
- DM35418_Dma_Check_Buffer_Used
 - DM35418 DMA Public Library Functions, [76](#)
- DM35418_Dma_Check_For_Error
 - DM35418 DMA Public Library Functions, [76](#)
- DM35418_Dma_Clear
 - DM35418 DMA Public Library Functions, [78](#)
- DM35418_Dma_Clear_Interrupt
 - DM35418 DMA Public Library Functions, [78](#)
- DM35418_Dma_Configure_Interrupts
 - DM35418 DMA Public Library Functions, [79](#)
- DM35418_DMA_Descriptor, [132](#)
 - buffer_start_offset, [133](#)
 - control_offset, [133](#)
 - num_buffers, [133](#)
- dm35418_dma_descriptor, [133](#)
 - buffer, [134](#)
 - buffer_size, [134](#)
 - bus_addr, [134](#)
 - channel, [134](#)
 - fb_num, [134](#)
 - list, [135](#)
 - virt_addr, [135](#)
- DM35418_Dma_Find_Interrupt
 - DM35418 DMA Public Library Functions, [80](#)
- DM35418_DMA_FUNCTIONS
 - DM35418 PCI Region Structures, [44](#)
- DM35418_Dma_Get_Current_Buffer_Count
 - DM35418 DMA Public Library Functions, [81](#)
- DM35418_Dma_Get_Errors
 - DM35418 DMA Public Library Functions, [81](#)
- DM35418_Dma_Get_Fifo_Counts
 - DM35418 DMA Public Library Functions, [82](#)
- DM35418_Dma_Get_Fifo_State
 - DM35418 DMA Public Library Functions, [83](#)
- DM35418_Dma_Get_Interrupt_Configuration
 - DM35418 DMA Public Library Functions, [83](#)
- DM35418_DMA_INITIALIZE
 - DM35418 PCI Region Structures, [44](#)
- DM35418_Dma_Initialize
 - DM35418 Board Access Public Library Functions, [101](#)
- DM35418_Dma_Pause
 - DM35418 DMA Public Library Functions, [84](#)
- DM35418_DMA_READ
 - DM35418 PCI Region Structures, [44](#)
- DM35418_Dma_Read
 - DM35418 Board Access Public Library Functions, [102](#)
- DM35418_Dma_Reset_Buffer
 - DM35418 DMA Public Library Functions, [84](#)
- DM35418_Dma_Setup
 - DM35418 DMA Public Library Functions, [85](#)
- DM35418_Dma_Setup_Set_Direction
 - DM35418 DMA Public Library Functions, [86](#)
- DM35418_Dma_Setup_Set_Used
 - DM35418 DMA Public Library Functions, [86](#)
- DM35418_Dma_Start
 - DM35418 DMA Public Library Functions, [87](#)
- DM35418_Dma_Status
 - DM35418 DMA Public Library Functions, [87](#)
- DM35418_Dma_Stop
 - DM35418 DMA Public Library Functions, [89](#)
- DM35418_DMA_WRITE
 - DM35418 PCI Region Structures, [44](#)
- DM35418_Dma_Write
 - DM35418 Board Access Public Library Functions, [103](#)
- DM35418_FIFO_EMPTY
 - DM35418 DMA Public Library Constants, [73](#)
- DM35418_FIFO_FULL
 - DM35418 DMA Public Library Constants, [73](#)
- DM35418_FIFO_HAS_DATA
 - DM35418 DMA Public Library Constants, [73](#)
- DM35418_Fifo_States
 - DM35418 DMA Public Library Constants, [72](#)
- DM35418_FIFO_UNKNOWN
 - DM35418 DMA Public Library Constants, [73](#)
- DM35418_Function_Block, [135](#)
 - control_offset, [136](#)
 - dma_channel, [136](#)
 - dma_offset, [136](#)
 - fb_num, [136](#)
 - fb_offset, [136](#)
 - num_dma_buffers, [137](#)
 - num_dma_channels, [137](#)
 - ordinal_fb_type_num, [137](#)
 - sub_type, [137](#)

- type, [137](#)
- type_revision, [138](#)
- DM35418_Function_Block_Open
 - DM35418 Board Library Public Functions, [96](#)
- DM35418_Function_Block_Open_Module
 - DM35418 Board Library Public Functions, [96](#)
- DM35418_Gains
 - DM35418 ADC Library Constants, [11](#)
- DM35418_Gbc_Ack_Interrupt
 - DM35418 Board Library Public Functions, [97](#)
- DM35418_Gbc_Board_Reset
 - DM35418 Board Library Public Functions, [97](#)
- DM35418_Gbc_Get_Format
 - DM35418 Board Library Public Functions, [98](#)
- DM35418_Gbc_Get_Fpga_Build
 - DM35418 Board Library Public Functions, [98](#)
- DM35418_Gbc_Get_Pdp_Number
 - DM35418 Board Library Public Functions, [99](#)
- DM35418_Gbc_Get_Revision
 - DM35418 Board Library Public Functions, [99](#)
- DM35418_Gbc_Get_Sys_Clock_Freq
 - DM35418 Board Library Public Functions, [99](#)
- DM35418_General_InstallISR
 - DM35418 Board Access Public Library Functions, [103](#)
- DM35418_General_RemoveISR
 - DM35418 Board Access Public Library Functions, [104](#)
- DM35418_General_SetISRPriority
 - DM35418 Board Access Public Library Functions, [104](#)
- DM35418_General_WaitForInterrupt
 - DM35418 Board Access Public Library Functions, [105](#)
- DM35418_Generate_Signal_Data
 - DM35418 Utility Library Functions, [124](#)
- DM35418_Get_Maskable
 - DM35418 Utility Library Functions, [124](#)
- DM35418_Get_Time_Diff
 - DM35418 Utility Library Functions, [125](#)
- DM35418_Input_Mode
 - DM35418 ADC Library Constants, [11](#)
- DM35418_Input_Ranges
 - DM35418 ADC Library Constants, [12](#)
- dm35418_ioctl_argument, [138](#)
 - dma, [138](#)
 - interrupt, [139](#)
 - modify, [139](#)
 - readwrite, [139](#)
- dm35418_ioctl_dma, [139](#)
 - buffer, [140](#)
 - buffer_ptr, [140](#)
 - buffer_size, [140](#)
 - channel, [140](#)
 - fb_num, [141](#)
 - function, [141](#)
 - num_buffers, [141](#)
 - pci, [141](#)
- dm35418_ioctl_interrupt_info_request, [142](#)
 - error_occurred, [142](#)
 - interrupt_fb, [142](#)
 - interrupts_remaining, [142](#)
 - valid_interrupt, [143](#)
- dm35418_ioctl_region_modify, [143](#)
 - access, [143](#)
 - mask, [144](#)
 - mask16, [144](#)
 - mask32, [144](#)
 - mask8, [144](#)
- dm35418_ioctl_region_readwrite, [145](#)
 - access, [145](#)
- dm35418_list_fb.c
 - main, [212](#)
 - program_name, [213](#)
- DM35418_Micro_Sleep
 - DM35418 Utility Library Functions, [125](#)
- DM35418_Modify
 - DM35418 Board Access Structures, [42](#)
- DM35418_OFFSET_FB_ADC_FIFO
 - DM35418 Register Offsets, [110](#)
- DM35418_Output_Ranges
 - DM35418 DAC Library Constants, [47](#)
- dm35418_pci_access_request, [145](#)
 - data, [146](#)
 - data16, [146](#)
 - data32, [146](#)
 - data8, [146](#)
 - offset, [146](#)
 - region, [147](#)
 - size, [147](#)
- dm35418_pci_region, [147](#)
 - allocated, [148](#)
 - io_addr, [148](#)
 - length, [148](#)
 - phys_addr, [148](#)
 - virt_addr, [148](#)
- DM35418_PCI_REGION_ACCESS_16
 - DM35418 PCI Region Structures, [45](#)
- DM35418_PCI_REGION_ACCESS_32
 - DM35418 PCI Region Structures, [45](#)
- DM35418_PCI_REGION_ACCESS_8
 - DM35418 PCI Region Structures, [45](#)
- dm35418_pci_region_access_dir
 - DM35418 Driver Enumerations, [90](#)
- DM35418_PCI_REGION_ACCESS_READ
 - DM35418 Driver Enumerations, [90](#)
- dm35418_pci_region_access_size
 - DM35418 PCI Region Structures, [44](#)
- DM35418_PCI_REGION_ACCESS_WRITE
 - DM35418 Driver Enumerations, [90](#)
- DM35418_PCI_REGION_FB
 - DM35418 PCI Region Structures, [45](#)
- DM35418_PCI_REGION_GBC
 - DM35418 PCI Region Structures, [45](#)
- DM35418_PCI_REGION_GBC2
 - DM35418 PCI Region Structures, [45](#)

- dm35418_pci_region_num
 - DM35418 PCI Region Structures, [45](#)
- DM35418_Read
 - DM35418 Board Access Structures, [42](#)
- DM35418_SAWTOOTH_WAVE
 - DM35418 Utility Library Functions, [123](#)
- DM35418_SINE_WAVE
 - DM35418 Utility Library Functions, [123](#)
- DM35418_SQUARE_WAVE
 - DM35418 Utility Library Functions, [123](#)
- DM35418_Syncbus_Clock_Source_Global_Get
 - DM35418 Sync Bus Library Public Functions, [112](#)
- DM35418_Syncbus_Clock_Source_Global_Set
 - DM35418 Sync Bus Library Public Functions, [113](#)
- DM35418_Syncbus_Disable
 - DM35418 Sync Bus Library Public Functions, [113](#)
- DM35418_Syncbus_Drive_Source_Get
 - DM35418 Sync Bus Library Public Functions, [114](#)
- DM35418_Syncbus_Drive_Source_Set
 - DM35418 Sync Bus Library Public Functions, [114](#)
- DM35418_Syncbus_Enable
 - DM35418 Sync Bus Library Public Functions, [115](#)
- DM35418_Syncbus_Get_Clk_Sel
 - DM35418 Sync Bus Library Public Functions, [116](#)
- DM35418_Syncbus_Get_PLL_Locked
 - DM35418 Sync Bus Library Public Functions, [116](#)
- DM35418_Syncbus_Open
 - DM35418 Sync Bus Library Public Functions, [116](#)
- DM35418_Syncbus_Set_Clk_Sel
 - DM35418 Sync Bus Library Public Functions, [117](#)
- DM35418_Syncbus_Set_Direction
 - DM35418 Sync Bus Library Public Functions, [117](#)
- DM35418_Syncbus_Termination_Disable
 - DM35418 Sync Bus Library Public Functions, [118](#)
- DM35418_Syncbus_Termination_Enable
 - DM35418 Sync Bus Library Public Functions, [119](#)
- dm35418_temperature.c
 - exit_program, [215](#)
 - main, [214](#)
 - program_name, [215](#)
 - sigint_handler, [215](#)
- DM35418_Temperature_Open
 - DM35418 Temperature, [120](#)
- DM35418_Temperature_Read
 - DM35418 Temperature, [120](#)
- dm35418_types.h
 - DM35418_CLK_BUS2, [248](#)
 - DM35418_CLK_BUS3, [248](#)
 - DM35418_CLK_BUS4, [248](#)
 - DM35418_CLK_BUS5, [248](#)
 - DM35418_CLK_BUS6, [248](#)
 - DM35418_CLK_BUS7, [248](#)
 - DM35418_CLK_SRC_BUS2, [248](#)
 - DM35418_CLK_SRC_BUS2_INV, [249](#)
 - DM35418_CLK_SRC_BUS3, [248](#)
 - DM35418_CLK_SRC_BUS3_INV, [249](#)
 - DM35418_CLK_SRC_BUS4, [248](#)
 - DM35418_CLK_SRC_BUS4_INV, [249](#)
 - DM35418_CLK_SRC_BUS5, [248](#)
 - DM35418_CLK_SRC_BUS5_INV, [249](#)
 - DM35418_CLK_SRC_BUS6, [248](#)
 - DM35418_CLK_SRC_BUS6_INV, [249](#)
 - DM35418_CLK_SRC_BUS7, [249](#)
 - DM35418_CLK_SRC_BUS7_INV, [249](#)
 - DM35418_CLK_SRC_CHAN_THRESH, [249](#)
 - DM35418_CLK_SRC_CHAN_THRESH_INV, [249](#)
 - DM35418_CLK_SRC_IMMEDIATE, [248](#)
 - DM35418_CLK_SRC_NEVER, [248](#)
 - DM35418_Clock_Buses, [248](#)
 - DM35418_Clock_Sources, [248](#)
- DM35418_Waveforms
 - DM35418 Utility Library Functions, [123](#)
- DM35418_Write
 - DM35418 Board Access Structures, [43](#)
- DM35418LIB_API
 - dm35418_board_access.h, [224](#)
- dma
 - dm35418_ioctl_argument, [138](#)
- dma_channel
 - DM35418_Function_Block, [136](#)
- dma_descr_list
 - dm35418_device_descriptor, [129](#)
- dma_has_error
 - dm35418_adc_continuous_dma.c, [166](#)
 - dm35418_adc_sync_trig_dma.c, [177](#)
 - dm35418_adc_thresh_dma.c, [186](#)
- dma_offset
 - DM35418_Function_Block, [136](#)
- dma_wait_queue
 - dm35418_device_descriptor, [129](#)
- DUMP_OPTION
 - DM35418 Example Programs Constants, [94](#)
- error_occurred
 - dm35418_ioctl_interrupt_info_request, [142](#)
- examples/dm35418_adc.c, [149](#)
- examples/dm35418_adc_basic_ints.c, [153](#)
- examples/dm35418_adc_continuous_dma.c, [158](#)
- examples/dm35418_adc_sync_trig_dma.c, [168](#)
- examples/dm35418_adc_thresh_dma.c, [179](#)
- examples/dm35418_adc_to_dac_dma.c, [187](#)
- examples/dm35418_dac.c, [196](#)
- examples/dm35418_dac_dma.c, [199](#)
- examples/dm35418_dac_syncbus.c, [203](#)
- examples/dm35418_dio.c, [209](#)
- examples/dm35418_list_fb.c, [211](#)
- examples/dm35418_temperature.c, [213](#)
- exit_program
 - dm35418_adc.c, [152](#)
 - dm35418_adc_basic_ints.c, [157](#)
 - dm35418_adc_continuous_dma.c, [166](#)
 - dm35418_adc_sync_trig_dma.c, [177](#)
 - dm35418_adc_thresh_dma.c, [186](#)
 - dm35418_adc_to_dac_dma.c, [195](#)
 - dm35418_dac_dma.c, [202](#)
 - dm35418_dac_syncbus.c, [208](#)
 - dm35418_temperature.c, [215](#)

EXTERNAL_OPTION

DM35418 Example Programs Constants, 94

fb_num

dm35418_dma_descriptor, 134
 DM35418_Function_Block, 136
 dm35418_ioctl_dma, 141

fb_offset

DM35418_Function_Block, 136

file_descriptor

DM35418_Board_Descriptor, 127

FILE_OPTION

DM35418 Example Programs Constants, 93

function

dm35418_ioctl_dma, 141

HELP_OPTION

DM35418 Example Programs Constants, 93

Help_Options

DM35418 Example Programs Constants, 93

HOURS_OPTION

DM35418 Example Programs Constants, 94

include/dm35418.h, 216

include/dm35418_adc_library.h, 217

include/dm35418_board_access.h, 223

include/dm35418_board_access_structs.h, 224

include/dm35418_dac_library.h, 225

include/dm35418_dio_library.h, 228

include/dm35418_dma_library.h, 229

include/dm35418_driver.h, 232

include/dm35418_examples.h, 233

include/dm35418_gbc_library.h, 236

include/dm35418_ioctl.h, 237

include/dm35418_os.h, 237

include/dm35418_registers.h, 238

include/dm35418_syncbus_library.h, 243

include/dm35418_temperature_library.h, 245

include/dm35418_types.h, 246

include/dm35418_util_library.h, 249

int_queue_count

dm35418_device_descriptor, 130

int_queue_in_marker

dm35418_device_descriptor, 130

int_queue_missed

dm35418_device_descriptor, 130

int_queue_out_marker

dm35418_device_descriptor, 130

int_wait_queue

dm35418_device_descriptor, 130

interrupt

dm35418_ioctl_argument, 139

interrupt_count

dm35418_adc.c, 152

interrupt_fb

dm35418_device_descriptor, 131
 dm35418_ioctl_interrupt_info_request, 142

interrupts_remaining

dm35418_ioctl_interrupt_info_request, 142

io_addr

dm35418_pci_region, 148

irq_number

dm35418_device_descriptor, 131

ISR

dm35418_adc_continuous_dma.c, 162
 dm35418_adc_sync_trig_dma.c, 172
 dm35418_adc_thresh_dma.c, 182
 dm35418_adc_to_dac_dma.c, 190

isr

DM35418_Board_Descriptor, 127

length

dm35418_pci_region, 148

list

dm35418_dma_descriptor, 135

local_buffer

dm35418_adc_continuous_dma.c, 167
 dm35418_adc_sync_trig_dma.c, 177
 dm35418_adc_thresh_dma.c, 186
 dm35418_adc_to_dac_dma.c, 195

LOW_THRESHOLD_OPTION

DM35418 Example Programs Constants, 94

main

dm35418_adc.c, 151
 dm35418_adc_basic_ints.c, 156
 dm35418_adc_continuous_dma.c, 162
 dm35418_adc_sync_trig_dma.c, 173
 dm35418_adc_thresh_dma.c, 183
 dm35418_adc_to_dac_dma.c, 190
 dm35418_dac.c, 198
 dm35418_dac_dma.c, 201
 dm35418_dac_syncbus.c, 206
 dm35418_dio.c, 210
 dm35418_list_fb.c, 212
 dm35418_temperature.c, 214

mask

dm35418_ioctl_region_modify, 144

mask16

dm35418_ioctl_region_modify, 144

mask32

dm35418_ioctl_region_modify, 144

mask8

dm35418_ioctl_region_modify, 144

MASTER_OPTION

DM35418 Example Programs Constants, 94

MINOR_OPTION

DM35418 Example Programs Constants, 93

MODE_OPTION

DM35418 Example Programs Constants, 94

modify

dm35418_ioctl_argument, 139

my_adc

dm35418_adc_basic_ints.c, 157
 dm35418_adc_continuous_dma.c, 167
 dm35418_adc_sync_trig_dma.c, 178
 dm35418_adc_thresh_dma.c, 186
 dm35418_adc_to_dac_dma.c, 195

- name
 - dm35418_device_descriptor, [131](#)
- next_buffer
 - dm35418_adc_continuous_dma.c, [167](#)
 - dm35418_adc_sync_trig_dma.c, [178](#)
 - dm35418_adc_thresh_dma.c, [187](#)
 - dm35418_adc_to_dac_dma.c, [195](#)
- NOSTOP_OPTION
 - DM35418 Example Programs Constants, [94](#)
- num_buffers
 - DM35418_DMA_Descriptor, [133](#)
 - dm35418_ioctl_dma, [141](#)
- NUM_BUFFERS_TO_USE
 - dm35418_dac_dma.c, [201](#)
 - dm35418_dac_syncbus.c, [205](#)
- num_dma_buffers
 - DM35418_Function_Block, [137](#)
- num_dma_channels
 - DM35418_Function_Block, [137](#)
- NUM_FB_TO_USE
 - dm35418_adc_to_dac_dma.c, [190](#)
- NUM_OPTION
 - DM35418 Example Programs Constants, [94](#)
- NUM_SAMPLES_PER_TRIGGER
 - dm35418_adc_sync_trig_dma.c, [172](#)
 - dm35418_adc_thresh_dma.c, [182](#)
- offset
 - dm35418_pci_access_request, [146](#)
- OFILE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- ordinal_fb_type_num
 - DM35418_Function_Block, [137](#)
- OUTPUT_ADC_OPTION
 - DM35418 Example Programs Constants, [94](#)
- output_channel_status
 - dm35418_adc_continuous_dma.c, [163](#)
 - dm35418_adc_sync_trig_dma.c, [173](#)
 - dm35418_adc_thresh_dma.c, [183](#)
 - dm35418_adc_to_dac_dma.c, [191](#)
- OUTPUT_RMS_OPTION
 - DM35418 Example Programs Constants, [94](#)
- PACKED_OPTION
 - DM35418 Example Programs Constants, [94](#)
- PATTERN_OPTION
 - DM35418 Example Programs Constants, [94](#)
- pci
 - dm35418_device_descriptor, [131](#)
 - dm35418_ioctl_dma, [141](#)
- pdev
 - dm35418_device_descriptor, [131](#)
- phys_addr
 - dm35418_pci_region, [148](#)
- pid
 - DM35418_Board_Descriptor, [128](#)
- PORT_OPTION
 - DM35418 Example Programs Constants, [94](#)
- program_name
 - dm35418_adc.c, [153](#)
 - dm35418_adc_basic_ints.c, [158](#)
 - dm35418_adc_continuous_dma.c, [167](#)
 - dm35418_adc_sync_trig_dma.c, [178](#)
 - dm35418_adc_thresh_dma.c, [187](#)
 - dm35418_adc_to_dac_dma.c, [196](#)
 - dm35418_dac.c, [199](#)
 - dm35418_dac_dma.c, [202](#)
 - dm35418_dac_syncbus.c, [209](#)
 - dm35418_dio.c, [211](#)
 - dm35418_list_fb.c, [213](#)
 - dm35418_temperature.c, [215](#)
- RANGE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- RATE_OPTION
 - DM35418 Example Programs Constants, [93](#)
- readwrite
 - dm35418_ioctl_argument, [139](#)
- RECEIVER_OPTION
 - DM35418 Example Programs Constants, [94](#)
- REF_NUM_OPTION
 - DM35418 Example Programs Constants, [94](#)
- REFCLK_OPTION
 - DM35418 Example Programs Constants, [94](#)
- reference_count
 - dm35418_device_descriptor, [132](#)
- REFILL_FIFO_OPTION
 - DM35418 Example Programs Constants, [94](#)
- region
 - dm35418_pci_access_request, [147](#)
- remove_isr_flag
 - dm35418_device_descriptor, [132](#)
- sample_count
 - dm35418_adc_basic_ints.c, [158](#)
- SAMPLES_OPTION
 - DM35418 Example Programs Constants, [94](#)
- SENDER_OPTION
 - DM35418 Example Programs Constants, [94](#)
- setup_adc
 - dm35418_adc_sync_trig_dma.c, [174](#)
 - dm35418_adc_thresh_dma.c, [184](#)
- setup_adcs
 - dm35418_adc_continuous_dma.c, [163](#)
 - dm35418_adc_to_dac_dma.c, [192](#)
- setup_ctrlc_handler
 - dm35418_adc_continuous_dma.c, [164](#)
- setup_dac
 - dm35418_adc_sync_trig_dma.c, [174](#)
 - dm35418_adc_thresh_dma.c, [184](#)
 - dm35418_dac_syncbus.c, [207](#)
- setup_dacs
 - dm35418_adc_continuous_dma.c, [164](#)
 - dm35418_adc_to_dac_dma.c, [192](#)
- setup_dio
 - dm35418_adc_sync_trig_dma.c, [175](#)
- setup_isr
 - dm35418_adc_to_dac_dma.c, [193](#)

- setup_slave_adcs
 - dm35418_adc_continuous_dma.c, [165](#)
- setup_syncbus
 - dm35418_adc_sync_trig_dma.c, [175](#)
 - dm35418_dac_syncbus.c, [207](#)
- sigint_handler
 - dm35418_adc.c, [152](#)
 - dm35418_adc_basic_ints.c, [157](#)
 - dm35418_adc_continuous_dma.c, [165](#)
 - dm35418_adc_sync_trig_dma.c, [176](#)
 - dm35418_adc_thresh_dma.c, [185](#)
 - dm35418_adc_to_dac_dma.c, [193](#)
 - dm35418_dac_dma.c, [202](#)
 - dm35418_dac_syncbus.c, [208](#)
 - dm35418_temperature.c, [215](#)
- size
 - dm35418_pci_access_request, [147](#)
- SIZE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- SLAVE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- START_OPTION
 - DM35418 Example Programs Constants, [93](#)
- start_slave_adcs
 - dm35418_adc_to_dac_dma.c, [193](#)
- STORE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- sub_type
 - DM35418_Function_Block, [137](#)
- SYNC_CONN_OPTION
 - DM35418 Example Programs Constants, [94](#)
- SYNC_TERM_OPTION
 - DM35418 Example Programs Constants, [94](#)
- SYNCBUS_OPTION
 - DM35418 Example Programs Constants, [94](#)
- SYNCBUS_TRIGGER_OFF_USEC
 - dm35418_adc_sync_trig_dma.c, [172](#)
- SYNCBUS_TRIGGER_ON_USEC
 - dm35418_adc_sync_trig_dma.c, [172](#)
- TERM_OPTION
 - DM35418 Example Programs Constants, [94](#)
- TEST_OPTION
 - DM35418 Example Programs Constants, [94](#)
- TOTAL_INT_SRC_MASK
 - dm35418_adc_basic_ints.c, [155](#)
- type
 - DM35418_Function_Block, [137](#)
- type_revision
 - DM35418_Function_Block, [138](#)
- USER_ID_OPTION
 - DM35418 Example Programs Constants, [94](#)
- valid_interrupt
 - dm35418_ioctl_interrupt_info_request, [143](#)
- VERBOSE_OPTION
 - DM35418 Example Programs Constants, [94](#)
- virt_addr
 - dm35418_dma_descriptor, [135](#)
 - dm35418_pci_region, [148](#)
- WAVE_OPTION
 - DM35418 Example Programs Constants, [94](#)